



Universidade do Minho

Escola de Engenharia

Licenciatura em Engenharia Informática

Unidade Curricular de Laboratórios de Informática IV

Ano Letivo de 2025/2026

Fix'n'Ride

Diogo Esteves – A104004

Diogo Fernandes – A104260

Jorge Fernandes – A104168

Rodrigo Fernandes – A104175

21 de maio de 2026

Data de Receção	
Responsável	
Avaliação	
Observações	

Fix'n'Ride

Resumo

O presente relatório documenta o ciclo completo de desenvolvimento do sistema Fix'n'Ride, uma solução de gestão integrada concebida para a oficina de micromobilidade elétrica MobiFix. No contexto atual de expansão das Smart Cities, a digitalização dos serviços de assistência técnica tornou-se um imperativo estratégico para superar a ineficiência dos processos manuais e a fragmentação de dados em folhas de cálculo.

O levantamento de necessidades foi realizado através de uma abordagem mista, integrando questionários, observação direta e entrevistas estruturadas com os principais stakeholders: administração, técnicos e operadores de loja. Deste processo resultou a especificação detalhada de 22 requisitos funcionais e 5 requisitos não funcionais, garantindo a cobertura de todo o fluxo operacional. O sistema permite a gestão integral de ordens de serviço baseadas num catálogo de intervenções com preços fixos, assegurando transparência orçamental ao cliente e eficiência na faturação.

A arquitetura implementada assenta numa stack de três camadas independentes: uma *Data API* em Node.js/Express com persistência em MongoDB, uma camada de Lógica de Negócios em ASP.NET Core (.NET 8) que orquestra as regras de domínio, e um frontend *Single Page Application* em React com Tailwind CSS, servindo quatro portais distintos por perfil de utilizador (Cliente, Mecânico, Operador e Administrador). A separação deliberada entre a camada de persistência e a camada de negócios, comunicando exclusivamente via HTTP com autenticação por chave interna, constitui a principal decisão arquitetural do sistema.

A solução cobre os fluxos operacionais críticos da MobiFix: diagnóstico e reparação de trotinetes com catálogo de intervenções, gestão de stock com alertas automáticos de reposição, venda direta ao balcão, *click & collect* de peças reservadas, faturação atómica com suporte a devoluções e notas de crédito, e um dashboard de KPIs financeiros e operacionais para a administração. A qualidade do sistema foi avaliada segundo o modelo ISO/IEC 25010, complementada por testes de integração com coleção Postman e testes de *stress* com métricas P50/P95/P99.

Área de Aplicação: Engenharia de Software, Sistemas de Informação, Gestão de Oficinas.

Palavras-Chave: LLM, .NET, C#, NoSQL, MongoDB, React, TanStack Query, UML, Gestão de Reparações, Inteligência Artificial Generativa.

Índice

1	Introdução e apresentação do trabalho	1
1.1	Contextualização	1
1.2	Motivação e Objetivos	2
1.3	Apresentação do Caso de Estudo	2
1.4	Análise da Viabilidade	3
1.5	Planeamento do Projeto	4
2	Engenharia de requisitos	6
2.1	Identificação de <i>Stakeholders</i>	6
2.2	Cenários de Utilização	7
2.2.1	Agendamento e Diagnóstico da Avaria	7
2.2.2	Gestão de Inventário e Reposição Automática	7
2.2.3	Fluxo de Venda Direta e Devolução	8
2.2.4	Análise de Performance e Rentabilidade	8
2.3	<i>User Stories</i>	9
2.3.1	Gestão de Oficina	9
2.3.2	Gestão de <i>Stock</i> e Compras	9
2.3.3	Administração e Faturação	9
2.3.4	Gestão de Conta e Veículos	10
2.3.5	Agendamento e Acompanhamento	10
2.3.6	Compras e Documentação	10
2.4	Métodos de Levantamento Utilizados	10
2.5	Entrevistas com Stakeholders	11
2.5.1	Entrevista com a Administradora Ana	11
2.5.2	Entrevista com o Operador de Loja João	12
2.5.3	Entrevista com o Mecânico Pedro	12
2.6	Requisitos Funcionais	12
2.7	Requisitos não Funcionais	34
2.8	Refinamento de Requisitos Ambíguos	38
2.9	Análise de Restrições	38
2.10	Especificação e Modelação do Sistema	39
2.10.1	Métodos de modelação adotados	39
2.10.2	Modelação do domínio do problema	40
2.10.3	Levantamento e especificação de Casos de Uso	44

3	Especificação do Software	66
3.1	Perspetiva do Produto	66
3.2	Ambiente Operacional	66
3.3	Diagramas de Atividades	67
3.3.1	Fluxos de Atendimento e Diagnóstico	67
3.3.2	Gestão de Inventário e Reposição	69
3.3.3	Execução e Faturação	71
3.4	Diagramas de Máquina de Estados	73
3.5	Reflexão acerca da utilização de LLMs na conceção e engenharia de requisitos do sistema Fix'N'Ride	75
4	Arquitetura e Design do Software	76
4.1	Introdução	76
4.2	Visão Geral da Arquitetura do Sistema	77
4.2.1	Padrões Arquiteturais Adotados	77
4.3	Justificação da <i>Stack</i> Tecnológica	78
4.4	Configuração do Ambiente de Desenvolvimento	80
4.4.1	Serviços Docker e Topologia da Rede Interna	81
4.4.2	Persistência de Dados e Inicialização Automática da Base de Dados	82
4.4.3	Gestão de Configurações e Variáveis de Ambiente	84
4.5	Síntese do Capítulo	85
5	Arquitetura da Camada de Dados	87
5.1	Justificação da Escolha do MongoDB	87
5.2	Estratégias de Modelação: <i>Embedding</i> vs. <i>Referencing</i>	88
5.3	Dicionário de Dados	91
5.3.1	Domínio de Utilizadores	91
5.3.2	Domínio de Serviços	93
5.3.3	Domínio Financeiro	95
5.3.4	Domínio de Stocks	97
6	Arquitetura e Design da Lógica de Negócios	100
6.1	Introdução ao Capítulo	100
6.2	Justificação da Separação Arquitetural: <i>Data API</i> vs. Lógica de Negócios	101
6.3	Organização Modular dos Serviços e Diagramas de Classes	102
6.3.1	Subsistema de Autenticação	102
6.3.2	Subsistema de Utilizadores	103
6.3.3	Subsistema de Catálogos	104
6.3.4	Subsistema de Serviços	104
6.3.5	Subsistema Financeiro	105
6.3.6	Subsistema de <i>Stocks</i>	105
6.3.7	Subsistema de Email	106
6.4	Comunicação HTTP com a <i>Data API</i>	107
6.5	Inventário de <i>Endpoints</i> da Lógica de Negócios	108

6.6	Segurança: Autenticação e Controlo de Acessos	113
6.6.1	Fluxo de Autenticação JWT	113
6.6.2	Gestão de Tokens: Lista Branca e Revogação	114
6.6.3	Modelo RBAC: Controlo de Acesso Baseado em Funções	115
6.7	Diagramas de Sequência dos Fluxos Operacionais	116
6.7.1	Fluxos de Autenticação e Gestão de Utilizadores	117
6.7.2	Fluxos de Serviços de Oficina	120
6.7.3	Fluxos Financeiros e Comerciais	124
6.7.4	Fluxos Administrativos	130
7	Implementação do <i>Frontend</i> (React + Tailwind CSS)	131
7.1	Arquitetura SPA com React Router e <i>Layouts</i> Aninhados	131
7.2	Mapeamento Completo de Rotas por Perfil	132
7.3	Camada de Serviços e <i>Hooks</i> com TanStack Query	134
7.4	Estilização com Tailwind CSS	136
7.5	<i>Design</i> , <i>Mockups</i> e Interfaces Implementadas	137
7.5.1	Processo de <i>Design</i> Assistido por IA	138
7.5.2	Interface Pública e Autenticação	138
7.5.3	Interface do Cliente (/FixNRide)	140
7.5.4	Interface do Operador (/FixNSell)	143
7.5.5	Interface do Mecânico (/FixNRepair)	144
7.5.6	Interface do Administrador (/FixNManage)	145
8	Geração de Código com Recurso a Inteligência Artificial	147
8.1	Introdução	147
8.2	Estratégia de Sessões e Gestão da Janela de Contexto	148
8.3	Desenvolvimento da Camada de Persistência e <i>Data API</i>	150
8.3.1	Modelação dos Schemas Mongoose	150
8.3.2	Geração dos Controladores e Rotas da <i>Data API</i>	151
8.4	Desenvolvimento da Camada de Lógica de Negócios	152
8.4.1	Estrutura do Servidor ASP.NET Core	152
8.4.2	Geração dos Services e Controllers .NET	152
8.5	Desenvolvimento da Camada de <i>Frontend</i>	154
8.5.1	Estrutura de Componentes e Portais	154
8.5.2	Geração de Componentes com TanStack Query	154
8.6	Iterações, Testes e Debug	155
8.6.1	Processo de Testes de Integração	155
8.6.2	Padrão de Iteração e Refinamento	156
8.7	Conclusão: Análise Crítica da Abordagem	157
8.7.1	Vantagens da Abordagem	158
8.7.2	Limitações e Riscos Identificados	158
8.7.3	Reflexão Final	159
9	Verificação, Validação e Avaliação da Qualidade do Software Produzido	161
9.1	Introdução ao Capítulo	161

9.2	Estratégia de Testes Adotada	161
9.2.1	Tipos de Testes e Âmbito de Cobertura	161
9.3	Testes Manuais da API: Casos de Teste Representativos	164
9.4	Testes de <i>Stress</i> com Postman Performance	167
9.4.1	Configuração dos Testes	168
9.4.2	Resultados Obtidos	168
9.4.3	Análise e Conclusões	169
9.5	Verificação da Satisfação dos Requisitos do SRS	169
9.6	Análise de Qualidade — ISO/IEC 25010	171
9.6.1	Adequação Funcional	171
9.6.2	Fiabilidade	171
9.6.3	Usabilidade	171
9.6.4	Eficiência de <i>Performance</i>	172
9.6.5	Manutenibilidade	172
9.6.6	Segurança	172
9.7	Análise Crítica do Código Gerado com LLM	173
9.7.1	O Papel das LLMs no Desenvolvimento	173
9.7.2	<i>Code Smells</i> Identificados e Refactorizações Aplicadas	173
9.8	Utilização de Inteligência Artificial	174
10	Conclusão e Trabalho Futuro	176
10.1	Síntese do Trabalho Realizado	176
10.2	Síntese das Decisões Arquiteturais	176
10.3	Avaliação do Grau de Cumprimento dos Requisitos	178
10.3.1	Requisitos Funcionais	178
10.3.2	Requisitos Não Funcionais	180
10.4	Reflexão Crítica sobre o Uso das LLMs	180
10.5	Trabalho Futuro	182
10.5.1	Melhorias de Segurança (Prioridade Alta)	182
10.5.2	Melhorias de Qualidade e Testabilidade (Prioridade Média)	182
10.5.3	Novas Funcionalidades (Prioridade Baixa)	183
10.6	Considerações Finais	184
11	Manual de Utilização e Instalação do Sistema Fix’n’Ride	185
	Parte I — Manual de Instalação e <i>Deployment</i>	187
11.1	Pré-requisitos	187
11.2	Configuração do Ambiente	188
11.3	Arranque do Sistema	190
11.4	Verificação da Instalação	192
11.5	Paragem e Limpeza do Ambiente	193
11.6	Deployment para Testes Externos (Remote Access)	195
	Parte II — Manual de Utilização por Perfil	197
11.7	Perfil Cliente	197
11.8	Perfil Operador	202
11.9	Perfil Mecânico	207

11.10	Perfil Administrador	212
	Lista de Siglas e Acrónimos	223
	Anexos	225
	Modelo de Domínio Completo	225
	Log de Prompts e Técnicas de Inteligência Artificial	226

Lista de Figuras

1.1	Diagrama de Gantt	5
2.1	Modelo de Dominio	40
2.2	Diagrama de casos de uso global	44
2.3	Diagrama de casos de uso das operações do cliente	45
2.4	Diagrama de casos de uso das operações do funcionário	46
2.5	Diagrama de casos de uso das operações do administrador	47
2.6	Diagrama de casos de uso das operações do operador de loja	48
2.7	Diagrama de casos de uso das operações do mecânico	49
3.1	Agendar Diagnóstico	68
3.2	Diagnóstico e Geração de Guia de Reparação	68
3.3	Encomenda de Reposição Inteligente	69
3.4	Administrador valida Encomenda	70
3.5	Operador de Loja receciona encomenda	70
3.6	Execução da Reparação	71
3.7	Venda Direta ao Balcão	72
3.8	Faturação e Levantamento da Trotinete	72
3.9	Serviço de Reparação	73
3.10	Encomenda de cliente	74
3.11	Encomenda de Reposição	74
4.1	Diagrama de componentes global do sistema Fix'n'Ride.	77
4.2	Diagrama de arquitetura de contentores do sistema Fix'n'Ride.	82
5.1	Diagrama do modelo de dados do sistema Fix'n'Ride em MongoDB.	90
6.1	Diagrama de classes do Subsistema de Autenticação.	103
6.2	Diagrama de classes do Subsistema de Utilizadores.	103
6.3	Diagrama de classes do Subsistema de Catálogos	104
6.4	Diagrama de classes do Subsistema de Serviços.	104
6.5	Diagrama de classes do Subsistema Financeiro	105
6.6	Diagrama de classes do Subsistema de <i>Stocks</i>	106
6.7	Diagrama de classes do Subsistema de Email.	107
6.8	Diagrama de sequência — Registo de Cliente.	117
6.9	Diagrama de sequência — Login de Cliente.	118
6.10	Diagrama de sequência — Login de Funcionário.	119

6.11 Diagrama de sequência — Registo de Trotinete.	120
6.12 Diagrama de sequência — Pedido de Serviço de Reparação.	121
6.13 Diagrama de sequência — Atualização de Serviço/Diagnóstico.	123
6.14 Diagrama de sequência — Levantamento de Trotinete.	124
6.15 Diagrama de sequência — Venda Direta em Loja.	125
6.16 Diagrama de sequência — Reserva de Peças (<i>Click & Collect</i>).	126
6.17 Diagrama de sequência — Levantamento de Encomenda de Cliente. . .	127
6.18 Diagrama de sequência — Devolução de Fatura.	129
6.19 Diagrama de sequência — Consulta de Estatísticas (Administrador). . .	130
7.1 <i>Landing page</i> pública do sistema Fix'n'Ride (HomePage).	138
7.2 Página de autenticação unificada (AuthPage) para clientes.	139
7.3 Página inicial da área reservada do cliente (HomeClientePage).	140
7.4 Página de agendamento de diagnóstico (AgendarDiagnosticoPage). . . .	141
7.5 Catálogo de peças do cliente (CatalogoPecasPage).	142
7.6 Módulo de venda direta ao balcão (VendaDiretaPage).	143
7.7 Página de trotinetes prontas para levantamento (TrotinetesProntasPage). .	144
7.8 <i>Dashboard</i> de diagnósticos do mecânico (Dashboard).	144
7.9 Página de reparações em curso do mecânico (Repairs).	145
7.10 <i>Dashboard</i> do administrador (AdminDashboard).	145
7.11 Módulo de gestão de encomendas de stock (EncomendasStock).	146
9.1 Resultado da execução da coleção Postman	167
11.1 Página de registo de cliente.	198
11.2 Página de <i>login</i> do cliente.	198
11.3 Página inicial do perfil cliente.	199
11.4 Página de gestão de trotinetes.	199
11.5 Formulário de agendamento de diagnóstico.	200
11.6 Catálogo de peças.	200
11.7 Painel de confirmação de reserva.	201
11.8 Página de venda direta.	203
11.9 <i>Checkout</i> de venda direta.	204
11.10 Página de levantamento de trotinetes.	205
11.11 Página de receção de encomendas de <i>stock</i>	206
11.12 Componente de seleção de intervenções.	208
11.13 Scanner de peças EAN.	209
11.14 Campo de relatório de diagnóstico.	210
11.15 Diálogo de agendamento de reparação.	211
11.16 <i>Dashboard</i> do administrador.	214
11.17 Página de encomendas de <i>stock</i>	215
11.18 Formulário de criação de encomenda de <i>stock</i>	215
11.19 Página de promoções.	216
11.20 Formulário de criação de promoção.	217
11.21 Página de gestão de funcionários.	218

11.22	Formulário de criação de funcionário.	218
11.23	Catálogo de peças.	219
11.24	Formulário de criação de peça.	220
11.25	Catálogo de intervenções.	221
11.26	Formulário de criação de intervenção.	221
11.1	Modelo de Domínio (Versão Ampliada)	225
11.2	Cenário de stress Base	233
11.3	Cenário de stress moderado	234
11.4	Cenário de stress intensivo	234

Lista de Tabelas

2.1	Requisito funcional 1: Registo do Cliente.	13
2.2	Requisito funcional 2: <i>Login</i> de Utilizador.	14
2.3	Requisito funcional 3: Registo de nova trotinete.	15
2.4	Requisito funcional 4: Realização de reserva de peças pelo cliente. . . .	16
2.5	Requisito funcional 5: Marcação de ato de diagnóstico.	17
2.6	Requisito funcional 6 : Consulta de histórico de faturas e reparações. . .	18
2.7	Requisito funcional 7 : Login de Funcionário.	19
2.8	Requisito funcional 8: Listar <i>stock</i> de peças.	20
2.9	Requisito funcional 9: Gerir promoções.	21
2.10	Requisito funcional 10: Edição do catálogo de peças.	22
2.11	Requisito funcional 11: Edição do catálogo de intervenções.	23
2.12	Requisito funcional 12 : Aprovação de ordens de encomenda pendentes e visualização de histórico de rececionadas.	24
2.13	Requisito funcional 13 : Consulta de dados operacionais.	25
2.14	Requisito funcional 14 : Registo de novo funcionário.	26
2.15	Requisito funcional 15: Preencher guia de reparação.	27
2.16	Requisito funcional 16: Registo do código de peça utilizada.	28
2.17	Requisito funcional 17: Alterar estado da reparação.	29
2.18	Requisito funcional 18 : Receção de encomenda de <i>stock</i>	30
2.19	Requisito funcional 19: Registo de venda e faturação.	31
2.20	Requisito funcional 20: Entrega de encomenda a cliente.	32
2.21	Requisito funcional 21: Registar levantamento de trotinete e pagamento. .	33
2.22	Requisito funcional 22: Processar devolução.	34
2.23	Requisito não funcional 1: Tempo de resposta (Desempenho).	35
2.24	Requisito não funcional 2: Suporte a múltiplos dispositivos de I/O.	35
2.25	Requisito não funcional 3: Fiabilidade e validação de dados.	36
2.26	Requisito não funcional 4: Restrições de implementação.	36
2.27	Requisito não funcional 5: Segurança e controlo de acessos.	37
2.29	Processo de Refinamento e Clarificação de Requisitos.	38
2.30	Dicionário de Entidades do Modelo de Domínio (Parte 1)	41
2.31	Dicionário de Entidades do Modelo de Domínio (Parte 2)	42
2.32	Descrição de Relacionamentos do Modelo de Domínio (Parte 1)	43
2.33	Descrição de Relacionamentos do Modelo de Domínio (Parte 2)	43
2.34	Descrição detalhada do caso de uso — Efetuar agendamento de diagnós- tico.	50
2.35	Descrição detalhada do caso de uso — Realizar Venda Direta ao Balcão	51

2.36	Descrição detalhada do caso de uso — Aprovar Encomenda de Reposição	51
2.37	Descrição detalhada do caso de uso — Faturar Serviço e Entregar Trotinete	52
2.38	Descrição detalhada do caso de uso — Consultar Relatórios Operacionais	53
2.39	Descrição detalhada do caso de uso — Inserir Novo Item no Catálogo . .	53
2.40	Descrição detalhada do caso de uso — Registrar Conta de Cliente. . . .	54
2.41	Descrição detalhada do caso de uso — Realizar encomenda.	55
2.42	Descrição detalhada do caso de uso — Gerir Promoções	55
2.43	Descrição detalhada do caso de uso — Registrar Novo Funcionário . . .	56
2.44	Descrição detalhada do caso de uso — Ajustar <i>Stock</i> Manualmente . . .	56
2.45	Descrição detalhada do caso de uso — Consultar Histórico de Faturas .	57
2.46	Descrição detalhada do caso de uso — Consultar Estado de Reparação.	57
2.47	Descrição detalhada do caso de uso — Consultar Agenda da Oficina . .	58
2.48	Descrição detalhada do caso de uso — Rececionar Encomenda de <i>Stock</i>	58
2.49	Descrição detalhada do caso de uso — Entregar Encomenda a Cliente .	59
2.50	Descrição detalhada do caso de uso — Processar Devolução	59
2.51	Descrição detalhada do caso de uso — Efetuar Login (Cliente).	60
2.52	Descrição detalhada do caso de uso — Registrar Nova Trotinete.	60
2.53	Descrição detalhada do caso de uso — Efetuar Login (Funcionário). . . .	61
2.54	Descrição detalhada do caso de uso — Alterar estado da intervenção . .	62
2.55	Descrição detalhada do caso de uso — Inativar acesso de Funcionário .	62
2.56	Descrição detalhada do caso de uso — Inativar item do catálogo	63
2.57	Descrição detalhada do caso de uso — Consultar Promoções	63
2.58	Descrição detalhada do caso de uso — Registrar Peça Utilizada	64
2.59	Descrição detalhada do caso de uso — Rececionar Encomenda de <i>Stock</i>	65
4.1	Decisões arquiteturais e justificação da <i>stack</i> tecnológica do sistema . .	80
4.2	Resumo dos serviços Docker do sistema Fix'n'Ride	82
4.3	Principais variáveis de ambiente do sistema Fix'n'Ride	85
5.1	Dicionário de dados — Modelo Cliente (coleção clientes)	91
5.2	Dicionário de dados — Modelo Funcionario (coleção funcionarios) . .	92
5.3	Dicionário de dados — Modelo Trotinete (coleção trotinetes)	92
5.4	Dicionário de dados — Modelo Servico (coleção servicos)	93
5.5	Dicionário de dados — Subdocumento <i>historicoIntervencoes</i> (por elemento)	94
5.6	Dicionário de dados — Modelo Agenda (coleção agenda)	94
5.7	Dicionário de dados — Modelo <i>IntervencaoCatalogo</i> (coleção <i>intervencoes_catalogo</i>	95
5.8	Dicionário de dados — Modelo Fatura (coleção faturas)	96
5.9	Dicionário de dados — Modelo Venda (coleção vendas)	96
5.10	Dicionário de dados — Modelo Promocao (coleção <i>promocoes</i>)	97
5.11	Dicionário de dados — Modelo Peca (coleção <i>pecas</i>)	98
5.12	Dicionário de dados — Modelo <i>EncomendaStock</i> (coleção <i>encomendas_stock</i>)	99
5.13	Dicionário de dados — Modelo <i>EncomendaCliente</i> (coleção <i>encomendas_cliente</i>)	99

6.1	Inventário de <i>endpoints</i> do servidor de Lógica de Negócios (ASP.NET Core)	108
6.2	Modelo RBAC do sistema Fix'n'Ride — mapeamento de perfis, módulos e permissões	115
7.1	Mapeamento completo de rotas da SPA Fix'n'Ride por perfil	132
7.2	Inventário dos principais <i>hooks</i> TanStack Query do <i>frontend</i> Fix'n'Ride	135
9.1	Resumo da cobertura de testes por módulo e tipo	163
9.2	Casos de teste manuais representativos da API Fix'n'Ride	164
9.3	Configuração dos cenários de testes de <i>stress</i>	168
9.4	Resultados dos testes de <i>stress</i> — métricas de performance	168
9.5	Verificação de satisfação dos requisitos funcionais do SRS	169
9.6	Avaliação qualitativa do Fix'n'Ride segundo ISO/IEC 25010	172
10.1	Balanco de cumprimento dos requisitos funcionais por área	179
11.1	Pré-requisitos de software para instalação do Fix'n'Ride	187
11.2	Sistemas operativos suportados e testados	188
11.3	Variáveis de ambiente do sistema Fix'n'Ride — obrigatórias vs. opcionais	190
11.4	URLs de acesso ao sistema Fix'n'Ride após arranque	193
11.5	Utilizadores pré-carregados para teste do sistema Fix'n'Ride	193
11.6	Funcionalidades do perfil Cliente	201
11.7	Funcionalidades do perfil Operador	206
11.8	Funcionalidades do perfil Mecânico	212
11.9	Funcionalidades do perfil Administrador	222
11.1	Resumo das ferramentas de IA utilizadas no projeto Fix'n'Ride por fase e tipo	235

1 Introdução e apresentação do trabalho

1.1 Contextualização

Na última década, o paradigma da mobilidade urbana sofreu uma transformação profunda e acelerada. A procura por transportes menos poluentes e o excesso de trânsito tornaram as trotinetes elétricas fundamentais para as cidades inteligentes. O que começou por ser um lazer é agora essencial para as deslocações diárias, criando um mercado que exige serviços de reparação e manutenção cada vez mais avançados. Contudo, esta expansão física e operacional não foi acompanhada por uma digitalização proporcional no setor dos serviços de assistência técnica. Muitas organizações, ao tentarem escalar as suas operações para responder a este volume massivo de procura, encontram-se limitadas por métodos de gestão tradicionais e sistemas de informação fragmentados.

O software, neste contexto, deixou de ser um mero utilitário administrativo para se tornar no orquestrador central de sistemas dinâmicos, em evolução contínua e fortemente orientados a dados. Paralelamente, a própria Engenharia de Software atravessa uma revolução impulsionada pela Inteligência Artificial. A emergência dos *Large Language Models* (LLM) alterou os métodos de idealização e desenvolvimento, permitindo que estes modelos atuem como agentes ativos em tarefas complexas, desde a Engenharia de Requisitos até à implementação. O presente trabalho insere-se na interseção destas duas realidades: a necessidade crítica de digitalização eficiente num setor de mobilidade em expansão e a utilização de agentes artificiais para acelerar e otimizar o ciclo de vida do software.

Prompt utilizado - Claude Sonnet 4.6

"Elabora a secção de Contextualização para a introdução de um projeto universitário de Engenharia de Software focado no desenvolvimento de um sistema de gestão para reparação de trotinetes elétricas. A secção deve ligar a evolução da micromobilidade urbana, a falta de digitalização no setor e o impacto da Inteligência Artificial (LLMs) no ciclo de desenvolvimento de software."

1.2 Motivação e Objetivos

A motivação principal para o desenvolvimento do sistema **Fix'n'Ride** reside na urgência de dotar a MobiFix de uma infraestrutura tecnológica capaz de sustentar o seu crescimento acelerado. A atual dependência de processos manuais, como folhas de cálculo e registos em papel para a abertura de ordens de serviço, e a inexistência de uma visão integrada dos dados operacionais conferem uma limitação à escalabilidade da empresa, onde o aumento do volume de clientes deixou de se traduzir num aumento proporcional de lucro, resultando em desorganização e desperdício de recursos.

A transição de uma gestão reativa para uma abordagem proativa e baseada em dados é o motor central deste projeto, visando eliminar custos ocultos e transformar a eficiência operacional numa vantagem competitiva. Neste âmbito, o projeto foca-se em atingir objetivos estratégicos claros, começando pela gestão eficiente do *stock* e redução de capital imobilizado.

Através da implementação de sistemas de gestão de inventário para peças, prevê-se uma redução de 15% nos custos de armazenamento e a eliminação de perdas por obsolescência. Simultaneamente, a digitalização do fluxo de trabalho técnico visa otimizar o ciclo de reparação, projetando um aumento de 25% na capacidade produtiva da oficina ao fornecer acesso imediato a históricos e manuais técnicos. No plano comercial, o desenvolvimento de um catálogo digital integrado pretende captar receita fora do horário físico, estimando-se um incremento de 20% na faturação de componentes no primeiro ano. Por fim, a melhoria da experiência do cliente será assegurada por sistemas de notificação e acompanhamento automáticos, visando reduzir em 40% o volume de contactos para ponto de situação, libertando recursos humanos e aumentando a taxa de fidelização através de uma maior transparência no serviço.

1.3 Apresentação do Caso de Estudo

A **MobiFix – Mobility Repair Services** atua no setor da assistência técnica especializada de micromobilidade urbana, operando com um modelo de negócio inspirado nas redes multimarca automóveis, mas adaptado às especificidades de trotinetes. O crescimento exponencial da procura transformou a organização num centro de serviços de alto débito, processando atualmente um volume massivo de peças e de trotinetes de fabricantes como Xiaomi, Segway ou Ninebot, como com intervenções que variam entre a manutenção preventiva e diagnósticos eletrónicos complexos em componentes críticos.

No entanto, a escalabilidade deste modelo encontra-se gravemente comprometida pela dependência de processos manuais e folhas de cálculo desconexas, o que impossibilita a sincronização eficiente entre o fluxo de entrada de veículos e a gestão da cadeia de abastecimento de peças. Esta desarticulação gera impactos negativos transversais a todos os intervenientes do ecossistema.

Para a administração, a falta de métricas em tempo real e a ausência de canais de venda online impedem o cálculo rigoroso da rentabilidade por serviço e a previsão da procura, resultando numa gestão de inventário ineficiente que oscila entre a imobilização de capital e ruturas de *stock* críticas. A equipa técnica enfrenta tempos de diagnóstico elevados e a necessidade constante de validação física de *stock* no armazém, dada a inexistência de um histórico acessível e de um sistema de reserva de peças, o que reduz drasticamente a produtividade da mão-de-obra.

Paralelamente, o cliente final experiencia um serviço opaco, sem acesso ao estado da reparação ou a ferramentas digitais para agendamento e consulta de catálogo, o que gera atrito na jornada de compra e aumenta a taxa de abandono. Financeiramente, estas ineficiências traduzem-se em custos operacionais elevados e na subutilização da capacidade produtiva da oficina. A impossibilidade de concluir serviços por falta de componentes ou a ocupação prolongada de bancadas por falhas de fluxo representam perdas de receita direta e custos de oportunidade significativos.

Neste cenário, o desenvolvimento do sistema **Fix'n'Ride** apresenta-se como um imperativo estratégico para desmaterializar e orquestrar todo o ciclo de vida do serviço — desde o agendamento e receção até à faturação e entrega — garantindo a sustentabilidade económica e a eficiência operacional da **MobiFix**.

1.4 Análise da Viabilidade

A decisão de avançar com o sistema **Fix'n'Ride** baseia-se numa análise rigorosa que confirma a exequibilidade e o benefício da solução.

No plano técnico, a arquitetura foi desenhada para garantir robustez através de uma *stack* tecnológica consolidada, utilizando a plataforma **.NET** (C#) e **Mongo** para assegurar a estabilidade das operações. Economicamente, a análise revela um cenário favorável com um retorno do investimento (ROI) projetado para um período inferior a 18 meses. Os ganhos de eficiência previstos na gestão de *stock* e na produtividade téc-

nica superam largamente o investimento inicial de desenvolvimento, tornando a solução financeiramente sustentável.

Operacionalmente, a solução integra-se de forma transparente nos processos atuais de entrada e saída de equipamentos, substituindo o papel e folhas de cálculo sem introduzir complexidade desnecessária para os utilizadores.

Além disso, a viabilidade temporal é assegurada pelo uso sistemático de LLMs durante a fase de desenvolvimento, o que permite acelerar a escrita de código e a validação do sistema, garantindo a entrega de uma solução robusta dentro do calendário estabelecido.

1.5 Planeamento do Projeto

O planeamento temporal do projeto *Fix'n'Ride* foi estruturado de acordo com as fases clássicas do ciclo de vida do desenvolvimento de software, adaptadas ao contexto académico da unidade curricular de Laboratórios de Informática IV. O Diagrama de Gantt apresentado permite visualizar a distribuição das tarefas ao longo do semestre, evidenciando as dependências entre fases e os marcos de entrega.

O desenvolvimento foi organizado em quatro fases sequenciais com sobreposição parcial. A fase de **Engenharia de Requisitos** ocupou as primeiras semanas do projeto, abrangendo o levantamento de necessidades junto dos stakeholders, a simulação de entrevistas, a definição dos requisitos funcionais e não funcionais, e a modelação do domínio. Esta fase é deliberadamente a mais longa da componente de análise, dada a complexidade operacional da MobiFix e o número de perfis de utilizador envolvidos.

A fase de **Especificação e Design** sucedeu-se em sobreposição parcial com a anterior, contemplando a elaboração dos diagramas de atividades, máquinas de estados e diagramas de sequência, bem como a definição da arquitetura em três camadas e das decisões tecnológicas principais.

As fases de **Implementação e Testes** decorreram em paralelo, refletindo a abordagem iterativa adotada: à medida que cada módulo era implementado, era imediatamente submetido a testes de integração via Postman. Este paralelismo foi possível graças ao uso sistemático de LLMs na geração de código, que reduziu significativamente o tempo de implementação de cada camada.

Por fim, a fase de **Documentação e Validação Final** concentrou-se nas últimas semanas, incluindo a avaliação ISO/IEC 25010, os testes de stress com métricas P50/P95/P99 e a elaboração do presente relatório.

A análise do diagrama revela que o caminho crítico do projeto passa pela implementação da camada de Lógica de Negócios em ASP.NET Core, dado que a camada de frontend depende dos endpoints desta camada para funcionar. Qualquer atraso na implementação dos *services* .NET propagar-se-ia diretamente ao portal React de cada perfil de utilizador.

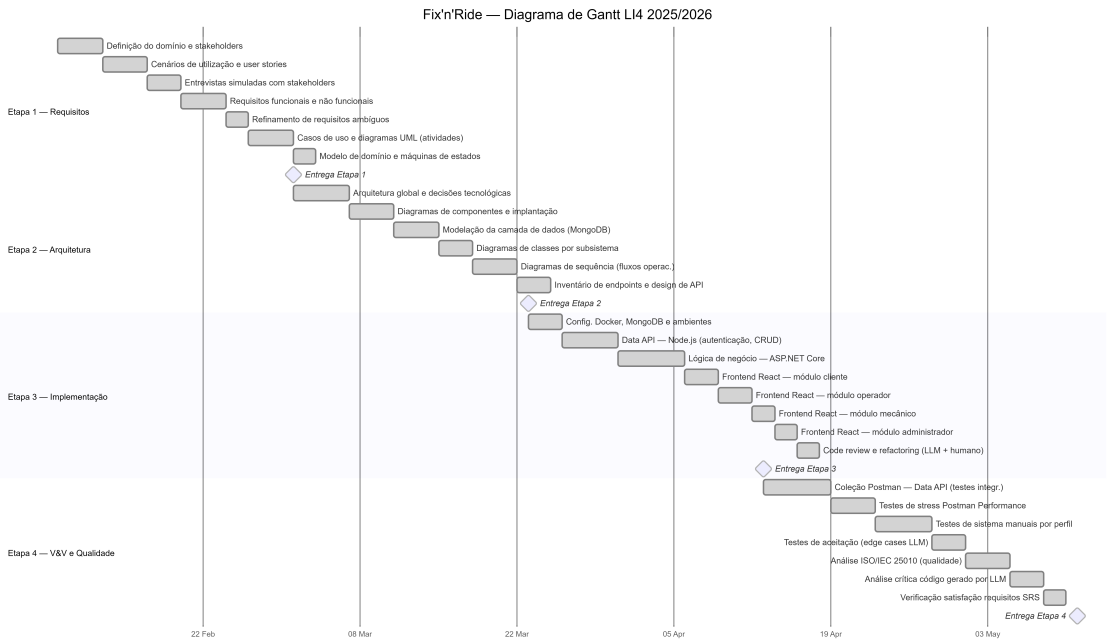


Figura 1.1: Diagrama de Gantt

2 Engenharia de requisitos

2.1 Identificação de *Stakeholders*

A definição dos intervenientes no sistema Fix'n'Ride é o pilar que sustenta toda a engenharia de requisitos, garantindo que a solução final mitiga as ineficiências operacionais da MobiFix.

No topo da pirâmide organizacional encontra-se a **Administração**, cujo foco principal reside na sustentabilidade económica e na obtenção de métricas factuais, como a rentabilidade por serviço, que atualmente são impossíveis de calcular devido à dependência de processos manuais.

No centro da operação técnica estão os **Mecânicos**, que necessitam de um acesso ágil ao histórico de reparações e manuais técnicos para reduzir tempos de diagnóstico e aumentar a produtividade.

Paralelamente, o **Operador de Loja** atua como o elo de ligação entre o cliente e a oficina, sendo responsável por orquestrar o fluxo de entrada de trotinetes e a receção de peças encomendadas para repôr o *stock* mínimo.

Por fim, não podemos esquecer o **Cliente Final**, que, embora não utilize o sistema diretamente para gestão, é o beneficiário direto da transparência e rapidez do serviço, fatores determinantes para reduzir a taxa de abandono e melhorar a experiência de utilização.

Prompt utilizado - Claude Sonnet 4.6

"Escreve a secção de Identificação de Stakeholders para o sistema Fix'n'Ride da MobiFix, uma oficina multimarca de trotinetes. Identifica os quatro intervenientes principais:

Administração (foco em métricas e rentabilidade), Mecânicos (acesso ágil ao histórico e manuais), Operador de Loja (elo entre cliente e oficina, gestão de stock) e Cliente Final (beneficiário da transparência do serviço). Para cada stakeholder, descreve o seu papel e necessidades no contexto de uma oficina com processos ainda manuais."

2.2 Cenários de Utilização

As histórias abaixo descrevem o fluxo operacional do sistema, desde o problema inicial até à resolução final, servindo de base para a extração dos requisitos técnicos

2.2.1 Agendamento e Diagnóstico da Avaria

A jornada começa quando um cliente deteta uma anomalia na sua trotinete e, em vez de recorrer a processos manuais ou chamadas telefónicas, acede ao portal **Fix'n'Ride** para efetuar uma marcação online.

Após iniciar sessão, o utilizador seleciona o veículo a intervir a partir do seu histórico pessoal e escolhe um horário disponível na agenda da oficina. No dia agendado, o mecânico Pedro consulta o seu *tablet* na bancada de trabalho para iniciar a triagem.

Após a inspeção física, Pedro utiliza o catálogo digital para selecionar as intervenções e peças necessárias, garantindo que o sistema aplica automaticamente os preços fixos de mão de obra e peças. O sistema gera então uma Guia de Reparação detalhada em formato PDF.

2.2.2 Gestão de Inventário e Reposição Automática

Enquanto a oficina opera, o sistema monitoriza silenciosamente a cadeia de abastecimento para evitar paragens por falta de material.

Quando o operador João realiza a venda de uma peça ou o mecânico Pedro utiliza uma peça na reparação, o inventário é decrementado em tempo real na base de dados.

Se o *stock* de um componente crítico atingir o limite mínimo configurado, o sistema dispara um alerta e cria automaticamente uma ordem de encomenda com o estado "Pendente". A administradora Ana, ao aceder ao seu painel de gestão, revê estas sugestões de reposição, ajustando as quantidades de acordo com o fluxo de caixa antes de clicar em "Aprovar".

Uma vez aprovada, a encomenda passa ao estado "Em Trânsito", permitindo que os mecânicos consultem a agenda e saibam exatamente quando as peças chegarão.

2.2.3 Fluxo de Venda Direta e Devolução

Na frente de loja, o sistema permite que o atendimento ao balcão seja rápido e livre de erros. Quando um cliente pretende adquirir apenas um acessório, como um capacete, o operador de loja João utiliza o módulo de Venda Direta, que ignora os fluxos complexos de oficina e permite processar o pagamento via *MBWay* ou Multibanco de imediato.

A fatura única, que engloba o material e garante a conformidade fiscal, é emitida no momento. No cenário de uma devolução, o processo é igualmente rigoroso: João insere o número da fatura original e o sistema carrega todos os itens associados.

Ao seleccionar o artigo a devolver, o sistema recalcula valores, emite a respetiva Nota de Crédito e repõe automaticamente a unidade no stock físico da loja, assegurando que o inventário e a caixa batem sempre certo ao final do dia.

2.2.4 Análise de Performance e Rentabilidade

No final do ciclo mensal, a administração consegue transformar dados operacionais em decisões estratégicas. O sistema gera gráficos e informação num dashboard com filtros complexos que permitem avaliar a situação financeira da empresa.

Através de gráficos interativos, o sistema revela a divisão exata da faturação entre serviços e venda de produtos, destacando as áreas de maior rentabilidade. Ao detetar, por exemplo, um excesso de stock em componentes de modelos antigos, Ana utiliza o módulo de Gestão de Promoções para criar uma campanha de desconto temporária.

O sistema passa então a aplicar esses descontos automaticamente em todos os orçamentos e faturas gerados durante o período da campanha, eliminando erros de cálculo manual e dinamizando o escoamento de inventário.

2.3 User Stories

Apresentamos de seguida quinze narrativas que ilustram o funcionamento desejado do sistema *Fix'n'Ride*. Estas histórias serviram de base para as discussões com os *stakeholders* e para o desenvolvimento de determinados requisitos.

2.3.1 Gestão de Oficina

US01: Como Mecânico, quero seleccionar os tipos de intervenção a efetuar a partir de um catálogo, para que o sistema aplique o preço fixo tabelado de mão-de-obra.

US02: Como Mecânico, quero consultar o diagnóstico, para garantir que apenas realize as reparações necessárias.

2.3.2 Gestão de *Stock* e Compras

US04: Como Administrador, quero que o sistema me alerta para reabastecimento quando uma peça atinge o *stock* mínimo, para evitar a interrupção de serviços por falta de material.

US05: Como Operador, quero registar a entrada de peças através de uma encomenda de reposição, para atualizar o inventário de forma fidedigna.

US06: Como Operador, quero realizar uma venda direta de peças ao balcão, para satisfazer clientes que não necessitam de serviço técnico.

2.3.3 Administração e Faturação

US07: Como Operador, quero gerar uma fatura única que englobe peças e mão-de-obra de um serviço, para finalizar o processo de entrega ao cliente.

US08: Como Administrador, quero visualizar dados financeiros de lucros e custos, para apoiar a tomada de decisão estratégica da oficina.

2.3.4 Gestão de Conta e Veículos

US09: Como Cliente, quero criar um registo pessoal no sistema, para que os meus dados de faturação e histórico fiquem guardados para futuras interações com a oficina.

US10: Como Cliente, quero registar a minha trotinete (marca, modelo e número de série) no meu perfil, para facilitar a identificação do veículo e manter um histórico de intervenções fidedigno.

2.3.5 Agendamento e Acompanhamento

US12: Como Cliente, quero agendar um diagnóstico *online* escolhendo um horário disponível, para garantir que a oficina me pode receber sem tempos de espera excessivos.

US13: Como Cliente, quero consultar o estado da minha reparação em tempo real, para saber exatamente quando a minha trotinete está pronta para levantamento sem ter de telefonar para a loja.

2.3.6 Compras e Documentação

US14: Como Cliente, quero comprar peças e acessórios através de um catálogo online, para poder garantir a reserva dos itens e efetuar o pagamento fisicamente de forma cómoda.

US15: Como Cliente, quero aceder ao meu histórico de faturas e descarregá-las em formato PDF, para poder gerir as garantias dos componentes e serviços adquiridos.

2.4 Métodos de Levantamento Utilizados

Após o esclarecimento do domínio do projeto, tornou-se essencial recolher informações precisas sobre os fluxos operacionais e as funcionalidades globais esperadas para o sistema *Fix'n'Ride*, de forma a garantir que a solução final responde efetivamente às necessidades de crescimento da *MobiFix*. Para esse fim, a equipa optou por uma abordagem mista de recolha de informação, recorrendo a três métodos complementares:

- **Questionários de formato livre:** Enviados aos vários operadores de loja, mecânicos e clientes, com o objetivo de recolher opiniões e sugestões sobre as principais funcionalidades desejadas e as dificuldades sentidas na atual dependência de processos manuais e folhas de cálculo.

- **Observação direta:** Acompanhamento do trabalho diário dos profissionais envolvidos na operação técnica e comercial da oficina, permitindo compreender melhor os fluxos de informação — desde a receção e diagnóstico da trotinete até à faturação — e identificar os bloqueios na escalabilidade atual.
- **Entrevistas estruturadas:** Entrevistas estruturadas realizadas com todos *stakeholders*, exceto os clientes, com o intuito de identificar requisitos estratégicos e prioridades de negócio que o sistema deverá suportar. .

Concluída a fase de recolha, procedeu-se à etapa de análise dos dados obtidos. Nesta fase, a equipa técnica compilou as respostas e observações para eliminar ambiguidades e redundâncias. As necessidades identificadas foram categorizadas em requisitos funcionais e não funcionais. Todos os requisitos levantados foram posteriormente validados em reuniões com a equipa e encontram-se agora escrutinados e organizados nas tabelas de especificação de requisitos.

2.5 Entrevistas com Stakeholders

Para garantir que o *Fix'n'Ride* resolve os problemas reais da *MobiFix*, foram realizadas entrevistas estruturadas com os principais intervenientes. Abaixo apresentam-se os pontos cruciais discutidos:

2.5.1 Entrevista com a Administradora Ana

Objetivo: Compreender as necessidades estratégicas e financeiras.

Equipa: Quais são os maiores "estrangulamentos" na gestão atual?

Ana: Perco muito dinheiro com peças paradas que ninguém usa e, ao mesmo tempo, perco clientes porque não temos pneus básicos em *stock*. Preciso que o sistema me avise o que comprar e que me diga, no final do mês, se a oficina está a ser rentável ou se estamos a gastar demasiado tempo em reparações baratas.

Equipa: Como imagina o controlo dessas compras?

Ana: O sistema deve sugerir a encomenda baseando-se no que saiu, mas eu quero dar o clique final de aprovação para controlar o fluxo de caixa.

2.5.2 Entrevista com o Operador de Loja João

Objetivo: Otimizar o atendimento ao cliente e vendas rápidas.

Equipa: O que mais atrasa o atendimento ao balcão?

João: A confusão nas entregas. Às vezes o mecânico acaba a trotinete, mas não me avisa. O cliente liga e eu não sei o que dizer. Além disso, vender um acessório simples, como um capacete, demora imenso porque tenho de preencher os mesmos dados de uma reparação complexa.

Equipa: O que facilitaria o seu dia-a-dia?

João: Uma forma de vender "direto" sem abrir processos longos e um painel que me mostre logo o que está pronto para o cliente levar.

2.5.3 Entrevista com o Mecânico Pedro

Objetivo: Melhorar a produtividade na oficina.

Equipa: Como é feito o registo das reparações atualmente?

Pedro: Usamos papel. O problema é que esqueço-me de anotar a peça que usei ou o papel desaparece e perdemos tempo a procurá-lo. Depois, na faturação, o valor nunca bate certo.

Equipa: Se tivesse um *tablet* na bancada, o que seria essencial?

Pedro: Preciso de uma lista simples onde selecione o que fiz (ex: 'Revisão Geral' ou 'Substituição de Travões') e o sistema já saiba o preço fixo, para evitar enganar.

2.6 Requisitos Funcionais

Nesta secção, encontram-se listados os requisitos funcionais do sistema, levantados através das entrevistas descritas na secção anterior.

Requisito de utilizador	Clientes são capazes de se registarem no sistema.
Fonte	Questionários
Área do sistema	Autenticação
Requisitos de sistema	1. Deve existir uma página de criação de conta de cliente. 2. Para um cliente realizar o registo, deve fornecer o seu nome completo, contacto telefónico, morada, NIF e email, bem como a definição de uma password de acesso. 3. Após a criação do novo registo, o sistema encaminha o cliente para a página de login, onde o cliente insere as suas credenciais.
Relevância para a aplicação	É essencial que o sistema permita a criação de contas de cliente, no sentido de ficar com o registo dos dados de faturação do mesmo, para efeitos de faturação, registo do histórico de compras / serviços e agendamento de diagnósticos.

Tabela 2.1: Requisito funcional 1: Registo do Cliente.

Requisito de utilizador	Os clientes devem conseguir fazer <i>login</i> no sistema.
Fonte	Questionários
Área do sistema	Autenticação
Requisitos de sistema	1. O sistema deve apresentar uma <i>homepage</i> com o catálogo das 4 peças mais vendidas, bem como a lista-gem dos serviços fornecidos. 2. A página deve conter uma área de acesso ao <i>login</i>. 3. Para o cliente realizar o <i>login</i>, deve introduzir o seu NIF e respetiva <i>password</i>. 4. Caso o sistema não aprove o início de sessão, deve informar o cliente relativamente ao erro nas credenciais, solicitando uma nova tentativa de <i>login</i>. 5. Após o <i>login</i>, o sistema encaminha o cliente para a sua área pessoal, onde este tem acesso ao histórico de compras, encomendas e serviços, bem como uma lista das trotinetes que registou no sistema.
Relevância para a aplicação	É de suma importância que o cliente tenha acesso à sua área pessoal, permitindo a gestão segura das suas encomendas e marcações, bem como a consulta do histórico individual, garantindo que as operações são associadas corretamente ao utilizador.

Tabela 2.2: Requisito funcional 2: *Login* de Utilizador.

Requisito de utilizador	Os clientes devem conseguir fazer o registo de uma nova trotinete.
Fonte	Questionários
Área do sistema	Catálogos
Requisitos de sistema	1. O sistema deve permitir ao cliente fazer o registo de uma trotinete, para efeitos de acompanhamento e agendamento de serviços e de recomendação de peças compatíveis. 2. Para efetuar o registo da trotinete, o cliente deve inserir o seu número de série, marca e modelo. 3. Após o registo, o sistema reencaminha o cliente de volta para a área pessoal, onde a nova trotinete já deve aparecer na listagem.
Relevância para a aplicação	Este registo é fundamental para permitir a personalização do suporte técnico e a marcação de serviços de manutenção. Ao associar veículos específicos a um utilizador, o sistema consegue manter um histórico de intervenções por equipamento, facilitando diagnósticos futuros e a gestão de garantias.

Tabela 2.3: Requisito funcional 3: Registo de nova trotinete.

Requisito de utilizador	Os clientes devem conseguir fazer a encomenda de peças comercializadas.
Fonte	Entrevista com Administradora Ana
Área do sistema	Finanças
Requisitos de sistema	1. O sistema deve permitir ao cliente visualizar um catálogo com os itens comercializados, e a sua disponibilidade em loja. 2. O sistema deve permitir ao cliente realizar a pesquisa da peça que deseja, a partir do seu nome ou código EAN. 3. Após a escolha das peças desejadas, o sistema deve encaminhar o cliente para a página de reserva, onde deverá apresentar ao cliente a lista das peças que este escolheu, bem como o respetivo preço total. 4. Após realizada a reserva das peças, o sistema avisa ao cliente que pode levanta-la em loja quando quiser e realizar o pagamento no momento. 5. Caso, após a reserva, o <i>stock</i> da(s) peça(s) atinja o valor mínimo, o sistema deverá criar uma nova encomenda automática de <i>stock</i>, para aprovação posterior da administração.
Relevância para a aplicação	A reserva de peças online constitui um fluxo de receita complementar aos serviços de reparação. Esta funcionalidade permite automatizar o processo comercial, diminuindo a necessidade de apoio ao cliente por questões de saber se temos certa peça. Adicionalmente, a integração imediata com a gestão de <i>stock</i> garante que a disponibilidade apresentada é real, prevenindo vendas de artigos indisponíveis e acionando automaticamente os mecanismos de reposição de inventário.

Tabela 2.4: Requisito funcional 4: Realização de reserva de peças pelo cliente.

Requisito de utilizador	Os clientes devem conseguir fazer a marcação do diagnóstico online.
Fonte	Questionários
Área do sistema	Serviços
Requisitos de sistema	1. O sistema deve apresentar ao cliente, caso este tenha adicionado a trotinete à sua conta, a opção de realizar uma marcação de diagnóstico referente a essa trotinete. 2. Após a escolha do horário, o sistema regista o serviço de diagnóstico na agenda da oficina.
Relevância para a aplicação	A gestão digital da agenda de diagnósticos é crucial para evitar a sobrelotação da oficina e garantir a disponibilidade de técnicos para a avaliação inicial. Este requisito permite planear a carga de trabalho diária.

Tabela 2.5: Requisito funcional 5: Marcação de ato de diagnóstico.

Requisito de utilizador	Os clientes devem conseguir ver o histórico de faturas emitidas e reparações realizadas.
Fonte	Questionários
Área do sistema	Finanças e Serviços
Requisitos de sistema	1. O sistema deve apresentar ao cliente, na sua área reservada (Perfil), um separador dedicado ao histórico de transações e outro dedicado ao estado atual de reparações concluídas e por concluir. 2. A lista apresentada no histórico de transações deve agregar cronologicamente todas as transações associadas ao cliente, distinguindo entre "Venda de Peças" e "Serviço de Reparação". 3. A lista apresentada no histórico de reparações deve distinguir entre reparações "Em Oficina" e reparações "Prontas", e também deve ter informação da descrição do diagnóstico, a queixa do próprio cliente, intervenções realizadas e o preço total. 4. Para cada registo, o sistema deve apresentar a data, o número da fatura, o valor total pago. 5. O sistema deve disponibilizar um botão para efetuar os <i>downloads</i> das faturas em PDF correspondentes a cada transação, para efeitos de garantia ou fiscalidade.
Relevância para a aplicação	A autonomia do cliente no acesso aos documentos fiscais reduz significativamente a carga administrativa da loja (menos pedidos de 2.ª via de faturas). Além disso, a transparência das reparações permite com que o cliente tenha conhecimento total da operação da loja em relação à sua trotinete.

Tabela 2.6: Requisito funcional 6 : Consulta de histórico de faturas e reparações.

Requisito de utilizador	Os funcionários da oficina (operadores, mecânicos e administradores) devem conseguir efetuar o <i>login</i> na aplicação.
Fonte	Entrevista com Administradora Ana
Área do sistema	Autenticação
Requisitos de sistema	1. O <i>login</i> de funcionário no sistema deve ser feito através do seu número mecânografico e <i>password</i> definidas previamente pelo administrador.
Relevância para a aplicação	A autenticação dos funcionários é crucial para garantir a segurança da informação e restringir o acesso à área de gestão da oficina apenas a pessoal autorizado. Além disso, permite que o sistema registre qual o funcionário responsável por cada operação, facilitando a auditoria e a gestão interna.

Tabela 2.7: Requisito funcional 7 : Login de Funcionário.

Requisito de utilizador	Todos os funcionários da oficina deverão ser capazes de fazer a listagem das peças em <i>stock</i> na oficina.
Fonte	Entrevista com Mecânico Pedro e Observação Direta
Área do sistema	Gestão de <i>Stocks</i> / Inventário
Requisitos de sistema	1. O sistema deve disponibilizar ao Administrador, Operador de Loja e Mecânico a capacidade de visualizar as peças em <i>stock</i> na loja por motivos de atualização de dados, venda direta e utilização de peças em intervenções respetivamente.
Relevância para a aplicação	O acesso rápido à listagem de peças é fundamental para que os mecânicos e operadores saibam a disponibilidade de material para as reparações em tempo real, evitando atrasos no serviço. Além disso, a funcionalidade de correção manual do <i>stock</i> é estritamente essencial para efeitos de inventário periódico e para acomodar discrepâncias físicas (como quebras, perdas ou peças danificadas), garantindo que a plataforma reflete sempre a realidade da oficina.

Tabela 2.8: Requisito funcional 8: Listar *stock* de peças.

Requisito de utilizador	Os administradores da empresa devem ser capazes de fazer a gestão das promoções dos produtos.
Fonte	Entrevista com Administradora Ana
Área do sistema	Finanças
Requisitos de sistema	1. O sistema deve disponibilizar aos Administradores um menu de gestão de promoções. 2. Este menu deve apresentar uma listagem do histórico de promoções (ativas, passadas e futuras), permitindo a sua consulta, 3. Na criação de uma nova promoção, o administrador deve definir: o nome da campanha, a percentagem de desconto, as datas de início e fim, e as peças a que se aplicam. 4. O sistema deve aplicar automaticamente as promoções ativas aos orçamentos e faturas gerados durante o período promocional, recalculando o Preço de Venda ao Público (PVP).
Relevância para a aplicação	A capacidade de criar e gerir promoções de forma autónoma é uma ferramenta comercial essencial. Permite à oficina escoar <i>stock</i> acumulado (ex: peças de modelos de trotinetes mais antigas), dinamizar serviços em épocas de menor afluência (ex: campanhas de "revisão de inverno") e fidelizar clientes. A aplicação automática dos descontos pelo sistema elimina falhas e cálculos manuais por parte dos mecânicos/operadores no momento da faturação.

Tabela 2.9: Requisito funcional 9: Gerir promoções.

Requisito de utilizador	Os administradores devem conseguir consultar e editar o catálogo de peças, incluindo a adição de novas entradas.
Fonte	Entrevista com Administradora Ana
Área do sistema	Catálogos
Requisitos de sistema	1. O sistema deve disponibilizar um menu de gestão do catálogo de peças da oficina. 2. O menu deve permitir listar e pesquisar todos as peças existentes, exibindo a sua referência, descrição, custo de aquisição, <i>stock</i> atual, categoria, Preço de Venda ao Público (PVP) e <i>status</i> de utilização em loja. 3. O sistema deve permitir a criação de um novo registo no catálogo. 4. Na criação ou edição de uma entrada, o sistema deve permitir definir ou atualizar informações como o custo de aquisição, margem de lucro, descrição, <i>stock</i> atual, nome e PVP. 5. O sistema deve permitir inativar itens do catálogo (ex: peças de trotinetes descontinuadas) para que não apareçam em novos orçamentos, mantendo o histórico em faturas antigas.
Relevância para a aplicação	A capacidade de gerir o catálogo de forma dinâmica é vital para a operação contínua da oficina. Ter um catálogo atualizado garante que todos os mecânicos e operadores orçamentam e faturam os serviços com base na mesma tabela de preços, evitando erros manuais, discrepâncias de valores e prejuízos para o negócio.

Tabela 2.10: Requisito funcional 10: Edição do catálogo de peças.

Requisito de utilizador	Os administradores devem conseguir consultar e editar o catálogo de intervenções, incluindo a adição de novas entradas.
Fonte	Entrevista com Administradora Ana
Área do sistema	Catálogos
Requisitos de sistema	1. O sistema deve disponibilizar um menu de gestão do catálogo de intervenções da oficina. 2. O menu deve permitir listar e pesquisar todos as intervenções existentes, exibindo o seu id, descrição, especialidade e preço. 3. O sistema deve permitir a criação de um novo registo no catálogo. 4. Na criação ou edição de uma entrada, o sistema deve permitir definir ou atualizar informações como o preço da intervenção, especialidade e a descrição do serviço. 5. O sistema deve permitir inativar itens do catálogo (ex: intervenções antiquadas) para que não apareçam em novos diagnósticos.
Relevância para a aplicação	A capacidade de gerir o catálogo de forma dinâmica é vital para a operação contínua da oficina. Ter um catálogo atualizado garante que todos os mecânicos e operadores orçamentam e faturam os serviços com base na mesma tabela de preços, evitando erros manuais, discrepâncias de valores e prejuízos para o negócio.

Tabela 2.11: Requisito funcional 11: Edição do catálogo de intervenções.

Requisito de utilizador	Os administradores da oficina devem ser capazes de re-ver e aprovar ordens de encomenda de peças que se encontrem pendentes, como também rececioná-las e visualizar o seu histórico.
Fonte	Entrevista com Administradora Ana
Área do sistema	Encomendas de Stock
Requisitos de sistema	1. O sistema deve monitorizar os níveis de inventário e gerar automaticamente uma ordem de encomenda com o estado "Pendente" sempre que uma peça atinge ou desce abaixo do seu limite de <i>stock</i> mínimo configurado. 2. O sistema deve disponibilizar à administração um menu de gestão de ordens de encomenda. 3. Neste menu, devem estar listadas as encomendas pendentes e em trânsito, detalhando a peça, o fornecedor, a quantidade sugerida para reposição e o custo estimado. 4. O sistema deve permitir ao administrador editar a quantidade a encomendar antes de tomar uma decisão. 5. O sistema deve disponibilizar a opção de "Aprovar" ou "Rejeitar/Cancelar" a ordem pendente. 6. O sistema deve permitir a receção de encomendas pelo administrador 7. O sistema deve permitir a visualização do histórico de encomendas rececionadas. 8. Ao ser aprovada, o estado da ordem de encomenda deve transitar obrigatoriamente de "Pendente" para "Em Trânsito".
Relevância para a aplicação	A criação automática de propostas de encomenda com base no <i>stock</i> mínimo evita ruturas de inventário, garantindo que a oficina tem sempre disponíveis as peças de maior desgaste (ex: pneus, pastilhas de travão, câmaras de ar) para não atrasar as reparações das trotinetes. Contudo, ao exigir a aprovação manual da administração, o sistema garante um controlo financeiro rigoroso, permitindo reter, agrupar ou ajustar encomendas consoante o fluxo de caixa (tesouraria) da empresa, antes de efetivar a compra junto do fornecedor.

Tabela 2.12: Requisito funcional 12 : Aprovação de ordens de encomenda pendentes e visualização de histórico de rececionadas.

Requisito de utilizador	Os administradores da oficina devem ser capazes de consultar dados operacionais detalhados sobre o desempenho do negócio.
Fonte	Entrevista com Administradora Ana
Área do sistema	Finanças
Requisitos de sistema	1. O sistema deve possuir uma área de <i>Dashboard</i> com filtros de pesquisa por período temporal (diário, semanal, mensal ou intervalo de datas personalizado). 2. O dashboard deve calcular e exibir o tempo médio de reparação das trotinetes (desde a entrada na oficina até ficarem prontas para levantamento), faturação, nº de serviços, nº de agendamentos, nº de vendas e montante devolvido. 3. O <i>dashboard</i> deve apresentar o volume de faturação operacional, segregando o valor gerado através de "Mão de Obra"(serviços) e o valor gerado através de "Peças" (produtos). 4. O sistema deve permitir a exportação dos dados do relatório para formatos padrão (ex: PDF) para análise externa.
Relevância para a aplicação	A capacidade de extrair e analisar estes indicadores (KPIs) é o "motor"da gestão da oficina. Saber o tempo médio de reparação permite identificar gargalos (ex: faltam mecânicos ou as peças demoram a chegar). Já a análise da faturação dividida entre peças e mão de obra permite à administração perceber onde está a verdadeira margem de lucro da empresa e ajustar a estratégia de preços do catálogo.

Tabela 2.13: Requisito funcional 13 : Consulta de dados operacionais.

Requisito de utilizador	Os administradores da oficina devem ser capazes de registar novos funcionários no sistema e atribuir-lhes os respetivos níveis de acesso.
Fonte	Observação Direta
Área do sistema	Utilizadores
Requisitos de sistema	1. O sistema deve disponibilizar aos utilizadores com perfil de administrador um menu de gestão de equipa/funcionários. 2. O sistema deve permitir a criação de um novo perfil de funcionário, exigindo o preenchimento de dados obrigatórios (ex: Nome, Email, Contacto telefónico). 3. O sistema deve permitir a atribuição de uma função/-perfil de acesso específico ao novo colaborador (ex: Administrador, Operador de Loja, Mecânico). 4. O sistema deve atribuir um número mecanográfico único a cada novo funcionário registado, que servirá de <i>username</i> para o <i>login</i>. 5. O sistema deve gerar ou permitir definir uma <i>password</i> para o primeiro acesso do funcionário.
Relevância para a aplicação	O registo rigoroso e centralizado da equipa é a base da segurança e da rastreabilidade na plataforma. Ao definir o cargo no momento do registo, o sistema garante que um mecânico acede à agenda técnica e ao <i>stock</i> , mas não tem permissão para ver relatórios financeiros ou aprovar encomendas de elevado valor. Além disso, ter cada funcionário devidamente identificado permite registar o histórico de quem efetuou cada intervenção numa trotinete, facilitando o controlo de qualidade.

Tabela 2.14: Requisito funcional 14 : Registo de novo funcionário.

Requisito de utilizador	Os mecânicos devem conseguir preencher e gerar uma guia de reparação após o diagnóstico inicial de uma trotinete agendada.
Fonte	Entrevista com Mecânico Pedro
Área do sistema	Serviços
Requisitos de sistema	1. O sistema deve permitir ao mecânico de triagem abrir a ficha de intervenção clicando na respetiva <i>slot</i> reservada na agenda. 2. Na abertura da ficha, o sistema deve exibir apenas a informação inicial submetida: o modelo da trotinete e os dados do cliente. 3. Após o diagnóstico, o sistema deve permitir ao mecânico selecionar os diferentes tipos de intervenções a executar, a sua descrição do diagnóstico e o <i>feedback</i> do cliente. 4. O sistema deve adicionar automaticamente o preço do somatório das intervenções, calculado a partir da tabela fixa de preço e peças pré-associadas. 5. Ao concluir o preenchimento, o sistema deve ser capaz de gerar o PDF da guia de reparação.
Relevância para a aplicação	Este fluxo constitui o ponto fulcral da transição entre o diagnóstico inicial e a execução técnica. Ao automatizar a geração da guia com base nas intervenções selecionadas, o sistema assegura o rigor orçamental e a rastreabilidade dos serviços propostos.

Tabela 2.15: Requisito funcional 15: Preencher guia de reparação.

Requisito de utilizador	Os mecânicos devem ser capazes de registar no sistema o código identificador específico das peças que efetivamente utilizaram durante a reparação.
Fonte	Entrevista com Mecânico Pedro
Área do sistema	Serviços e Catálogos
Requisitos de sistema	1. O sistema deve permitir ao mecânico aceder a um serviço que se encontre com o estado "Em Execução". 2. Para cada peça, o sistema deve disponibilizar um campo para inserção manual do código único da peça física instalada. 3. O sistema deve validar se a peça lida corresponde ao modelo orçamentado e se existe em <i>stock</i>. 4. O sistema deve descontar a peça específica do inventário e associar permanentemente esse código ao histórico daquela trotinete.
Relevância para a aplicação	O registo exato da peça aplicada é vital para a rastreabilidade e gestão de garantias. Se uma bateria ou um motor avariar meses depois, a oficina consegue provar ao fabricante exatamente qual foi a peça instalada na trotinete daquele cliente, facilitando a troca. Além disso, garante um rigor absoluto no inventário, evitando que as peças orçamentadas sejam diferentes das peças que saíram fisicamente da prateleira do armazém.

Tabela 2.16: Requisito funcional 16: Registo do código de peça utilizada.

Requisito de utilizador	O sistema deve conseguir alterar automaticamente o estado do serviço na plataforma.
Fonte	Entrevista com Operador João
Área do sistema	Serviços
Requisitos de sistema	1. O sistema atualiza o serviço para "Em Execução" a partir do momento em que o mecânico realiza a primeira intervenção. 2. O sistema deve registar a data e hora exatas em que o serviço transita para o estado "Em Execução" (início) e para "Concluído" (fim). 3. Se for a última intervenção o sistema deve alterar o estado do respetivo Serviço para "Finalizado" e o sistema envia automaticamente um <i>email</i> para o respetivo cliente.
Relevância para a aplicação	Manter o estado da reparação atualizado é essencial para a organização interna da oficina e para a transparência do serviço. Permite aos operadores de loja e administradores saberem em tempo real o volume de trabalho em curso nas bancadas. A notificação automática de conclusão poupa tempo valioso aos funcionários, eliminando a necessidade de chamadas telefónicas manuais, e melhora significativamente a experiência do cliente, que é informado no exato momento em que a sua trotinete está pronta a circular.

Tabela 2.17: Requisito funcional 17: Alterar estado da reparação.

Requisito de utilizador	Os operadores de loja devem conseguir registar a receção física de uma encomenda de peças que se encontrava em trânsito.
Fonte	Entrevista com Operador João
Área do sistema	Encomendas de Stock
Requisitos de sistema	1. O sistema deve apresentar ao operador uma lista com todas as encomendas que estão com o estado "Em Trânsito". 2. O sistema deve permitir seleccionar a encomenda que acabou de chegar e confirmar as quantidades de peças que foram efetivamente entregues pelo fornecedor. 3. Após a confirmação da receção, o sistema deve alterar o estado da encomenda para "Rececionada". 4. O sistema deve somar automaticamente a quantidade de peças recebidas ao <i>stock</i> atual da oficina.
Relevância para a aplicação	A receção correta das encomendas na loja é o momento crucial em que o inventário da aplicação passa a corresponder à realidade física das prateleiras. Fechar este ciclo garante que o material recém-chegado fica imediatamente disponível para a equipa técnica, permitindo desbloquear reparações de trotinetes que estavam paradas à espera de peças e melhorando o tempo de resposta ao cliente.

Tabela 2.18: Requisito funcional 18 : Receção de encomenda de *stock*.

Requisito de utilizador	Os operadores de loja devem conseguir registar a venda avulsa de peças.
Fonte	Entrevista com Operador João
Área do sistema	Finanças / <i>stocks</i>
Requisitos de sistema	1. O sistema deve permitir ao operador iniciar uma nova venda direta para artigos de balcão. 2. O sistema deve permitir pesquisar os artigos por nome ou código EAN. 3. O sistema deve calcular o valor total dos artigos. 4. Após a confirmação do pagamento, o sistema deve emitir a fatura/recibo correspondente.
Relevância para a aplicação	Por motivos legais é necessária fatura. E por motivos de gestão de stock.

Tabela 2.19: Requisito funcional 19: Registo de venda e faturação.

Requisito de utilizador	Os operadores de loja devem conseguir registar a entrega de uma encomenda de peças que foi previamente solicitada por um cliente para levantamento ao balcão.
Fonte	Observação Direta
Área do sistema	Gestão de <i>stocks</i>
Requisitos de sistema	1. O sistema deve permitir ao operador pesquisar e listar as encomendas de clientes que se encontrem com o estado "Pronto para Levantamento" (ex: número de fatura ou número da encomenda). 2. O sistema deve apresentar os detalhes dos artigos reservados para o cliente. 3. O sistema deve permitir ao operador registar a entrega física da mercadoria. 4. Após o pagamento, o sistema deve alterar o estado da encomenda do cliente para "Entregue".
Relevância para a aplicação	Muitas vezes, os clientes não precisam de assistência técnica na oficina, pretendendo apenas encomendar peças específicas (ex: um carregador novo, um capacete ou um pneu) para recolha na loja. Ter um fluxo dedicado para gerir e entregar estas encomendas garante que o <i>stock</i> reservado para um cliente não é vendido acidentalmente a outra pessoa no balcão. Além disso, torna o processo de levantamento muito mais rápido, proporcionando um excelente serviço ao cliente.

Tabela 2.20: Requisito funcional 20: Entrega de encomenda a cliente.

Requisito de utilizador	Os operadores de loja devem conseguir registar o levantamento físico de uma trotinete reparada e o respetivo pagamento, utilizando o NIF do cliente.
Fonte	Entrevista com Operador João
Área do sistema	Serviços
Requisitos de sistema	1. O sistema deve permitir ao operador visualizar as trotinetes prontas para levantamento destacadas com o NIF do cliente. 2. O sistema deve apresentar um ecrã de resumo com o valor total a pagar, detalhando peças aplicadas e mão de obra, de acordo com o que foi registado pelo mecânico. 3. O sistema deve permitir o registo do método de pagamento e emitir a fatura correspondente. 4. Após a validação do pagamento, o sistema deve alterar o estado da reparação para "Fechado".
Relevância para a aplicação	Este é o momento final da jornada do cliente na oficina. A exigência de apresentação do NIF acelera drasticamente o atendimento ao balcão e evita falhas humanas na identificação da trotinete correta a entregar (especialmente útil se houver várias trotinetes do mesmo modelo na loja). Além disso, o bloqueio do sistema até que o pagamento seja registado garante que nenhuma trotinete sai das instalações sem que a receita correspondente dê entrada em caixa.

Tabela 2.21: Requisito funcional 21: Registar levantamento de trotinete e pagamento.

Requisito de utilizador	Os operadores de loja devem conseguir processar a devolução de artigos, mediante a apresentação da respetiva fatura pelo cliente.
Fonte	Observação Direta
Área do sistema	Gestão de Serviços / Gestão de <i>Stocks</i>
Requisitos de sistema	1. O sistema deve permitir ao operador visualizar as transações original a partir do número da fatura apresentada pelo cliente. 2. O sistema deve dar a opção ao operador de realizar uma devolução total. 3. O sistema deve calcular automaticamente o valor a reembolsar, tendo em conta eventuais promoções ou descontos aplicados na altura da compra original. 4. O sistema deve emitir a respetiva nota de crédito e repor a quantidade das peças devolvidas no inventário da oficina.
Relevância para a aplicação	O processo de devolução é uma exigência legal (proteção do consumidor) e uma parte normal do retalho (ex: o cliente comprou um suporte de telemóvel que afinal não cabe no guiador da sua trotinete). Permitir que o operador selecione exatamente o que vai ser devolvido a partir da fatura original impede erros de cálculo em reembolsos e fraudes. A reposição automática do artigo no sistema garante que a peça volta a estar imediatamente disponível para o próximo cliente.

Tabela 2.22: Requisito funcional 22: Processar devolução.

2.7 Requisitos não Funcionais

Os requisitos não funcionais especificam ou restringem as características do sistema como um todo, não se focando em serviços específicos, mas sim em propriedades emergentes (como fiabilidade e desempenho) e em restrições de implementação. Estes requisitos foram definidos tendo em conta o contexto académico do projeto e as tecnologias selecionadas.

Requisito não funcional	O tempo de resposta do sistema para operações de leitura na base de dados (ex: carregamento do catálogo de peças ou pesquisa de clientes) não deve exceder os 2 segundos, em condições normais de rede.
Fonte	Equipa Técnica de Desenvolvimento
Relevância para a aplicação	O desempenho (<i>response time</i>) é uma propriedade emergente crítica. A rapidez do sistema evita estrangulamentos no atendimento ao balcão, garantindo que o operador não faz o cliente esperar enquanto o sistema processa a informação.

Tabela 2.23: Requisito não funcional 1: Tempo de resposta (Desempenho).

Requisito não funcional	A interface do utilizador deve ser adaptável (<i>responsive</i>), suportando adequadamente a interação através de dispositivos de <i>input/output</i> distintos: monitores de <i>desktop</i> (na receção) e ecrãs táteis de <i>tablets</i> (nas bancadas de reparação).
Fonte	Observação Direta dos fluxos de trabalho
Relevância para a aplicação	Define uma restrição clara sobre as capacidades dos dispositivos de I/O. Garante que os mecânicos podem utilizar a aplicação diretamente na zona técnica, sem necessidade de se deslocarem a um terminal fixo. Será assegurado pelo uso de <i>Tailwind CSS</i> .

Tabela 2.24: Requisito não funcional 2: Suporte a múltiplos dispositivos de I/O.

Requisito não funcional	O sistema deve validar todos os <i>inputs</i> inseridos pelos utilizadores (ex: formatos de email, NIF, ou <i>stock</i> negativo) e apresentar mensagens de erro claras, não permitindo que a aplicação encerre de forma inesperada (<i>crash</i>).
Fonte	Boas práticas de Engenharia de Software
Relevância para a aplicação	Relaciona-se com a fiabilidade (<i>reliability</i>) do sistema. Sendo operado por vários funcionários com diferentes níveis de literacia digital, o sistema tem de ser robusto o suficiente para recuperar de erros de introdução de dados sem corromper a base de dados.

Tabela 2.25: Requisito não funcional 3: Fiabilidade e validação de dados.

Requisito não funcional	O sistema deve ser desenvolvido obrigatoriamente utilizando a <i>framework</i> React.js para a interface, .NET para a lógica de negócios, e MongoDB para o armazenamento e representação de dados.
Fonte	Decisão Arquitetural da Equipa Técnica
Relevância para a aplicação	Define as restrições de implementação do sistema. A escolha destas tecnologias impõe um limite claro à forma como os dados serão estruturados e como a interface comunicará com a base de dados, garantindo a coesão técnica do projeto.

Tabela 2.26: Requisito não funcional 4: Restrições de implementação.

Requisito não funcional	O acesso às diferentes áreas do sistema deve ser condicionado pelo perfil do utilizador (RBAC), e as credenciais (<i>passwords</i>) devem ser armazenadas com recurso a algoritmos de <i>hashing</i> (<i>standard</i> do .NET Identity).
Fonte	Administração da MobiFix
Relevância para a aplicação	Restrição de segurança e representação de dados. Garante a integridade da informação financeira (visível apenas para administradores) e impede que os dados fiquem expostos em texto limpo diretamente na base de dados.

Tabela 2.27: Requisito não funcional 5: Segurança e controlo de acessos.

2.8 Refinamento de Requisitos Ambíguos

Durante a análise das entrevistas e questionários, foram identificadas diversas declarações ambíguas que necessitaram de refinamento técnico para evitar falhas na implementação. A tabela seguinte ilustra o processo de transformação dessas necessidades em requisitos claros.

Necessidade Inicial (Ambígua)	Ambiguidade Identificada	Requisito Refinado / Especificação Técnica
"O sistema deve ser rápido."	O que define "rápido"? Depende da ligação ou do servidor?	O tempo de resposta para leituras na BD não deve exceder os 2 segundos (RNF1).
"Controlar melhor o <i>stock</i> ."	Quais as ações? Manual ou automático?	Geração automática de ordens de encomenda pendentes ao atingir o <i>stock</i> mínimo (RF12).
"Permitir vender peças."	Inclui serviços ou apenas itens físicos?	Diferenciação no catálogo entre "Peça"(com <i>stock</i>) e "Serviço"(sem <i>stock</i> físico).

Tabela 2.29: Processo de Refinamento e Clarificação de Requisitos.

2.9 Análise de Restrições

A implementação do sistema Fix'n'Ride é condicionada por um conjunto de fatores técnicos, operacionais e estratégicos que delimitam o âmbito da solução. No plano tecnológico, o desenvolvimento é obrigatoriamente centralizado no ecossistema .NET, com interface em React e persistência de dados em MongoDB.

A interface de utilizador deve ser responsiva, utilizando o framework Tailwind CSS para garantir operabilidade tanto em terminais fixos de balcão como em tablets nas bancadas de reparação.

Operacionalmente, a performance é uma restrição crítica: o tempo de resposta para leituras de dados não deve exceder os 2 segundos. A segurança da informação é salva-

guardada por um sistema de controlo de acessos baseado em perfis (RBAC) e cifragem de credenciais através de algoritmos de hashing.

O projeto visa um retorno de investimento num período inferior a 12 meses, cumprindo rigorosamente o calendário de entrega estabelecido.

Prompt utilizado - Claude Sonnet 4.6

"Escreve a secção de Análise de Restrições para o sistema Fix'n'Ride, abordando três planos: tecnológico (ecossistema .NET, React, MongoDB, Tailwind CSS), operacional (tempo de resposta máximo de 2 segundos, RBAC e hashing de credenciais).O retorno de investimento em menos de 12 meses, cumprimento do calendário de entrega académico)."

2.10 Especificação e Modelação do Sistema

2.10.1 Métodos de modelação adotados

A transição da análise de necessidades para a construção do software exige a adoção de uma linguagem formal que permita abstrair a complexidade do sistema e comunicar inequivocamente a arquitetura proposta.

Com base no levantamento dos requisitos funcionais e não-funcionais apresentado no capítulo anterior, procedeu-se à etapa de especificação do comportamento do sistema. Esta etapa materializou-se na identificação e detalhe dos **Casos de Uso**, os quais descrevem as interações entre os atores e a aplicação, garantindo o alinhamento com os objetivos de negócio da MobiFix.

Simultaneamente, foi elaborada a modelação estrutural através do **Modelo de Domínio**. Este modelo visa estabelecer um vocabulário comum e representar as entidades do mundo real, as suas propriedades e as relações que estabelecem entre si, servindo de base lógica para o posterior desenho do esquema da base de dados relacional.

Prompt utilizado - Claude Sonnet 4.6

"Escreve a introdução à secção de Especificação e Modelação do Sistema para o Fix'n'Ride. Justifica a adoção de Casos de Uso para mapear as interações entre atores e o sistema, e do Modelo de Domínio para representar as entidades de negócio e as suas relações. Explica como estes dois modelos servem de base para o desenho da arquitetura e do esquema da base de dados."

2.10.2 Modelação do domínio do problema

O Modelo de domínio representa a estrutura estática concetual do sistema, focando-se nas entidades de negócio e capturando a semântica operacional da MobiFix. Ilustra como os conceitos fundamentais de negócio se relacionam para suportar os fluxos de oficina e venda. A Figura 2.1 apresenta o diagrama do modelo de domínio desenvolvido de acordo com as especificações levantadas.

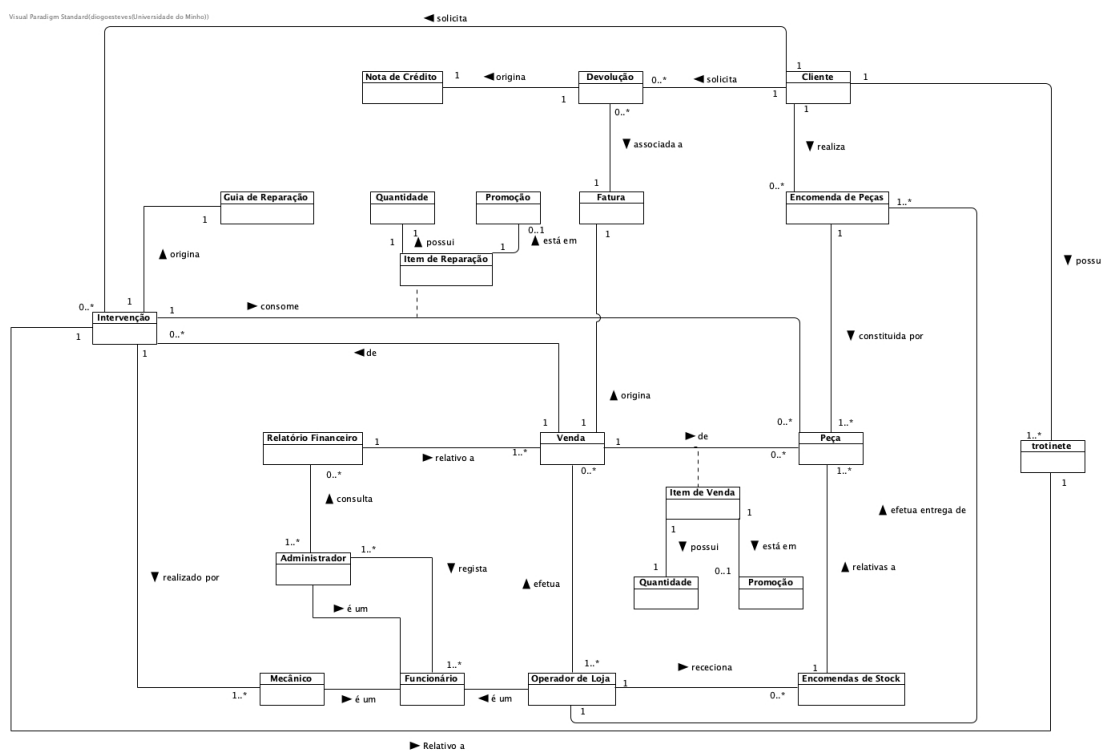


Figura 2.1: Modelo de Domínio(Ver versão ampliada no Anexo)

De forma a sistematizar a interpretação do diagrama, apresentam-se nas subsecções seguintes a descrição das entidades e dos relacionamentos estabelecidos.

Dicionário de Entidades

A Tabela 2.30 e a Tabela 2.31 descrevem as principais classes identificadas no domínio do problema.

Entidade	Descrição
Cliente	Utilizador final detentor das trotinetes e responsável financeiro.
Trotinete	Veículo sujeito a intervenção técnica e histórico de reparações.
Serviço de Reparação	Entidade central que agrega intervenções.
Intervenção	Entidade que corresponde a operações realizadas na trotinete e agrega as peças utilizadas.
Venda	Registo de comercialização de peças ou acessórios ao balcão.
Peça	Artigo físico passível de ser vendido ou incorporado num serviço.
Fatura	Documento fiscal que formaliza a cobrança de vendas ou serviços.
Nota de Crédito	Documento retificativo para anulação total ou parcial de faturas.

Tabela 2.30: Dicionário de Entidades do Modelo de Domínio (Parte 1)

Entidade	Descrição
Devolução	Processo de retorno de artigos ou serviços, originando crédito ao cliente.
Encomenda de Peças	Registo de pedido de componentes para aprovisionamento técnico.
Guia de Reparação	Documento operacional que orienta o técnico na execução do serviço.
Funcionário	Colaborador da empresa com credenciais de acesso ao sistema.
Operador de Loja	Especialização de funcionário focada em vendas e receção.
Mecânico	Especialização de funcionário focada em diagnóstico e reparação.
Administrador	Perfil com privilégios elevados para gestão e parametrização.

Tabela 2.31: Dicionário de Entidades do Modelo de Domínio (Parte 2)

Descrição de Relacionamentos

As tabelas seguintes explicitam as regras de negócio e cardinalidades extraídas do modelo de domínio atualizado.

Relacionamento	Regra de Negócio / Cardinalidade
Cliente <i>possui</i> Trotinete	1:0..*. Um cliente detém múltiplas trotinetes; cada trotinete pertence a apenas um dono.
Cliente <i>solicita</i> Devolução	0..1:0..*. Devoluções são iniciadas por clientes; um cliente pode ter várias devoluções históricas.
Cliente <i>realiza</i> Encomenda	1:0..*. Cada encomenda de peças está vinculada a um único cliente.
Trotinete <i>tem</i> Serviço	1:0..*. Uma trotinete acumula um histórico de intervenções técnicas ao longo do seu ciclo de vida.
Serviço <i>origina</i> Guia	1:1. Cada ordem de reparação gera obrigatoriamente um documento de guia técnica.
Serviço <i>realizado por</i> Mecânico	1:1..*. Uma reparação pode exigir a intervenção de um ou mais técnicos mecânicos.
Serviço <i>composto por</i> Intervenções	1:1..*. Serviços são compostos por um diagnóstico e outros tipos de intervenções.
Intervenção <i>utiliza</i> Peça	1:0..*. Peças podem ser necessárias para realizar intervenção.

Tabela 2.32: Descrição de Relacionamentos do Modelo de Domínio (Parte 1)

Relacionamento	Regra de Negócio / Cardinalidade
Venda <i>origina</i> Fatura	1:1. Cada transação comercial gera exatamente um documento fiscal de faturação.
Devolução <i>origina</i> N. Crédito	1:1. Um processo de devolução concluído resulta na emissão de uma Nota de Crédito.
Devolução <i>associa-se</i> Fatura	0..*:1. Uma devolução refere-se obrigatoriamente a uma fatura previamente emitida.
Operador <i>efetua</i> Venda	1..*:0..*. Vendas são processadas por operadores de loja (um ou mais por transação).
Venda <i>contém</i> Peça	1:1..*. Uma venda pode envolver múltiplos artigos físicos de <i>stock</i> .

Tabela 2.33: Descrição de Relacionamentos do Modelo de Domínio (Parte 2)

2.10.3 Levantamento e especificação de Casos de Uso

A próxima fase da modelação do problema consiste na identificação e documentação das interações fundamentais entre os diversos atores e o sistema. Para tal, procedeu-se ao levantamento de *Use Cases* (Casos de Uso), uma etapa crucial que permite mapear os requisitos funcionais da plataforma de partilha de trotinetes a partir da perspectiva de quem com ela interage.

Nesta secção, são detalhados os cenários de utilização previstos para os diferentes perfis de utilizador. Estes incluem, tipicamente, o cliente comum (focado no registo, aluguer, pagamento e localização de trotinetes) e o administrador ou equipa de manutenção (responsáveis pela gestão da frota, registo de reparações e monitorização global do sistema).

Através da apresentação dos diagrama de casos de uso e das respetivas descrições textuais, pretende-se clarificar o comportamento esperado do sistema em cada operação, estabelecendo assim a base estrutural necessária para o subsequente desenho da arquitetura e desenvolvimento do software. Os diagramas estão sub-divididos por ator envolvido, no sentido de facilitar a leitura do mesmo.

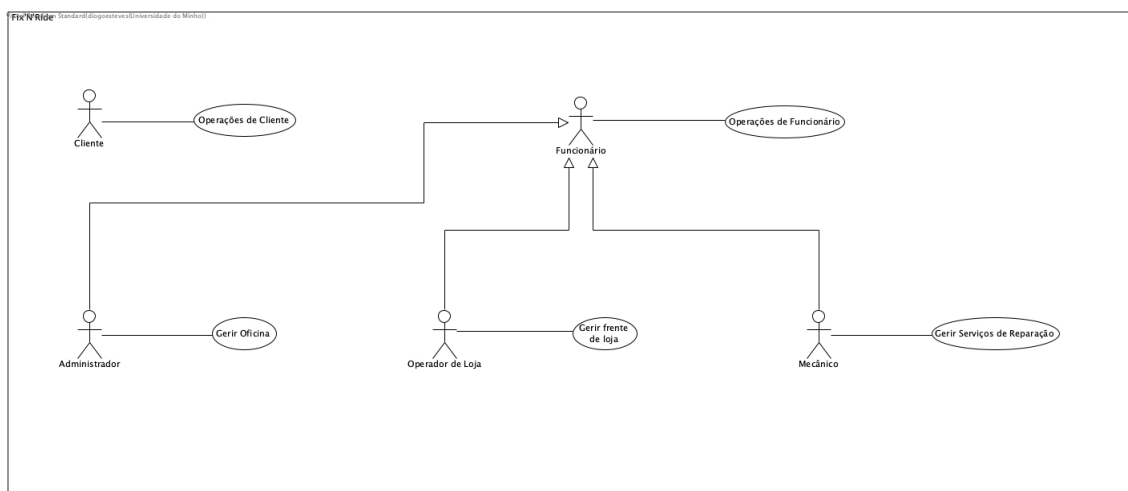


Figura 2.2: Diagrama de casos de uso global do sistema Fix'n'Ride

O primeiro diagrama é referente ao diagrama global de casos de uso. Este diagrama ilustra a visão macrouniversal das interações com o sistema *Fix'n'Ride*, separando claramente os fluxos direcionados ao utilizador externo das operações de gestão interna.

Um aspeto central deste modelo é a utilização da relação de generalização (herança) para modelar a equipa da oficina: o ator genérico "Funcionário" agrega os comportamentos transversais a todo o *staff* (representados pelas "Operações de Funcionário", como a autenticação no sistema). Este ator é depois especializado em três perfis de acesso distintos, consoante as suas competências práticas: o "Administrador", responsável por "Gerir Oficina" (relatórios, promoções e aprovações); o "Operador de Loja", focado em "Gerir frente de loja" (receção, vendas diretas e entregas); e o "Mecânico", encarregue de "Gerir Serviços de Reparação" (triagem técnica e execução de intervenções).

Em paralelo, o "Cliente" atua como um ator independente, interagindo exclusivamente com o pacote de "Operações de Cliente", o qual engloba ações como o agendamento de diagnósticos e a consulta de histórico.

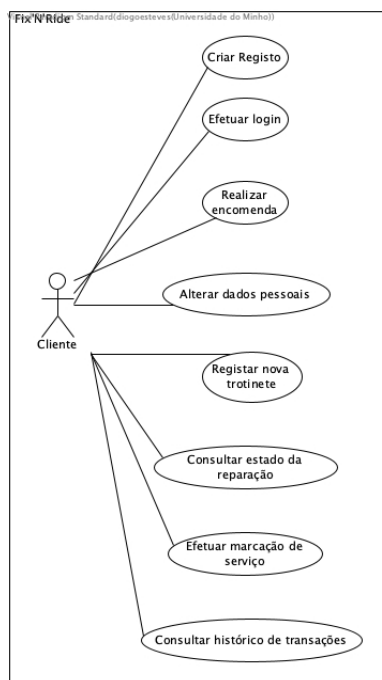


Figura 2.3: Diagrama de casos de uso das operações do cliente no sistema Fix'n'Ride

O segundo diagrama foca-se especificamente no pacote de "Operações de Cliente", detalhando as interações que o utilizador final pode realizar na sua área reservada do sistema Fix'n'Ride. O ator principal, o "Cliente", possui autonomia para gerir o seu perfil através dos casos de uso "Criar Registo", "Efetuar login" e "Alterar dados pessoais". No que diz respeito à componente comercial, o cliente tem a capacidade de "Realizar encomenda" de peças diretamente pelo catálogo online e "Consultar histórico de transações", permitindo-lhe gerir as suas faturas passadas. Relativamente à assistência técnica, o diagrama evidencia o fluxo de "Registar nova trotinete" para associar veículos ao seu perfil,

"Efetuar marcação de serviço" para agendar diagnósticos e "Consultar estado da reparação", garantindo que o utilizador acompanha o progresso das intervenções em tempo real sem necessitar de contactar a oficina.

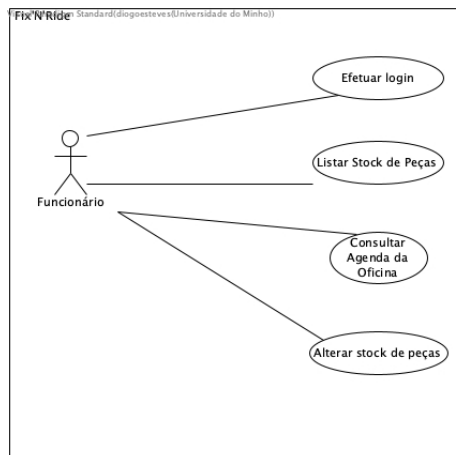


Figura 2.4: Diagrama de casos de uso das operações do funcionário no sistema Fix'n'Ride

O terceiro diagrama foca-se no ator genérico "Funcionário". Este perfil encapsula as operações de base partilhadas por todos os membros do *staff* da oficina, independentemente da sua especialização. As ações incluem a autenticação na plataforma através do caso de uso "Efetuar login", a verificação rápida de inventário com o "Listar Stock de Peças" e o ajuste de discrepâncias com o "Alterar stock de peças". Adicionalmente, este ator tem acesso transversal à organização do trabalho através do caso de uso "Consultar Agenda da Oficina", permitindo a qualquer colaborador visualizar as reparações e entregas planeadas para o dia.

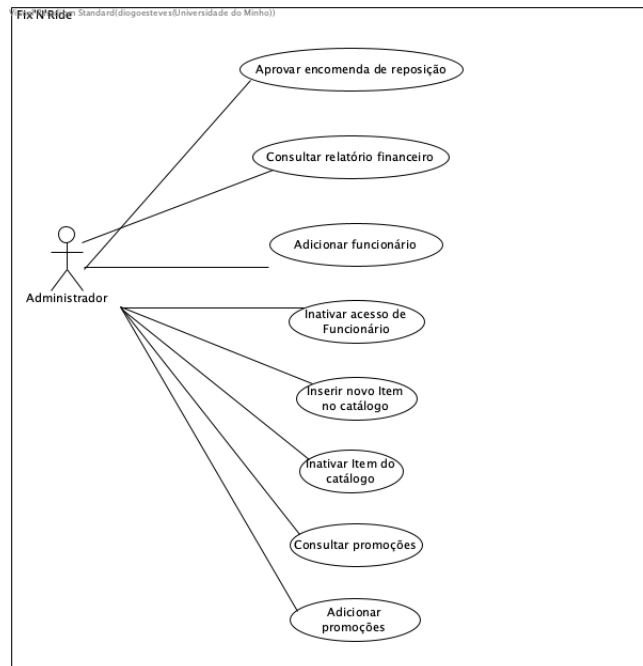


Figura 2.5: Diagrama de casos de uso das operações do administrador no sistema Fix'n'Ride

O quarto diagrama detalha as interações do "Administrador", o perfil com os privilégios mais elevados no sistema. Este ator é responsável pela gestão e parametrização do negócio. No âmbito financeiro e de *stock*, o administrador interage com os casos de uso "Aprovar encomenda de reposição" e "Consultar relatório financeiro". Na vertente comercial, gere os preços e as tabelas através dos fluxos "Inserir novo Item no catálogo", "Inativar Item do catálogo", "Consultar promoções" e "Adicionar promoções". Por fim, assegura o controlo de acessos e a organização da equipa através dos casos de uso "Adicionar funcionário" e "Inativar acesso de Funcionário".

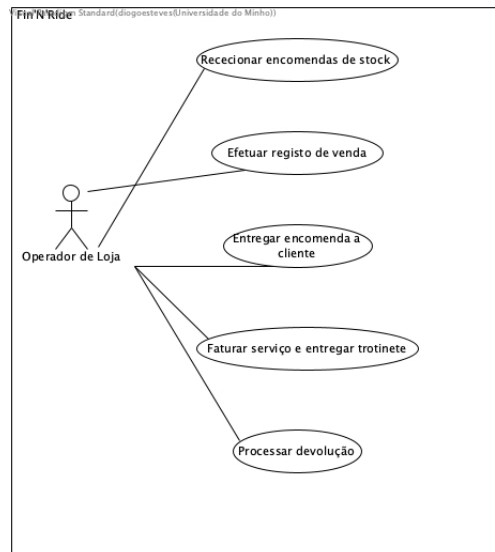


Figura 2.6: Diagrama de casos de uso das operações do operador de loja no sistema Fix'n'Ride

O quinto diagrama ilustra as responsabilidades do "Operador de Loja", que atua na linha da frente comercial e no controlo de entradas físicas de material. As suas interações refletem o atendimento direto ao balcão e a logística diária. Este ator é responsável por "Rececionar encomendas de stock" provenientes dos fornecedores. No atendimento ao público, conduz transações rápidas através de "Efetuar registo de venda" e "Entregar encomenda a cliente". Além disso, é o responsável por finalizar os processos da oficina perante o cliente, utilizando os casos de uso "Faturar serviço e entregar trotinete" e, caso seja necessário, "Processar devolução".

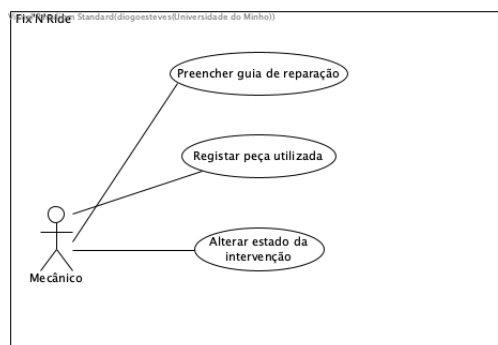


Figura 2.7: Diagrama de casos de uso das operações do mecânico no sistema Fix'n'Ride

O sexto e último diagrama de casos de uso está centrado no "Mecânico", o ator responsável pela execução técnica e intervenção direta nas trotinetes. As suas operações estão desenhadas para serem realizadas a partir da bancada de trabalho, minimizando a carga administrativa. O fluxo inicia-se com a triagem do veículo, onde o mecânico interage com o sistema para "Preencher guia de reparação". Durante a execução do serviço, este utiliza o caso de uso "Registrar peça utilizada" para garantir a rastreabilidade dos componentes instalados e deduzi-los do inventário. Ao longo de todo o processo, o ator gere o progresso do trabalho através do caso de uso "Alterar estado da intervenção", garantindo que o cliente e o restante *staff* se mantêm atualizados sobre a reparação.

Especificação Detalhada dos Casos de Uso

Nesta secção, apresenta-se a descrição detalhada dos principais Casos de Uso (Use Cases) do sistema, focando no fluxo interativo entre os diferentes atores e a aplicação para satisfazer os requisitos operacionais da MobiFix.

Use Case	UC01 – Efetuar agendamento de diagnóstico
Cenários	O Cliente faz o agendamento do diagnóstico para a sua trottinete.
Descrição	O cliente escolhe um horário disponível na agenda da oficina para entregar a sua trottinete para diagnóstico.
Pré-condições	- O Cliente tem sessão iniciada no sistema. - O Cliente tem, pelo menos, uma trottinete registada no seu perfil.

Continua na próxima página

Continuação da Tabela 2.34 — UC01 – Efetuar agendamento de diagnóstico

Use Case	UC01 – Efetuar agendamento de diagnóstico
Pós-condições	• A agenda é registrada com slot do tipo "Diagnóstico" e estado "Reservado". • O serviço é registrado na agenda da oficina com o estado "Agendado" e um intervenção de diagnóstico. • O horário selecionado fica indisponível para outros clientes.
Fluxo normal	1. O cliente acede à sua área reservada e seleciona a opção de "Agendar Diagnóstico". 2. O sistema lista as trotinetes associadas ao cliente. 3. O cliente seleciona a trotinete a intervir.ionar. 4. O sistema informa o cliente que o preço final depende do diagnóstico presencial do técnico. 5. O sistema apresenta as <i>slots</i> de horário disponíveis na oficina. 6. O cliente seleciona o dia e a hora desejados. 7. O sistema registra o diagnóstico na agenda da oficina.

Tabela 2.34: Descrição detalhada do caso de uso — Efetuar agendamento de diagnóstico.

Use Case	UC02 – Realizar Venda Direta ao Balcão
Cenários	• O Operador de Loja efetua a venda de peças diretamente a um cliente.
Descrição	O cliente adquire peças em loja sem necessitar de intervenção técnica por parte dos mecânicos, procedendo ao pagamento imediato.
Pré-condições	• O sistema está operacional. • O Operador de Loja iniciou sessão no sistema. • O artigo solicitado existe no catálogo do sistema.
Pós-condições	• O <i>stock</i> das peças vendidas é decrementado. • A fatura da venda é emitida no sistema. • O valor financeiro é adicionado ao <i>dashboard</i> do dia.
Fluxo normal	1. O Operador de Loja pesquisa e seleciona as peças solicitadas no catálogo. 2. O Operador adiciona os artigos à funcionalidade de Venda Direta. 3. O sistema apresenta o valor a pagar. 4. O Operador confirma o recebimento do pagamento. 5. O sistema emite a fatura. 6. O sistema baixa as unidades ao <i>stock</i> do inventário. 7. O sistema não deixa o <i>stock</i> ir para negativo. 8. O sistema registra o lucro na operação financeira.

Continua na próxima página

Continuação da Tabela 2.35 — UC03 – Realizar Venda Direta ao Balcão

Use Case	UC02 – Realizar Venda Direta ao Balcão
Fluxo alternativo 1	[Geração de encomenda automática] (passo 6): 6.1 O sistema verifica que a venda fez o <i>stock</i> atingir o nível mínimo configurado. 6.2 O sistema cria uma Encomenda de <i>Stock</i> pendente para a Administração. 6.3 O fluxo prossegue normalmente para o passo 7.

Tabela 2.35: Descrição detalhada do caso de uso — Realizar Venda Direta ao Balcão

Use Case	UC03 – Aprovar Encomenda de Stock
Cenários	O Administrador analisa e aprova uma ordem de reposição de <i>stock</i> gerada pelo sistema.
Descrição	O sistema alerta para a necessidade de repor <i>stock</i> de uma peça. O administrador revê a nota de encomenda pendente, ajusta as quantidades se necessário e aprova a compra.
Pré-condições	- O sistema está operacional. - O utilizador com perfil de Administrador iniciou sessão. - O <i>stock</i> de pelo menos uma peça atingiu o limite mínimo, gerando uma pendência.
Pós-condições	O estado da encomenda passa a "Em Trânsito".
Fluxo normal	- O administrador acede ao menu de gestão de encomendas. - O sistema lista as encomendas com estado "Pendente". - O administrador seleciona a encomenda pendente a rever. - O sistema exibe os detalhes (peça, fornecedor, quantidade sugerida, custo estimado). - O administrador verifica a informação e clica em "Aprovar". - O sistema atualiza o registo para "Em Trânsito".
Fluxo alternativo 1	[Ajuste de quantidade] (passo 4): 4.1. O administrador decide encomendar uma quantidade diferente da sugerida. 4.2. O administrador altera o valor no campo de quantidade. 4.3. O sistema recalcula instantaneamente o custo estimado. 4.4. O fluxo regressa ao passo 5.
Fluxo de exceção 1	[Rejeitar Encomenda] (passo 5): 5.1. O administrador decide não avançar com a compra e seleciona "Rejeitar/Cancelar". 5.2. O sistema cancela a encomenda pendente e remove-a da lista. 5.3. O fluxo termina.

Tabela 2.36: Descrição detalhada do caso de uso — Aprovar Encomenda de Reposição

Use Case	UC04 – Faturar Serviço e Entregar Trotinete
Cenários	• O Operador de Loja recebe o cliente, emite a fatura e entrega a trotinete reparada.
Descrição	Conclusão do ciclo de reparação, onde o serviço dado como "Concluído" pelo mecânico é pago pelo cliente e faturado.
Pré-condições	• O sistema está operacional. • O Operador de Loja iniciou sessão. • O serviço correspondente encontra-se no estado "Fechado".
Pós-condições	• A Ordem de Serviço transita para "Fechado". • O documento fiscal (fatura única) é gerado no sistema e enviado para o cliente.
Fluxo normal	1. O cliente apresenta-se na receção para levantar o equipamento. 2. O Operador pesquisa pela trotine associada ao cliente. 3. O sistema apresenta os detalhes do serviço concluído e o valor final (peças e mão de obra). 4. O Operador seleciona a via de pagamento e confirma a transação. 5. O sistema emite a fatura única para o cliente. 6. O Operador entrega a trotinete fisicamente. 7. O sistema altera o estado do serviço para "Fechado" e arquiva-a, enviando também a fatura para a área do cliente.
Fluxo de exceção 1	[Falha no pagamento] (passo 4): 4.1. O pagamento (ex: cartão ou <i>MBWay</i>) é rejeitado. 4.2. O Operador cancela a operação no sistema. 4.3. O sistema mantém o serviço no estado "Concluído". 4.4. A entrega da trotinete é suspensa.

Tabela 2.37: Descrição detalhada do caso de uso — Faturar Serviço e Entregar Trotinete

Use Case	UC05 – Consultar <i>Dashboard</i> de Dados Operacionais
Cenários	• O Administrador acede aos <i>dashboards</i> para analisar a performance da oficina e métricas de faturação.
Descrição	Geração de infográficos com indicadores de desempenho (KPIs), como tempo médio de reparação, faturação e montante devolvido.
Pré-condições	• O sistema está operacional e contém dados históricos registados. • O utilizador com perfil de Administrador iniciou sessão.
Pós-condições	• Nenhuma (operação exclusiva de leitura/consulta).

Continua na próxima página

Continuação da Tabela 2.38 — UC05 – Consultar Dashboard de Dados Operacionais

Use Case	UC05 – Consultar Dashboard de Dados Operacionais
Fluxo normal	1. O administrador navega até ao módulo de Dashboard. 2. O administrador seleciona o intervalo de datas pretendido. 3. O sistema processa os dados da base de dados (Vendas, Serviços e peças). 4. O sistema apresenta o <i>dashboard</i> com gráficos e totais agregados.

Tabela 2.38: Descrição detalhada do caso de uso — Consultar Relatórios Operacionais

Use Case	UC06 – Inserir Novo Item no Catálogo
Cenários	• O Administrador regista uma nova peça para passar a estar disponível na oficina.
Descrição	Adição de um novo registo (peça ou serviço) ao catálogo central do sistema, estipulando os seus custos e preço de venda.
Pré-condições	• O sistema está operacional. • O utilizador com perfil de Administrador iniciou sessão.
Pós-condições	• O novo item é gravado na base de dados e passa a estar imediatamente disponível para venda e orçamentação.
Fluxo normal	1. O administrador acede ao menu de gestão de Catálogo de Peças e Serviços. 2. O administrador seleciona a opção "Novo Registo". 3. O sistema apresenta o formulário de inserção. 4. O administrador preenche os dados (Categoria, Referência, Descrição, Fornecedor, Custos e PVP). 5. O administrador clica em "Guardar". 6. O sistema valida a formatação dos dados e grava a nova entidade. 7. O sistema atualiza e exibe a grelha do catálogo com a nova entrada.
Fluxo de exceção 1	[Referência Duplicada] (passo 6): 6.1. O sistema deteta que a Referência/Código EAN introduzido já existe na base de dados. 6.2. O sistema interrompe a gravação e exibe uma notificação de erro ao administrador. 6.3. O fluxo regressa ao passo 4 para correção do campo.

Tabela 2.39: Descrição detalhada do caso de uso — Inserir Novo Item no Catálogo

Use Case	UC07 – Registrar Conta de Cliente
Cenários	• Um novo utilizador cria uma conta no sistema para poder agendar serviços e comprar peças online.
Descrição	O utilizador fornece os seus dados pessoais e de faturação para criar um perfil de acesso ao portal do cliente.
Pré-condições	• O sistema está operacional.
Pós-condições	• É criado um novo registo na tabela de Clientes. • O cliente fica apto a efetuar <i>login</i> na plataforma.
Fluxo normal	1. O utilizador acede à página de criação de conta. 2. O utilizador preenche os dados solicitados (Nome, Email, Contacto, Morada, NIF e Password). 3. O sistema valida os dados e aplica <i>hashing</i> à <i>password</i>. 4. O sistema guarda o registo e apresenta uma mensagem de sucesso. 5. O sistema redireciona o utilizador para a página de <i>login</i>.
Fluxo alternativo 1	[NIF já registado] (passo 4): 4.1. O sistema deteta que o NIF introduzido já existe na base de dados. 4.2. O sistema impede a criação da conta e exibe uma mensagem de erro ("NIF já em uso"). 4.3. O fluxo regressa ao passo 2 para o utilizador corrigir o NIF ou fazer <i>login</i>.

Tabela 2.40: Descrição detalhada do caso de uso — Registrar Conta de Cliente.

Use Case	UC08 – Realizar Encomenda de Peças
Cenários	• O Cliente adquire peças autonomamente através do catálogo online da loja.
Descrição	Fluxo comercial onde o cliente seleciona peças para posterior levantamento físico.
Pré-condições	• O sistema está operacional. • O Cliente iniciou sessão na sua conta. • Existe <i>stock</i> das peças desejadas para a realização da encomenda.
Pós-condições	• O <i>stock</i> das peças adquiridas é decrementado.
Fluxo normal	1. O cliente acede ao catálogo na <i>landing page</i> e pesquisa peças. 2. O cliente adiciona as peças desejadas ao carrinho. 3. O cliente clica em reservar em peças 4. O sistema deduz as quantidades no inventário geral.

Continua na próxima página

Continuação da Tabela 2.41 — UC08 – Realizar encomenda

Use Case	UC08 – Realizar encomenda
Fluxo alternativo 1	[Atingido <i>stock</i> mínimo de peça] (passo 7): 7.1. O sistema verifica que foi atingido o <i>stock</i> mínimo de uma peça envolvida na transação. 7.2. O sistema cria uma encomenda de novo <i>stock</i> para aprovação da administração.

Tabela 2.41: Descrição detalhada do caso de uso — Realizar encomenda.

Use Case	UC9 – Adicionar Promoções
Cenários	• O Administrador cria uma campanha de desconto para um conjunto de peças.
Descrição	Criação de regras de desconto temporárias que o sistema aplicará automaticamente nas vendas e orçamentos durante um período específico.
Pré-condições	• O sistema está operacional. • O utilizador com perfil de Administrador iniciou sessão.
Pós-condições	• A promoção fica registada e ativa nas datas estipuladas.
Fluxo normal	1. O administrador acede ao menu de gestão de promoções. 2. O administrador seleciona "Nova Promoção". 3. O administrador preenche os dados: percentagem de desconto, datas de início/fim e itens abrangidos. 4. O administrador clica em "Guardar". 5. O sistema valida que as datas são coerentes. 6. O sistema guarda a promoção e atualiza a listagem de campanhas ativas.
Fluxo de exceção 1	[Datas Inválidas] (passo 5): 5.1. O sistema verifica que a data de fim é anterior à data de início. 5.2. O sistema bloqueia a gravação e apresenta um aviso ao utilizador. 5.3. O fluxo regressa ao passo 3.

Tabela 2.42: Descrição detalhada do caso de uso — Gerir Promoções

Use Case	UC10 – Registrar Novo Funcionário
Cenários	• O Administrador adiciona um novo funcionário à equipa no sistema.
Descrição	Criação de credenciais de acesso para funcionários, atribuindo-lhes o nível de permissões adequado às suas funções (RBAC).

Continua na próxima página

Continuação da Tabela 2.43 — UC10 – Registrar Novo Funcionário

Use Case	UC10 – Registrar Novo Funcionário
Pré-condições	• O utilizador com perfil de Administrador iniciou sessão.
Pós-condições	• É gerado um número mecanográfico único. • O novo funcionário fica com acesso ao sistema.
Fluxo normal	1. O administrador acede ao menu de gestão de equipa e clica em "Novo Funcionário". 2. O administrador insere os dados pessoais (Nome, Email, Contacto). 3. O administrador seleciona o perfil de acesso (ex: Mecânico de diagnóstico). 4. O Administrador define a <i>password</i> e clica em "Guardar". 5. O sistema valida os dados e gera um número mecanográfico único. 6. O sistema guarda o registo na base de dados.

Tabela 2.43: Descrição detalhada do caso de uso — Registrar Novo Funcionário

Use Case	UC11 – Alterar <i>Stock</i> de peças
Cenários	• Um funcionário deteta uma discrepância entre o inventário do sistema e o armazém físico e corrige o valor.
Descrição	Alteração manual das quantidades de peças no sistema para refletir perdas, danos ou erros de contagem.
Pré-condições	• O Administrador iniciou sessão.
Pós-condições	• A quantidade da peça é atualizada na base de dados. • É registado um <i>log</i> de alteração manual de inventário.
Fluxo normal	1. O funcionário acede ao menu de gestão de <i>stocks</i>. 2. O sistema apresenta a listagem das peças e o seu respetivo <i>stock</i>. 3. O funcionário clica em "Ajustar <i>Stock</i>". 4. O funcionário insere a quantidade real correta, 5. O funcionário guarda as alterações. 6. O sistema atualiza o valor no inventário.

Tabela 2.44: Descrição detalhada do caso de uso — Ajustar *Stock* Manualmente

Use Case	UC12 – Consultar histórico de faturas
Cenários	• O Cliente consulta as suas faturas passadas.

Continua na próxima página

Continuação da Tabela 2.45 — UC12 – Consultar histórico de faturas

Use Case	UC12 – Consultar histórico de faturas
Descrição	Permite ao cliente visualizar cronologicamente todas as suas compras e serviços, permitindo o download das faturas em PDF.
Pré-condições	• O Cliente iniciou sessão na sua área pessoal.
Pós-condições	• O sistema disponibiliza o documento PDF para transferência.
Fluxo normal	1. O cliente acede ao separador "Faturas". 2. O sistema lista as transações (Vendas e Serviços) de forma cronológica. 3. O cliente clica no botão de download de uma fatura. 4. O sistema recupera e transfere o ficheiro PDF da fatura.

Tabela 2.45: Descrição detalhada do caso de uso — Consultar Histórico de Faturas

Use Case	UC13 – Consultar Estado de Reparação
Cenários	• O Cliente verifica o ponto de situação da sua trotinete em oficina.
Descrição	O cliente acompanha em tempo real o estado do serviço (ex: diagnóstico e reparação).
Pré-condições	• Existe um serviço ativo associado ao cliente.
Pós-condições	• O cliente obtém informação sobre a disponibilidade para levantamento.
Fluxo normal	1. O cliente consulta a lista de serviços em andamento na área reservada. 2. O sistema apresenta o tipo de serviço e o estado atual (ex: "Em Execução"). 3. Quando o estado muda para "Pronto", o sistema exibe a nota de disponibilidade para levantamento.

Tabela 2.46: Descrição detalhada do caso de uso — Consultar Estado de Reparação.

Use Case	UC14 – Consultar Agenda da Oficina
Cenários	• Funcionários consultam o planeamento de reparações e entregas de <i>stock</i>.
Descrição	Visualização centralizada de todos os agendamentos diários, semanais ou mensais da oficina.
Pré-condições	• O Funcionário está autenticado no sistema.
Pós-condições	Nenhuma (operação de consulta).

Continua na próxima página

Continuação da Tabela 2.47 — UC14 – Consultar Agenda da Oficina

Use Case	UC14 – Consultar Agenda da Oficina
Fluxo normal	1. O utilizador acede ao menu "Agenda". 2. O sistema apresenta as vistas (diária, semanal ou mensal). 3. O sistema carrega os dados de agendamentos e entregas de fornecedores. 4. O utilizador clica num evento para ver detalhes (cliente, trotinete, peças ou guia de reparação).

Tabela 2.47: Descrição detalhada do caso de uso — Consultar Agenda da Oficina

Use Case	UC15 – Rececionar Encomenda de <i>Stock</i>
Cenários	• O Operador de Loja ou o Administrador regista a chegada física de material encomendado.
Descrição	Atualização imediata do inventário após conferência de mercadoria entregue por fornecedores.
Pré-condições	• Existe uma encomenda com estado "Em Trânsito".
Pós-condições	• O <i>stock</i> das peças é incrementado automaticamente. • O estado da encomenda transita para "Rececionada".
Fluxo normal	1. O operador ou administrador visualiza a lista de encomendas "Em Trânsito". 2. Seleciona a encomenda que acaba de chegar. 3. Confirma as quantidades efetivamente entregues. 4. O sistema atualiza o inventário geral.

Tabela 2.48: Descrição detalhada do caso de uso — Rececionar Encomenda de *Stock*

Use Case	UC16 – Entregar Encomenda a Cliente
Cenários	• O Cliente levanta peças que reservou previamente sem serviço técnico.
Descrição	Fluxo de entrega ao balcão de material reservado, garantindo que o <i>stock</i> não é vendido a terceiros.
Pré-condições	• Encomenda no estado "Pronto para Levantamento".
Pós-condições	• Encomenda transita para "Entregue".

Continua na próxima página

Continuação da Tabela 2.49 — UC16 – Entregar Encomenda a Cliente

Use Case	UC16 – Entregar Encomenda a Cliente
Fluxo normal	1. O operador pesquisa a encomenda pelo NIF do cliente. 2. O sistema lista os artigos e valores pendentes. 3. O sistema permite o pagamento por <i>MBWay</i> ou Multibanco 4. O operador confirma a entrega física e o pagamento. 5. O sistema altera o estado da encomenda para "Entregue".

Tabela 2.49: Descrição detalhada do caso de uso — Entregar Encomenda a Cliente

Use Case	UC17 – Processar Devolução
Cenários	• O Cliente deseja devolver um artigo mediante fatura. • O cliente apresenta a fatura para levantamento.
Descrição	Reversão total ou parcial de uma venda, com emissão de nota de crédito e reposição de <i>stock</i> .
Pré-condições	• Apresentação da fatura original válida.
Pós-condições	• Emissão de Nota de Crédito. • Reposição automática no inventário.
Fluxo normal	1. O operador procura o número da fatura original.. 2. O operador seleciona a fatura a devolver. 3. O sistema recalcula o valor (considerando promoções originais). 4. O sistema emite a Nota de Crédito e atualiza o <i>stock</i>, bem como as margens de lucro do dia.

Tabela 2.50: Descrição detalhada do caso de uso — Processar Devolução

Use Case	UC18 – Efetuar <i>Login</i> (Cliente)
Cenários	• O Cliente acede à sua área reservada.
Descrição	O sistema valida as credenciais introduzidas e redireciona o utilizador para a sua área pessoal.
Pré-condições	• O cliente possui uma conta ativa no sistema.
Pós-condições	• O cliente é redirecionado para a página da área reservada. • É criada uma sessão segura e autenticada.
Fluxo normal	1. O utilizador insere as suas credenciais (NIF e <i>Password</i>). 2. O sistema valida os dados contra a base de dados. 3. O cliente é redirecionado para a página da área pessoal.

Continua na próxima página

Continuação da Tabela 2.51 — UC18 – Efetuar Login (cliente)

Use Case	UC18 – Efetuar Login (Cliente)
Fluxo de exceção 1	[Credenciais Inválidas] (passo 2): 2.1. O sistema deteta erro nos dados introduzidos. 2.2. O sistema exibe uma mensagem de erro e solicita nova tentativa.

Tabela 2.51: Descrição detalhada do caso de uso — Efetuar Login (Cliente).

Use Case	UC19 – Registrar Nova Trotinete
Cenários	O Cliente adquiriu uma nova trotinete e quer associá-la ao seu histórico.
Descrição	O cliente insere os dados identificadores do veículo (marca, modelo e n.º série) para facilitar futuras reparações.
Pré-condições	O Cliente iniciou sessão.
Pós-condições	O veículo fica disponível na lista de equipamentos do cliente.
Fluxo normal	- O utilizador seleciona "Adicionar Veículo" na sua área pessoal. - O utilizador insere a Marca, Modelo e Número de Série. - O sistema valida se o número de série já existe (evitando duplicados). - O sistema confirma o sucesso do registo.
Fluxo de exceção 1	[Número de série duplicado] (passo 3): 3.1. O sistema verifica que o cliente já tem uma trotinete com o número de série introduzido. 3.2. O sistema informa o cliente que não é permitida a inserção de trotinetes duplicadas. 3.3. A operação termina.

Tabela 2.52: Descrição detalhada do caso de uso — Registrar Nova Trotinete.

Use Case	UC20 – Efetuar Login (Funcionário)
Cenários	O funcionário acede à sua área pessoal.
Descrição	O sistema valida as credenciais introduzidas e redireciona o utilizador para o respetivo dashboard, consoante a sua função.
Pré-condições	O funcionário possui uma conta ativa no sistema.
Pós-condições	- O funcionário é redirecionado para a respetiva área pessoal. - É criada uma sessão segura e autenticada.

Continua na próxima página

Continuação da Tabela 2.53 — UC20 – Efetuar Login (Funcionário)

Use Case	UC20 – Efetuar Login (Funcionário)
Fluxo normal	1. O utilizador insere as suas credenciais (Nº mecanográfico e <i>Password</i>). 2. O sistema valida os dados contra a base de dados. 3. O funcionário é redirecionado para a área pessoal correspondente.
Fluxo de exceção 1	[Credenciais Inválidas] (passo 2): 2.1. O sistema deteta erro nos dados introduzidos. 2.2. O sistema exibe uma mensagem de erro e solicita nova tentativa.

Tabela 2.53: Descrição detalhada do caso de uso — Efetuar Login (Funcionário).

Use Case	UC21 – Alterar estado da intervenção
Cenários	• O mecânico altera o estado de uma intervenção na interface detalhada.
Descrição	O sistema permite ao mecânico atualizar o estado da intervenção no respetivo módulo de diagnóstico e de reparação. O sistema regista a data e hora exata (<i>timestamps</i>) de fim da reparação. Quando as datas de fim de todas as intervenções do serviço estiverem preenchidas, o sistema notifica muda automaticamente o estado da reparação na página do cliente e recalcula o tempo estimado das intervenções fazendo a média com o tempo real apurado.
Pré-condições	• O mecânico possui uma sessão ativa e autenticada no sistema. • Existe um serviço de reparação registado e acessível para alteração.
Pós-condições	• O estado do serviço é atualizado na base de dados. • A <i>timestamp</i> de início e/ou fim é devidamente registada. • A vista do cliente é atualizada em conformidade. • (Se Concluído) O tempo estimado de serviço na plataforma é atualizado, fazendo a média com o novo tempo real da reparação.

Continua na próxima página

Continuação da Tabela 2.54 — UC21 – Alterar estado da intervenção

Use Case	UC21 – Alterar estado da intervenção
Fluxo normal	1. O mecânico acede à interface detalhada da reparação. 2. O mecânico seleciona a intervenção que está a realizar. 3. O mecânico interage com o <i>dropdown</i> para validar a sua realização. 4. O sistema atualiza o registo na tabela de "Serviços" na base de dados. 5. O sistema grava a <i>timestamp</i> exata da transição (início ou fim). 6. O sistema atualiza a vista do cliente. 7. O sistema calcula o tempo real da reparação acabada de concluir e atualiza o tempo estimado de serviço do sistema, fazendo a média com este novo tempo.

Tabela 2.54: Descrição detalhada do caso de uso — Alterar estado da intervenção

Use Case	UC22 – Inativar acesso de Funcionário
Cenários	• O administrador revoga o acesso de um funcionário ao sistema (ex: por término de contrato).
Descrição	O sistema inativa a conta do funcionário para impedir futuros <i>logins</i> , mantendo intacto o histórico de serviços e operações realizadas pelo mesmo para efeitos de auditoria.
Pré-condições	• O utilizador com perfil de Administrador tem sessão iniciada. • A conta do funcionário a inativar existe no sistema.
Pós-condições	• A conta transita para o estado "Inativo". • O funcionário perde a capacidade de autenticação no sistema.
Fluxo normal	1. O administrador acede ao menu de gestão de funcionários. 2. O sistema lista os funcionários ativos e inativos. 3. O administrador pesquisa e seleciona o funcionário a alterar. 4. O administrador clica na opção "Inativar Conta". 5. O sistema solicita a confirmação da operação. 6. O administrador confirma a inativação. 7. O sistema atualiza o estado na base de dados e termina ativamente qualquer sessão que o funcionário possa ter aberta.

Tabela 2.55: Descrição detalhada do caso de uso — Inativar acesso de Funcionário

Use Case	UC23 – Inativar item do catálogo
Cenários	• O Administrador marca uma peça descontinuada ou um serviço obsoleto como inativo.

Continua na próxima página

Continuação da Tabela 2.56 — UC23 – Inativar item do catálogo

Use Case	UC23 – Inativar item do catálogo
Descrição	O sistema oculta o item selecionado de futuras listagens comerciais e de orçamentação, assegurando, contudo, a sua persistência no histórico de faturas e guias de reparação antigas.
Pré-condições	• O Administrador iniciou sessão no sistema.
Pós-condições	• O item deixa de estar visível para seleção por parte dos mecânicos ou para compra online pelos clientes.
Fluxo normal	1. O administrador acede ao menu de gestão do Catálogo de Peças. 2. O administrador pesquisa o item que pretende descontinuar. 3. O administrador clica na opção "Inativar Item" na respetiva linha. 4. O sistema exibe um aviso de confirmação. 5. O administrador confirma a ação. 6. O sistema atualiza o estado do registo e remove o item da visualização pública e das opções de nova orçamentação.

Tabela 2.56: Descrição detalhada do caso de uso — Inativar item do catálogo

Use Case	UC24 – Consultar Promoções
Cenários	• O Administrador verifica a listagem de campanhas promocionais a decorrer ou já terminadas.
Descrição	Visualização e pesquisa do histórico de todas as promoções (ativas, passadas e futuras) criadas no sistema.
Pré-condições	• O utilizador com perfil de Administrador iniciou sessão.
Pós-condições	É apresentada a lista das promoções passadas, correntes ou futuras ao administrador.
Fluxo normal	1. O administrador acede ao menu de gestão de promoções. 2. O sistema carrega a listagem de todas as campanhas registadas. 3. O administrador aplica filtros (ex: "Apenas Ativas" ou pesquisa por intervalo de datas). 4. O sistema apresenta os resultados, detalhando as percentagens de desconto e os itens abrangidos por cada campanha.

Tabela 2.57: Descrição detalhada do caso de uso — Consultar Promoções

Use Case	UC25 – Registar Peça Utilizada
Cenários	• O Mecânico instala uma nova bateria e regista o seu número na ficha da reparação.

Continua na próxima página

Continuação da Tabela 2.58 — UC25 – Registrar Peça Utilizada

Use Case	UC25 – Registrar Peça Utilizada
Descrição	Inserção manual do código identificador da peça física aplicada na trotinete, promovendo a rastreabilidade para efeitos de garantia e ajustando o inventário.
Pré-condições	• O mecânico iniciou sessão no sistema. • O serviço selecionado encontra-se no estado "Em Execução".
Pós-condições	• O <i>stock</i> daquela peça é decrementado. • O código único lido fica permanentemente associado ao histórico daquela trotinete.
Fluxo normal	1. O mecânico acede à interface do serviço em curso. 2. O sistema lista as peças previstas no orçamento original. 3. O mecânico introduz o código da peça física que vai instalar. 4. O sistema valida se a peça corresponde ao modelo orçamentado e se existe <i>stock</i> da mesma. 5. O sistema regista o código na base de dados e desconta uma unidade no inventário geral.
Fluxo de exceção 1	[Peça Inválida ou Sem <i>Stock</i>] (passo 4): 4.1. O sistema deteta que a peça não existe no armazém ou não corresponde à orçamentada. 4.2. O sistema bloqueia a inserção e exibe uma notificação de erro ao mecânico. 4.3. O fluxo termina.

Tabela 2.58: Descrição detalhada do caso de uso — Registrar Peça Utilizada

Use Case	UC26 – Rececionar Encomenda de <i>Stock</i>
Cenários	• O Operador de loja ou Administrador recebe o material físico de um fornecedor e dá entrada do mesmo no sistema.
Descrição	O operador ou administrador confere as mercadorias entregues face à encomenda pendente, validando as quantidades e incrementando o inventário da loja.
Pré-condições	• O Operador de loja ou Administrador iniciou sessão no sistema. • Existe pelo menos uma encomenda no estado "Em Trânsito".
Pós-condições	• O estado da encomenda passa a "Rececionada". • O <i>stock</i> das peças recebidas é atualizado de forma automática.

Continua na próxima página

Continuação da Tabela 2.59 — UC26 – Rececionar Encomenda de *Stock*

Use Case	UC26 – Rececionar Encomenda de <i>Stock</i>
Fluxo normal	1. O operador ou administrador acede ao menu de gestão de encomendas e filtra as "Em Trânsito". 2. O operador seleciona a encomenda correspondente à guia de entrega do fornecedor. 3. O sistema exibe os itens esperados e as respetivas quantidades. 4. O operador verifica e confirma que as quantidades entregues coincidem com a lista. 5. O sistema altera o estado da encomenda para "Rececionada". 6. O sistema soma as unidades entregues ao inventário central de <i>stock</i>.

Tabela 2.59: Descrição detalhada do caso de uso — Rececionar Encomenda de *Stock*

3 Especificação do Software

A especificação do sistema *Fix'n'Ride* segue as diretrizes estabelecidas pelas normas IEEE 830 e ISO/IEC/IEEE 29148 para a Especificação de Requisitos de Software (SRS). Esta secção detalha a perspetiva do produto, o ambiente operacional e as restrições tecnológicas que guiarão as fases de desenho arquitetural e implementação.

3.1 Perspetiva do Produto

O *Fix'n'Ride* é um sistema de informação *Web-based* independente, concebido para substituir o atual ecossistema fragmentado de folhas de cálculo e processos manuais da *MobiFix*. O sistema atuará como a plataforma central da oficina, orquestrando desde a receção e diagnóstico de trotinetes até à gestão de inventário e faturação, garantindo uma fonte única de verdade (*Single Source of Truth*) para todos os dados operacionais e financeiros. A solução adota uma arquitetura multi-tier, garantindo a independência entre a interface do utilizador, a lógica de processamento e a persistência de dados.

Prompt utilizado - Claude Sonnet 4.6

"Escreve a secção 'Perspetiva do Produto' para a SRS do sistema *Fix'n'Ride*, seguindo as normas IEEE 830 e ISO/IEC/IEEE 29148. O sistema é uma aplicação Web independente que substitui o ecossistema fragmentado de folhas de cálculo e processos manuais da *MobiFix*. Deve atuar como plataforma central da oficina, desde a receção e diagnóstico de trotinetes até à gestão de inventário e faturação, garantindo uma *Single Source of Truth* para todos os dados operacionais e financeiros. Refere a arquitetura multi-tier adotada."

3.2 Ambiente Operacional

A aplicação será disponibilizada num modelo cliente-servidor através da Web, exigindo apenas um *browser* moderno (Google Chrome, Mozilla Firefox, Microsoft Edge ou Safari)

para o seu acesso, eliminando a necessidade de instalações locais nos postos de trabalho da oficina. O sistema deverá ser totalmente responsivo, garantindo uma utilização fluida tanto nos computadores da receção como nos *tablets* ou *smartphones* utilizados pelos mecânicos nas bancadas de reparação.

Prompt utilizado - Claude Sonnet 4.6

"Descreve o Ambiente Operacional do sistema Fix'n'Ride. O sistema deve ser disponibilizado num modelo cliente-servidor via Web, acessível apenas através de browser moderno (Chrome, Firefox, Edge, Safari), sem necessidade de instalações locais nos postos de trabalho da oficina. Deve ser totalmente responsivo para garantir utilização fluida tanto nos computadores da receção como nos tablets e smartphones dos mecânicos nas bancadas de reparação."

3.3 Diagramas de Atividades

Os diagramas de atividades apresentados nesta secção detalham o comportamento dinâmico do sistema, ilustrando o fluxo de controlo e de dados entre os diferentes intervenientes e a plataforma. Estes diagramas complementam a especificação de Casos de Uso, garantindo que os processos operacionais da MobiFix sejam executados de forma coerente e eficiente.

3.3.1 Fluxos de Atendimento e Diagnóstico

O ciclo de vida de uma reparação inicia-se com o agendamento por parte do cliente, conforme ilustrado na Figura 3.1. Após a autenticação, o utilizador seleciona uma trotinete do seu histórico e reserva uma slot disponível na agenda da oficina.

No dia agendado, ocorre o processo de triagem técnica (Figura 3.2). O mecânico realiza o diagnóstico físico e seleciona as intervenções necessárias através do catálogo digital, o que permite ao sistema gerar automaticamente uma Guia de Reparação em formato PDF para validação do cliente.

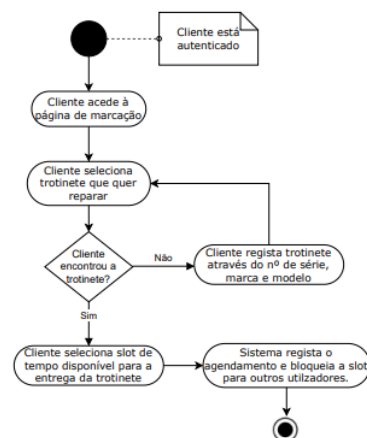


Figura 3.1: Agendar Diagnóstico

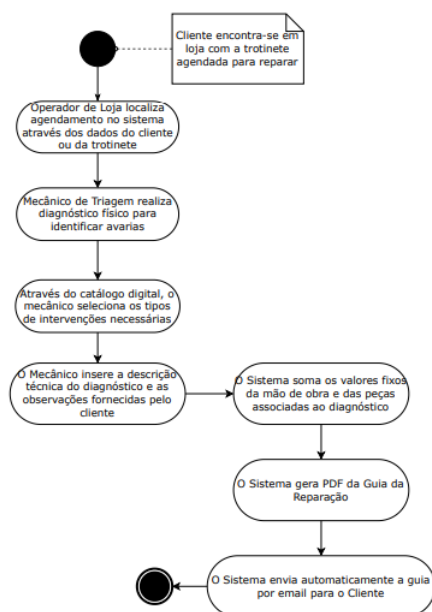


Figura 3.2: Diagnóstico e Geração de Guia de Reparação

3.3.2 Gestão de Inventário e Reposição

A robustez da cadeia de abastecimento é garantida por fluxos de reposição inteligente. A Figura 3.3 descreve o mecanismo de monitorização automática de stock: ao atingir o limite mínimo, o sistema cria uma encomenda com o estado "Pendente". Esta deve ser revista e aprovada pela administração (Figura 3.4) para controlo financeiro antes da transição para o estado "Em Trânsito".

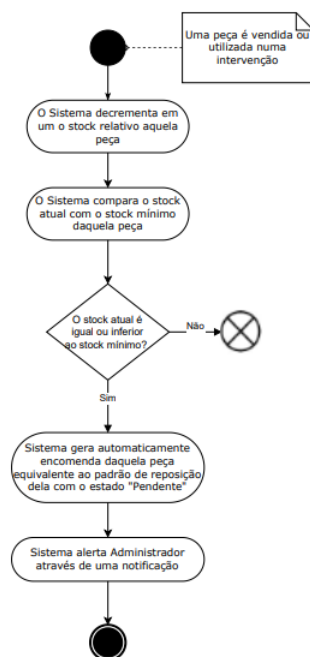


Figura 3.3: Encomenda de Reposição Inteligente

A receção física das peças pelo operador de loja (Figura 3.5) encerra o ciclo de aprovisionamento, incrementando automaticamente o inventário disponível para a oficina.

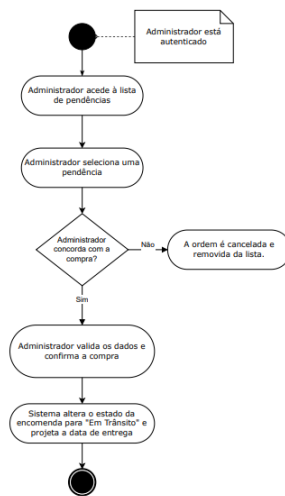


Figura 3.4: Administrador valida Encomenda

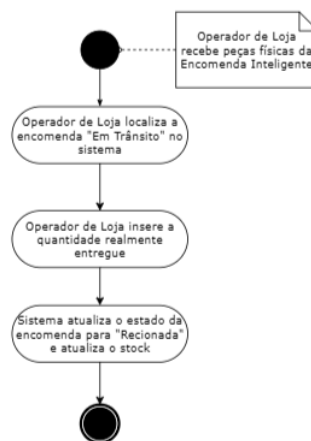


Figura 3.5: Operador de Loja recepciona encomenda

3.3.3 Execução e Faturação

Durante a execução da reparação (Figura 3.6), o mecânico regista os códigos únicos das peças utilizadas para garantir a rastreabilidade e as timestamps de início e fim do serviço. A conclusão de todas as tarefas aciona uma notificação automática ao cliente.

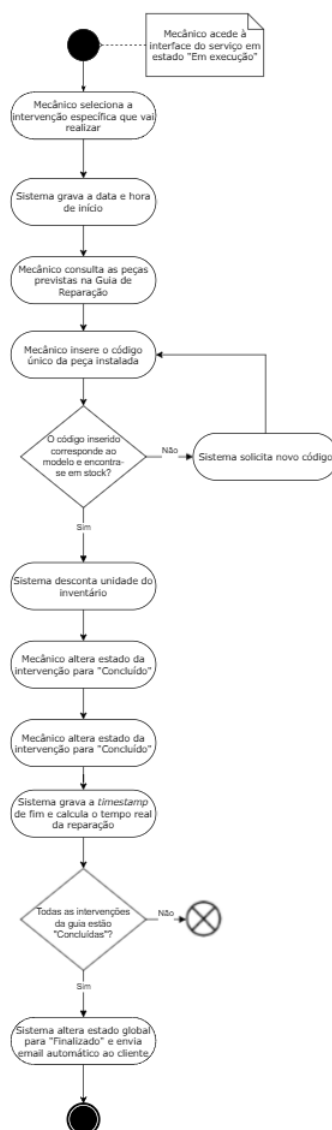


Figura 3.6: Execução da Reparação

Finalmente, o sistema suporta tanto a Venda Direta ao Balcão (Figura 3.7) quanto o levantamento de veículos reparados (Figura 3.11), assegurando a emissão das respetivas faturas e a conformidade fiscal de todas as transações.

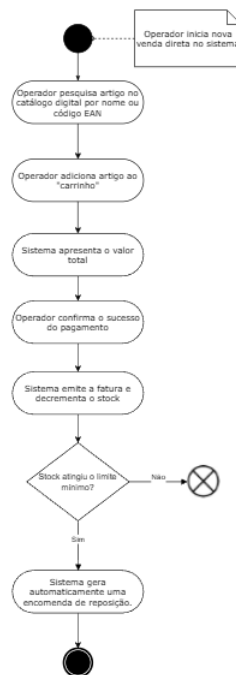


Figura 3.7: Venda Direta ao Balcão

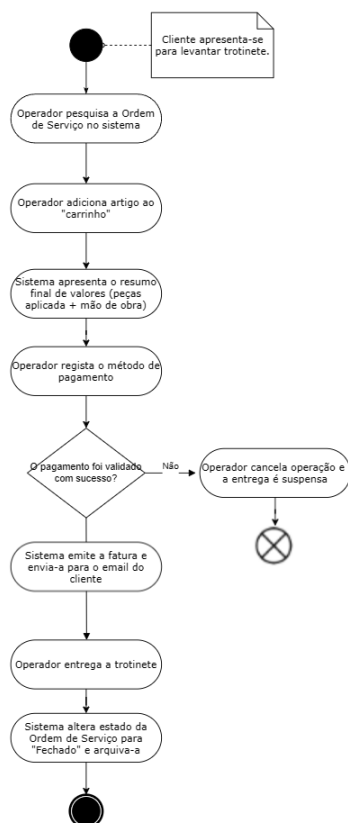


Figura 3.8: Faturação e Levantamento da Trotinete

3.4 Diagramas de Máquina de Estados

Para uma melhor compreensão do funcionamento das 3 operações-chave do sistema, recorreu-se à elaboração de Diagramas de Máquina de Estados. Estes diagramas ilustram as dinâmicas da plataforma *Fix'n'Ride*, mapeando todos os estados pelos quais as principais entidades transitam desde a sua criação até à sua conclusão ou arquivo, bem como os eventos que despoletam essas transições.

Nesta secção, apresentam-se os modelos comportamentais de três fluxos fundamentais para a operação da *MobiFix*:

- **Serviço de Reparação (Ordem de Serviço):** Acompanha a jornada técnica e comercial da trotinete, iniciando-se com o agendamento do diagnóstico pelo cliente, passando pela execução mecânica na bancada, até ao estado de conclusão e posterior faturação e levantamento ao balcão.
- **Encomenda de Reposição (Gestão de Stock):** Modela a cadeia de abastecimento automatizada, desde o momento em que o sistema alerta para a necessidade de repor *stock* e gera uma pendência, passando pela aprovação financeira da administração para "Em Trânsito", até à atualização do inventário após a conferência da mercadoria entregue pelo fornecedor.
- **Encomenda de Cliente (Venda e Levantamento):** Ilustra o fluxo comercial de aquisição autónoma de peças online (*Click & Collect*), onde a encomenda transita do estado de "Pronto para Levantamento" para "Entregue" após a recolha física e verificação na loja.

Abaixo encontram-se representados os respetivos diagramas que formalizam estas lógicas de transição.

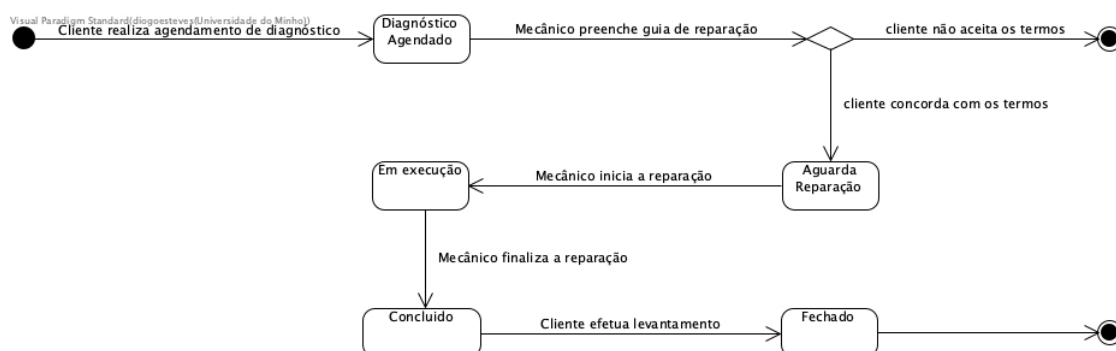


Figura 3.9: Diagrama de máquina de estados do serviço de reparação

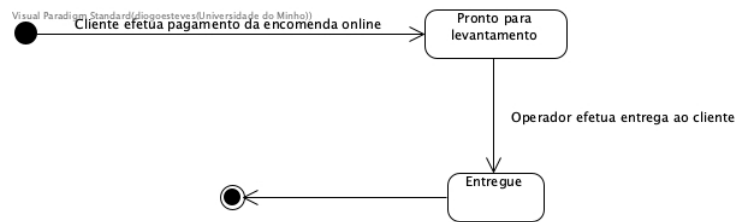


Figura 3.10: Diagrama de máquina de estados da encomenda de cliente

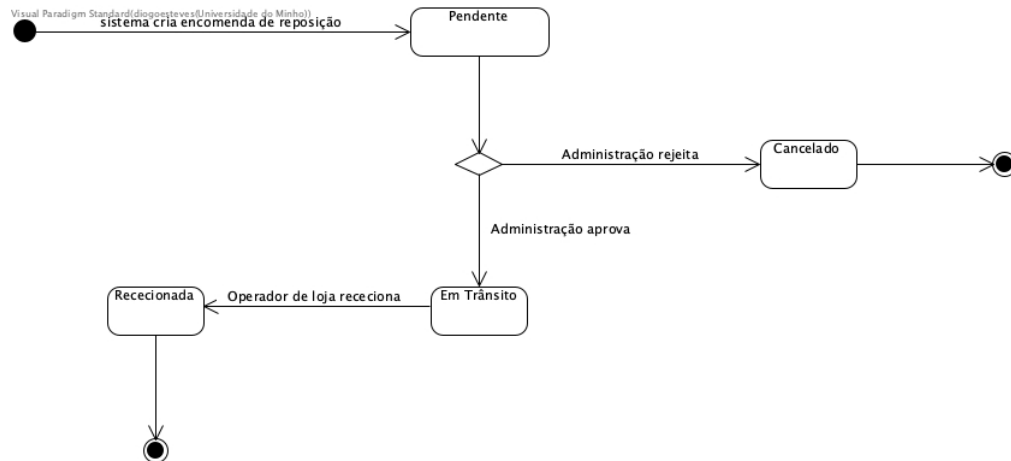


Figura 3.11: Diagrama de máquina de estados da encomenda de reposição

3.5 Reflexão acerca da utilização de LLMs na concepção e engenharia de requisitos do sistema Fix’N’Ride

A adoção de Grandes Modelos de Linguagem (LLMs) na primeira fase de concepção e engenharia de requisitos do sistema Fix’n’Ride assumiu um papel estrito de assistência e aceleração, e não de substituição da análise humana. É imperativo salientar que a Inteligência Artificial não foi responsável pela identificação nem pelo levantamento dos requisitos operacionais ou de negócio. A definição das regras, necessidades e restrições do sistema foi integralmente conduzida pela equipa técnica com base no estudo das operações da MobiFix. A utilização das LLMs centrou-se, exclusivamente, na agilização da materialização e documentação desse esforço analítico.

Neste contexto, as ferramentas de IA revelaram-se valiosas na estruturação inicial do documento, prestando apoio na geração da contextualização do projeto e na pesquisa exploratória sobre o panorama atual da micromobilidade. Foram também utilizadas para transformar as notas recolhidas pela equipa em narrativas coesas, auxiliando na redação das simulações das entrevistas aos *stakeholders* e na formulação padronizada das *User Stories*, garantindo que a comunicação das necessidades se mantinha clara e perceptível.

Adicionalmente, as LLMs atuaram como um co-piloto na escrita deste capítulo, otimizando o tempo de redação. A sua capacidade de processamento de linguagem natural foi aplicada no refinamento e na articulação de parágrafos mais densos e técnicos, assegurando a coesão, a correção gramatical e o rigor vocabular exigidos num documento formal de Engenharia de Software.

Por fim, estabeleceu-se uma fronteira metodológica clara: não houve qualquer recurso a LLMs para o desenho ou geração dos diagramas UML, nem na identificação dos requisitos. Toda a modelação visual — incluindo o Modelo de Domínio, os Diagramas de Casos de Uso e os Diagramas de Atividades — resultou exclusivamente do esforço analítico, lógico e concetual da equipa de desenvolvimento. O levantamento dos requisitos foi realizado tendo por base o conhecimento e a observação da equipa em cenários reais, utilizando exemplos de oficinas de reparação consolidadas (como a Norauto) e procurando espelhar o seu funcionamento no contexto da micromobilidade. Esta abordagem garantiu que a espinha dorsal arquitetural do sistema Fix’n’Ride reflete fielmente as restrições da realidade, sem introduzir alucinações ou artefactos genéricos na estrutura do sistema.

4 Arquitetura e Design do Software

4.1 Introdução

O presente capítulo documenta as decisões de arquitetura e de *design* de *software* que sustentam o sistema **Fix'n'Ride**. Partindo dos requisitos funcionais e não funcionais especificados no capítulo anterior, procedeu-se à definição da arquitetura global do sistema, à seleção fundamentada da *stack* tecnológica e à documentação das principais decisões estruturais — com recurso a ferramentas de Inteligência Artificial Generativa (LLMs) como assistentes de revisão arquitetural e de produção de artefactos UML.

O capítulo encontra-se organizado em três eixos principais. Em primeiro lugar, apresenta-se a **visão geral da arquitetura**, descrevendo o padrão estrutural adotado e a forma como os quatro serviços do sistema se articulam entre si. Em segundo lugar, justifica-se a **seleção tecnológica**, relacionando cada escolha com as exigências operacionais da **MobiFix**. Por fim, documenta-se a **configuração do ambiente**, incluindo a orquestração de contentores Docker, a estratégia de persistência de dados e a gestão de configurações sensíveis.

É importante notar que os diagramas de classes e de sequência detalhados por módulo — que aprofundam o comportamento interno de cada camada — são apresentados nos capítulos subsequentes, dedicados às respetivas camadas do sistema.

Prompt utilizado - Claude Sonnet 4.6

"Escreve a introdução ao capítulo de Arquitetura e Design do Software do sistema Fix'n'Ride. O capítulo deve contextualizar as decisões tomadas na segunda etapa do projeto, referindo que se parte dos requisitos do capítulo anterior para definir a arquitetura global, a *stack* tecnológica e as decisões estruturais. Organiza a introdução em três eixos: visão geral da arquitetura (quatro serviços em Docker), justificação da seleção tecnológica, e configuração do ambiente. Menciona o uso de LLMs como assistentes de revisão arquitetural e produção de artefactos UML."

4.2 Visão Geral da Arquitetura do Sistema

O sistema **Fix'n'Ride** foi concebido segundo uma arquitetura de **múltiplas camadas** (*Multi-Tier Architecture*), com separação física e lógica clara entre a camada de apresentação, a camada de lógica de negócios e a camada de persistência de dados. Esta abordagem materializa o princípio de *Separation of Concerns*, garantindo que cada componente possui uma responsabilidade bem delimitada — o que facilita a manutenibilidade, a testabilidade independente de cada camada e a escalabilidade horizontal do sistema.

A arquitetura global é composta por quatro serviços independentes, cada um empacotado num contentor Docker dedicado: o servidor de base de dados (mongodb), a *Data API* (data-api), o servidor de Lógica de Negócios (1n) e o servidor de *Frontend* (frontend). A comunicação entre camadas segue um modelo **cliente-servidor unidirecional e hierárquico**: o *Frontend* React comunica exclusivamente com o servidor de Lógica de Negócios, que por sua vez delega as operações de persistência na *Data API*. O MongoDB é acessível apenas por este último serviço, nunca diretamente pelas camadas superiores, garantindo o encapsulamento da lógica de acesso a dados e a proteção do motor de persistência.

A Figura 4.1 apresenta o diagrama de componentes global do sistema, ilustrando esta topologia e os fluxos de comunicação entre os quatro serviços.

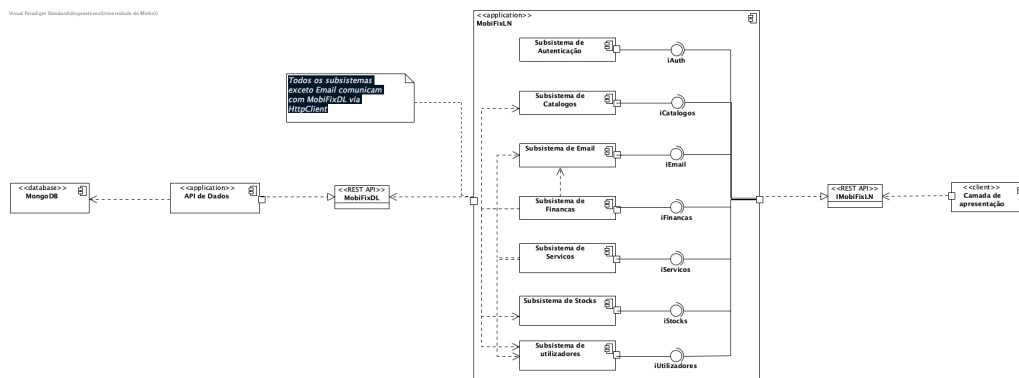


Figura 4.1: Diagrama de componentes global do sistema Fix'n'Ride.

4.2.1 Padrões Arquiteturais Adotados

A arquitetura do **Fix'n'Ride** é governada por um padrão estrutural transversal a ambos os servidores de *backend*, cuja aplicação garante a coesão interna de cada serviço e o baixo acoplamento entre eles.

Tanto na camada de Lógica de Negócios (.NET) como na *Data API* (Node.js/Express),

foi adotado o padrão **MVC** (*Model-View-Controller*), com uma separação clara entre três responsabilidades distintas. Os **Controllers** recebem os pedidos HTTP e devolvem as respostas adequadas, sem conter lógica de negócio. Os **Services** (no servidor .NET) e os **Controllers** de domínio (na *Data API*) encapsulam a lógica de processamento de cada módulo funcional — utilizadores, serviços de oficina, stocks e finanças. Os **Models**, materializados sob a forma de DTOs (*Data Transfer Objects*) em ambas as camadas, definem os contratos de dados trocados entre serviços, garantindo que nenhuma estrutura interna é exposta inadvertidamente ao exterior.

Na *Data API*, os *Schemas* Mongoose assumem adicionalmente o papel de Modelos de domínio, acrescentando validação declarativa, tipagem e referências entre coleções (*Referencing*) diretamente ao nível da definição da estrutura de dados. Toda esta organização é segmentada por domínio funcional, garantindo a modularidade e a navegabilidade da base de código.

Prompt utilizado - Claude Sonnet 4.6

"Descreve a visão geral da arquitetura do sistema Fix'n'Ride e os padrões arquiteturais adotados. O sistema segue uma Multi-Tier Architecture com separação física e lógica entre apresentação, lógica de negócios e persistência. É composto por quatro serviços Docker independentes: mongodb, data-api (Node.js/Express), In (ASP.NET Core) e frontend (React). O padrão MVC é adotado em ambos os backends, com Controllers, Services e DTOs. Na Data API, os Schemas Mongoose assumem o papel de Modelos de domínio com validação declarativa. Toda a organização é segmentada por domínio funcional."

4.3 Justificação da *Stack* Tecnológica

A *stack* tecnológica implementada resultou de um conjunto de decisões deliberadas, fundamentadas nas exigências operacionais da **MobiFix** e nas características do domínio do problema. A Tabela 4.1 detalha cada componente tecnológico, a camada que ocupa na arquitetura e a justificação da sua adoção.

Tecnologia	Decisão Arquitetural e Justificação
MongoDB + Mon-goose	Camada: Persistência de dados. A natureza heterogênea e evolutiva das entidades do domínio beneficia do modelo de documentos BSON. Entidades como <i>Servico</i> — que agrega intervenções e peças em <i>arrays</i> embutidas — e <i>Fatura</i> — que embute subdocumentos de devoluções com notas de crédito — são representadas de forma nativa, eliminando a necessidade de tabelas de junção e operações JOIN. O Mongoose acrescenta validação de <i>schemas</i>, tipagem e referências entre coleções (<i>Referencing</i>), aproximando o desenvolvimento NoSQL das boas práticas da orientação a objetos.
Node.js + Express	Camada: <i>Data API</i> (CRUD). A afinidade natural do ecossistema JavaScript com o formato JSON produzido nativamente pelo MongoDB, aliada ao modelo de I/O não-bloqueante e orientado a eventos do Node.js, torna esta combinação adequada para servir um volume elevado de pedidos CRUD concorrentes com baixa latência. O ecossistema npm disponibilizou imediatamente dependências críticas como <i>mongoose</i>, <i>jsonwebtoken</i> e <i>morgan</i>.
ASP.NET (.NET 8) Core	Camada: Lógica de negócios. As regras de negócio complexas da MobiFix — orquestração do levantamento de trotinetes, <i>hashing</i> de <i>passwords</i> via BCrypt, geração e validação de tokens JWT — exigem um ambiente fortemente tipado, com injeção de dependências nativa e documentação automática de APIs via Swagger/OpenAPI. A adoção deliberada de uma tecnologia distinta face à <i>Data API</i> reforça a separação de responsabilidades e impede o acesso direto à base de dados pela camada de negócios.
React + TanStack Query	Camada: <i>Frontend</i> (SPA). A arquitetura baseada em componentes reutilizáveis do React, conjugada com a biblioteca TanStack Query — que abstrai a comunicação assíncrona com a API, disponibilizando <i>caching</i> automático, revalidação e sincronização de estado com o servidor —, garante uma interface reativa e performante. A navegação entre portais é gerida pelo React Router, com proteção de rotas baseada no perfil de utilizador extraído do token JWT.
Tailwind CSS	Camada: Estilização da interface. A abordagem <i>utility-first</i> acelera o desenvolvimento da interface e garante consistência visual entre os quatro portais do sistema (cliente, mecânico, operador de loja e administrador), sem a sobrecarga de folhas de estilos globais de difícil manutenção e com uma pegada de CSS mínima em produção graças ao mecanismo de <i>purging</i> automático.

Continua na próxima página

Continuação da Tabela 4.1 — Decisões Arquiteturais e *Stack* Tecnológica

Tecnologia	Decisão Arquitetural e Justificação
Docker + Docker Compose	Camada: Orquestração de serviços. Garante a portabilidade e reprodutibilidade do ambiente de execução, eliminando inconsistências entre ambientes de desenvolvimento e produção. O isolamento de cada serviço numa rede interna privada, com exposição ao exterior apenas das portas estritamente necessárias, reforça a postura de segurança do sistema. A declaração de dependências de arranque assegura que os serviços iniciam na ordem correta, prevenindo falhas de conexão em cadeia.

Tabela 4.1: Decisões arquiteturais e justificação da *stack* tecnológica do sistema

Prompt utilizado - Claude Sonnet 4.6

"Gera uma tabela de decisões arquiteturais e justificação da *stack* tecnológica para o sistema Fix'n'Ride. Para cada tecnologia, indica a camada da arquitetura que ocupa e justifica a sua adoção no contexto das operações da MobiFix. Tecnologias a cobrir: MongoDB + Mongoose (persistência, estrutura de documentos BSON para entidades como Serviço e Fatura), Node.js + Express (Data API CRUD, afinidade JSON, I/O não-bloqueante), ASP.NET Core .NET 8 (lógica de negócios, tipagem forte, JWT, Swagger), React + TanStack Query (SPA, caching, React Router com proteção de rotas por JWT), Tailwind CSS (utility-first, quatro portais), Docker + Docker Compose (portabilidade, rede interna privada, ordem de arranque)."

Deltas: A IA justificou inicialmente a escolha do MongoDB com argumentos genéricos sobre flexibilidade NoSQL. Foi necessário reformular com exemplos concretos do domínio Fix'n'Ride, como a entidade Serviço com arrays embutidos de intervenções e a Fatura com subdocumentos de devoluções. A justificação da separação entre Data API (Node.js) e Lógica de Negócios (.NET) como decisão deliberada para impedir o acesso direto à base de dados também não foi contemplada pela IA e foi introduzida manualmente.

4.4 Configuração do Ambiente de Desenvolvimento

A infraestrutura de suporte ao desenvolvimento e execução do **Fix'n'Ride** é integralmente definida através de um único ficheiro de orquestração, o `docker-compose.yml`, localizado na raiz do repositório. Este ficheiro declara os quatro serviços que compõem o sistema, as suas dependências de arranque, os volumes de persistência de dados e as variáveis de ambiente injetadas em cada componente.

4.4.1 Serviços Docker e Topologia da Rede Interna

O `docker-compose.yml` define quatro serviços interdependentes, cuja comunicação interna é efetuada através de uma rede *bridge* privada gerida automaticamente pelo Docker Engine. Nesta rede, cada serviço é resolvível pelo nome do seu contentor como *host-name* — por exemplo, o serviço `ln` acede à *Data API* através do endereço `http://data-api:3001`, sem necessidade de expor essa porta ao exterior. O acesso externo é restrito às portas mapeadas explicitamente na configuração.

A cadeia de dependências declarada (`mongodb` → `data-api` → `ln` → `frontend`) garante que o Docker Compose respeita a ordem de arranque dos serviços, prevenindo falhas de ligação por tentativas prematuras de conexão a serviços ainda não disponíveis. A Tabela 4.2 resume os quatro serviços, as suas responsabilidades, portas expostas e dependências de arranque.

Serviço	Responsabilidade, Porta e Dependências
<code>mongodb</code>	Responsabilidade: Servidor de base de dados NoSQL. Persiste os documentos da base de dados <code>fixnride_db</code> num volume Docker dedicado (<code>mongodb_data</code>). Executa automaticamente o <code>script init.sh</code> no primeiro arranque, importando as doze coleções de dados iniciais e criando os índices de performance. Porta: Não exposta ao exterior (acesso apenas pela rede interna Docker). Depende de: —
<code>data-api</code>	Responsabilidade: API RESTful de acesso direto à base de dados, implementada em Node.js/Express. Abstrai o MongoDB, expondo os dados através de <i>endpoints</i> prefixados com <code>/api/*</code> , organizados por domínio funcional. Comunica com o <code>mongodb</code> pela rede interna via <code>MONGO_URI</code> . Porta: <code>\${DATA_API_PORT}:3001</code> Depende de: <code>mongodb</code>
<code>ln</code>	Responsabilidade: Servidor de Lógica de Negócios em ASP.NET Core (.NET 8). Aloja as regras de negócio complexas, a autenticação JWT e a comunicação inter-serviços com a <code>data-api</code> , autenticada via <code>INTERNAL_API_KEY</code> . Expõe documentação interativa via Swagger UI em ambiente de desenvolvimento. Porta: <code>\${LN_API_PORT}:5001</code> Depende de: <code>data-api</code>
<code>frontend</code>	Responsabilidade: Servidor de desenvolvimento da interface React, executado via <code>npm run dev</code> sobre a imagem <code>node:22-alpine</code> . Serve a SPA ao <i>browser</i> do utilizador e delega todas as chamadas de API para o serviço <code>ln</code> . O volume <code>bind mount ./Frontend:/app</code> permite <i>hot-reload</i> em tempo real durante o desenvolvimento. Porta: <code>3000:3000</code> Depende de: <code>ln</code>

Continua na próxima página

Continuação da Tabela 4.2 — Serviços Docker

Serviço	Responsabilidade, Porta e Dependências
---------	--

Tabela 4.2: Resumo dos serviços Docker do sistema Fix'n'Ride

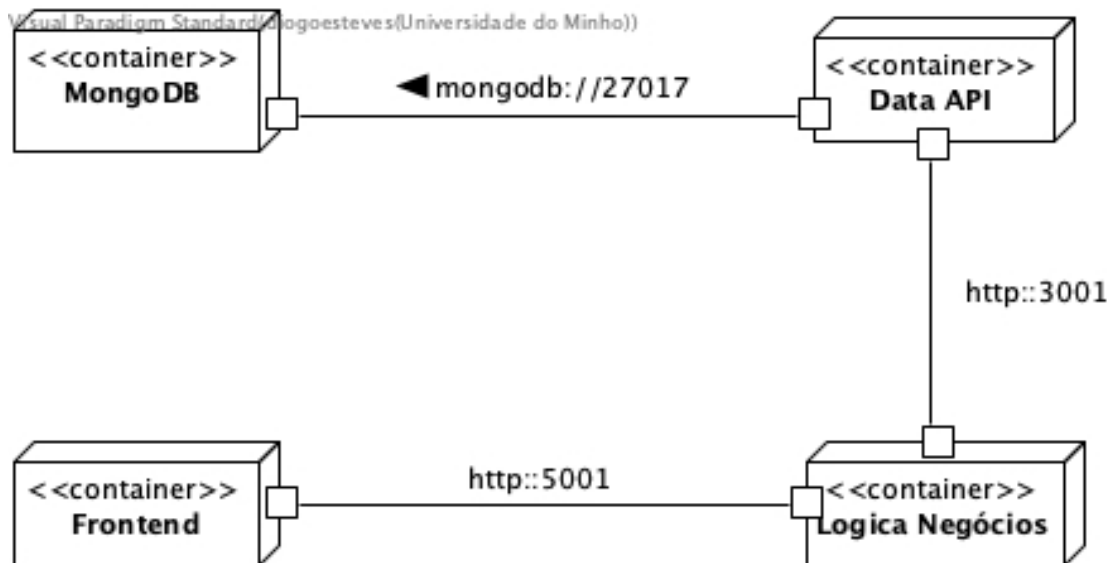


Figura 4.2: Diagrama de arquitetura de contêineres do sistema Fix'n'Ride.

4.4.2 Persistência de Dados e Inicialização Automática da Base de Dados

A persistência dos dados do MongoDB é assegurada através de um **Docker Volume** nomeado (`mongodb_data`), montado em `/data/db` no interior do contêiner. Este mecanismo garante que os dados da base de dados `fixnride_db` sobrevivem ao ciclo de vida dos contêineres, não sendo perdidos em caso de reconstrução ou reinício do serviço.

O serviço `mongodb` utiliza uma imagem personalizada, construída a partir de `mongo:latest`, que incorpora as ferramentas `mongodb-database-tools` e copia um conjunto de *scripts* para o diretório `/docker-entrypoint-initdb.d/`. No primeiro arranque do contêiner, o *script* `init.sh` é executado automaticamente pelo *entrypoint* oficial da imagem MongoDB, procedendo à importação das doze coleções de dados de *seed* e à criação dos índices de unicidade e performance:

Listing 4.1: Excerto do *script* de inicialização e criação de índices da base de dados (init.sh)

```
# Importacao das colecoes de dados iniciais
importar "funcionarios"          "funcionarios.json"
importar "clientes"              "clientes.json"
importar "trotinetes"           "trotinetes.json"
importar "pecas"                 "pecas.json"
importar "intervencoes_catalogo" "intervencoes_catalogo.json"
importar "servicos"              "servicos.json"
importar "agenda"                "agenda.json"
importar "vendas"                "vendas.json"
importar "faturas"               "faturas.json"
importar "encomendas_cliente"    "encomendas_cliente.json"
importar "encomendas_stock"      "encomendas_stock.json"
importar "promocoes"             "promocoes.json"

# Criacao de indices via mongosh
db.funcionarios.createIndex({ email: 1 }, { unique: true });
db.clientes.createIndex({ email: 1 }, { unique: true });
db.trotinetes.createIndex({ clienteId: 1 });
db.servicos.createIndex({ trotineteId: 1 });
db.servicos.createIndex({ estado: 1 });
db.faturas.createIndex({ clienteId: 1 });
db.agenda.createIndex({ mecanicoId: 1, dataHoraInicio: 1 });
```

Esta abordagem de *Infrastructure as Code* (IaC) garante que qualquer elemento da equipa pode reconstituir o ambiente completo — com dados de teste consistentes e índices de performance aplicados — através de um único comando (`docker compose up -build`).

Prompt utilizado - Claude Sonnet 4.6

"Descreve a estratégia de persistência de dados e inicialização automática da base de dados do Fix'n'Ride. O MongoDB usa um Docker Volume nomeado (`mongodb_data`) montado em `/data/db` para garantir que os dados sobrevivem ao ciclo de vida dos conteúdos. A imagem personalizada incorpora `mongodb-database-tools` e executa automaticamente o script `init.sh` no primeiro arranque, importando as doze coleções de seed e criando índices de unicidade e performance. Esta abordagem de *Infrastructure as Code* permite reconstituir o ambiente completo com um único comando (`docker compose up -build`)."

4.4.3 Gestão de Configurações e Variáveis de Ambiente

Seguindo os princípios de *DevSecOps* e as recomendações da metodologia **Twelve-Factor App**, todas as configurações sensíveis são externalizadas num ficheiro `.env`, localizado na raiz do projeto e explicitamente excluído do controlo de versões via `.gitignore`. Os serviços `data-api` e `ln` referenciam este ficheiro através da diretiva `env_file: .env` no `docker-compose.yml`, sendo as variáveis injetadas nos contentores no momento do seu arranque.

Esta prática garante que credenciais críticas — como a chave secreta de assinatura de tokens JWT, a chave de autenticação inter-serviços e a *connection string* do MongoDB — não são incluídas de forma estática no código fonte (*hardcoded*), mitigando o risco de exposição acidental em repositórios de código. A Tabela 4.3 detalha as variáveis de ambiente mais relevantes do sistema.

Variável	Serviço(s) Consumidor(es) e Finalidade
MONGO_URI	Serviço: data-api. URI de ligação ao MongoDB pela rede interna Docker. Utiliza o nome do contentor como <i>hostname</i> : <code>mongodb://mongodb:27017/fixnride_db</code> .
MONGO_INITDB_DATABASE	Serviço: mongodb. Define o nome da base de dados a criar no primeiro arranque do contentor (<code>fixnride_db</code>).
DATA_API_PORT	Serviço: data-api. Porta exposta ao <i>host</i> pelo serviço data-api, configurável sem necessidade de alterar o <code>docker-compose.yml</code> .
LN_API_PORT	Serviço: ln. Porta exposta ao <i>host</i> pelo servidor de Lógica de Negócios (.NET).
INTERNAL_API_KEY	Serviços: data-api e ln. Chave de API partilhada entre serviços internos. Autentica os pedidos do serviço ln junto da data-api, prevenindo chamadas diretas não autorizadas às rotas de sistema protegidas (e.g., <code>/api/auth/funcionario/:numero</code>).
JwtSettings_SecretKey	Serviços: ln e data-api. Chave criptográfica para assinatura e validação de tokens JWT. Partilhada entre os dois serviços para garantir consistência na verificação de tokens emitidos pelo servidor .NET.
DataApiSettings_BaseUrl	Serviço: ln. URL base da <i>Data API</i> acessível pela rede interna (<code>http://data-api:3001</code>), utilizado pelo HttpClient do serviço .NET em todas as chamadas inter-serviços.
ASPNETCORE_ENVIRONMENT	Serviço: ln. Define o ambiente de execução do ASP.NET Core (Development ou Production), controlando a ativação do Swagger UI e o nível de detalhe nas mensagens de erro devolvidas ao cliente.

Continua na próxima página

Continuação da Tabela 4.3 — Variáveis de Ambiente

Variável	Serviço(s) Consumidor(es) e Finalidade
CHOKIDAR _USEPOLLING	Serviço: frontend. Ativa o modo de <i>polling</i> para detecção de alterações de ficheiros pelo servidor Vite, necessário em ambientes Docker no Windows e macOS onde os eventos de sistema de ficheiros nativos não são propagados para o interior dos contentores.

Tabela 4.3: Principais variáveis de ambiente do sistema Fix’n’Ride

Prompt utilizado - Claude Sonnet 4.6

"Descreve a gestão de configurações e variáveis de ambiente do Fix’n’Ride segundo os princípios de DevSecOps e a metodologia Twelve-Factor App. Todas as configurações sensíveis são externalizadas num ficheiro .env excluído do controlo de versões via .gitignore. Detalha as variáveis mais relevantes: MONGO_URI, MONGO_INITDB_DATABASE, DATA_API_PORT, LN_API_PORT, INTERNAL_API_KEY (autenticação inter-serviços entre In e data-api), JwtSettings__SecretKey (partilhada entre os dois backends), Data-ApiSettings__BaseUrl, ASPNETCORE_ENVIRONMENT e CHOKIDAR_USEPOLLING (para hot-reload em Docker no Windows/macOS)."

Deltas: A IA gerou uma lista de variáveis de ambiente genéricas sem contextualizar o propósito específico de cada uma no Fix’n’Ride. Foi necessário introduzir manualmente a explicação da INTERNAL_API_KEY como mecanismo de autenticação inter-serviços que previne chamadas diretas não autorizadas à Data API, e a justificação da CHOKIDAR_USEPOLLING como solução para uma limitação específica do Docker em sistemas operativos Windows e macOS, que a IA desconhecia.

4.5 Síntese do Capítulo

A arquitetura definida para o **Fix’n’Ride** reflete um conjunto de decisões conscientes e justificadas, orientadas por três princípios fundamentais: a **separação de responsabilidades** entre serviços com tecnologias e objetivos distintos; a **segurança por camadas**, com autenticação JWT para os utilizadores, chave de API interna para a comunicação inter-serviços e externalização de credenciais sensíveis; e a **reprodutibilidade do ambiente**, garantida pelo paradigma *Infrastructure as Code* implementado via Docker Compose.

A separação entre a *Data API* (Node.js, responsável pelo acesso direto ao MongoDB) e o servidor de Lógica de Negócios (.NET, responsável pela orquestração de processos

complexos) constitui o alicerce desta arquitetura: permite que cada camada evolua independentemente, que as regras de negócio da oficina sejam testadas de forma isolada e que a base de dados nunca seja exposta diretamente à internet. Os capítulos seguintes aprofundam cada uma destas camadas, detalhando os modelos de dados, os padrões de implementação e os fluxos de interação que concretizam os requisitos funcionais da **MobiFix**.

5 Arquitetura da Camada de Dados

5.1 Justificação da Escolha do MongoDB

A seleção do **MongoDB** como sistema de gestão de base de dados do **Fix’n’Ride** resultou de uma análise cuidada das características do domínio operacional da **MobiFix** e das limitações inerentes ao modelo relacional clássico face a esse domínio.

O primeiro fator determinante prende-se com a **natureza hierárquica e agregada dos dados**. Entidades centrais do sistema, como o *Serviço*, encapsulam estruturas de dados intrinsecamente aninhadas: um serviço de reparação contém um histórico de intervenções realizadas, cada uma das quais agrega a sua própria lista de peças utilizadas, o mecânico responsável e os tempos de execução. Numa base de dados relacional, esta estrutura exigiria no mínimo três tabelas distintas com operações de junção (*JOIN*) em cada leitura. Com o MongoDB, todo este agregado é persistido num único documento, tornando a leitura e a escrita atómicas e semanticamente coerentes com a unidade de negócio.

O segundo fator reside na **flexibilidade do esquema**. Ao longo do ciclo de vida da oficina, entidades como a *Fatura* podem adquirir novas propriedades — por exemplo, um subdocumento de devolução com nota de crédito — sem que seja necessário executar migrações de esquema que bloqueariam a base de dados em contexto de produção. A biblioteca **Mongoose** mitiga o risco de inconsistência associado a esta flexibilidade, impondo validações, tipagem e enumerações definidas ao nível da aplicação.

O terceiro fator é a **performance em operações de leitura do domínio**. As consultas mais frequentes do sistema — listar os serviços em execução de um mecânico, obter o histórico de reparações de uma trotinete, ou consultar as encomendas pendentes de um cliente — beneficiam da colocação de dados relacionados no mesmo documento, eliminando *round-trips* adicionais à base de dados. Os índices criados automaticamente pelo *script* de inicialização (*init.sh*) reforçam esta vantagem para os campos de filtragem mais utilizados.

Prompt utilizado - Claude Sonnet 4.6

"Justifica a escolha do MongoDB como sistema de gestão de base de dados para o Fix'n'Ride, no contexto das operações da MobiFix. Aborda três fatores determinantes: (1) a natureza hierárquica e agregada dos dados, exemplificando com a entidade Serviço que embute intervenções com peças e timestamps; (2) a flexibilidade do esquema para acomodar novas propriedades como subdocumentos de devolução em Fatura sem migrações bloqueantes; (3) a performance em leituras frequentes do domínio, como listar serviços em execução de um mecânico ou o histórico de reparações de uma trotinete. Menciona o papel do Mongoose na mitigação dos riscos da flexibilidade do esquema."

5.2 Estratégias de Modelação: *Embedding* vs. *Referencing*

A modelação das coleções do sistema recorreu deliberadamente a uma combinação das duas estratégias fundamentais do MongoDB: ***Embedding*** (subdocumentos embutidos) e ***Referencing*** (referências entre documentos). A escolha entre as duas estratégias em cada relação foi guiada por três critérios: a frequência de acesso conjunto aos dados, a cardinalidade da relação e a necessidade de acesso independente à entidade relacionada.

***Embedding* — Dados Fortemente Acoplados.** Optou-se pela estratégia de *embedding* nas situações em que os dados relacionados não possuem existência nem utilidade fora do contexto do documento pai, e em que a cardinalidade é limitada e previsível. Os casos mais significativos são:

- `Fatura` $\supset$ `devolucoes`: Cada devolução — contendo data, motivo e a respetiva nota de crédito com valor creditado — é embutida como subdocumento no array `devolucoes` da fatura. Uma devolução não existe nem faz sentido fora da fatura a que pertence, tornando o *embedding* a escolha natural.
- `Servico` $\supset$ `historicoIntervencoes` $\supset$ `pecasUtilizadas`: O histórico de intervenções de um serviço, incluindo as peças consumidas em cada uma, é embutido diretamente no documento do serviço numa estrutura de dois níveis de aninhamento. Esta decisão permite que a leitura completa do estado de uma reparação seja satisfeita com uma única operação `findById`, sem qualquer *join*.
- `Venda` $\supset$ `itensVenda`: Os itens de uma venda direta (peça e quantidade) são embutidos como array no documento da venda, pela mesma lógica de agregação atômica.

- `EncomendaCliente` $\supset$ `itens`: As listas de itens encomendados pelo cliente seguem o mesmo padrão, dado que os itens não têm existência autónoma fora da encomenda.
- `Promocao` $\supset$ `peçasAplicaveisIds`: Os EANs das peças às quais uma promoção se aplica são armazenados como um array de identificadores embutido no documento da promoção, evitando uma coleção de ligação para uma relação de baixa cardinalidade.

Referencing — Entidades com Existência Autónoma.

A estratégia de *referencing* foi adotada para todas as relações entre entidades que possuem um ciclo de vida e uma identidade própria, independente do documento que as referencia. A diretiva `ref` do Mongoose formaliza estas referências, permitindo o uso de `populate()` quando se pretende resolver a relação. Os casos principais são:

- `Trotinete` $\rightarrow$ `Cliente`: O campo `clienteId` armazena o NIF do cliente proprietário. Um cliente existe independentemente das suas trotinetes e é acedido de forma autónoma em múltiplos contextos (autenticação, faturação, perfil).
- `Servico` $\rightarrow$ `Trotinete`: O campo `trotineteId` referencia a trotinete pelo número de série. A trotinete é uma entidade de longa duração com estado próprio (`emServico`).
- `Fatura` $\rightarrow$ `Cliente`, `Servico`, `Venda`: A fatura mantém referências ao cliente e, opcionalmente, ao serviço ou venda que a originou, permitindo navegar o histórico financeiro em qualquer direção.
- `Agenda` $\rightarrow$ `Funcionario`, `Servico`: Um slot de agenda referencia o mecânico responsável e o serviço associado, entidades que existem e são geridas de forma completamente independente.

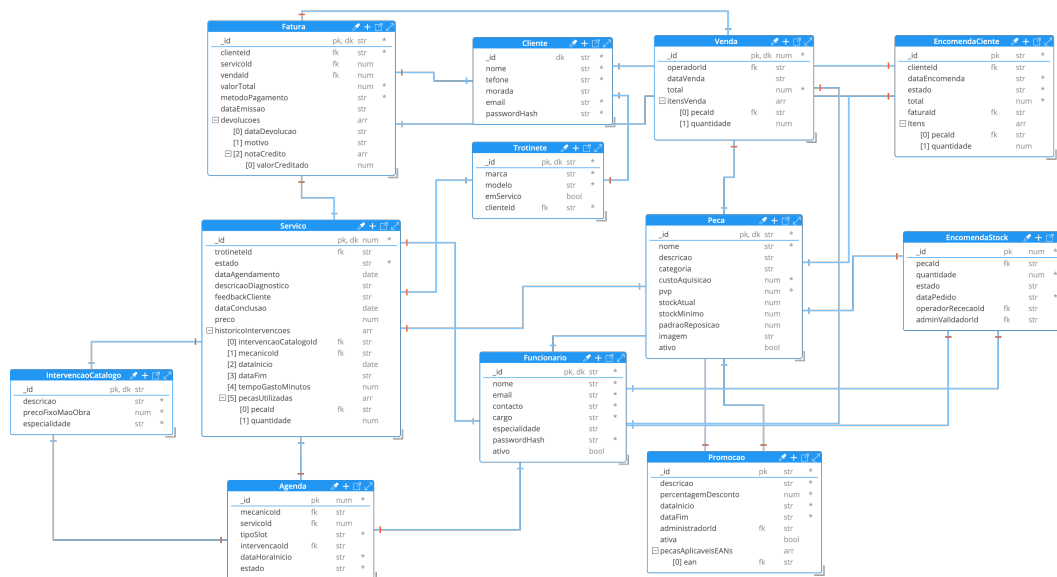


Figura 5.1: Diagrama do modelo de dados do sistema Fix'n'Ride em MongoDB.

Prompt utilizado - Claude Sonnet 4.6

"Descreve as estratégias de modelação de dados do Fix'n'Ride em MongoDB, distinguindo entre Embedding e Referencing. Para o Embedding, explica os casos em que os dados não têm existência fora do documento pai e a cardinalidade é previsível: Fatura com subdocumentos de devoluções e nota de crédito, Serviço com historicoIntervencoes aninhado com pecasUtilizadas (dois níveis), Venda com itensVenda, EncomendaCliente com itens, e Promocao com array de pecasAplicaveisIds. Para o Referencing, explica as relações entre entidades com ciclo de vida autónomo: Trocinete → Cliente, Serviço → Trocinete, Fatura → Cliente/Serviço/Venda, e Agência → Funcionário/Serviço."

Deltas: A IA propôs inicialmente uma estratégia de normalização excessiva, sugerindo referências externas para entidades como os itens de venda e as intervenções do serviço, que a equipa tinha deliberadamente optado por embutir. Foi necessário impor explicitamente no prompt os critérios de decisão — frequência de acesso conjunto, cardinalidade e necessidade de acesso independente — e corrigir manualmente a classificação de cada relação para alinhar com as decisões arquiteturais da equipa.

5.3 Dicionário de Dados

As subsecções seguintes apresentam o dicionário de dados completo do sistema, detalhando cada campo dos doze modelos Mongoose implementados. As coleções encontram-se agrupadas por domínio funcional. A notação **Ref**: na descrição de um campo indica uma referência a outro modelo (*Referencing*); a notação **Emb.** indica um subdocumento embutido (*Embedding*).

5.3.1 Domínio de Utilizadores

Modelo Cliente — Coleção clientes

Campo	Tipo, Restrições e Descrição
_id	Tipo: String Obrigatório: Sim Único: Sim NIF do cliente (ex: 250123456). Serve como identificador primário da coleção, dispensando a geração de um ID sintético.
nome	Tipo: String Obrigatório: Sim Nome completo do cliente para efeitos de identificação e faturação.
telefone	Tipo: String Obrigatório: Sim Número de contacto telefónico do cliente.
morada	Tipo: String Obrigatório: Não Default: null Morada de faturação. Pode ser nula até o cliente a preencher no seu perfil.
email	Tipo: String Obrigatório: Sim Único: Sim Endereço de correio eletrónico. Sujeito a índice de unicidade na coleção, utilizado também como credencial de acesso.
passwordHash	Tipo: String Obrigatório: Sim Resumo criptográfico (BCrypt) da palavra-passe. Nunca armazenada em texto simples.

Tabela 5.1: Dicionário de dados — Modelo Cliente (coleção clientes)

Modelo Funcionario — Coleção funcionarios

Campo	Tipo, Restrições e Descrição
_id	Tipo: String Obrigatório: Sim Único: Sim Número mecanográfico do funcionário (ex: ADM001, MEC001). Identificador natural que reflete o cargo e a sequência de registo.
nome	Tipo: String Obrigatório: Sim Nome completo do funcionário.

Continua na próxima página

Continuação da Tabela 5.2 — Modelo Funcionario

Campo	Tipo, Restrições e Descrição
email	Tipo: String Obrigatório: Sim Único: Sim Endereço de correio eletrónico institucional. Sujeito a índice de unicidade.
contacto	Tipo: String Obrigatório: Sim Número de contacto telefónico do funcionário.
cargo	Tipo: String Obrigatório: Sim Enum: ADMINISTRADOR OPERADOR MECANICO Perfil de acesso do funcionário. Determina o conjunto de permissões (<i>role</i>) no sistema RBAC e as rotas do <i>frontend</i> acessíveis.
especialidade	Tipo: String Obrigatório: Não Default: null Área técnica de especialização. Utilizado exclusivamente para funcionários com cargo MECANICO, no contexto de atribuição de intervenções.
passwordHash	Tipo: String Obrigatório: Sim Resumo criptográfico (BCrypt) da palavra-passe de acesso ao sistema.
ativo	Tipo: Boolean Obrigatório: Não Default: true Indicador de conta ativa. Permite desativar o acesso de um funcionário sem eliminar o seu registo histórico nas coleções relacionadas.

Tabela 5.2: Dicionário de dados — Modelo Funcionario (coleção funcionarios)

Modelo Trotinete — Coleção trotinetes

Campo	Tipo, Restrições e Descrição
_id	Tipo: String Obrigatório: Sim Único: Sim Número de série do veículo. Identificador físico único do equipamento, gravado na sua placa de identificação.
marca	Tipo: String Obrigatório: Sim Marca do fabricante da trotinete (ex: Xiaomi, Segway).
modelo	Tipo: String Obrigatório: Sim Modelo específico do veículo, relevante para a verificação de compatibilidade de peças.
emServico	Tipo: Boolean Obrigatório: Não Default: false Indica se o veículo se encontra atualmente em processo de reparação na oficina. Controlado automaticamente pelas transições de estado do serviço associado.
clienteId	Tipo: String Obrigatório: Sim Ref: Cliente NIF do cliente proprietário do veículo. Referência à coleção clientes.

Tabela 5.3: Dicionário de dados — Modelo Trotinete (coleção trotinetes)

5.3.2 Domínio de Serviços

Modelo Servico — Coleção `servicos`

Campo	Tipo, Restrições e Descrição
<code>_id</code>	Tipo: Number Obrigatório: Sim Único: Sim Identificador numérico sequencial do serviço de reparação.
<code>trotineteId</code>	Tipo: String Obrigatório: Sim Ref: Trotinete Número de série da trotinete sujeita ao serviço.
<code>estado</code>	Tipo: String Obrigatório: Sim Enum: AGENDADO EXECUCAO CONCLUIDO FECHADO Estado atual do ciclo de vida do serviço. A transição CONCLUIDO → FECHADO ocorre no momento do levantamento, com emissão de fatura.
<code>dataAgendamento</code>	Tipo: Date Obrigatório: Não Default: Date.now Data e hora de criação e agendamento do serviço.
<code>descricaoDiagnostico</code>	Tipo: String Obrigatório: Não Default: null Texto de diagnóstico técnico preenchido pelo mecânico após inspeção inicial ao veículo.
<code>feedbackCliente</code>	Tipo: String Obrigatório: Não Default: null Descrição da avaria reportada pelo cliente no momento de entrega do veículo à oficina.
<code>dataConclusao</code>	Tipo: Date Obrigatório: Não Default: null Data e hora em que o serviço transitou para o estado CONCLUIDO.
<code>preco</code>	Tipo: Number Obrigatório: Não Default: 0 Min: 0 Valor total do serviço em euros, calculado pela Lógica de Negócios com base nas intervenções realizadas e peças consumidas.
<code>historicoIntervencoes</code>	Tipo: Array Emb. (subdocumentos aninhados) Lista de intervenções técnicas realizadas no âmbito do serviço. Cada elemento contém os campos detalhados na Tabela 5.5.

Tabela 5.4: Dicionário de dados — Modelo Servico (coleção `servicos`)

Campo	Tipo, Restrições e Descrição
<code>intervencaoCatalogoId</code>	Tipo: Number Obrigatório: Não Ref: IntervencaoCatalogo Referência ao tipo de intervenção do catálogo de serviços da oficina.
<code>mecanicoId</code>	Tipo: String Obrigatório: Não Ref: Funcionario Número mecanográfico do mecânico que executou esta intervenção específica.
<code>dataInicio</code>	Tipo: Date Obrigatório: Não Momento de início da execução da intervenção técnica.
<code>dataFim</code>	Tipo: Date Obrigatório: Não Momento de conclusão da intervenção. Permanece nulo enquanto a intervenção se encontra em curso.

Continua na próxima página

Continuação da Tabela 5.5 — Subdocumento `historicoIntervencoes`

Campo	Tipo, Restrições e Descrição
<code>tempoGastoMinutos</code>	Tipo: Number Obrigatório: Não Duração total da intervenção em minutos, para fins de rastreabilidade operacional e análise de produtividade.
<code>peçasUtilizadas</code>	Tipo: Array Emb. (subdocumentos aninhados) Lista de peças consumidas nesta intervenção. Cada elemento contém <code>pecaId</code> (Ref: <code>Peca</code>) e quantidade (Default: 1, Min: 1).

Tabela 5.5: Dicionário de dados — Subdocumento `historicoIntervencoes` (por elemento)

Modelo Agenda — Coleção `agenda`

Campo	Tipo, Restrições e Descrição
<code>_id</code>	Tipo: Number Obrigatório: Sim Único: Sim Identificador numérico do slot de agenda.
<code>mecanicoId</code>	Tipo: String Obrigatório: Sim Ref: <code>Funcionario</code> Número mecanográfico do mecânico a quem o slot está atribuído. Utilizado no índice composto com <code>dataHoraInicio</code> para consultas de agenda por mecânico e data.
<code>servicoId</code>	Tipo: Number Obrigatório: Sim Ref: <code>Servico</code> Identificador do serviço associado ao slot agendado.
<code>tipoSlot</code>	Tipo: String Obrigatório: Sim Enum: DIAGNÓSTICO REPARACAO Natureza do trabalho a realizar no slot: diagnóstico inicial ou execução de reparação.
<code>intervencaoId</code>	Tipo: Number Obrigatório: Não Default: null Ref: <code>IntervencaoCatalogo</code> Referência à intervenção do catálogo. Preenchida apenas nos slots de tipo REPARACAO.
<code>dataHoraInicio</code>	Tipo: String Obrigatório: Não Default: now Data e hora de início do slot, no formato ISO 8601 truncado (YYYY-MM-DDTHH:mm).
<code>estado</code>	Tipo: String Obrigatório: Não Default: RESERVADO Estado atual do slot. Valores utilizados: RESERVADO (agendado mas não executado) e CONCLUIDO.

Tabela 5.6: Dicionário de dados — Modelo Agenda (coleção `agenda`)

Modelo `IntervencaoCatalogo` — Coleção `intervencoes_catalogo`

Campo	Tipo, Restrições e Descrição
_id	Tipo: Number Obrigatório: Sim Único: Sim Identificador numérico da intervenção no catálogo da oficina.
descricao	Tipo: String Obrigatório: Sim Descrição textual do tipo de trabalho (ex: “Substituição de Pneu”, “Revisão de Travões”).
precoFixoMaoDeObra	Tipo: Number Obrigatório: Sim Min: 0 Custo fixo de mão-de-obra associado a este tipo de intervenção, em euros, independente das peças utilizadas. Base de cálculo do orçamento apresentado ao cliente.
especialidade	Tipo: String Obrigatório: Sim Área técnica de especialização exigida para executar a intervenção. Utilizado para filtrar os mecânicos elegíveis na atribuição de tarefas.

Tabela 5.7: Dicionário de dados — Modelo IntervencaoCatalogo (coleção intervencoes_catalogo)

5.3.3 Domínio Financeiro

Modelo Fatura — Coleção faturas

Campo	Tipo, Restrições e Descrição
_id	Tipo: String Obrigatório: Sim Único: Sim Número da fatura no formato alfanumérico (ex: FT-123456). Gerado aleatoriamente pela Lógica de Negócios no momento da emissão.
clienteId	Tipo: String Obrigatório: Sim Ref: Cliente NIF do cliente titular da fatura. Utilizado no índice de performance para consultas do histórico financeiro por cliente.
servicoId	Tipo: Number Obrigatório: Não Default: null Ref: Servico Referência ao serviço de reparação que originou a fatura. Mutuamente exclusivo com vendaId: apenas um dos dois pode estar preenchido.
vendaId	Tipo: Number Obrigatório: Não Default: null Ref: Venda Referência à venda direta ao balcão que originou a fatura. Mutuamente exclusivo com servicoId.
valorTotal	Tipo: Number Obrigatório: Sim Min: 0 Valor total faturado em euros.
metodoPagamento	Tipo: String Obrigatório: Sim Enum: MBWAY MULTIBANCO NUMERARIO Método de pagamento utilizado na transação.
dataEmissao	Tipo: String Obrigatório: Não Default: now Data e hora de emissão da fatura, no formato ISO 8601 truncado (YYYY-MM-DDTHH:mm).

Continua na próxima página

Continuação da Tabela 5.8 — Modelo Fatura

Campo	Tipo, Restrições e Descrição
devolucoes	Tipo: Array Emb. (subdocumentos aninhados) Lista de devoluções associadas a esta fatura. Cada elemento contém: dataDevolucao (Default: now), motivo (String, obrigatório) e notaCredito.valorCreditado (Number, obrigatório, Min: 0).

Tabela 5.8: Dicionário de dados — Modelo Fatura (coleção faturas)

Modelo Venda — Coleção vendas

Campo	Tipo, Restrições e Descrição
_id	Tipo: Number Obrigatório: Sim Único: Sim Identificador numérico sequencial da venda direta ao balcão.
operadorId	Tipo: String Obrigatório: Sim Ref: Funcionario Número mecanográfico do operador que processou a venda. Permite rastreabilidade por funcionário nos relatórios financeiros.
dataVenda	Tipo: String Obrigatório: Não Default: now Data e hora de registo da venda, no formato ISO 8601 truncado.
total	Tipo: Number Obrigatório: Sim Min: 0 Valor total da venda em euros, calculado com base nos itens e promoções ativas no momento da transação.
itensVenda	Tipo: Array Emb. (subdocumentos aninhados) Lista de itens vendidos. Cada elemento contém pecaId (Ref: Peca) e quantidade (Obrigatório , Min: 1).

Tabela 5.9: Dicionário de dados — Modelo Venda (coleção vendas)

Modelo Promocao — Coleção promocoess

Campo	Tipo, Restrições e Descrição
_id	Tipo: Number Obrigatório: Sim Único: Sim Identificador numérico da promoção.
descricao	Tipo: String Obrigatório: Sim Texto descritivo da campanha promocional (ex: “Promoção de Verão”).
percentagemDesconto	Tipo: Number Obrigatório: Sim Min: 0 Max: 100 Percentagem de desconto a aplicar sobre o PVP das peças abrangidas pela promoção.
dataInicio	Tipo: String Obrigatório: Sim Data de início de vigência da promoção, no formato ISO 8601.
dataFim	Tipo: String Obrigatório: Sim Data de término de vigência da promoção, no formato ISO 8601.

Continua na próxima página

Continuação da Tabela 5.10 — Modelo Promocao

Campo	Tipo, Restrições e Descrição
administradorId	Tipo: String Obrigatório: Sim Ref: Funcionario Número mecanográfico do administrador que criou e é responsável pela promoção.
ativa	Tipo: Boolean Obrigatório: Não Default: true Indica se a promoção se encontra atualmente em vigor. Pode ser desativada manualmente antes do término da data de vigência.
peçasAplicaveisIds	Tipo: Array[String] Obrigatório: Não Ref: Peca Lista de códigos EAN das peças às quais o desconto se aplica. Array vazio indica que a promoção não se encontra associada a nenhuma peça.

Tabela 5.10: Dicionário de dados — Modelo Promocao (coleção promocoas)

5.3.4 Domínio de Stocks

Modelo Peca — Coleção peças

Campo	Tipo, Restrições e Descrição
_id	Tipo: String Obrigatório: Sim Único: Sim Código EAN da peça. Identificador universal de produto, compatível com leitura por código de barras.
nome	Tipo: String Obrigatório: Sim Designação comercial da peça (ex: “Pneu Traseiro 8.5”).
descricao	Tipo: String Obrigatório: Não Default: null Descrição técnica detalhada da peça e respetivas compatibilidades de modelo.
categoria	Tipo: String Obrigatório: Não Default: null Categoria de agrupamento no catálogo (ex: “Pneumáticos”, “Travões”). Utilizada para filtragem e navegação no portal do cliente.
custoAquisicao	Tipo: Number Obrigatório: Sim Min: 0 Custo de aquisição unitário ao fornecedor, em euros. Utilizado para cálculo de margem nos relatórios financeiros.
pvp	Tipo: Number Obrigatório: Sim Min: 0 Preço de venda ao público unitário, em euros. Base de cálculo para vendas e orçamentos.
stockAtual	Tipo: Number Obrigatório: Não Default: 0 Min: 0 Quantidade de unidades disponíveis em armazém. Decrementado automaticamente após confirmação de vendas e reparações pela Lógica de Negócios.
stockMinimo	Tipo: Number Obrigatório: Não Default: 5 Min: 0 Limiar de stock abaixo do qual é gerado um alerta de reposição no painel de administração.

Continua na próxima página

Continuação da Tabela 5.11 — Modelo Peca

Campo	Tipo, Restrições e Descrição
padraoReposicao	Tipo: Number Obrigatório: Não Default: 5 Quantidade padrão a encomendar nas reposições automáticas de stock sugeridas pelo sistema.
imagem	Tipo: String Obrigatório: Não Default: null Nome do ficheiro de imagem associado à peça para apresentação no catálogo público.
ativo	Tipo: Boolean Obrigatório: Não Default: true Indica se a peça se encontra disponível para venda e utilização em reparações. Permite descontinuar artigos sem eliminar o histórico associado.

Tabela 5.11: Dicionário de dados — Modelo Peca (coleção pecas)

Modelo EncomendaStock — Coleção encomendas_stock

Campo	Tipo, Restrições e Descrição
_id	Tipo: Number Obrigatório: Sim Único: Sim Identificador numérico da encomenda de reposição de stock ao fornecedor.
pecaId	Tipo: String Obrigatório: Sim Ref: Peca Código EAN da peça a repor no inventário.
quantidade	Tipo: Number Obrigatório: Sim Min: 1 Número de unidades a encomendar ao fornecedor.
estado	Tipo: String Obrigatório: Não Default: PENDENTE Enum: PENDENTE TRANSITO RECECIONADA Ciclo de vida da encomenda: criada pelo sistema (PENDENTE), aprovada pelo administrador e enviada ao fornecedor (TRANSITO), rececionada fisicamente em armazém (RECECIONADA).
dataPedido	Tipo: String Obrigatório: Não Default: now Data e hora de criação da encomenda, no formato ISO 8601 truncado.
operadorRececaoId	Tipo: String Obrigatório: Não Default: null Ref: Funcionario Número mecanográfico do operador que rececionou fisicamente a encomenda em armazém. Permanece nulo até à transição para o estado RECECIONADA.
adminValidadorId	Tipo: String Obrigatório: Não Default: null Ref: Funcionario Número mecanográfico do administrador que aprovou a encomenda e despoletou a transição para TRANSITO. Permanece nulo até à aprovação.

Continua na próxima página

Continuação da Tabela 5.12 — Modelo EncomendaStock

Campo	Tipo, Restrições e Descrição
--------------	-------------------------------------

Tabela 5.12: Dicionário de dados — Modelo EncomendaStock (coleção encomendas_stock)

Modelo EncomendaCliente — **Coleção** encomendas_cliente

Campo	Tipo, Restrições e Descrição
_id	Tipo: Number Obrigatório: Sim Único: Sim Identificador numérico da encomenda do cliente (<i>Click & Collect</i>).
clienteId	Tipo: String Obrigatório: Sim Ref: Cliente NIF do cliente que efetuou a encomenda. Utilizado no índice de performance para consultas por cliente.
dataEncomenda	Tipo: String Obrigatório: Não Default: now Data e hora de submissão da encomenda pelo cliente, no formato ISO 8601 truncado.
estado	Tipo: String Obrigatório: Não Default: PRONTO PARA LEVANTAMENTO Enum: PRONTO PARA LEVANTAMENTO ENTREGUE Estado atual da encomenda. Transita para ENTREGUE no momento da entrega ao balcão pelo operador, com emissão simultânea de fatura.
total	Tipo: Number Obrigatório: Sim Min: 0 Valor total da encomenda em euros, calculado no momento da sua criação.
faturaId	Tipo: String Obrigatório: Não Default: null Ref: Fatura Referência à fatura emitida no momento do levantamento pelo operador. Permanece nula até à entrega efetiva.
itens	Tipo: Array Emb. (subdocumentos aninhados) Lista de itens encomendados pelo cliente. Cada elemento contém pecaId (Ref: Peca, obrigatório) e quantidade (Obrigatório , Min: 1).

Tabela 5.13: Dicionário de dados — Modelo EncomendaCliente (coleção encomendas_cliente)

6 Arquitetura e Design da Lógica de Negócios

6.1 Introdução ao Capítulo

O presente capítulo documenta as decisões de arquitetura e de *design* de *software* que sustentam o sistema **Fix'n'Ride**, consolidadas na segunda etapa do projeto. Partindo dos requisitos funcionais e não funcionais especificados no capítulo anterior, procedeu-se à definição da arquitetura global do sistema, à seleção fundamentada da *stack* tecnológica e à documentação das principais decisões estruturais — com recurso a ferramentas de Inteligência Artificial Generativa como assistentes de revisão arquitetural e de produção de artefactos UML. Em particular, o agente autónomo **Antigravity** (Google DeepMind) foi utilizado para gerar o *scaffolding* inicial dos quatro serviços Docker, revisar a coerência entre o *docker-compose.yml* e a documentação, e propor melhorias à estrutura de variáveis de ambiente — atuando sempre sob supervisão e revisão crítica da equipa

O capítulo encontra-se organizado em seis secções principais. A primeira justifica a decisão arquitetural de separar fisicamente a lógica de negócios do acesso a dados. A segunda apresenta a organização modular dos serviços e os respetivos diagramas de classes por subsistema. A terceira documenta o mecanismo de comunicação HTTP inter-serviços. A quarta inventaria os *endpoints* expostos ao *frontend*. A quinta detalha o modelo de segurança adotado, incluindo o fluxo JWT e o modelo RBAC. Por fim, a sexta apresenta os diagramas de sequência dos fluxos operacionais mais críticos do sistema.

Prompt utilizado - Claude Sonnet 4.6

"Escreve a introdução ao capítulo de Arquitetura e Design da Lógica de Negócios do Fix'n'Ride. O capítulo cobre as decisões tomadas na segunda etapa do projeto. Organiza a introdução em seis eixos: justificação da separação arquitetural Data API vs. Lógica de Negócios, organização modular dos serviços com diagramas de classes por subsistema, mecanismo de comunicação HTTP inter-serviços, inventário de endpoints expostos ao frontend, modelo de segurança JWT e RBAC, e diagramas de sequência dos fluxos operacionais mais críticos."

6.2 Justificação da Separação Arquitetural: *Data API* vs. Lógica de Negócios

Uma das decisões arquiteturais mais determinantes do sistema **Fix’n’Ride** foi a separação explícita entre o servidor de acesso a dados (*Data API*, Node.js) e o servidor de Lógica de Negócios (LN, ASP.NET Core), materializados em dois contentores Docker independentes. Esta separação não é meramente tecnológica — resulta de um princípio de **separação de responsabilidades** (*Separation of Concerns*) aplicado à camada de serviços, e justifica-se por quatro razões fundamentais.

Complexidade das Regras de Negócio. A *Data API* limita-se a operações CRUD simples sobre o MongoDB, sem qualquer conhecimento do domínio da oficina. A Lógica de Negócios, pelo contrário, implementa processos que envolvem múltiplas entidades e passos interdependentes. O exemplo mais elucidativo é o processo de levantamento de uma trotinete concluída: o servidor LN (1) consulta o serviço na *Data API*, (2) determina o NIF do cliente proprietário da trotinete, (3) cria a Fatura via *Data API* e (4) fecha o serviço numa sequência atômica — tudo isto orquestrado num único pedido do *frontend*. Sem esta camada, esta orquestração teria de residir no cliente React, o que comprometeria gravemente a segurança e a testabilidade.

Segurança e Validação Centralizada. O servidor LN é a única entidade que emite tokens JWT. A validação de *passwords* com **BCrypt**, a geração de tokens com **HMAC-SHA256** e a verificação de identidade via *claims* — operações críticas de segurança — estão concentradas num ambiente fortemente tipado e auditável (.NET), e nunca expostas ao JavaScript do *browser*. Adicionalmente, a *Data API* permanece inacessível ao exterior, aceitando apenas pedidos provenientes da rede Docker interna e portadores da *INTERNAL_API_KEY*.

Escalabilidade e Manutenibilidade Independentes. A separação em dois servidores permite que cada um escale de forma autónoma. Em cenários de carga elevada, o serviço LN pode ser replicado sem necessidade de replicar a *Data API*. Do ponto de vista da manutenibilidade, as alterações às regras de negócio afetam exclusivamente o código .NET, sem risco de introduzir regressões na camada de persistência.

Adequação Tecnológica. O .NET 8 oferece suporte nativo a injeção de dependências, documentação automática via Swagger/OpenAPI, autenticação JWT com *Microsoft.AspNetCore.Authentication.JwtBearer* e tipagem estática rigorosa. O Node.js/Express, por sua vez, é mais adequado para o papel de *gateway* de dados leve e de alta concorrência que a *Data API* desempenha.

Prompt utilizado - Claude Sonnet 4.6

"Justifica a decisão arquitetural de separar fisicamente a Lógica de Negócios (ASP.NET Core) da Data API (Node.js) no Fix'n'Ride em dois contentores Docker independentes. Apresenta quatro razões fundamentais: (1) complexidade das regras de negócio — exemplifica com o processo atômico de levantamento de trotinete com quatro passos coordenados; (2) segurança e validação centralizada — autenticação BCrypt, geração JWT com HMAC-SHA256 e INTERNAL_API_KEY para isolar a Data API do exterior; (3) escalabilidade e manutenibilidade independentes por serviço; (4) adequação tecnológica — .NET 8 para lógica tipada e Node.js como gateway de dados leve."

6.3 Organização Modular dos Serviços e Diagramas de Classes

Em lugar de um único serviço monolítico, a Lógica de Negócios do **Fix'n'Ride** foi implementada segundo um modelo de **módulos de serviço especializados**, cada um responsável por um domínio funcional distinto. Esta organização segue o princípio da responsabilidade única ao nível dos serviços, e é suportada pela infraestrutura de *Dependency Injection* do ASP.NET Core. Cada módulo é definido por uma interface (prefixada com *I*) e uma implementação concreta, registadas no contentor de DI do *Program.cs* através do mecanismo *AddHttpClient<TInterface, TImplementation>*.

O sistema encontra-se organizado em sete subsistemas, descritos nas subsecções seguintes.

6.3.1 Subsistema de Autenticação

O subsistema de Autenticação é responsável pelo fluxo completo de gestão de identidade do sistema. O *AuthService* implementa a verificação de credenciais com BCrypt, a geração de tokens JWT com *claims* de identidade e *role*, e o registo de novos clientes com *hashing* de *password*. Os *endpoints* de *login* e registo são públicos — não requerem um token JWT válido, uma vez que são precisamente os responsáveis pela sua emissão. O *endpoint* de *logout* (*POST /api/Auth/logout*) é, pelo contrário, protegido com *[Authorize]*, pois necessita da identidade do utilizador (extraída do token) para revogar o *claim jti* correspondente.

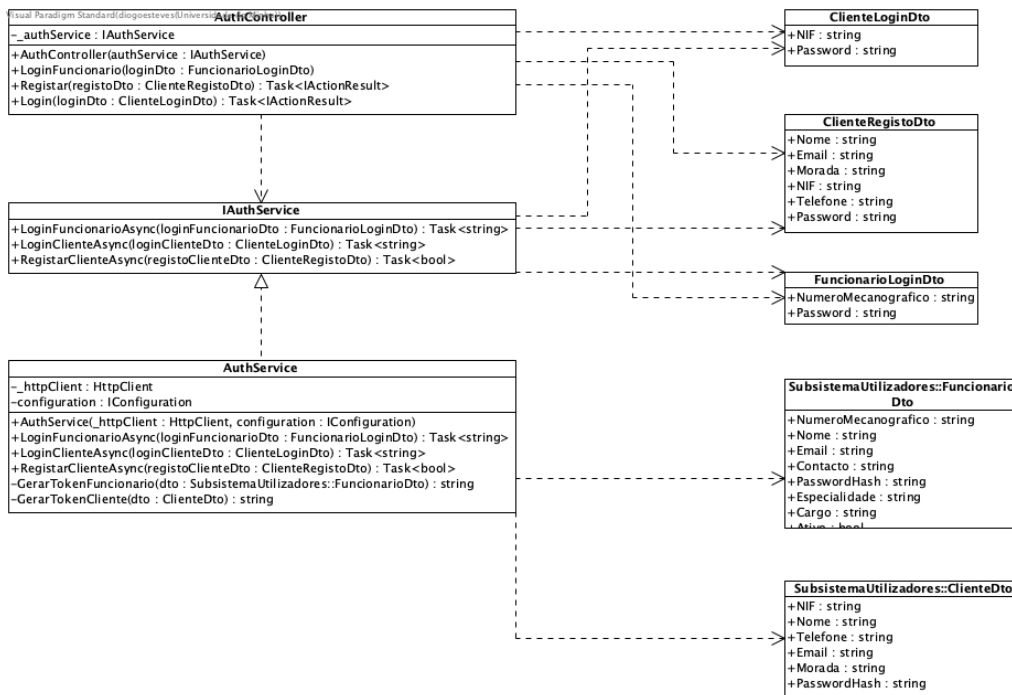


Figura 6.1: Diagrama de classes do Subsistema de Autenticação.

6.3.2 Subsistema de Utilizadores

O subsistema de Utilizadores centraliza a gestão de identidades e perfis operacionais da oficina. Abrange os serviços *FuncionarioService* e *TrotineteService*, responsáveis pela criação de funcionários — com *hashing* da *password* antes da persistência —, atualização de perfis e gestão da frota de trotinetes associada a cada cliente.

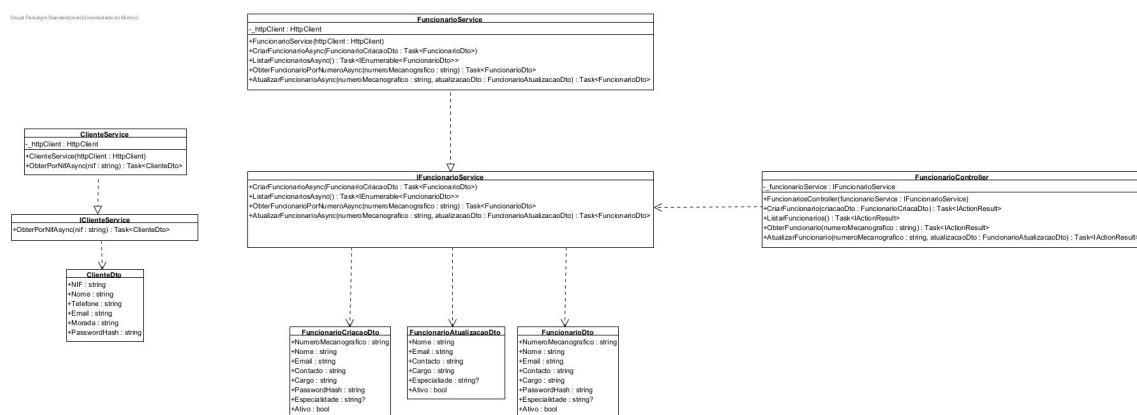


Figura 6.2: Diagrama de classes do Subsistema de Utilizadores.

6.3.3 Subsistema de Catálogos

O subsistema de Catálogos gere as entidades de suporte ao funcionamento da oficina consultadas com elevada frequência: o catálogo de peças (*PecaService*) e o catálogo de intervenções técnicas (*IntervencaoCatalogoService*). O *IntervencaoCatalogoService* gere os tipos de intervenção disponíveis com o respetivo preço fixo de mão-de-obra, utilizados pelos mecânicos no preenchimento das ordens de serviço.

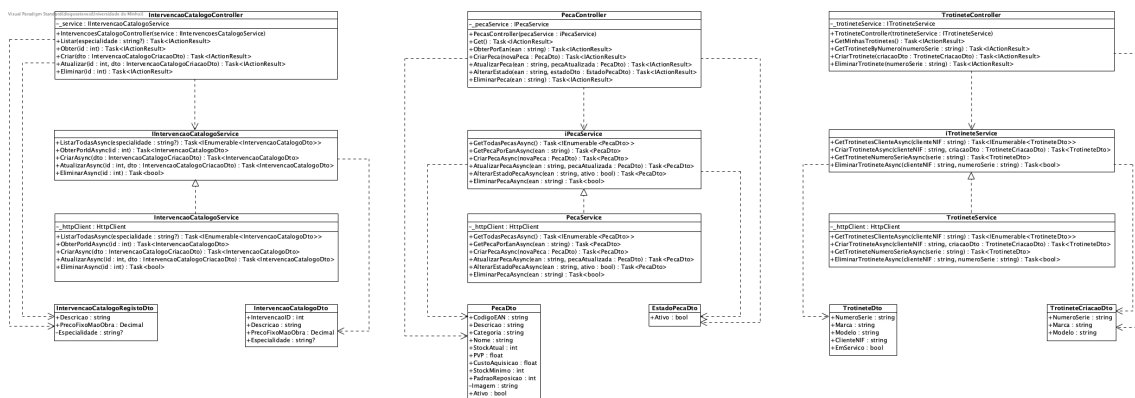


Figura 6.3: Diagrama de classes do Subsistema de Catálogos

6.3.4 Subsistema de Servicios

O subsistema de Serviços constitui o núcleo operacional da **MobiFix**, suportando o ciclo de vida completo das reparações. O *ServicoService* gere a criação de diagnósticos, as transições de estado e o processo atômico *LevantarComFaturaAsync*. O *AgendaService* complementa este subsistema, gerindo os *slots* de agenda dos mecânicos.

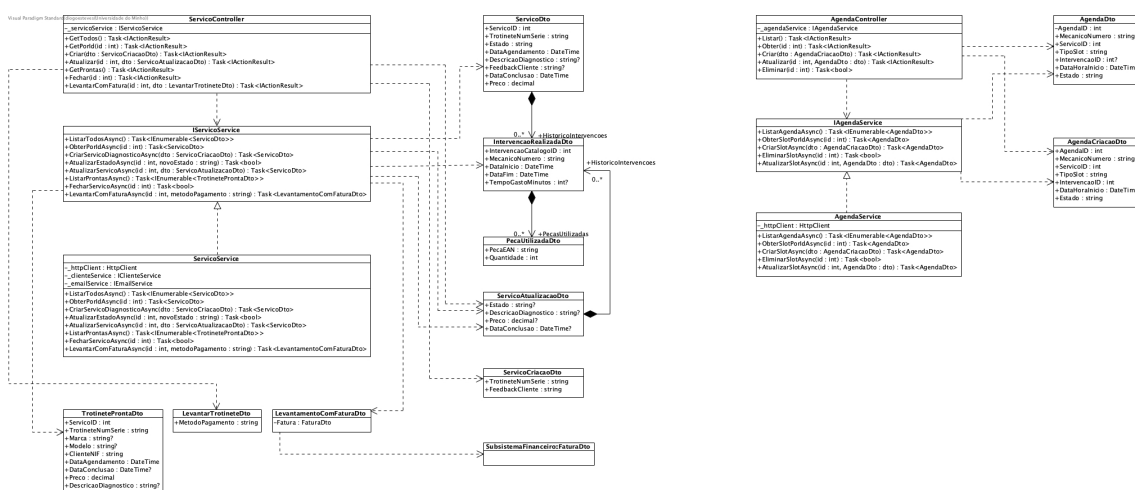


Figura 6.4: Diagrama de classes do Subsistema de Servicos.

6.3.5 Subsistema Financeiro

O subsistema Financeiro centraliza toda a inteligência comercial da plataforma. O *VendaService* processa vendas diretas ao balcão, recalculando o total no servidor. O *FaturaService* emite documentos fiscais e gere devoluções. O *EncomendaClienteService* trata as reservas *Click & Collect* com *Task.WhenAll*. O *PromocaoService* gere campanhas de desconto. O *EstatisticasService* calcula os KPIs financeiros por período.

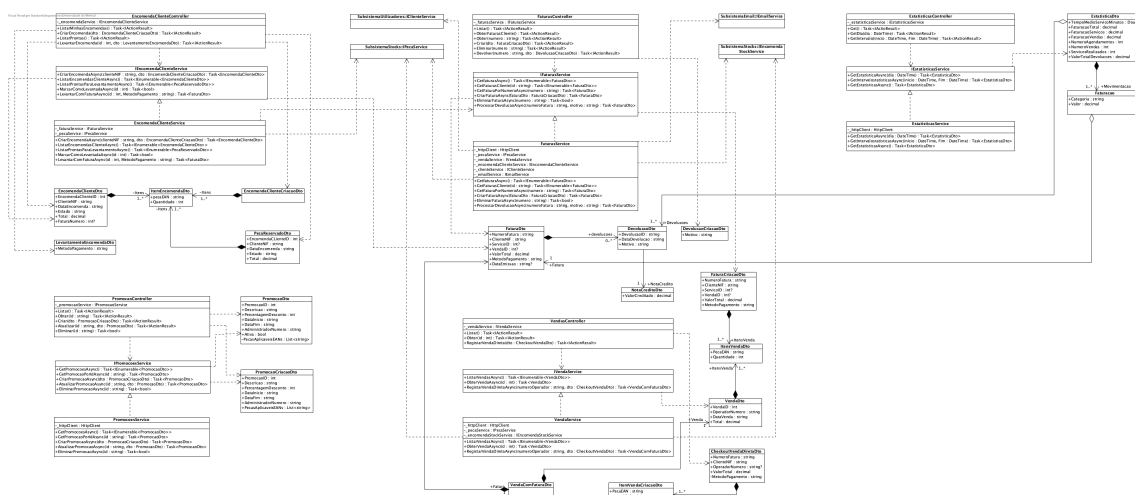


Figura 6.5: Diagrama de classes do Subsistema Financeiro

6.3.6 Subsistema de *Stocks*

O subsistema de *Stocks* gere o fluxo de aprovação e receção de encomendas de reposição. O *EncomendaStockService* implementa as transições *PENDENTE* → *TRANSITO* → *RECECIONADA*. A transição para *RECECIONADA* despoleta automaticamente o incremento do *stockAtual* da peça correspondente na *Data API*.

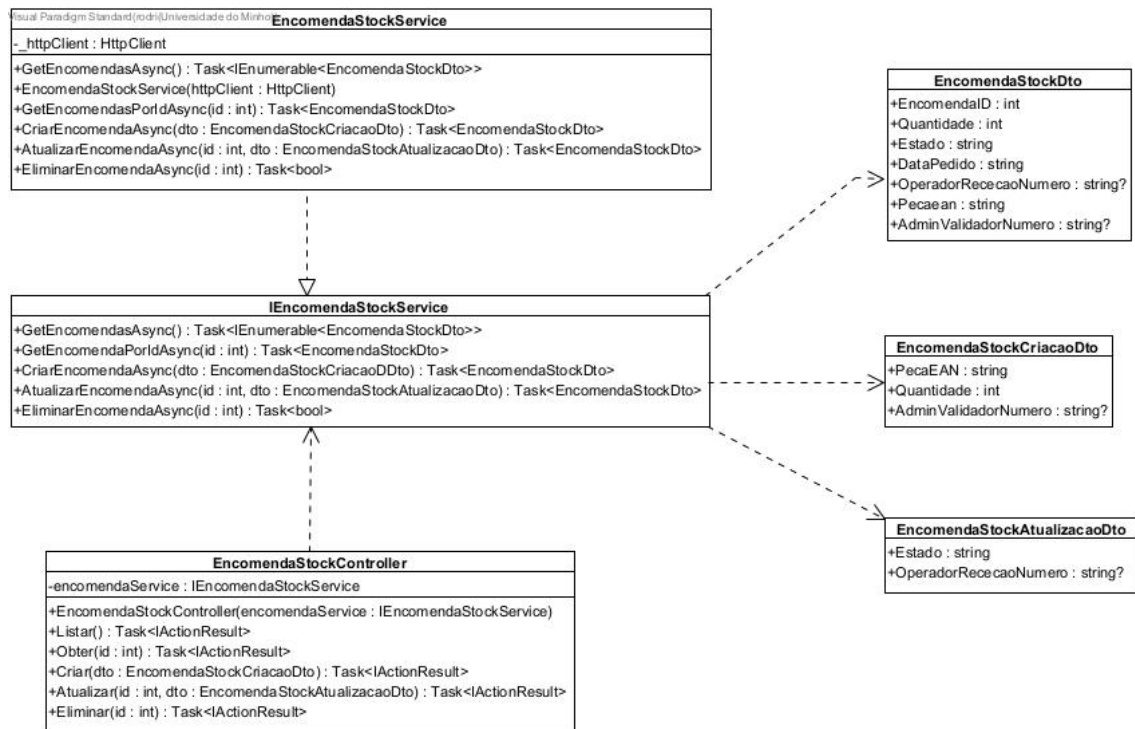


Figura 6.6: Diagrama de classes do Subsistema de *Stocks*.

6.3.7 Subsistema de Email

O subsistema de Email providenciará a camada de comunicação assíncrona com os clientes. A separação deste serviço num módulo independente garante que eventuais falhas no envio não bloqueiam o fluxo operacional principal.

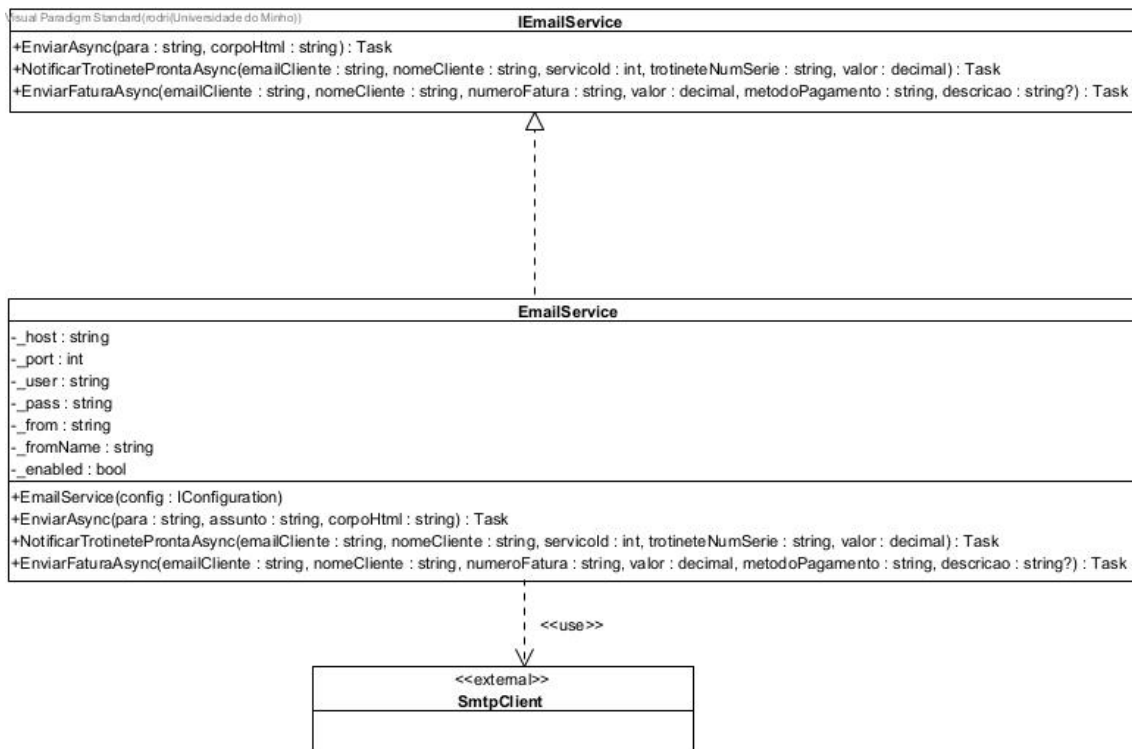


Figura 6.7: Diagrama de classes do Subsistema de Email.

6.4 Comunicação HTTP com a *Data API*

Todos os módulos de serviço da LN comunicam com a *Data API* exclusivamente através do mecanismo **Typed HttpClient** do ASP.NET Core, configurado centralmente no *Program.cs*.

Configuração Centralizada via *ConfigureDefaultClient*. Esta função é aplicada a *todos* os clientes HTTP registados no contentor de DI. Configura dois aspetos em cada instância: o **endereço base**, lido de *DATA_API_URL* (*http://data-api:3001* na rede Docker interna), com exceção lançada em arranque se ausente (*fail-fast*); e o **cabeçalho *x-api-key*** com o valor da *INTERNAL_API_KEY*, adicionado automaticamente a todos os pedidos. As chamadas utilizam os métodos de extensão *GetFromJsonAsync*, *PostAsJsonAsync*, *PutAsJsonAsync* e *DeleteAsync*, com *PropertyNamingPolicy = null* para preservar os campos PascalCase.

Resiliência e Tratamento de Erros. Cada chamada HTTP é encapsulada em verificação de *IsSuccessStatusCode*. Em caso de falha, os serviços devolvem *null* ou cole-

ções vazias ao *Controller*, que traduz estes resultados em respostas HTTP semânticas (404 Not Found, 400 Bad Request).

Orquestração Paralela. Em operações que requerem dados de múltiplas coleções simultaneamente, o *EncomendaClienteService* utiliza *Task.WhenAll* para executar chamadas à *Data API* em paralelo, reduzindo a latência total face a chamadas sequenciais.

Prompt utilizado - Claude Sonnet 4.6

"Descreve o mecanismo de comunicação HTTP entre o servidor de Lógica de Negócios (.NET) e a Data API (Node.js) do Fix'n'Ride. Cobre três aspetos: (1) configuração centralizada via *ConfigureDefaultClient* — endereço base lido de *DATA_API_URL* com fail-fast em arranque, e cabeçalho *x-api-key* com *INTERNAL_API_KEY* adicionado automaticamente a todos os pedidos; (2) resiliência e tratamento de erros — encapsulamento em *IsSuccessStatusCode* com devolução de null ou coleções vazias ao *Controller* que traduz em respostas HTTP semânticas; (3) orquestração paralela via *Task.WhenAll* para chamadas a múltiplas coleções simultaneamente no *EncomendaClienteService*."

6.5 Inventário de *Endpoints* da Lógica de Negócios

O servidor LN expõe os seus *endpoints* RESTFUL ao *frontend* React através de *Controllers* ASP.NET Core decorados com *[Authorize]*, com exceção dos *endpoints* de *login* e registo do *AuthController*, que são públicos. O *endpoint* de *logout* requer autenticação. A Tabela 6.1 apresenta o inventário completo, organizado por módulo funcional.

Tabela 6.1: Inventário de <i>endpoints</i> do servidor de Lógica de Negócios (ASP.NET Core)	
Método e Rota	Serviço Interno e Descrição Funcional
Módulo de Autenticação (<i>AuthController</i>) — Login e Registo Públicos; Logout Protegido (<i>[Authorize]</i>)	
POST <i>/api/Auth/login/funcionario</i>	Serviço: <i>AuthService</i> Autentica um funcionário. Valida as credenciais com BCrypt e devolve um token JWT com <i>claims</i> de cargo (<i>ADMINISTRADOR</i> , <i>OPERADOR</i> ou <i>MECANICO</i>).
POST <i>/api/Auth/login/cliente</i>	Serviço: <i>AuthService</i> Autentica um cliente pelo NIF e <i>password</i> . Devolve token JWT com <i>claim</i> de <i>role</i> <i>Cliente</i> .
	<i>Continua na próxima página</i>

Continuação da Tabela 6.1 — Endpoints da Lógica de Negócios

Método e Rota	Serviço Interno e Descrição Funcional
POST <i>/api/Auth/register/cliente</i>	Serviço: <i>AuthService</i> Regista um novo cliente, aplicando BCrypt à <i>password</i> antes da persistência na <i>Data API</i> .
POST <i>/api/Auth/logout</i>	Serviço: <i>AuthController</i> Termina a sessão do utilizador autenticado: revoga o <i>jwt</i> do token na <i>TokenListService</i> (movendo-o para a lista negra) e elimina o <i>cookie mobifix_auth</i> . Requer <i>[Authorize]</i> .
Módulo de Serviços	
GET <i>/api/Servicos</i>	Serviço: <i>ServicoService</i> Lista todos os serviços de reparação com o histórico de intervenções embutido.
GET <i>/api/Servicos/{id}</i>	Serviço: <i>ServicoService</i> Obtém o detalhe completo de um serviço pelo ID, incluindo o histórico de intervenções e peças utilizadas.
POST <i>/api/Servicos</i>	Serviço: <i>ServicoService</i> Cria um novo serviço de diagnóstico associado a uma trotinete, com estado inicial <i>AGENDADO</i> .
PUT <i>/api/Servicos/{id}</i>	Serviço: <i>ServicoService</i> Atualiza o estado, o diagnóstico, o preço ou as intervenções de um serviço existente.
GET <i>/api/Servicos/prontas</i>	Serviço: <i>ServicoService</i> Lista as trotinetes com serviço no estado <i>CONCLUIDO</i> a aguardar levantamento pelo operador.
PUT <i>/api/Servicos/{id}</i> <i>/levantar</i>	Serviço: <i>ServicoService</i> Processo atômico de levantamento: consulta o serviço, determina o cliente proprietário, emite a fatura e transita o serviço para <i>FECHADO</i> numa sequência coordenada de chamadas à <i>Data API</i> .
PUT <i>/api/Servicos/{id}</i> <i>/fechar</i>	Serviço: <i>ServicoService</i> Fecha diretamente um serviço sem emissão de fatura.
GET <i>/api/Agenda</i>	Serviço: <i>AgendaService</i> Lista todos os <i>slots</i> de agenda da oficina, com suporte a filtro por mecânico e data.
POST <i>/api/Agenda</i>	Serviço: <i>AgendaService</i> Cria um novo <i>slot</i> de agendamento para um mecânico, associado a um serviço e tipo de trabalho (<i>DIAGNOSTICO</i> ou <i>REPARACAO</i>).
GET <i>/api/Agenda/{id}</i>	Serviço: <i>AgendaService</i> Obtém os dados de um <i>slot</i> de agenda pelo ID.
PUT <i>/api/Agenda/{id}</i>	Serviço: <i>AgendaService</i> Atualiza o estado ou os dados de um <i>slot</i> existente (e.g., transição para <i>CONCLUIDO</i>).

Continua na próxima página

Continuação da Tabela 6.1 — Endpoints da Lógica de Negócios

Método e Rota	Serviço Interno e Descrição Funcional
DELETE <i>/api/Agenda/{id}</i>	Serviço: <i>AgendaService</i> Remove um <i>slot</i> de agenda.
Módulo Financeiro	
GET <i>/api/Vendas</i>	Serviço: <i>VendaService</i> Lista todas as vendas diretas registadas no sistema.
GET <i>/api/Vendas/{id}</i>	Serviço: <i>VendaService</i> Obtém uma venda específica pelo ID, com os itens vendidos.
POST <i>/api/Vendas/direta</i>	Serviço: <i>VendaService</i> Processo atómico de venda direta ao balcão: cria o registo de <i>Venda</i> e a <i>Fatura</i> correspondente num único pedido. O total é recalculado no servidor com base no PVP e promoções ativas. O operador é identificado pelo JWT.
GET <i>/api/Faturas</i>	Serviço: <i>FaturaService</i> Lista todas as faturas emitidas. Suporta filtro por NIF de cliente via <i>query parameter</i> .
GET <i>/api/Faturas/{numero}</i>	Serviço: <i>FaturaService</i> Obtém os dados completos de uma fatura pelo número, incluindo devoluções e notas de crédito embutidas.
POST <i>/api/Faturas/devolucao/{numero}</i>	Serviço: <i>FaturaService</i> Regista uma devolução com nota de crédito numa fatura existente, embutindo o subdocumento no array <i>devolucoes</i> .
GET <i>/api/Promocoas</i>	Serviço: <i>PromocaoService</i> Lista todas as promoções registadas, ativas e inativas.
GET <i>/api/Promocoas/{id}</i>	Serviço: <i>PromocaoService</i> Obtém uma promoção específica pelo ID, com a lista de peças aplicáveis.
POST <i>/api/Promocoas</i>	Serviço: <i>PromocaoService</i> Cria uma nova campanha promocional com período de vigência, percentagem de desconto e lista de EANs de peças abrangidas.
PUT <i>/api/Promocoas/{id}</i>	Serviço: <i>PromocaoService</i> Atualiza os dados de uma promoção existente.
PATCH <i>/api/Promocoas/{id}</i> <i>/estado</i>	Serviço: <i>PromocaoService</i> Altera o estado ativo/inativo de uma promoção sem alterar os restantes dados.
DELETE <i>/api/Promocoas/{id}</i>	Serviço: <i>PromocaoService</i> Remove permanentemente uma promoção.
GET <i>/api/EncomendaCliente</i>	Serviço: <i>EncomendaClienteService</i> Lista as encomendas do cliente autenticado (NIF extraído do JWT).

Continua na próxima página

Continuação da Tabela 6.1 — Endpoints da Lógica de Negócios

Método e Rota	Serviço Interno e Descrição Funcional
POST <i>/api/EncomendaCliente</i>	Serviço: <i>EncomendaClienteService</i> Cria uma reserva <i>Click & Collect</i> . O total é calculado no servidor com base no PVP das peças.
GET <i>/api/EncomendaCliente/prontas</i>	Serviço: <i>EncomendaClienteService</i> Lista encomendas no estado <i>PRONTO PARA LEVANTAMENTO</i> , com dados enriquecidos das peças, obtidos em paralelo via <i>Task.WhenAll</i> .
PUT <i>/api/EncomendaCliente/{id}/levantar</i>	Serviço: <i>EncomendaClienteService</i> Marca uma encomenda de cliente como <i>ENTREGUE</i> e regista a fatura associada.
GET <i>/api/Estatisticas</i>	Serviço: <i>EstatisticasService</i> Calcula e devolve os KPIs globais de faturação, número de serviços concluídos e devoluções.
GET <i>/api/Estatisticas/dia</i>	Serviço: <i>EstatisticasService</i> Calcula estatísticas filtrando por um dia específico, passado via <i>query parameter</i> (<i>?dia=</i>).
GET <i>/api/Estatisticas/intervalo</i>	Serviço: <i>EstatisticasService</i> Calcula estatísticas para um intervalo de datas (<i>?inicio=&fim=</i>).
Módulo de Stocks	
GET <i>/api/EncomendaStock</i>	Serviço: <i>EncomendaStockService</i> Lista todas as encomendas de reposição de stock, independentemente do estado.
POST <i>/api/EncomendaStock</i>	Serviço: <i>EncomendaStockService</i> Cria uma nova encomenda de reposição com estado inicial <i>PENDENTE</i> , aguardando aprovação administrativa.
PUT <i>/api/EncomendaStock/{id}</i>	Serviço: <i>EncomendaStockService</i> Atualiza o estado da encomenda. Na transição para <i>RECECIONADA</i> , incrementa automaticamente o <i>stockAtual</i> da peça correspondente na <i>Data API</i> .
DELETE <i>/api/EncomendaStock/{id}</i>	Serviço: <i>EncomendaStockService</i> Remove uma encomenda de stock pendente rejeitada.
Módulo de Catálogos e Utilizadores	
GET <i>/api/Pecas</i>	Serviço: <i>PecaService</i> Lista todas as peças do catálogo, com suporte a filtros por categoria e estado.
GET <i>/api/Pecas/{ean}</i>	Serviço: <i>PecaService</i> Obtém uma peça pelo código EAN.
POST <i>/api/Pecas</i>	Serviço: <i>PecaService</i> Cria uma nova peça no catálogo com os dados técnicos e financeiros.

Continua na próxima página

Continuação da Tabela 6.1 — Endpoints da Lógica de Negócios

Método e Rota	Serviço Interno e Descrição Funcional
PUT <i>/api/Pecas/{ean}</i>	Serviço: <i>PecaService</i> Atualiza os dados de uma peça existente (preço, stock mínimo, descrição).
PATCH <i>/api/Pecas/{ean}/estado</i>	Serviço: <i>PecaService</i> Ativa ou desativa uma peça no catálogo sem a eliminar.
GET <i>/api/Trotinetes</i>	Serviço: <i>TrotineteService</i> Lista as trotinetes associadas ao cliente autenticado (NIF extraído do JWT).
POST <i>/api/Trotinetes</i>	Serviço: <i>TrotineteService</i> Regista uma nova trotinete no perfil do cliente autenticado.
DELETE <i>/api/Trotinetes</i> <i>/numSerie}</i>	Serviço: <i>TrotineteService</i> Remove uma trotinete do perfil do cliente.
GET <i>/api/Funcionarios</i>	Serviço: <i>FuncionarioService</i> Lista todos os funcionários registados no sistema.
POST <i>/api/Funcionarios</i>	Serviço: <i>FuncionarioService</i> Cria um novo funcionário, aplicando BCrypt à <i>password</i> antes da persistência.
PUT <i>/api/Funcionarios</i> <i>/numero}</i>	Serviço: <i>FuncionarioService</i> Atualiza os dados de um funcionário existente (cargo, especialidade, contacto).
GET <i>/api/IntervencoesCatalogo</i>	Serviço: <i>IntervencaoCatalogoService</i> Lista as intervenções do catálogo, com suporte a filtro por especialidade técnica.
POST <i>/api/IntervencoesCatalogo</i>	Serviço: <i>IntervencaoCatalogoService</i> Cria uma nova intervenção no catálogo com preço de mão-de-obra e especialidade requerida.
PUT <i>/api/IntervencoesCatalogo/{id}</i>	Serviço: <i>IntervencaoCatalogoService</i> Atualiza os dados de uma intervenção do catálogo (descrição, preço, especialidade).
DELETE <i>/api/IntervencoesCatalogo/{id}</i>	Serviço: <i>IntervencaoCatalogoService</i> Remove uma intervenção do catálogo.

Prompt utilizado - Claude Sonnet 4.6

"Gera o inventário completo de endpoints RESTful do servidor de Lógica de Negócios do Fix'n'Ride, organizado por módulo funcional. Para cada endpoint, indica o método HTTP, a rota completa, o serviço interno invocado e a descrição funcional. Módulos a cobrir: Autenticação (público, sem JWT), Serviços (incluindo o endpoint atómico de levantamento e o de listagem de trotinetes prontas), Agenda, Financeiro (Vendas, Faturas com devolução, Promoções, EncomendaCliente, Estatísticas com filtros por dia e por

intervalo), Stocks (EncomendaStock com as três transições de estado), e Catálogos e Utilizadores (Pecas com PATCH de estado, Trotinetes, Funcionarios, IntervencoesCatalogo)."

Deltas: A IA gerou um inventário com rotas insuficientes. Foi necessário adicionar algumas rotas manualmente que não foram contempladas pela IA.

6.6 Segurança: Autenticação e Controlo de Acessos

A estratégia de segurança adotada combina o padrão **JSON Web Token (JWT)** para autenticação sem estado (*stateless*) com um modelo de **Controlo de Acesso Baseado em Funções** (RBAC), implementado de forma distribuída entre o servidor LN (.NET) e o *frontend* React.

6.6.1 Fluxo de Autenticação JWT

1. **Submissão de Credenciais:** O utilizador submete as credenciais através do formulário de *login*. Para funcionários, o identificador é o número mecanográfico; para clientes, o NIF. O *frontend* invoca o *endpoint* `POST /api/Auth/login/*` do servidor LN.
2. **Recuperação do Registo na Data API:** O *AuthService* invoca as rotas de sistema protegidas da *Data API*, incluindo a `INTERNAL_API_KEY` no cabeçalho `x-api-key`. A *Data API* devolve o documento completo do utilizador, incluindo o *hash* BCrypt da *password*.
3. **Verificação Criptográfica:** O *AuthService* invoca `BCrypt.Net.BCrypt.Verify`. Em caso de falha, devolve imediatamente `HTTP 401 Unauthorized`.
4. **Geração do Token JWT:** Em caso de sucesso, é gerado um token JWT assinado com **HMAC-SHA256** e a `JwtSettings__SecretKey`, com validade de **8 horas**. O token encapsula os *claims* *jti* (identificador único de instância, UUID v4), *id*, *nome* e *cargo* (acrescido de *email* no caso dos clientes). O *jti* é a chave de identidade de cada emissão de token, indispensável para o mecanismo de revogação descrito nos passos seguintes.
5. **Emissão de Cookie e Resposta:** O servidor executa dois efeitos simultâneos. Primeiro, emite um *cookie* **HttpOnly** denominado `mobifix_auth`, com as diretivas `SameSite=Strict`, `Secure` (ativo quando HTTPS) e expiração sincronizada com a

do JWT, tornando-o inacessível ao JavaScript do *browser* e resistente a CSRF. Segundo, regista o *jti* na lista branca da *TokenListService*. O token é também devolvido no corpo da resposta (`{ token }`), permitindo ao *frontend* descodificar os *claims* localmente se necessário. Nas chamadas subsequentes, o *middleware JwtBearer* lê o token preferencialmente do cabeçalho *Authorization: Bearer {token}* e, na sua ausência, extrai-o automaticamente do *cookie mobifix_auth* através do evento *OnMessageReceived*.

6. **Validação Contínua:** Em cada pedido a *endpoints* protegidos, o *middleware* JWT Bearer valida assinatura, *issuer*, *audience* e expiração. Adicionalmente, o evento *OnTokenValidated* consulta a *TokenListService* (*IsAllowed(jti)*): o token é rejeitado com *HTTP 401* se o *jti* não constar da lista branca ou se constar da lista negra, independentemente de a assinatura ser válida. Esta verificação ocorre antes de qualquer lógica de *Controller*.
7. **Terminação de Sessão:** O *logout* invoca *POST /api/Auth/logout* no servidor LN, que (1) extrai o *jti* do token via *User.FindFirst*, (2) revoga-o na *TokenListService* (movendo a entrada da lista branca para a lista negra), e (3) elimina o *cookie mobifix_auth* via *Response.Cookies.Delete*. A partir desse instante, qualquer pedido com o token revogado é rejeitado com *HTTP 401*, mesmo que o token ainda não tenha expirado naturalmente. No *frontend*, a cache do TanStack Query é limpa via *queryClient.clear()* e o utilizador é redirecionado para */auth*.

Prompt utilizado - Claude Sonnet 4.6

"Descreve o fluxo completo de autenticação JWT do Fix'n'Ride em sete passos sequenciais: (1) submissão de credenciais pelo frontend (número mecanográfico para funcionários, NIF para clientes); (2) recuperação do registo na Data API com INTERNAL_API_KEY; (3) verificação criptográfica BCrypt com devolução imediata de HTTP 401 em caso de falha; (4) geração de token JWT assinado com HMAC-SHA256 e validade de 8 horas com claims de id, nome e cargo; (5) armazenamento no localStorage sob a chave 'token'; (6) validação contínua pelo middleware JWT Bearer em cada pedido protegido; (7) terminação de sessão com remoção do token, limpeza da cache TanStack Query e redirecionamento para /auth."

6.6.2 Gestão de Tokens: Lista Branca e Revogação

O serviço *TokenListService*, registado como *singleton* no contentor de DI, colmata a limitação fundamental do JWT *stateless*: a impossibilidade nativa de invalidar um token antes da sua expiração natural. A solução mantém em memória um registo parti-

lhado por todos os pedidos, baseado em duas estruturas concorrentes *ConcurrentDictionary<string, DateTimeOffset>*:

- **Lista branca (*whitelist*)**: mapeia o *jti* de cada token emitido para a respetiva data de expiração. Apenas tokens cujo *jti* conste desta lista são aceites pelo *middleware* (verificação em *OnTokenValidated*).
- **Lista negra (*blacklist*)**: regista os *jti* de tokens explicitamente revogados via *logout*. A sua presença resulta em rejeição imediata com *HTTP 401*, mesmo que a assinatura e a expiração sejam válidas.

A operação *Whitelist(jti, expiresAt)* é invocada no momento da emissão do token, no método privado *EmitirCookieEWhitelist* do *AuthController*. A operação *Revoke(jti)* é invocada no *endpoint* de *logout*, removendo a entrada da lista branca e inserindo-a na lista negra. Uma rotina de limpeza automática (*TryCleanup*) elimina entradas expiradas de ambas as listas a cada 10 minutos, prevenindo o crescimento ilimitado da memória.

6.6.3 Modelo RBAC: Controlo de Acesso Baseado em Funções

O sistema implementa um modelo RBAC com quatro perfis, cujos privilégios são definidos pela *claim cargo* embutida no token JWT. O componente *ProtectedRoute* do React Router verifica a presença e validade do token e o *cargo* face ao array *allowedRoles* da rota, redirecionando para */auth* em caso de acesso não autorizado.

Tabela 6.2: Modelo RBAC do sistema Fix'n'Ride — mapeamento de perfis, módulos e permissões

Perfil (<i>Role</i>)	Módulo, Prefixo de Rota e Permissões Funcionais
Público (sem token)	Rotas: /, /auth, /staff Acesso à <i>landing page</i> e formulários de autenticação. Sem acesso a qualquer dado do sistema.
CLIENTE	Módulo: /FixNRide/* Consultar e agendar diagnósticos; gerir trotinetes associadas ao perfil; consultar faturas e histórico de transações; navegar no catálogo e reservar peças (<i>Click & Collect</i>).
OPERADOR	Módulo: /FixNSell/* Processar vendas diretas ao balcão; consultar e entregar trotinetes concluídas; gerir peças reservadas por clientes; recepcionar encomendas de stock; consultar faturas emitidas.
Continua na próxima página	

Continuação da Tabela 6.2 — Modelo RBAC

Perfil (Role)	Módulo, Prefixo de Rota e Permissões Funcionais
MECANICO	Módulo: <i>/FixNRepair/*</i> Consultar agenda de diagnósticos e reparações; registrar intervenções e peças utilizadas; atualizar o estado dos serviços em execução.
ADMINISTRADOR	Módulos: <i>/FixNManage/*</i> , <i>/FixNSell/*</i> , <i>/FixNRepair/*</i> Acesso total: todas as operações dos perfis Operador e Mecânico, acrescentadas de gestão de utilizadores, aprovação de encomendas de stock, gestão de promoções, gestão do catálogo de peças e acesso ao <i>dashboard</i> de estatísticas financeiras.

Prompt utilizado - Claude Sonnet 4.6

"Descreve o modelo RBAC do Fix'n'Ride com quatro perfis de acesso. Para cada perfil, indica o módulo de frontend correspondente, o prefixo de rota e as permissões funcionais: Público (sem token — landing page e autenticação), CLIENTE (*/FixNRide/** — agendamentos, trotinetes, faturas, catálogo e Click & Collect), OPERADOR (*/FixNSell/** — vendas, entregas, receção de encomendas, faturas), MECANICO (*/FixNRepair/** — agenda, intervenções, estado de serviços) e ADMINISTRADOR (*/FixNManage/** mais acesso total aos módulos de Operador e Mecânico — utilizadores, encomendas de stock, promoções, catálogo, dashboard financeiro). Explica o papel do componente *ProtectRoute* do React Router na verificação do cargo extraído do JWT."

6.7 Diagramas de Sequência dos Fluxos Operacionais

Os diagramas de sequência apresentados nesta secção modelam o comportamento dinâmico dos fluxos operacionais mais críticos do sistema **Fix'n'Ride**, ilustrando a orquestração de chamadas entre o *frontend* React, o servidor de Lógica de Negócios (ASP.NET Core), a *Data API* (Node.js/Express) e a base de dados MongoDB. Os fluxos encontram-se agrupados por domínio funcional.

6.7.1 Fluxos de Autenticação e Gestão de Utilizadores

Registo de Cliente

O fluxo de registo permite que um novo cliente crie a sua conta no portal **Fix’n’Ride**. O *frontend* submete os dados do formulário para o *endpoint* público *POST /api/Auth/register/cliente*. O *AuthService* do servidor LN aplica *BCrypt* à *password* antes de persistir o novo documento na coleção *clientes* via *Data API*, garantindo que a *password* nunca é armazenada em texto simples.

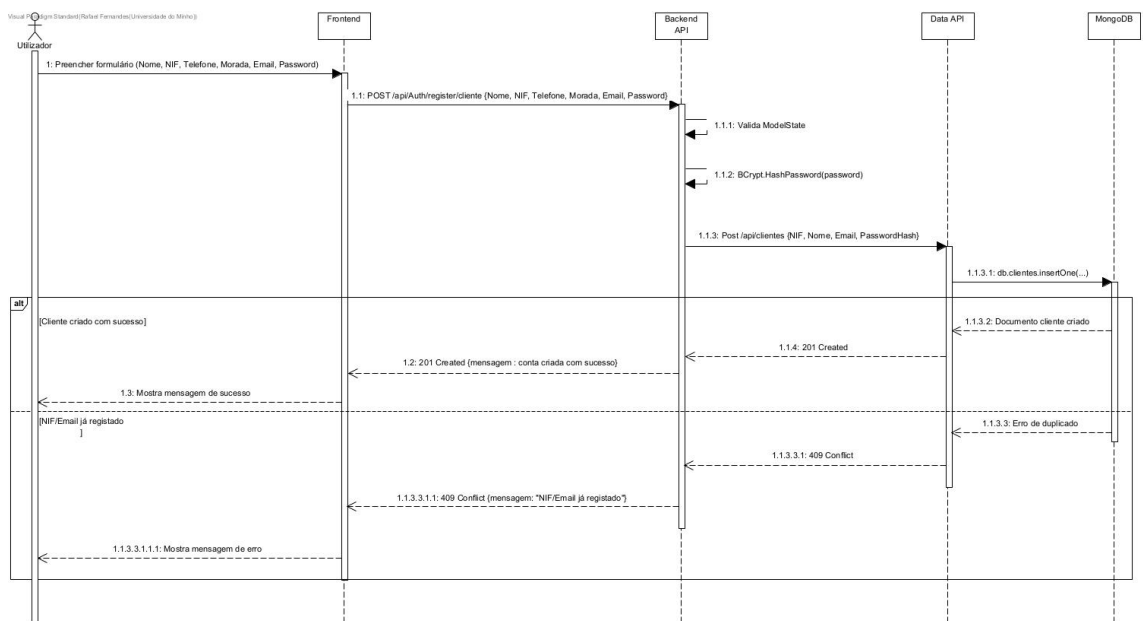


Figura 6.8: Diagrama de sequência — Registo de Cliente.

Login de Cliente

O processo de autenticação de um cliente valida as credenciais (NIF e *password*) e devolve um token JWT com a *claim* de *role Cliente*, que o *frontend* armazena em *localStorage* e utiliza em todas as chamadas subsequentes.

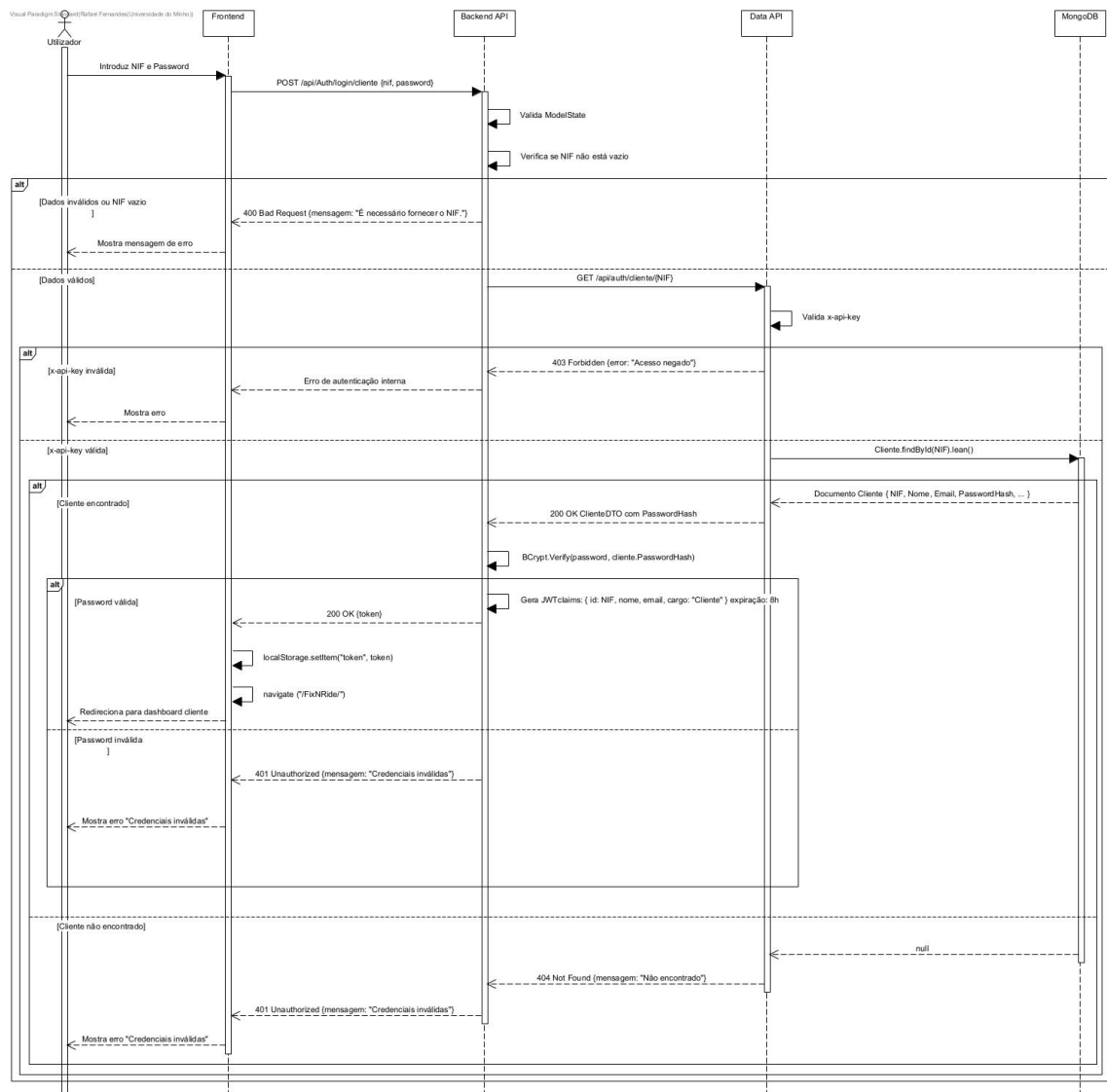


Figura 6.9: Diagrama de sequência — Login de Cliente.

Login de Funcionário

O fluxo de autenticação de funcionários é análogo ao de clientes, mas utiliza o número mecanográfico como identificador e devolve um token JWT com a *claim* de *role* correspondente ao cargo do funcionário (*ADMINISTRADOR*, *OPERADOR* ou *MECANICO*), determinando as rotas acessíveis no *frontend*.

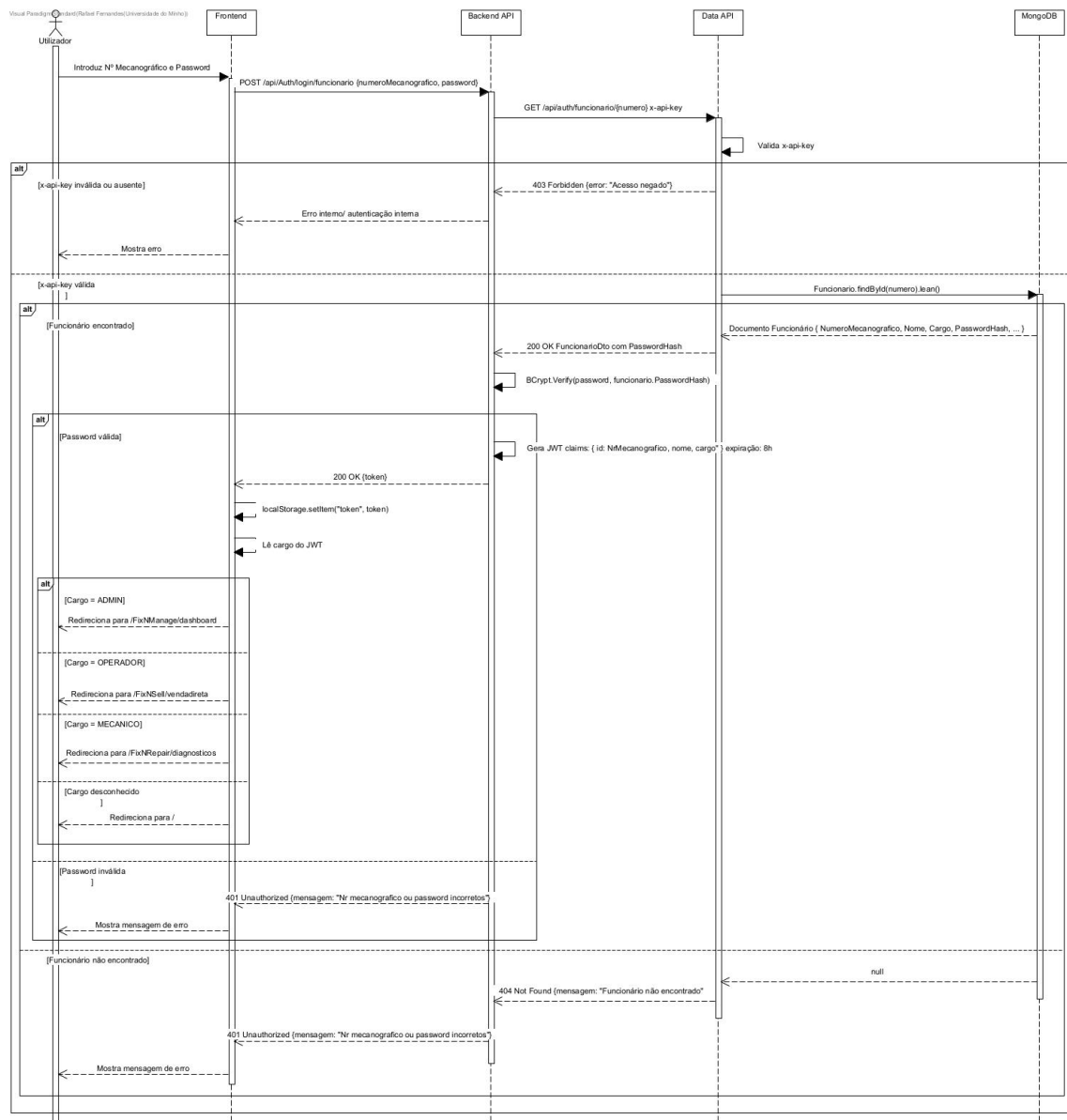


Figura 6.10: Diagrama de sequência — Login de Funcionário.

Registo de Trotinete

Um cliente autenticado pode associar uma nova trotinete ao seu perfil. O servidor LN extrai o NIF do cliente diretamente do token JWT, garantindo que um cliente nunca pode registar trotinetes em nome de outro utilizador.

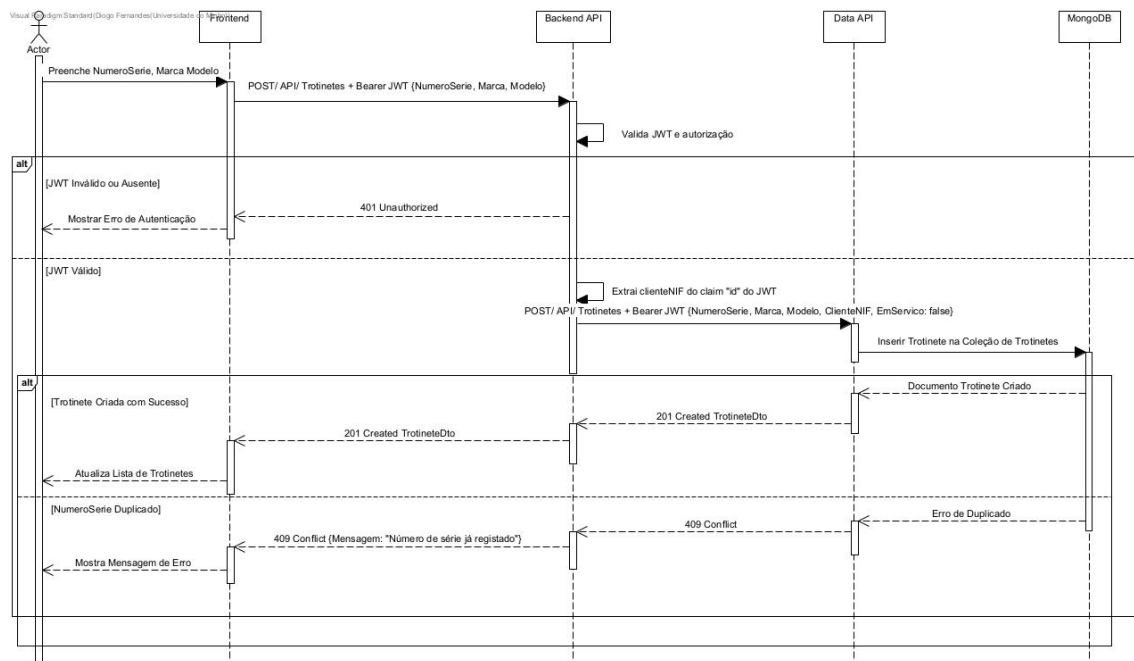


Figura 6.11: Diagrama de sequência — Registro de Trotinete.

6.7.2 Fluxos de Serviços de Oficina

Pedido de Serviço de Reparação

O cliente inicia o processo de reparação agendando um diagnóstico no portal. O sistema cria um novo serviço com estado *AGENDADO* e regista o *slot* correspondente na agenda do mecânico disponível.

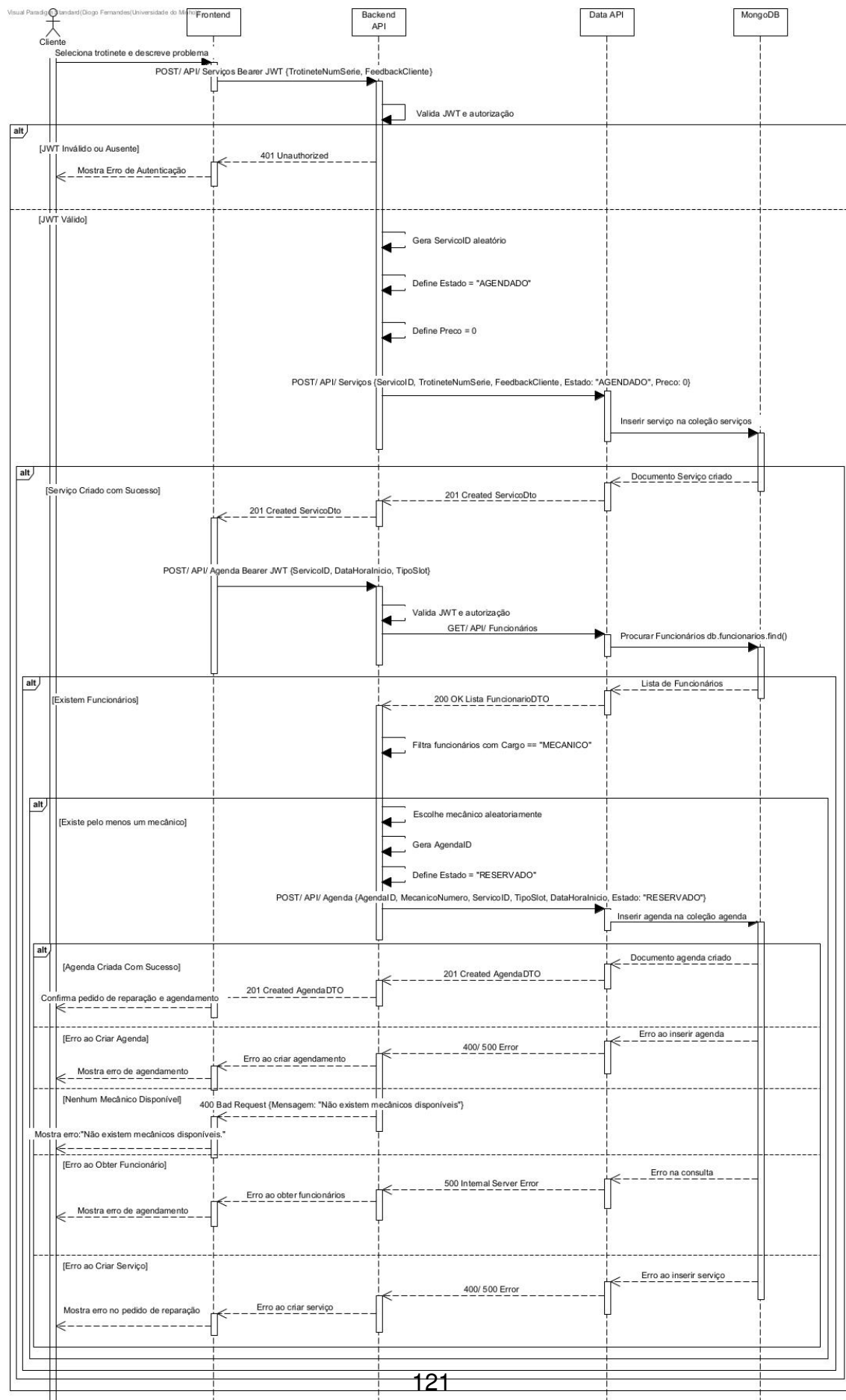


Figura 6.12: Diagrama de sequência — Pedido de Serviço de Reparação.

Atualização de Serviço/Diagnóstico

O mecânico, após a inspeção inicial da trotinete, preenche o diagnóstico técnico e registra as intervenções necessárias. O sistema atualiza o estado do serviço e calcula o preço total com base nas intervenções selecionadas do catálogo e nas peças associadas.

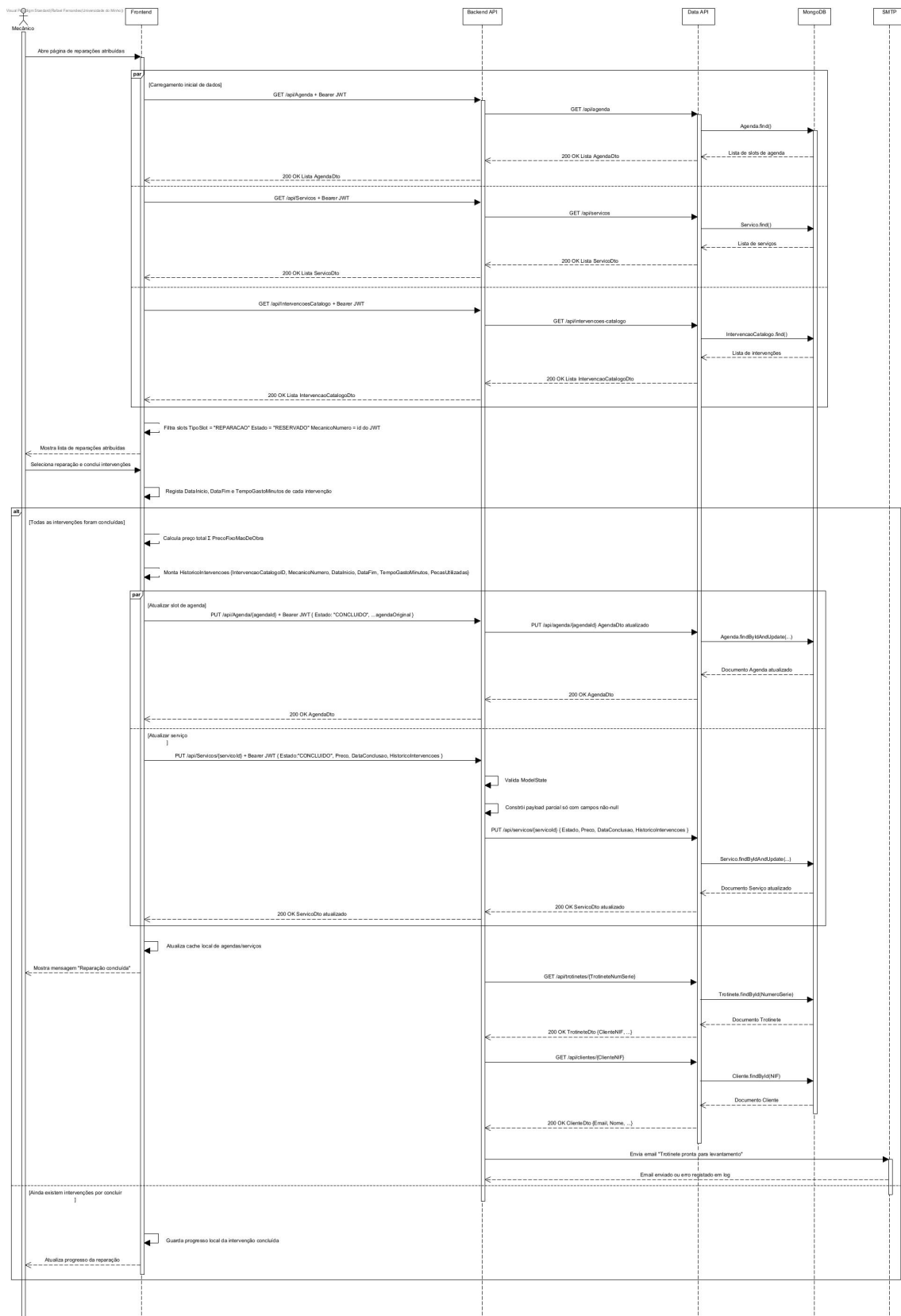


Figura 6.13: Diagrama de sequência — Atualização de Serviço/Diagnóstico.

Levantamento de Trotinete

O levantamento de uma trotinete reparada constitui o fluxo mais complexo do sistema, envolvendo a orquestração atômica de múltiplas operações: consulta do serviço, identificação do cliente proprietário, emissão de fatura e fecho do serviço.

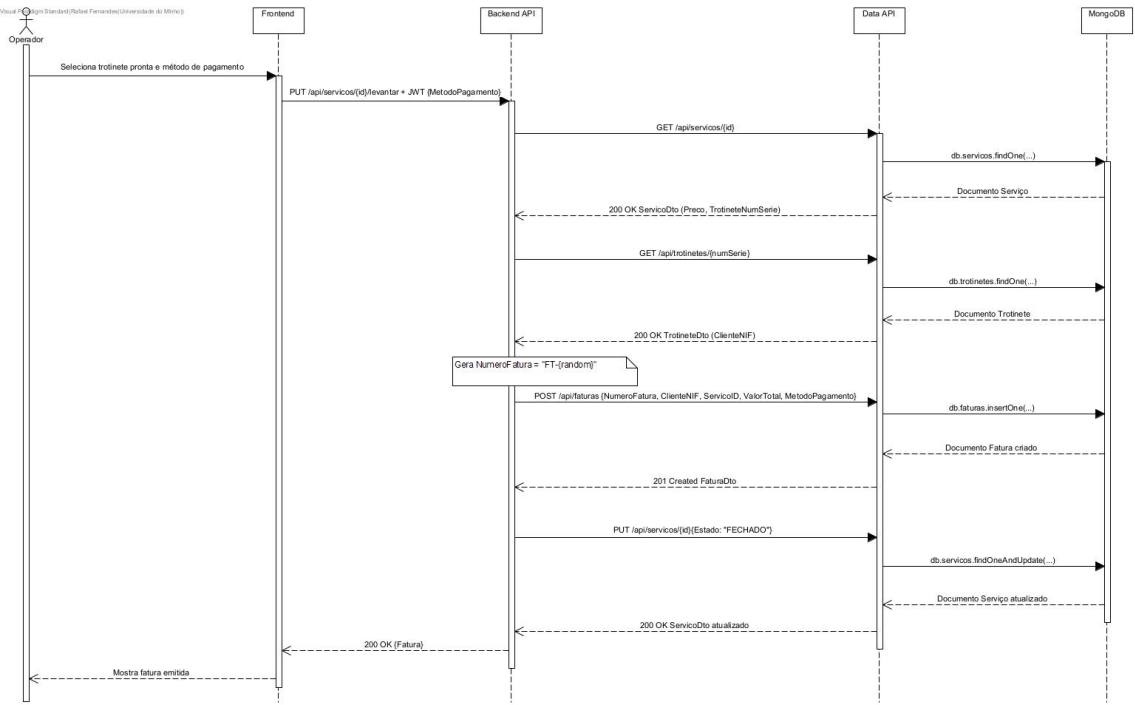


Figura 6.14: Diagrama de sequência — Levantamento de Trotinete.

6.7.3 Fluxos Financeiros e Comerciais

Venda Direta em Loja

O operador de loja processa a venda direta de peças ao balcão. O servidor LN recalcula o total no lado do servidor com base no PVP atual e nas promoções ativas, prevenindo manipulação de preços pelo *frontend*, e cria atomicamente o registo de venda e a fatura correspondente.

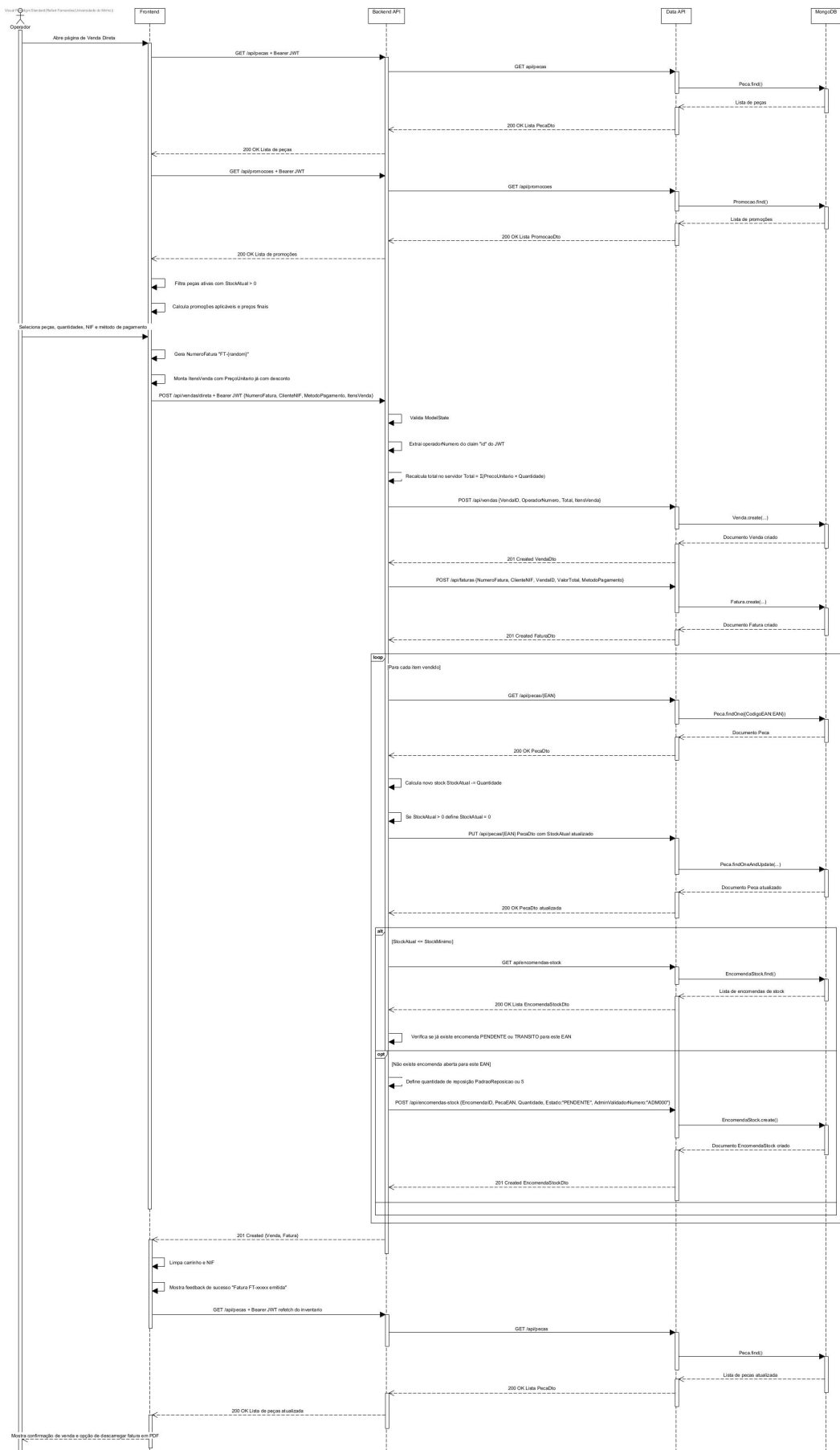


Figura 6.15: Diagrama de sequência — Venda Direta em Loja.

Reserva de Peças (Click & Collect)

O cliente autenticado pode reservar peças do catálogo para levantamento em loja. O servidor LN calcula o total com base no PVP atual e cria a encomenda com estado **PRONTO PARA LEVANTAMENTO**.

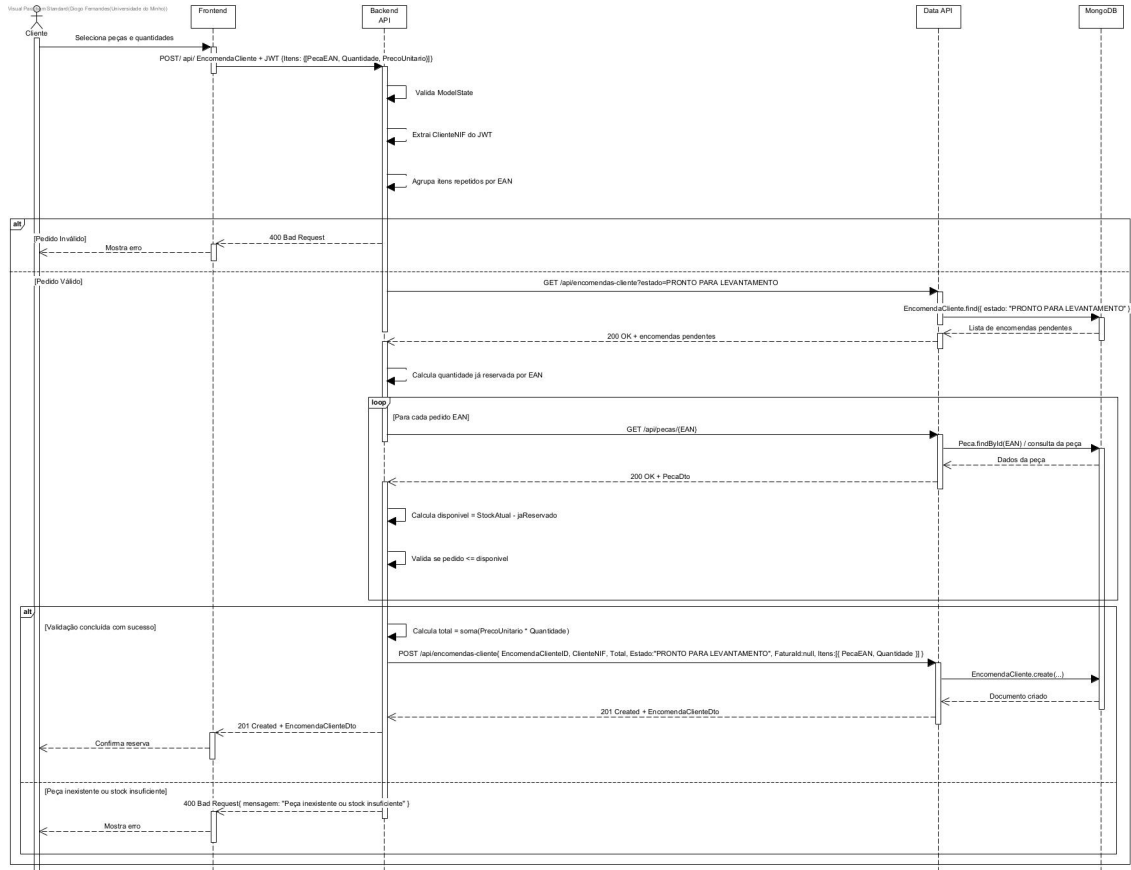


Figura 6.16: Diagrama de sequência — Reserva de Peças (Click & Collect).

Levantamento de Encomenda de Cliente

O operador de loja entrega as peças reservadas pelo cliente. O sistema atualiza o estado da encomenda para **ENTREGUE** e emite a fatura correspondente.

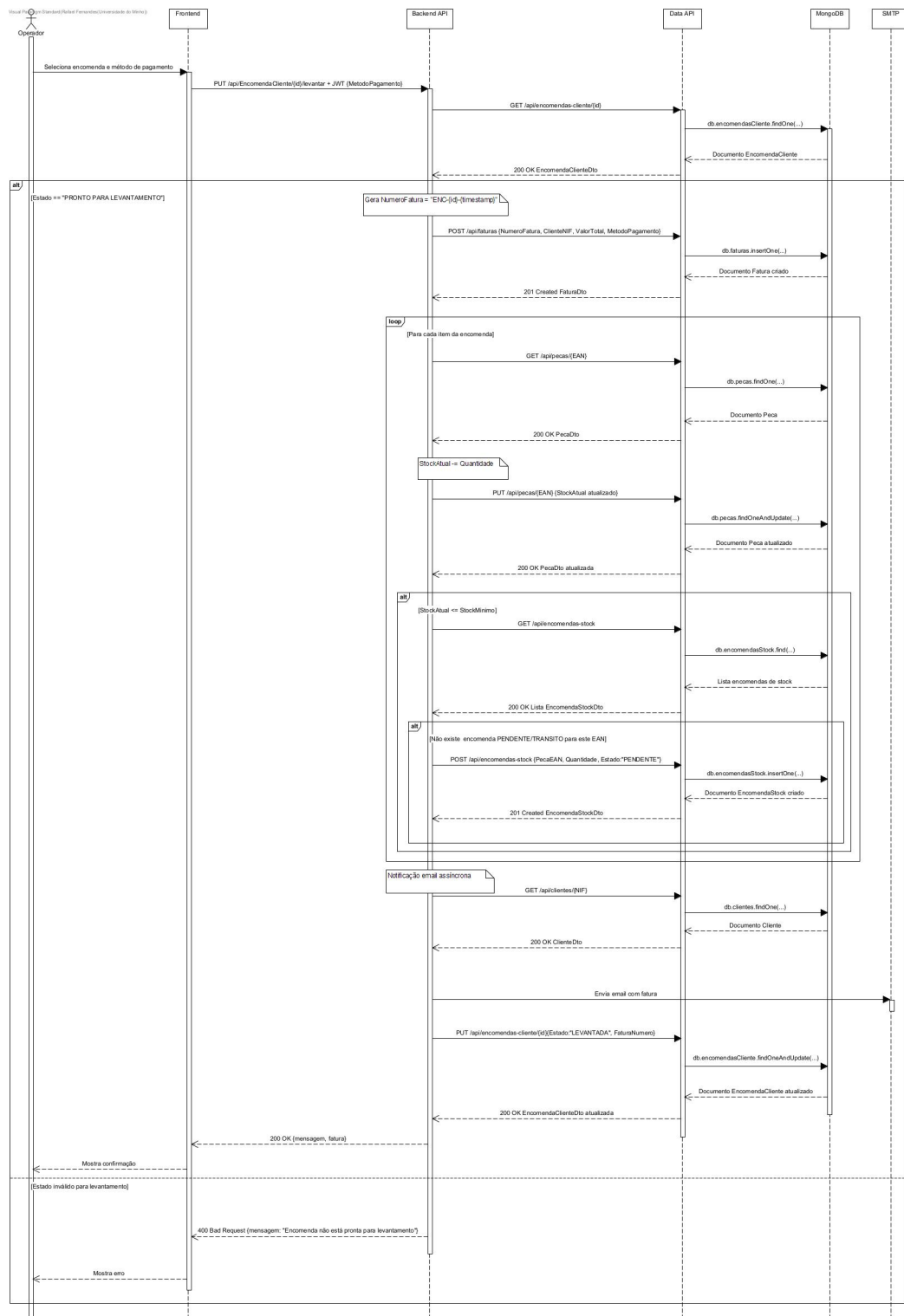


Figura 6.17: Diagrama de sequência — Levantamento de Encomenda de Cliente.

Devolução de Fatura

O operador regista uma devolução associada a uma fatura existente. O sistema emite a nota de crédito correspondente, embutindo o subdocumento de devolução diretamente no array *devolucoes* da fatura original, preservando a rastreabilidade fiscal sem apagar o histórico.

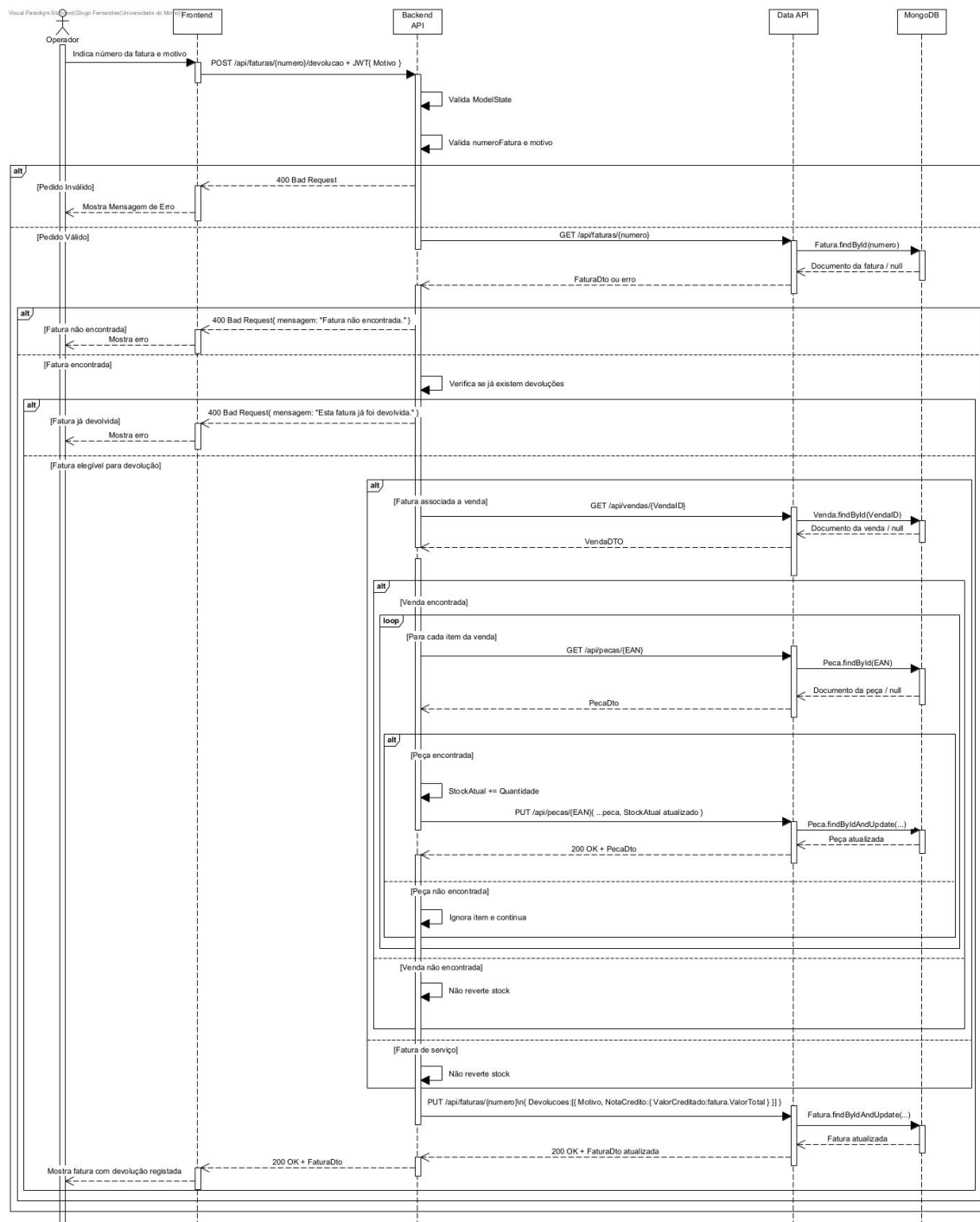


Figura 6.18: Diagrama de sequência — Devolução de Fatura.

6.7.4 Fluxos Administrativos

Consulta de Estatísticas (Administrador)

O administrador acede ao *dashboard* financeiro para monitorizar os KPIs da oficina. O *EstatisticasService* agrega dados de múltiplas coleções — faturas, vendas, serviços e devoluções — calculando os indicadores globais ou filtrados por período.

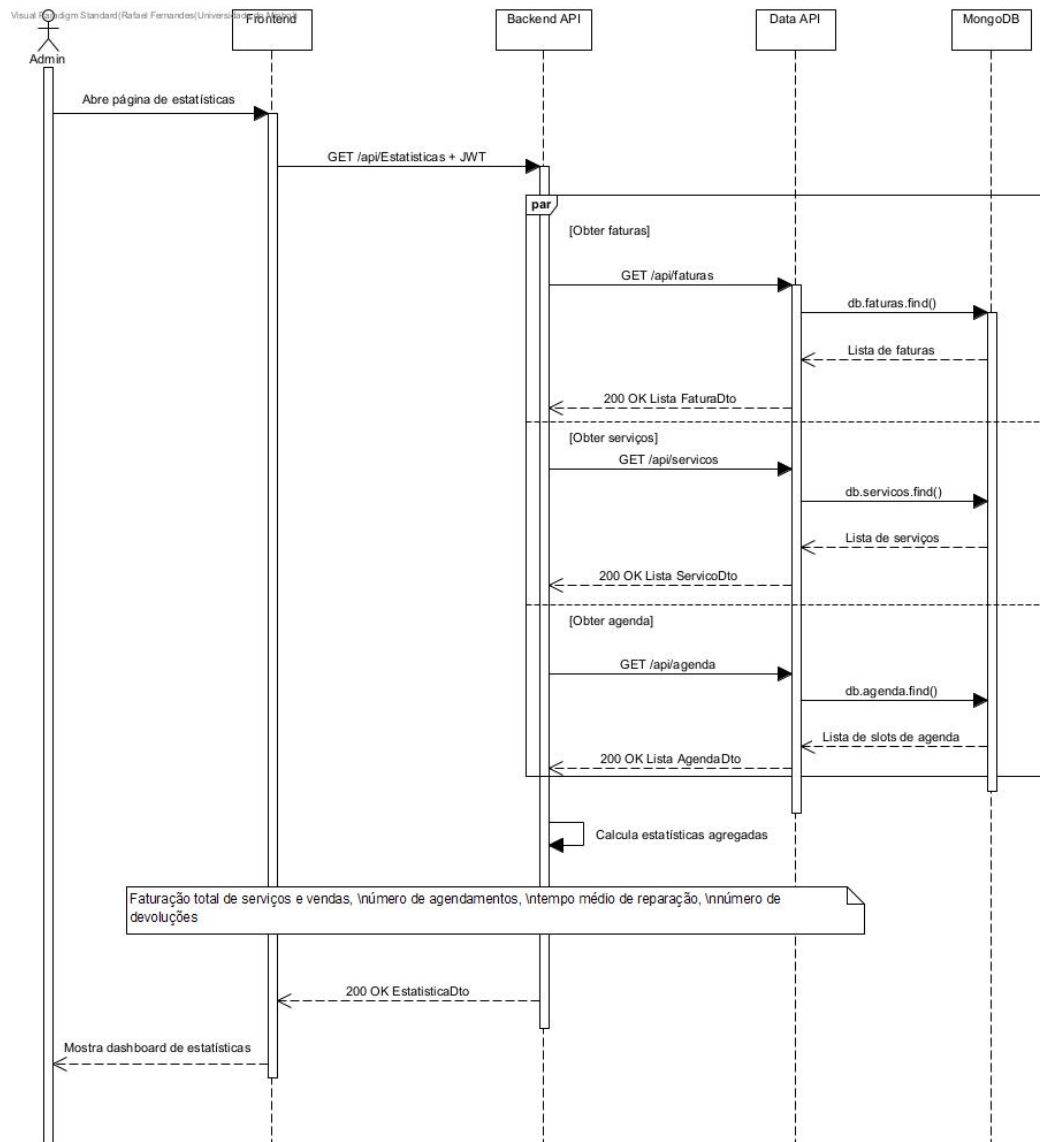


Figura 6.19: Diagrama de sequência — Consulta de Estatísticas (Administrador).

7 Implementação do *Frontend* (React + Tailwind CSS)

7.1 Arquitetura SPA com React Router e *Layouts* Aninhados

A interface do utilizador do sistema **Fix’n’Ride** foi implementada como uma **SPA** (*Single Page Application*), utilizando **React 18** como biblioteca de renderização e **React Router v6** como motor de navegação *client-side*. Neste modelo, o servidor de *frontend* serve um único ficheiro HTML ao *browser* no carregamento inicial; toda a navegação subsequente entre vistas é gerida inteiramente em memória pelo React Router, sem recarregamentos de página, o que confere uma experiência de utilizador fluida e responsiva.

O ponto de entrada do sistema de navegação é o componente `AppRouter`, declarado em `src/router/AppRouter.jsx`, que envolve toda a aplicação num `BrowserRouter` e define a hierarquia de rotas. A estrutura de navegação assenta em dois conceitos-chave do React Router v6: **rotas aninhadas** e o componente `Outlet`.

Rotas Aninhadas e `Outlet`. Cada módulo funcional da aplicação — área do cliente, do operador, do mecânico e do administrador — é estruturado como um conjunto de rotas filho aninhadas sob uma rota pai que define o *layout* partilhado do módulo. O componente pai renderiza o esqueleto visual (barra lateral de navegação, cabeçalho, rodapé) e inclui o componente `<Outlet />` no ponto onde o conteúdo específico de cada rota filho deve ser renderizado. Esta abordagem garante que os elementos de navegação e identidade visual de cada módulo são instanciados uma única vez, sem re-renderização desnecessária ao navegar entre subpáginas do mesmo módulo.

Por exemplo, o módulo do operador (`/FixNSell`) usa o componente `Layout` como pai — que inclui a *sidebar* com os menus de navegação e o botão de *logout* — e as páginas `VendaDireta`, `TrotinetesProntas` e `PecasReservadas`, entre outras, são renderizadas como filhos dentro do `<Outlet />` desse *layout*.

Proteção de Rotas. As rotas privadas são protegidas pelo componente `ProtectedRoute`, descrito em detalhe na Secção 6.6. Este componente atua como um *guard* que verifica a presença e validade do token JWT antes de renderizar o `<Outlet />` dos filhos, redirecionando para `/auth` em caso de acesso não autorizado.

Instância HTTP Centralizada. Toda a comunicação com o servidor LN é realizada através de uma instância **Axios** centralizada, definida em `src/services/api.js`. Esta instância configura a `baseUrl` como `http://localhost:5001/api` e regista um *interceptor* de pedidos que injeta automaticamente o token JWT do `localStorage` no cabeçalho `Authorization: Bearer` de todos os pedidos, eliminando a necessidade de gestão manual de autenticação em cada chamada de serviço.

Prompt utilizado - Claude Sonnet 4.6

"Descreve a arquitetura SPA do Fix'n'Ride implementada com React 18 e React Router v6. Explica o papel do componente `AppRouter` como ponto de entrada da navegação, o conceito de rotas aninhadas com o componente `Outlet` para partilha de layouts entre subpáginas do mesmo módulo (exemplificando com o módulo do operador `/FixNSell`), a proteção de rotas pelo componente `ProtectedRoute` com verificação de JWT e redirecionamento para `/auth`, e a instância `Axios` centralizada em `src/services/api.js` com *interceptor* que injeta automaticamente o token JWT em todos os pedidos."

7.2 Mapeamento Completo de Rotas por Perfil

A Tabela 7.1 apresenta o inventário completo das rotas da SPA, organizadas por perfil de acesso, com indicação do componente renderizado, do *layout* pai e do nível de proteção.

Tabela 7.1: Mapeamento completo de rotas da SPA Fix'n'Ride por perfil			
Perfil	Rota	Componente / Página	Layout Pai
	Público (sem autenticação)		
Público	/	HomePage	Nenhum
Público	/auth	AuthPage	Nenhum
Público	/staff	LoginPage	Nenhum
Público	* (wildcard)	Redireciona para /	—
	Cliente (allowedRoles=['Cliente'])		
Cliente	/FixNRide/	HomeClientePage	ProtectedRoute

Continua na próxima página

Continuação da Tabela 7.1 — Mapeamento de Rotas por Perfil

Perfil	Rota	Componente / Página	Layout Pai
Cliente	/FixNRide/trotinetes	MinhasTrotinetesPage	ProtectedRoute
Cliente	/FixNRide/faturas	FaturasPage	ProtectedRoute
Cliente	/FixNRide/agendar	AgendarDiagnosticoPage	ProtectedRoute
Cliente	/FixNRide/catalogo	CatalogoPecasPage	ProtectedRoute
Operador e Administrador (allowedRoles=['OPERADOR', 'ADMINISTRADOR'])			
Operador	/FixNSell (index)	VendaDiretaPage	Layout
Operador	/FixNSell/vendadireta	VendaDiretaPage	Layout
Operador	/FixNSell/trotinetes-prontas	TrotinetesProntasPage	Layout
Operador	/FixNSell/pecas-reservadas	PecasReservadas	Layout
Operador	/FixNSell/rececao-encomendas	RececaoEncomendasPage	Layout
Operador	/FixNSell/faturas	FaturasPage (Operador)	Layout
Mecânico e Administrador (allowedRoles=['MECANICO', 'ADMINISTRADOR'])			
Mecânico	/FixNRepair/ (index)	Dashboard	RepairsLayout
Mecânico	/FixNRepair/diagnosticos	Dashboard	RepairsLayout
Mecânico	/FixNRepair/reparacoes	Repairs	RepairsLayout
Administrador (allowedRoles=['ADMINISTRADOR'])			
Administrador	/FixNManage/ (index)	AdminDashboard	AdminLayout
Administrador	/FixNManage/dashboard	AdminDashboard	AdminLayout
Administrador	/FixNManage/encomendas	EncomendasStock	AdminLayout
Administrador	/FixNManage/users	GestaoUsers	AdminLayout
Administrador	/FixNManage/promocoos	Promocoos	AdminLayout
Administrador	/FixNManage/pecas	PecasManagement	AdminLayout

Prompt utilizado - Claude Sonnet 4.6

"Gera o inventário completo de rotas da SPA Fix'n'Ride, organizado por perfil de acesso. Para cada rota, indica o perfil (Público, Cliente, Operador/Administrador, Mecânico/Administrador, Administrador exclusivo), a rota completa, o componente ou página renderizado e o layout pai. Cobre todos os módulos: rotas públicas (/ , /auth, /staff, wildcard), rotas do cliente (/FixNRide/*), rotas do operador (/FixNSell/*), rotas do mecânico (/FixNRepair/*), e rotas exclusivas do administrador (/FixNManage/*). Indica os allowedRoles de cada grupo de rotas protegidas."

7.3 Camada de Serviços e *Hooks* com TanStack Query

A gestão de estado assíncrono e de comunicação com a API no *frontend* do **Fix’n’Ride** é inteiramente suportada pela biblioteca **TanStack Query** (anteriormente conhecida como React Query). Esta biblioteca abstrai a complexidade de pedidos HTTP, *caching*, sincronização e revalidação de dados, permitindo que os componentes React se foquem exclusivamente na renderização.

Organização em Duas Camadas. A arquitetura do *frontend* organiza a comunicação com a API em duas camadas distintas e complementares:

1. **Camada de Serviços** (`src/services/`): Cada domínio funcional possui um módulo de serviço dedicado (ex: `agendaService.js`, `pecaService.js`, `operatorService.js`). Estes módulos são responsáveis exclusivamente pela invocação dos *endpoints* HTTP via Axios, construção dos *payloads* e devolução dos dados. São funções puras e reutilizáveis, sem estado.
2. **Camada de *Hooks*** (`src/hooks/`): Cada domínio possui um conjunto de *hooks* React personalizados (ex: `useAgenda.js`, `usePecas.js`, `useServicos.js`) que encapsulam as chamadas à camada de serviços dentro de `useQuery` (leitura) ou `useMutation` (escrita). Os componentes de página consomem exclusivamente estes *hooks*, sem conhecimento da camada de serviços subjacente.

Padrão `useQuery` para Leitura de Dados. Cada *hook* de leitura usa `useQuery` com uma *query key* única e estável que identifica o recurso em *cache*. A TanStack Query gere automaticamente os estados de `isLoading`, `isError` e `data`, utilizados pelos componentes para renderizar indicadores de carregamento e mensagens de erro de forma declarativa. O parâmetro `staleTime` é configurado por domínio: dados voláteis como as peças têm `staleTime: 1`, forçando revalidação frequente, enquanto dados mais estáveis como as faturas de cliente usam `staleTime: 1000 * 60 * 5` (cinco minutos).

Padrão `useMutation` para Escrita e Invalidação de *Cache*. As operações de escrita são encapsuladas em *hooks* baseados em `useMutation`. O *callback* `onSuccess` de cada mutação invoca `queryClient.invalidateQueries` com a *query key* correspondente, sinalizando à TanStack Query que os dados em *cache* estão desatualizados e devem ser revalidados na próxima leitura. Em cenários de alta frequência de atualização — como a lista de serviços no módulo do mecânico — complementa-se a invalidação com uma **atualização otimista da *cache*** via `queryClient.setQueryData`: o item modificado é substituído imediatamente no array em *cache* com o objeto devolvido pelo

servidor, eliminando o *delay* visual entre a confirmação da operação e a atualização da lista.

A Tabela 7.2 resume os principais *hooks* implementados por domínio.

Tabela 7.2: Inventário dos principais <i>hooks</i> TanStack Query do <i>frontend</i> Fix'n'Ride		
Ficheiro	Hook	Operação / Serviço Invocado
Domínio de Agenda		
useAgenda.js	useAgendas	useQuery — lista todos os <i>slots</i> de agenda.
useAgenda.js	useCriarAgenda	useMutation — cria <i>slot</i> ; invalida ['agendas'].
useAgenda.js	useAtualizarAgenda	useMutation — atualiza <i>slot</i> ; atualização otimista da <i>cache</i> .
Domínio de Serviços		
useServicos.js	useServicos	useQuery — lista todos os serviços.
useServicos.js	useCriarServico	useMutation — cria serviço; invalida ['servicos'].
useServicos.js	useAtualizarServico	useMutation — atualiza serviço; atualização otimista da <i>cache</i> .
Domínio de Peças		
usePecas.js	usePecas	useQuery — lista peças (staleTime: 1).
usePecas.js	useCriarPeca	useMutation — cria peça; invalida ['pecas'].
usePecas.js	useAtualizarPeca	useMutation — atualiza dados de uma peça.
usePecas.js	useAlterarEstadoPeca	useMutation — ativa/desativa peça via PATCH.
Domínio Financeiro		
useFaturas.js	useFaturas	useQuery — faturas do cliente (staleTime: 5 min).
Domínio de Utilizadores		
useFuncionarios.js	useFuncionarios	useQuery — lista todos os funcionários.
useFuncionarios.js	useCriarFuncionario	useMutation — cria funcionário; invalida ['funcionarios'].
useFuncionarios.js	useAtualizarFuncionario	useMutation — atualiza dados de um funcionário.
Domínio de Promoções		
usePromocao.js	usePromocoes	useQuery — lista promoções (staleTime: 1).
		<i>Continua na próxima página</i>

Continuação da Tabela 7.2 — Hooks TanStack Query

Ficheiro	Hook	Operação / Serviço Invocado
usePromocao.js	useCriarPromocao	useMutation — cria promoção; invalida ['promocoes'].
usePromocao.js	useAlterarEstadoPromocao	useMutation — ativa/desativa promoção.
Domínio do Operador		
useOperator.js	usePecasOperator	useQuery — catálogo para o operador (staleTime: 30s).
useOperator.js	useEncomendasStockOperator	useQuery — encomendas de stock (staleTime: 15s).
useOperator.js	useConfirmarRececao	useMutation — confirma receção; invalida chaves de stock e encomendas.
Domínio de Catálogo		
useIntervencoesCatalogo.js	useIntervencoesCatalogo	useQuery — lista catálogo com filtro opcional por especialidade.

Prompt utilizado - Claude Sonnet 4.6

"Descreve a camada de serviços e de hooks com TanStack Query do frontend Fix'n'Ride. Explica a organização em duas camadas: (1) módulos de serviço em src/services/ com funções puras que invocam endpoints HTTP via Axios; (2) hooks React personalizados em src/hooks/ que encapsulam useQuery e useMutation. Detalha o padrão de staleTime diferenciado por domínio (staleTime: 1 para peças e promoções, 5 minutos para faturas, 30 segundos para o catálogo do operador), a invalidação de cache em onSuccess das mutações, e a atualização otimista via queryClient.setQueryData para cenários de alta frequência como a lista de serviços do mecânico."

7.4 Estilização com Tailwind CSS

A estilização da interface do **Fix'n'Ride** é integralmente baseada no *framework* **Tailwind CSS**, seguindo a abordagem *utility-first*: em vez de escrever folhas de estilo CSS separadas, as classes de estilo são aplicadas diretamente no *markup* JSX dos componentes, a partir de um vocabulário extenso de utilitários atômicos predefinidos.

Esta abordagem traduz-se em vantagens práticas concretas no contexto do projeto. A eliminação da troca de contexto entre ficheiros .jsx e .css acelera o desenvolvimento e torna o estilo de cada componente imediatamente legível no código. A ausência de

colisões de nomes de classes e de especificidade CSS (*specificity wars*) — problemas comuns em projetos de média dimensão com CSS global — é eliminada por definição, dado que cada classe utilitária tem um efeito único e bem definido.

Cada módulo da aplicação define a sua própria linguagem visual através das classes Tailwind, mantendo coerência interna: o módulo de cliente (`/FixNRide`) adota uma estética *mobile-first* com cores azuis e roxas, *cards* arredondados e uma barra de navegação inferior (`BottomNav`) adequada a utilização em *smartphone*; o módulo do operador (`/FixNSell`) e do administrador (`/FixNManage`) utilizam uma *sidebar* fixa lateral adaptada a ecrãs de secretária; e o módulo do mecânico (`/FixNRepair`) adota um design mais denso em informação, com cartões de diagnóstico e painéis de detalhes lado a lado.

O projeto não utiliza um ficheiro de configuração Tailwind customizado com *design tokens* específicos da marca, optando pelas classes base do Tailwind. Em produção recomenda-se a definição de uma paleta de cores e uma escala tipográfica proprietária no `tailwind.config.js`, garantindo maior consistência de marca.

Prompt utilizado - Claude Sonnet 4.6

"Descreve a estratégia de estilização do Fix'n'Ride com Tailwind CSS, seguindo a abordagem utility-first. Explica as vantagens práticas no contexto do projeto: eliminação da troca de contexto entre ficheiros JSX e CSS, ausência de colisões de especificidade, e legibilidade do estilo diretamente no markup. Detalha a linguagem visual distinta de cada módulo: cliente (`/FixNRide`) com estética *mobile-first*, cores azuis e roxas, *cards* arredondados e `BottomNav`; operador e administrador com *sidebar* lateral fixa para *desktop*; mecânico com design denso em informação com painéis lado a lado.

7.5 Design, Mockups e Interfaces Implementadas

A presente secção documenta o processo de *design* da interface — desde os *mockups* iniciais até às capturas de ecrã do produto final — por perfil de utilizador. Esta abordagem permite confrontar diretamente a proposta de *design* com a implementação, evidenciando o grau de fidelidade alcançado.

7.5.1 Processo de *Design* Assistido por IA

Previamente ao início da implementação, a equipa elaborou um conjunto de *mockups* de alta fidelidade para as principais páginas do sistema, com o objetivo de validar a proposta de experiência de utilizador (UX) e definir a linguagem visual antes do desenvolvimento do código. A ferramenta utilizada foi o **Figma**, complementada com o *plugin* **Figma AI (First Draft)**, que permitiu gerar esboços iniciais de baixa fidelidade a partir de *prompts* textuais descritivos de cada portal. Estes esboços foram depois refinados manualmente pela equipa no Figma, definindo a paleta de cores, tipografia e hierarquia visual de cada módulo.

O fluxo de trabalho adotado foi o seguinte: (1) geração de *wireframe* com Figma AI a partir de *prompt* descritivo; (2) revisão e ajuste manual pela equipa no Figma; (3) exportação do *mockup* de alta fidelidade; (4) utilização do **Antigravity** (Google DeepMind) para gerar o código React/Tailwind correspondente a partir da imagem do *mockup* e da descrição do componente, acelerando o *handoff design-para-código*. Esta abordagem reduziu significativamente o tempo entre a aprovação do *design* e a disponibilidade de uma primeira implementação funcional.

7.5.2 Interface Pública e Autenticação

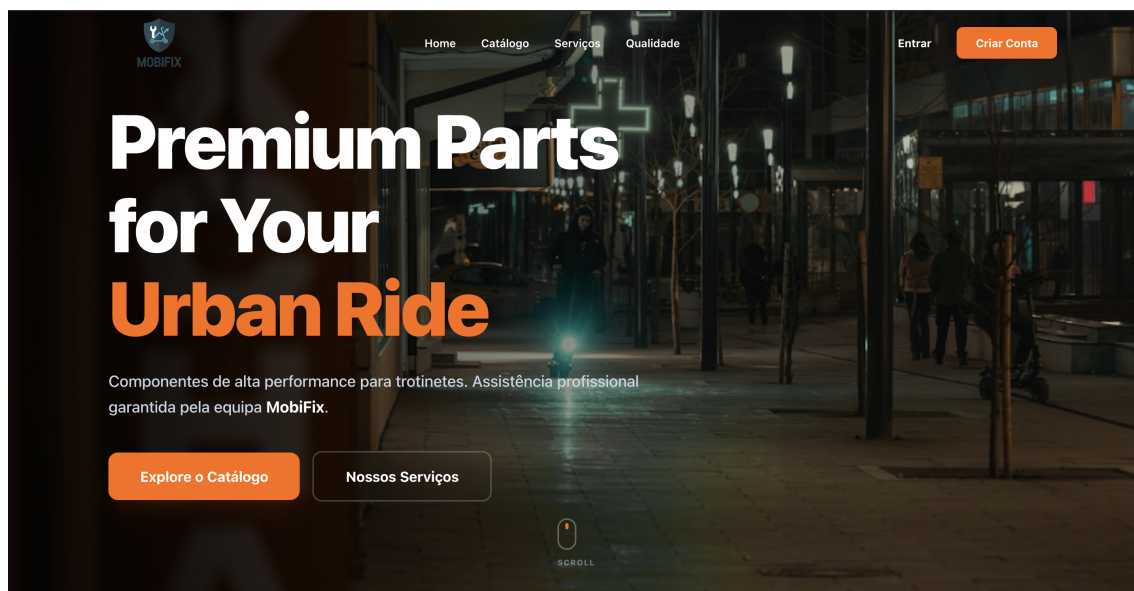


Figura 7.1: *Landing page* pública do sistema Fix'n'Ride (HomePage).



**Bem-vindo de volta à
revolução urbana.**

Aceda à sua conta para gerir as suas reservas e
histórico de reparações.

[← Voltar ao Início](#)

Login de Cliente

Introduza o seu NIF e password para entrar.

NIF

123456789

Palavra-passe



[Esqueceu-se da password?](#)

Entrar

Ainda não tem conta? [Registe-se aqui](#)

Figura 7.2: Página de autenticação unificada (AuthPage) para clientes.

7.5.3 Interface do Cliente (/FixNRide)

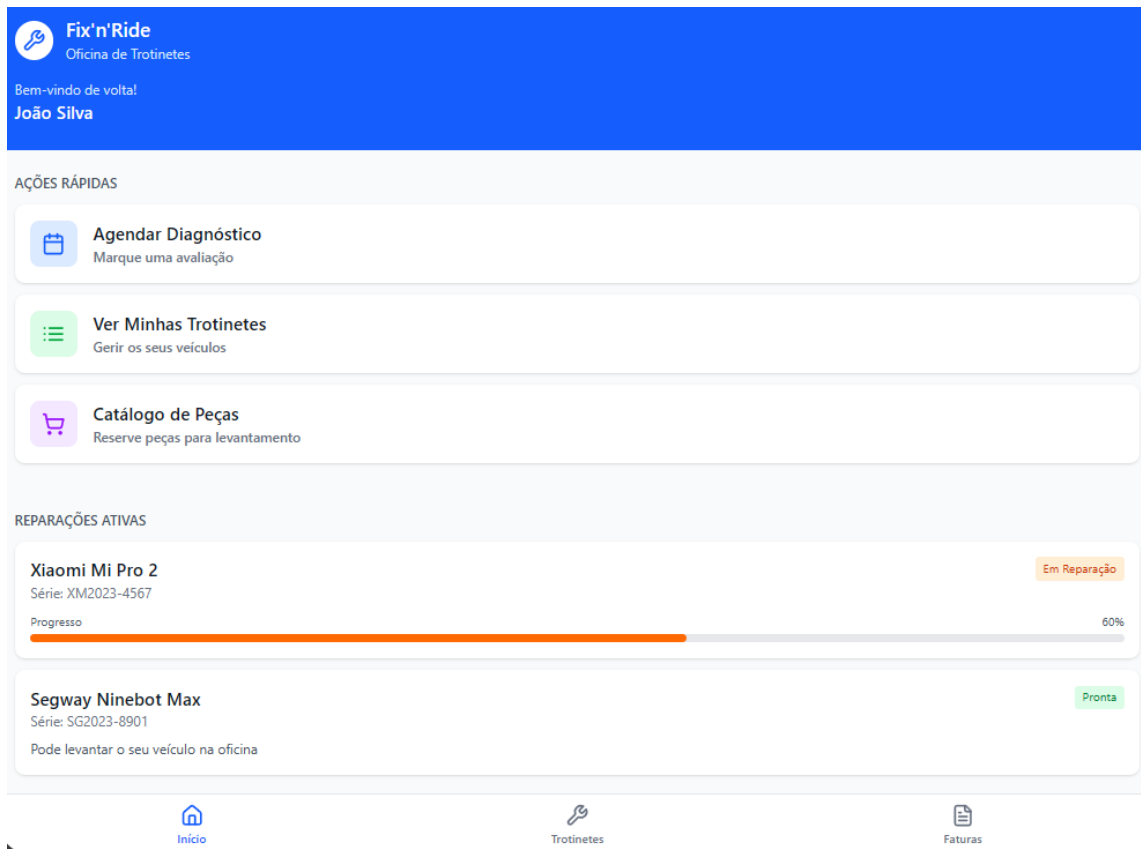


Figura 7.3: Página inicial da área reservada do cliente (HomeClientePage).

← Agendar Diagnóstico

ⓘ Agende um diagnóstico gratuito. Os nossos técnicos irão avaliar o seu veículo e fornecer um orçamento detalhado.

Selecione a Trotinete

Escolha um veículo... ▾

📅 Data do Diagnóstico

30/02/0025

🕒 Horário Disponível

09:00

10:00

11:00

14:00

15:00

16:00

17:00

📝 Descrição do Problema

Descreva o problema que a sua trotinete apresenta...

🏠 Início

🔧 Trotinetes

📄 Faturas

Figura 7.4: Página de agendamento de diagnóstico (AgendarDiagnosticoPage).

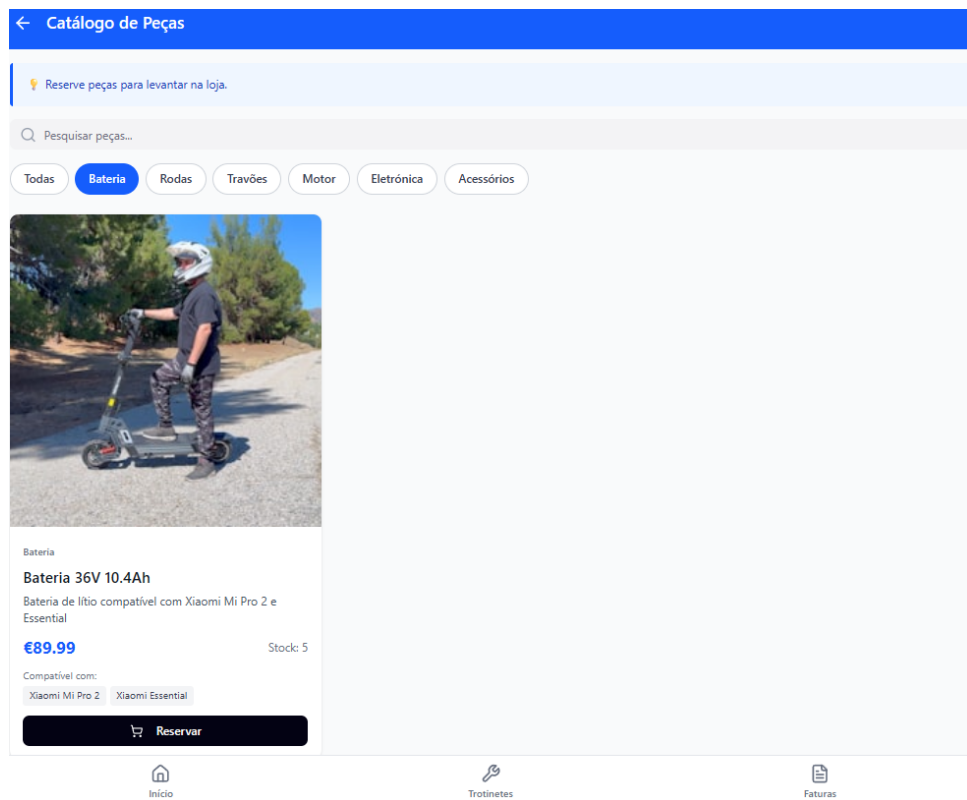


Figura 7.5: Catálogo de peças do cliente (CatalogoPecasPage).

7.5.4 Interface do Operador (/FixNSell)

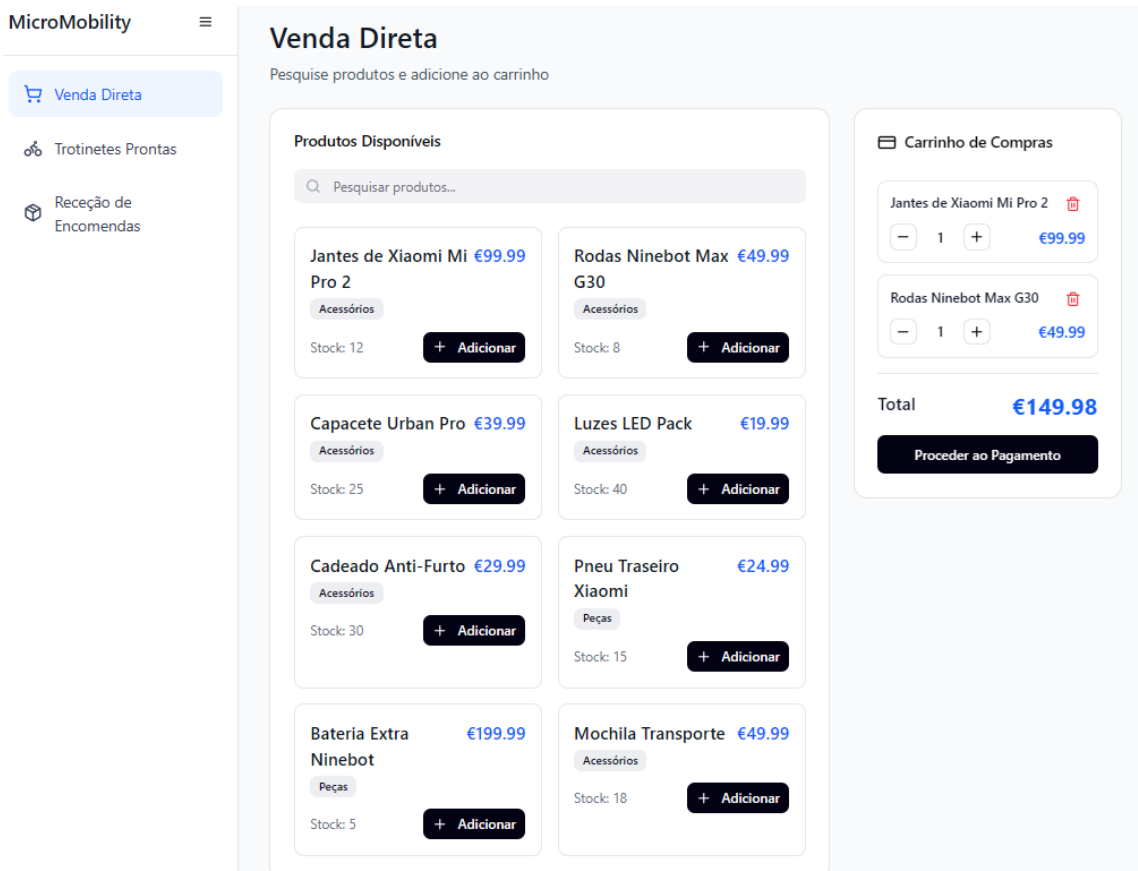


Figura 7.6: Módulo de venda direta ao balcão (VendaDiretaPage).

Trotinetes Prontas

Trotinetes reparadas pendentes de levantamento

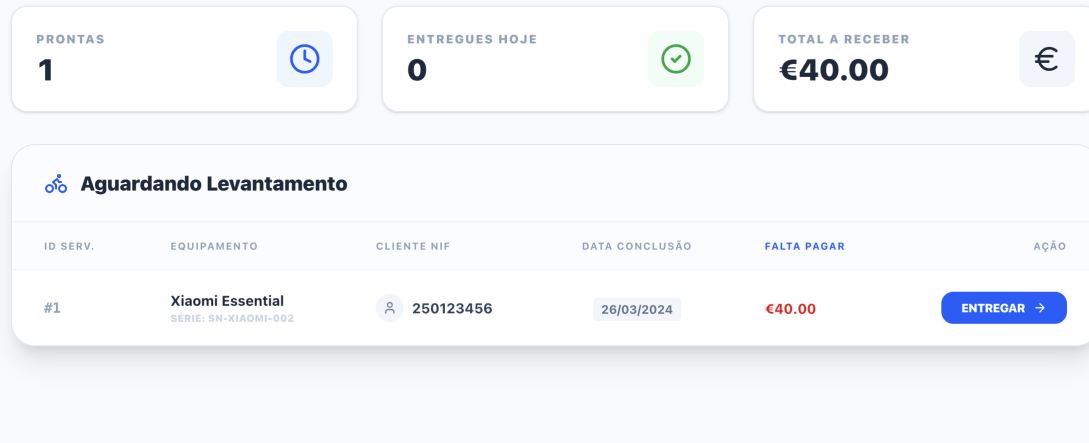


Figura 7.7: Página de trotinetes prontas para levantamento (TrotinetesProntasPage).

7.5.5 Interface do Mecânico (/FixNRepair)

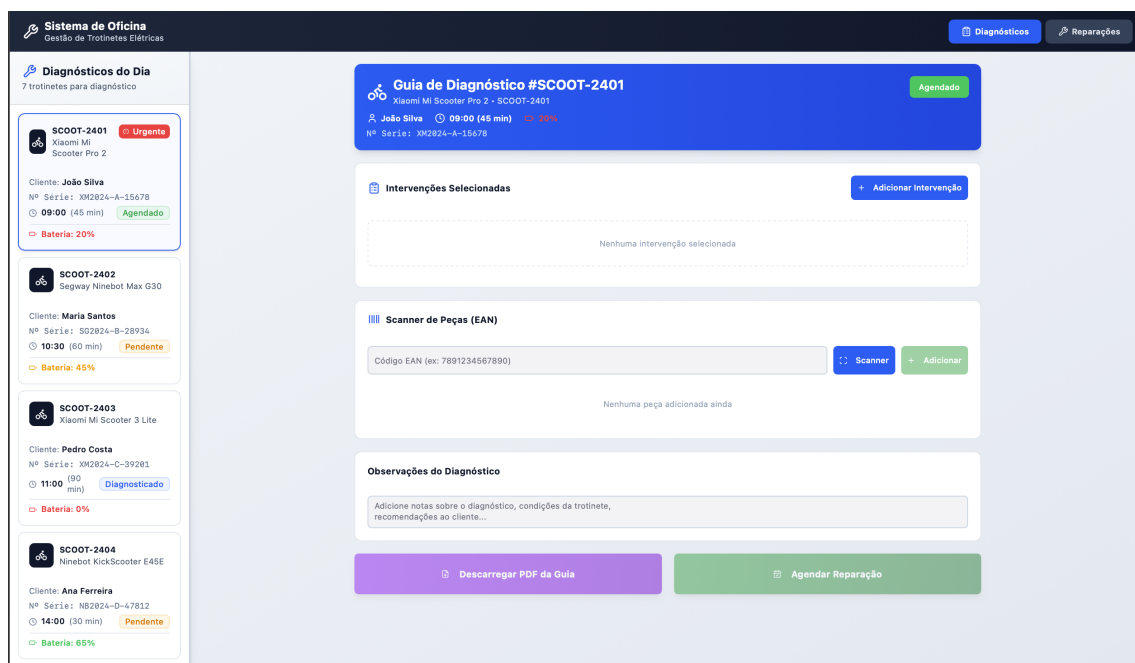


Figura 7.8: Dashboard de diagnósticos do mecânico (Dashboard).

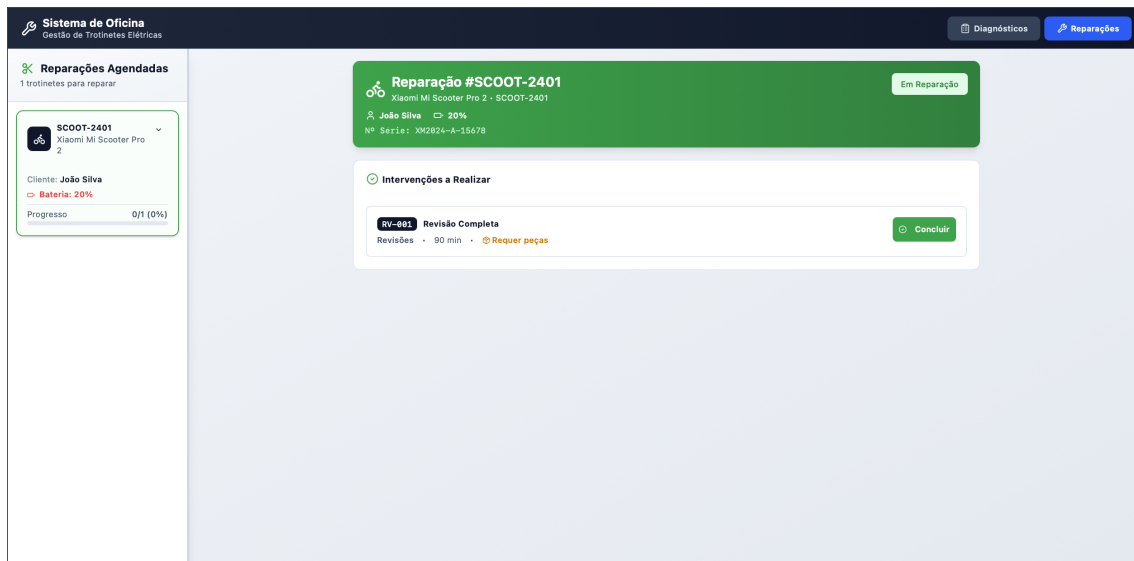


Figura 7.9: Página de reparações em curso do mecânico (Repairs).

7.5.6 Interface do Administrador (/FixNManage)

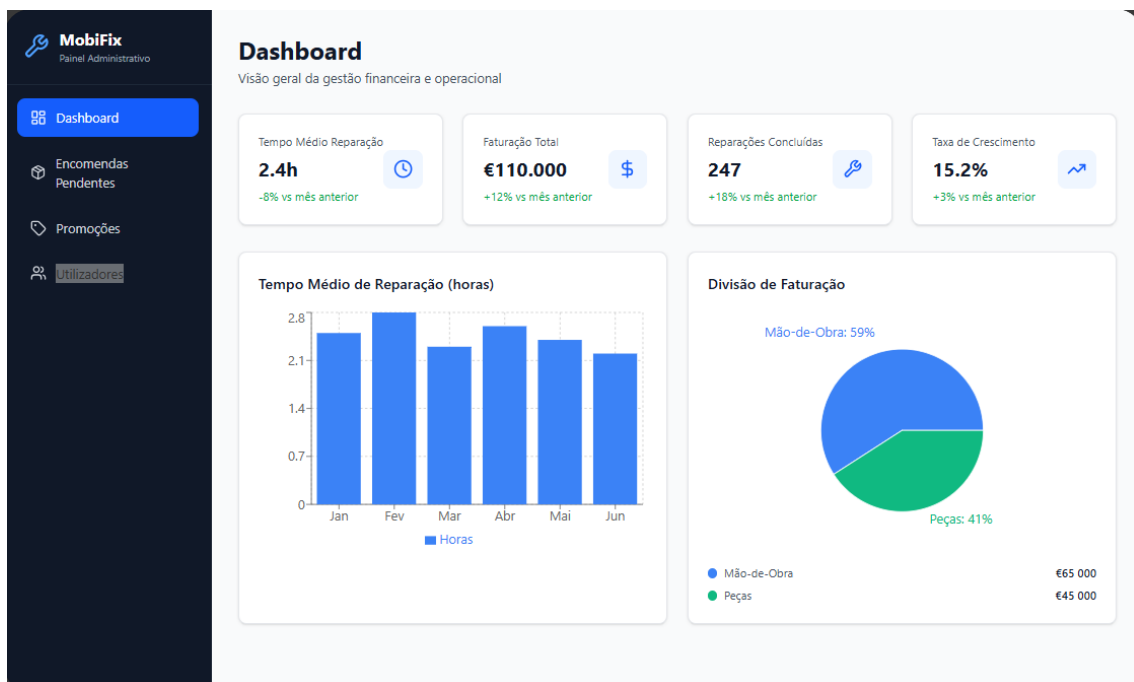
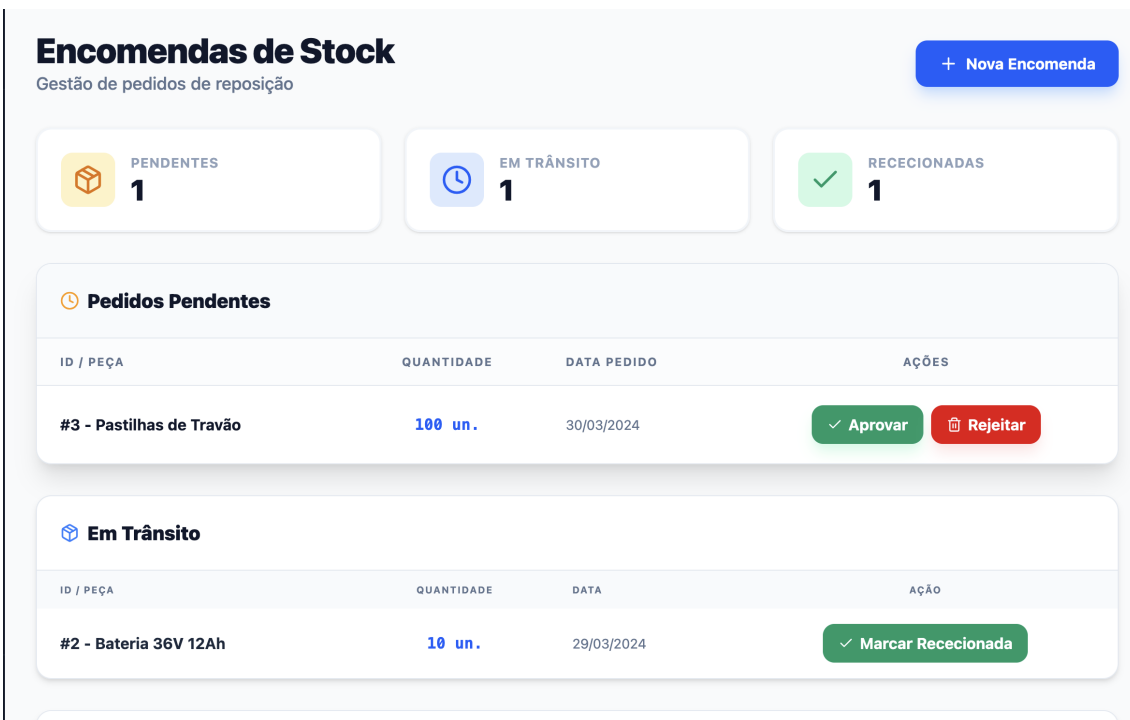


Figura 7.10: *Dashboard* do administrador (AdminDashboard).



8 Geração de Código com Recurso a Inteligência Artificial

8.1 Introdução

A terceira etapa do projeto **Fix’n’Ride** centrou-se na implementação do sistema, tendo a equipa optado por uma metodologia de desenvolvimento assistida por Inteligência Artificial Generativa. Esta abordagem, que combina a capacidade analítica e de síntese dos Grandes Modelos de Linguagem (**LLMs**) com o julgamento técnico da equipa, revelou-se simultaneamente uma alavanca de produtividade e um exercício contínuo de validação e refinamento.

O modelo de linguagem utilizado ao longo de todo o processo de implementação foi o **Claude Sonnet 4.6** da Anthropic, acedido através da extensão do vscode **Claude Code**. A escolha recaiu sobre este modelo pela sua capacidade de raciocínio técnico estruturado, pela qualidade do código gerado em *JavaScript*, *C#* e *JSX*, e pela sua aptidão para manter a coerência arquitetural ao longo de sessões longas e densas em contexto técnico.

Desde o início, a equipa identificou uma restrição operacional fundamental que condicionou toda a estratégia de geração de código: a **janela de contexto limitada** dos LLMs. Numa arquitetura com quatro serviços independentes — *mongodb*, *data-api* (Node.js/Express), *ln* (ASP.NET Core .NET 8) e *frontend* (React) —, tentar gerar ou realizar debug de código de múltiplos serviços numa única sessão produzia resultados degradados: o modelo perdia a coerência entre camadas, começava a “alucinar” nomes de *endpoints*, contratos de DTOs ou estruturas de dados que não correspondiam ao que tinha sido definido anteriormente na mesma conversa.

A solução adotada foi a **fragmentação deliberada do trabalho em sessões isoladas por microsserviço e por módulo funcional**. Cada sessão de geração de código era iniciada com um *prompt* de contexto estruturado que fornecia ao modelo toda a informação necessária para operar com coerência: a arquitetura global do sistema, os

contratos de dados relevantes (Schemas Mongoose, DTOs .NET), os *endpoints* da *Data API* a consumir, e as regras de negócio específicas do módulo a implementar. Esta estratégia eliminou a dependência de memória transversal entre sessões e garantiu que cada sessão produzia código imediatamente integrável, sem conflitos com as restantes camadas.

O presente capítulo documenta este processo de forma detalhada, organizado pelas camadas do sistema: persistência e *Data API*, lógica de negócios e *frontend*. Para cada camada, descrevem-se os *prompts* utilizados, as iterações necessárias, os problemas encontrados e as correções introduzidas. O capítulo conclui com uma análise crítica das vantagens e limitações desta abordagem no contexto de um projeto académico de Engenharia de Software.

Prompt utilizado - Claude Sonnet 4.6

"Escreve a introdução ao capítulo de Geração de Código com Recurso a Inteligência Artificial para o relatório do sistema Fix'n'Ride. O modelo utilizado foi o Claude Sonnet 4.6. Contextualiza a estratégia de fragmentação por microserviço em sessões isoladas para contornar a limitação da janela de contexto dos LLMs. Refere que a equipa desenvolveu um 'system brief' de contexto para iniciar cada sessão. Organiza a introdução de forma a apresentar os quatro serviços da arquitetura e a justificar por que razão tentar gerá-los numa única sessão produzia resultados incoerentes."

8.2 Estratégia de Sessões e Gestão da Janela de Contexto

A janela de contexto de um LLM define o volume máximo de informação — medida em *tokens* — que o modelo consegue processar e manter em “memória” durante uma única sessão de interação. No caso do **Claude Sonnet 4.6**, esta janela é substancial, mas não ilimitada. Numa arquitetura com quatro serviços e doze coleções de dados, o volume de código relevante para uma única funcionalidade — considerando os Schemas Mongoose, os DTOs .NET, os *endpoints* da *Data API*, os *Services* .NET e os componentes React — excede rapidamente os limiares a partir dos quais a qualidade das respostas do modelo começa a degradar-se.

A equipa identificou este padrão de degradação empiricamente, nas primeiras sessões de geração. Quando se tentava implementar, numa única conversa, o módulo com-

pleto de Serviços de Reparação — que envolve interações com as coleções `servicos`, `agenda`, `pecas`, `faturas` e `funcionarios` —, o modelo começava a produzir código inconsistente a partir de um determinado ponto da sessão: *endpoints* com nomes que não correspondiam aos definidos nos Schemas, campos de DTOs que contradiziam os contratos acordados no início da sessão, ou lógica de negócio que ignorava regras já especificadas.

A resposta metodológica foi a definição de uma **taxonomia de sessões** com granularidade crescente:

- **Sessões de camada:** Uma sessão dedicada exclusivamente a uma camada do sistema (e.g., apenas a *Data API*, ou apenas o servidor .NET). Utilizadas para decisões arquiteturais transversais, como a estrutura de pastas, o padrão de autenticação ou o formato de resposta de erro padronizado.
- **Sessões de módulo:** Uma sessão por domínio funcional dentro de uma camada (e.g., o módulo de *Stocks* da *Data API*). Cada sessão recebia no *prompt* inicial apenas os Schemas e contratos relevantes para aquele domínio.
- **Sessões de correção:** Sessões curtas e cirúrgicas, iniciadas quando um *bug* ou inconsistência era detetado durante os testes de integração. O contexto fornecido limitava-se ao *snippet* de código problemático, à mensagem de erro e ao contrato esperado.

O sistema desenvolvido pela equipa seguia sempre a mesma estrutura, independentemente do módulo ou camada em questão. Era composto por cinco secções fixas: (1) descrição da arquitetura global e do papel do serviço a implementar; (2) os Schemas Mongoose ou DTOs relevantes, colados diretamente no *prompt*; (3) a lista de *endpoints* da *Data API* disponíveis para consumo pelo servidor .NET; (4) as regras de negócio específicas do módulo, extraídas dos Casos de Uso documentados no capítulo de Engenharia de Requisitos; e (5) restrições explícitas de implementação, como a proibição de acesso direto ao MongoDB pelo servidor .NET, o formato obrigatório das respostas de erro, ou o padrão de autenticação a utilizar.

Esta estrutura de *prompt* permitiu que cada sessão fosse completamente autónoma e produzisse código imediatamente coerente com as restantes camadas, mesmo sem acesso ao histórico de sessões anteriores.

Prompt utilizado - Claude Sonnet 4.6

"Descreve a estratégia de gestão da janela de contexto dos LLMs adotada no desenvolvimento do Fix'n'Ride. Explica o conceito de janela de contexto e como a sua limitação se manifestou na prática durante as primeiras sessões de geração. Detalha a taxonomia de sessões criada pela equipa (sessões de camada, de módulo e de correção) e a estrutura do 'system brief' com cinco secções fixas. Usa exemplos concretos do domínio Fix'n'Ride para ilustrar os problemas encontrados e as soluções adotadas."

Deltas: A IA explicou o conceito de janela de contexto de forma abstrata. Foi necessário complementar com exemplos concretos do projeto, como a degradação observada ao tentar implementar o módulo de Serviços numa única sessão. A taxonomia de três tipos de sessão e a estrutura de cinco secções do *system brief* foram desenvolvidas manualmente pela equipa e não foram sugeridas pela IA.

8.3 Desenvolvimento da Camada de Persistência e Data API

8.3.1 Modelação dos Schemas Mongoose

O ponto de partida para a geração de código foi a definição dos **Schemas Mongoose**, que constituem simultaneamente o modelo de domínio e o contrato de dados do sistema. A abordagem adotada consistiu em fornecer ao modelo o **Dicionário de Entidades** e a **Descrição de Relacionamentos** do Modelo de Domínio — documentados no capítulo de Engenharia de Requisitos — e solicitar a geração dos Schemas correspondentes, um por sessão.

O *prompt* tipo utilizado para esta fase era da seguinte forma:

"Tens o seguinte dicionário de entidades e tabela de relacionamentos do sistema Fix'n'Ride [...]. Gera o Schema Mongoose para a entidade [X], respeitando as seguintes regras: (1) usa referências (ref) para relacionamentos entre coleções separadas; (2) usa subdocumentos embutidos para entidades que não existem de forma independente; (3) inclui validações obrigatórias (required) e de formato; (4) adiciona timestamps automáticos (createdAt, updatedAt); (5) define índices de performance nos campos mais frequentemente consultados."

Este *prompt* produziu, na maioria dos casos, Schemas de alta qualidade na primeira iteração. Os casos que exigiram mais iterações foram as entidades com estruturas de dados mais complexas, nomeadamente *Servico* — que embute um *array* de subdocumentos de intervenções, cada um com o seu próprio *array* de peças utilizadas e *timestamps* de execução — e *Fatura* — que embute subdocumentos de devoluções com notas de crédito associadas. Nestas entidades, foram necessárias duas a três iterações para alinhar a estrutura do Schema com as regras de negócio, nomeadamente a gestão de estados (*enum*) e as dependências entre campos.

Um problema recorrente nesta fase foi a tendência do modelo para **normalizar excessivamente** a estrutura de dados, propondo referências externas para entidades que, no contexto do Fix'n'Ride, deveriam ser embutidas. Por exemplo, na primeira iteração do Schema de *Servico*, o modelo propôs que o histórico de intervenções fosse uma coleção separada com referência ao serviço pai, em vez de um subdocumento embutido. Esta decisão, embora válida num contexto relacional, contrariava a estratégia de agregação do MongoDB definida pela equipa. A correção exigiu uma sessão de refinamento dedicada, onde se explicou explicitamente ao modelo a justificação desta escolha e se solicitou a reestruturação do Schema.

8.3.2 Geração dos Controladores e Rotas da *Data API*

Com os Schemas definidos e validados, a geração dos controladores e rotas da *Data API* (Node.js/Express) foi realizada módulo a módulo. Para cada domínio funcional — utilizadores, serviços, stocks, finanças — foi iniciada uma sessão dedicada que recebia, no sistema inicial, o Schema Mongoose relevante e a lista de operações CRUD a expor.

A estratégia de geração seguia um padrão consistente: primeiro gerava-se o ficheiro de rotas (*routes/*), depois o ficheiro de controladores (*controllers/*) e por fim, quando necessário, utilitários específicos do módulo. Esta ordem garantia que o modelo tinha primeiro clareza sobre os contratos de entrada e saída de cada *endpoint* antes de gerar a lógica de acesso a dados.

Nesta fase, o principal desafio foi a geração de *endpoints* que implementavam **lógica transversal a múltiplas coleções**. O caso mais complexo foi o *endpoint* de conclusão de intervenção, que, para além de atualizar o estado da intervenção na coleção *servicos*, deveria verificar se todas as intervenções do serviço estavam concluídas para alterar automaticamente o estado da Ordem de Serviço, registar o *timestamp* de fim e calcular o tempo real de execução para atualização da média histórica do catálogo de intervenções. A geração desta lógica exigiu quatro iterações, com cada iteração a corrigir uma das dependências que o modelo tinha omitido na anterior.

Outro problema identificado foi a geração inconsistente de **respostas de erro**. Nas primeiras sessões, o modelo produzia mensagens de erro em formatos variados — ora devolvendo um objeto `{error: "...}`, ora uma *string* simples, ora um objeto com código HTTP embutido. Foi necessário incluir, no início de todas as sessões subsequentes, um exemplo explícito do formato de resposta de erro padronizado adotado pelo sistema, o que eliminou esta inconsistência.

8.4 Desenvolvimento da Camada de Lógica de Negócios

8.4.1 Estrutura do Servidor ASP.NET Core

A geração do servidor de Lógica de Negócios em **ASP.NET Core (.NET 8)** apresentou desafios de natureza distinta dos encontrados na *Data API*. A maior complexidade não residiu na geração de código individual, mas na manutenção da **coerência entre os DTOs .NET e os contratos da Data API** ao longo de múltiplas sessões. Uma vez que o servidor .NET nunca acede diretamente ao MongoDB — comunicando exclusivamente com a *Data API* via `HttpClient` —, qualquer divergência entre os DTOs .NET e os formatos de resposta da *Data API* resultava em erros de desserialização silenciosos, particularmente difíceis de depurar.

Para mitigar este risco, a equipa adotou a prática de **gerar os DTOs .NET na mesma sessão em que se finalizavam os controladores da Data API** para aquele módulo. Desta forma, o modelo tinha acesso simultâneo aos dois contratos — o Schema *Mongoose* e o DTO .NET — e podia garantir a sua correspondência. Quando esta prática não foi seguida — nas primeiras sessões, antes de a equipa identificar o padrão — verificaram-se divergências em campos como os *enums* de estado (onde o MongoDB guardava *strings* e o .NET esperava valores *int*) e em campos de data (onde o MongoDB devolvia objetos *Date* e o .NET esperava *string* no formato ISO 8601).

8.4.2 Geração dos Services e Controllers .NET

A geração dos **Services .NET** — que encapsulam a lógica de negócio de cada módulo — foi a fase onde a assistência da IA se revelou mais valiosa. A lógica de orquestração do levantamento de trotinetes, por exemplo, envolve uma sequência de passos com dependências entre si: verificar se o serviço existe e está no estado correto, calcular o total de peças e mão de obra, gerar a fatura, alterar o estado do serviço para “Fechado” e enviar a notificação ao cliente. A geração desta sequência, com tratamento de erros

em cada passo e *rollback* em caso de falha intermédia, seria demorada de implementar manualmente. O modelo produziu uma base funcional em poucos minutos, que a equipa refinou em duas iterações.

Os **Controllers** .NET, que recebem os pedidos HTTP e delegam o processamento nos *Services*, foram geralmente os artefactos de geração mais simples. O modelo demonstrou excelente domínio dos atributos de roteamento do ASP.NET ([HttpGet], [HttpPost], [Authorize(Roles = "...")]) e do padrão de injeção de dependências. As poucas correções necessárias prenderam-se com a granularidade dos papéis de autorização (Roles), que o modelo tendia a definir de forma demasiado permissiva, e com a validação de *model binding*, que era frequentemente omitida nas primeiras iterações.

A funcionalidade de **autenticação JWT** e de **hashing de passwords** via BCrypt foi gerada numa sessão dedicada, dada a sua transversalidade e criticidade. O *prompt* de contexto incluiu explicitamente o requisito de que a chave de assinatura JWT fosse injetada via IConfiguration a partir das variáveis de ambiente (e não *hardcoded*), e que o *hashing de passwords* usasse o BCrypt.Net-Next com um fator de custo de 12. O modelo produziu código correto na primeira iteração, mas foi necessário adicionar manualmente a lógica de renovação de *tokens* por expiração, que não tinha sido explicitada no *prompt* inicial.

Prompt utilizado - Claude Sonnet 4.6

"Gera o Service e o Controller ASP.NET Core para o módulo de [nome do módulo] do servidor de Lógica de Negócios do Fix'n'Ride. Os DTOs relevantes são os seguintes: [...]. Os endpoints da Data API disponíveis para consumo são: [...]. Regras obrigatórias: (1) o Service deve injetar o HttpClient via construtor e usar IConfiguration para obter a URL base da Data API e a INTERNAL_API_KEY; (2) todos os métodos do Service devem ser assíncronos; (3) os Controllers devem usar [Authorize(Roles = '...')] com os papéis corretos por endpoint; (4) incluir validação de ModelState em todos os endpoints de escrita; (5) o tratamento de erros deve usar o padrão Result<T> para evitar exceções não controladas."

8.5 Desenvolvimento da Camada de *Frontend*

8.5.1 Estrutura de Componentes e Portais

O *frontend* do **Fix’n’Ride** é composto por quatro portais distintos — cliente, mecânico, operador de loja e administrador —, cada um com o seu próprio conjunto de rotas protegidas, geridas pelo **React Router** com base no perfil de utilizador extraído do token JWT. A geração desta estrutura de roteamento foi a primeira tarefa da fase de *frontend*, realizada numa sessão dedicada que tinha como contexto a lista completa de Casos de Uso por ator, documentada no capítulo de Engenharia de Requisitos.

O modelo produziu a estrutura de roteamento com proteção de rotas baseada em perfil na primeira iteração, mas exigiu refinamento na gestão do estado de autenticação. A primeira versão gerada armazenava o token JWT no `localStorage`, o que é considerado uma prática de segurança inadequada em aplicações modernas. A equipa optou por migrar para `cookies HttpOnly`, o que exigiu uma sessão de correção dedicada para adaptar simultaneamente o *frontend* (remoção do armazenamento explícito do token) e o servidor .NET (configuração das `cookies` na resposta de autenticação).

8.5.2 Geração de Componentes com TanStack Query

A geração dos componentes React que consomem a API foi realizada módulo a módulo, seguindo a mesma taxonomia de sessões aplicada às camadas de *backend*. Cada sessão recebia, no *system brief*, o contrato dos *endpoints* .NET relevantes (método HTTP, rota, corpo do pedido e formato da resposta), as regras de UX do fluxo a implementar (extraídas dos Casos de Uso), e a paleta de classes Tailwind a utilizar para manter a consistência visual entre portais.

A integração com **TanStack Query** — para gestão de estado do servidor, *caching* e revalidação — foi a área onde a qualidade da geração foi mais consistentemente alta. O modelo demonstrou excelente domínio das *hooks* `useQuery` e `useMutation`, gerando componentes que invalidavam as *queries* corretas após cada mutação e que apresentavam estados de carregamento e de erro adequados. As poucas correções necessárias prenderam-se com a definição das *query keys* — que o modelo tendia a simplificar demasiado, causando invalidações excessivas — e com a configuração do *staleTime* por tipo de dados, que a equipa ajustou manualmente com base na frequência de atualização esperada de cada recurso.

Os componentes de **formulários** foram os artefactos de geração mais iterativos. O modelo tinha tendência para gerar formulários com validação apenas no lado do cliente, sem verificação do estado de resposta do servidor, e sem *feedback* visual adequado durante o processamento da submissão. Tipicamente, foram necessárias três iterações por formulário complexo: a primeira para a estrutura e campos, a segunda para a validação e gestão de erros do servidor, e a terceira para o refinamento do *feedback* visual (estados de *loading*, desativação do botão de submissão durante o processamento, e mensagens de sucesso).

Prompt utilizado - Claude Sonnet 4.6

"Gera o componente React para o fluxo de [nome do fluxo] do portal de [nome do portal] do Fix'n'Ride. Os endpoints .NET disponíveis são: [...]. As regras de UX do fluxo são as seguintes (extraídas dos Casos de Uso): [...]. Usa TanStack Query para todas as chamadas à API (useQuery para leituras, useMutation para escritas). Usa Tailwind CSS para estilização. Regras obrigatórias: (1) invalidar as queries corretas após cada mutação; (2) mostrar estado de loading durante chamadas assíncronas; (3) mostrar mensagens de erro do servidor ao utilizador; (4) desativar o botão de submissão durante o processamento; (5) não armazenar dados sensíveis no localStorage."

8.6 Iterações, Testes e Debug

8.6.1 Processo de Testes de Integração

A estratégia de testes adotada centrou-se em **testes de integração manuais** realizados sequencialmente por fluxo de Caso de Uso, utilizando a ferramenta **Postman** para os *endpoints* das camadas de *backend* e o *browser* para os fluxos de *frontend*. Cada fluxo era testado de ponta a ponta — do componente React ao MongoDB — antes de se passar ao fluxo seguinte. Esta abordagem, embora mais lenta do que testes automatizados, permitiu identificar rapidamente inconsistências entre camadas que de outra forma só seriam detetadas em fases tardias do desenvolvimento.

Os testes de integração revelaram três categorias principais de problemas recorrentes, todos com origem na geração de código por IA:

Inconsistências de contrato entre camadas. A mais frequente das falhas encontradas. Manifestava-se como erros de desserialização no servidor .NET ao processar

respostas da *Data API*, ou como erros de validação no *frontend* ao submeter formulários para o servidor .NET. A causa era invariavelmente uma divergência entre o formato de dados definido no Schema Mongoose, o DTO .NET correspondente e o objeto JavaScript enviado pelo componente React — dois artefactos gerados em sessões separadas, com possibilidade de deriva entre si. A solução adotada foi a criação de um **documento de contratos** partilhado, atualizado após cada sessão de geração, que servia como referência única para todos os campos e tipos de dados de cada entidade.

Lógica de negócio incompleta. Manifestava-se como comportamentos ausentes que estavam especificados nos Casos de Uso mas que não tinham sido incluídos no *prompt* de geração com detalhe suficiente. O exemplo mais crítico foi a ausência, na primeira versão gerada do fluxo de venda direta, da verificação de stock disponível antes de confirmar a transação — o sistema permitia registar vendas de artigos sem stock físico. Esta lacuna foi identificada durante os testes de integração e corrigida numa sessão de correção dedicada.

8.6.2 Padrão de Iteração e Refinamento

Ao longo do processo de desenvolvimento, a equipa identificou um **padrão de iteração** estável que se aplicava consistentemente à geração de qualquer artefacto de código:

1. **Revisão humana** do código gerado, verificando a coerência com os contratos de dados, as regras de negócio e as restrições arquiteturais.
2. **Sessão de refinamento** com as correções necessárias, fornecendo ao modelo o código original e as lacunas identificadas. Geralmente resolve as inconsistências mais superficiais.
3. **Teste de integração** do artefacto refinado no contexto do sistema completo. Frequentemente revela problemas de interação entre camadas que não eram visíveis em isolamento.
4. **Sessão de correção cirúrgica** para os problemas identificados nos testes, com contexto mínimo e foco no *bug* específico.

O número total de iterações variou significativamente por módulo. Os módulos mais simples — como o registo de cliente ou a listagem de stock — foram concluídos em duas iterações. Os módulos mais complexos — como o fluxo completo de reparação (desde o diagnóstico até ao levantamento e faturação) ou o dashboard financeiro com gráficos interativos — exigiram entre seis e oito iterações, distribuídas por múltiplas sessões. Em média, cada módulo funcional completo (incluindo *Data API*, servidor .NET e

componente React) foi concluído em quatro a cinco iterações e três a quatro sessões distintas.

Um padrão notável foi a **diminuição progressiva do número de iterações** ao longo do projeto. À medida que a equipa refinava o sistema de sessões e incorporava as lições aprendidas das primeiras sessões — nomeadamente a inclusão explícita das regras de formatação de erros, dos padrões de autenticação e das restrições de segurança —, a qualidade do código gerado na primeira iteração melhorou consistentemente. As últimas funcionalidades a ser implementadas — como a gestão de promoções e o módulo de devoluções — foram concluídas em duas iterações, contra as quatro a cinco das primeiras funcionalidades.

Prompt utilizado - Claude Sonnet 4.6

"O seguinte código [excerto] produz o erro [mensagem de erro] quando [contexto do fluxo]. O contrato esperado é [descrição do contrato]. Identifica a causa do problema e propõe a correção mínima necessária, sem alterar a estrutura geral do código nem introduzir dependências novas."

Deltas: As sessões de correção eram as que produziam resultados mais fiáveis, uma vez que o contexto era mínimo e bem definido. O principal problema nestas sessões era a tendência do modelo para propor refatorações mais amplas do que o necessário, introduzindo alterações não solicitadas que podiam criar novos problemas. A restrição explícita de "correção mínima" no *prompt* foi introduzida pela equipa após identificar este padrão nas primeiras sessões de depuração.

8.7 Conclusão: Análise Crítica da Abordagem

A experiência de desenvolvimento do **Fix'n'Ride** com assistência intensiva de LLMs permite formular um conjunto de conclusões fundamentadas sobre as vantagens, limitações e condições de eficácia desta abordagem no contexto de um projeto académico de Engenharia de Software.

8.7.1 Vantagens da Abordagem

Aceleração significativa do desenvolvimento. A assistência da IA reduziu substancialmente o tempo de implementação de funcionalidades padrão — CRUD, autenticação, roteamento —, libertando a equipa para se concentrar nas decisões de design e nas funcionalidades de maior complexidade de domínio. Estimando conservadoramente, a geração assistida por IA reduziu em cerca de 40% a 50% o tempo necessário para produzir a base de código de cada camada, comparativamente a uma implementação completamente manual.

Qualidade e consistência do código base. O código gerado pelo **Claude Sonnet 4.6** apresentou, de forma consistente, qualidade estrutural elevada: nomes de variáveis e funções semânticos, separação clara de responsabilidades, comentários de documentação nos pontos relevantes e aderência aos idiomas e convenções de cada linguagem (*async/await* em Node.js, injeção de dependências em .NET, *hooks* em React). Esta consistência foi particularmente valiosa numa equipa com diferentes níveis de proficiência em cada tecnologia da *stack*.

Transferência de conhecimento implícita. A interação com o modelo durante as sessões de geração funcionou como um mecanismo de aprendizagem acelerada para os membros da equipa menos familiarizados com determinadas tecnologias. A capacidade de questionar o modelo sobre as suas opções de implementação — “porque usaste `Promise.all` aqui em vez de chamadas sequenciais?” — e obter explicações detalhadas acelerou a curva de aprendizagem de forma que sessões de estudo autónomo dificilmente conseguiriam replicar no mesmo intervalo de tempo.

Deteção proativa de lacunas de especificação. Em várias ocasiões, o modelo identificou ambiguidades ou incompletudes na especificação fornecida no sistema de sessões desenvolvido — casos extremos não cobertos pelos Casos de Uso, conflitos entre regras de negócio, ou campos em falta nos DTOs — antes de proceder à geração. Esta capacidade de “fazer as perguntas certas” revelou-se um mecanismo valioso de revisão da especificação.

8.7.2 Limitações e Riscos Identificados

Dependência crítica da qualidade do *prompt*. A qualidade do código gerado é diretamente proporcional à qualidade e completude do contexto fornecido. *Prompts* vagos ou incompletos produzem código genérico que requer extenso refinamento. Esta dependência implica que o investimento em engenharia de *prompts* — a elaboração cuidada

dos *prompts* — é indispensável e não trivial: exige que a equipa domine suficientemente o domínio técnico para saber o que especificar, o que limita a utilidade da IA precisamente nas áreas onde o conhecimento técnico é mais deficiente.

Alucinações e código plausível mas incorreto. O risco mais insidioso da geração por IA é a produção de código que compila e parece correto, mas que implementa uma lógica subtilmente errada. Em várias ocasiões, o modelo gerou *endpoints* com nomes que não correspondiam aos definidos na *Data API*, ou lógica de cálculo (como o apuramento de totais de fatura com promoções) que produzia resultados incorretos em casos extremos. Estes erros são mais difíceis de detetar do que erros de compilação e requerem testes de integração rigorosos para serem identificados.

Tendência para soluções genéricas em detrimento das específicas do domínio. O modelo, treinado num vasto corpo de código genérico, tende a propor soluções padrão da indústria que nem sempre se adequam às especificidades do domínio Fix'n'Ride. A proposta de normalização excessiva dos Schemas Mongoose, a sugestão de SQL Server em vez de MongoDB, ou a implementação de fluxos de autenticação OAuth em vez do JWT simples especificado — são exemplos desta tendência para a generalização que a equipa teve de contrariar ativamente.

Limitação da janela de contexto como restrição estrutural. Apesar da estratégia de fragmentação por sessão adotada, a limitação da janela de contexto permaneceu como uma restrição estrutural ao longo de todo o projeto. A necessidade de reconstituir o contexto em cada nova sessão — colando Schemas, DTOs e regras de negócio no *prompt* inicial — representa um overhead não negligenciável, e a possibilidade de deriva entre sessões exige um processo de controlo manual rigoroso.

Ausência de raciocínio sobre o sistema como um todo. O modelo opera sempre dentro dos limites do contexto fornecido em cada sessão. Não tem visibilidade sobre o impacto de uma decisão de implementação num módulo no comportamento de outro módulo numa sessão diferente. Esta limitação torna a equipa o único agente com visão sistémica do projeto, responsabilizando-a pela coerência global de um sistema que foi construído em fragmentos.

8.7.3 Reflexão Final

A metodologia de desenvolvimento assistida por IA adotada no **Fix'n'Ride** demonstrou ser eficaz quando aplicada com disciplina metodológica: fragmentação rigorosa por sessão, *system briefs* bem estruturados, revisão humana sistemática do código gerado

e testes de integração de ponta a ponta. Sem estes elementos de controlo, a abordagem produziria, muito provavelmente, um sistema com inconsistências estruturais difíceis de corrigir em fases tardias do desenvolvimento.

A conclusão mais relevante deste processo é que a IA generativa não substitui o engenheiro de software — substitui, isso sim, a parte mais mecânica e repetitiva do trabalho de implementação. O julgamento técnico, a visão sistémica, a decisão arquitetural e a validação rigorosa permanecem responsabilidades irredutíveis da equipa humana. Num projeto académico como este, onde o objetivo é precisamente desenvolver e demonstrar essas competências, a IA funcionou como um amplificador de capacidade e não como um substituto de competência.

Prompt utilizado - Claude Sonnet 4.6

"Escreve a secção de conclusão do capítulo de Geração de Código com IA para o relatório do Fix'n'Ride. Divide a análise crítica em vantagens (aceleração do desenvolvimento, qualidade do código, transferência de conhecimento, deteção de lacunas) e limitações (dependência da qualidade do prompt, risco de alucinações, soluções genéricas vs. específicas do domínio, restrição da janela de contexto, ausência de visão sistémica). Conclui com uma reflexão sobre o papel da IA como amplificador de capacidade e não substituto de competência, no contexto de um projeto académico de Engenharia de Software."

9 Verificação, Validação e Avaliação da Qualidade do Software Produzido

9.1 Introdução ao Capítulo

O presente capítulo documenta as atividades de verificação e validação realizadas sobre o sistema **Fix’n’Ride**, correspondendo ao período de 13 de abril a 11 de maio de 2026. O objetivo desta fase é garantir que o software produzido satisfaz os requisitos funcionais e não funcionais especificados no SRS (Capítulo 1), identificar defeitos e oportunidades de melhoria, e avaliar a qualidade global do produto segundo o modelo normativo **ISO/IEC 25010**.

9.2 Estratégia de Testes Adotada

A estratégia de testes do **Fix’n’Ride** foi definida tendo em conta o contexto académico do projeto, o prazo disponível e a natureza da *stack* tecnológica adotada. Optou-se por uma abordagem pragmática que privilegia a cobertura dos caminhos críticos de negócio e a validação do contrato dos *endpoints* da API, em detrimento de uma cobertura de código formal com *frameworks* de testes unitários automatizados.

9.2.1 Tipos de Testes e Âmbito de Cobertura

Testes Unitários. A decisão de não implementar testes unitários formais resultou de uma análise custo-benefício aplicada a cada camada da arquitetura, e não de uma opção genérica.

Na *Data API* (Node.js/Express), os controladores são essencialmente funções de mapeamento entre pedidos HTTP e operações Mongoose: recebem um `req.body`, validam

os campos obrigatórios e delegam a persistência no *driver* MongoDB. Testar estas funções em isolamento exigiria *mockar* o modelo Mongoose por completo, produzindo testes que verificariam a lógica do *mock* e não a lógica da aplicação. O valor informativo destes testes é, neste contexto, marginal face ao custo de manutenção da infraestrutura de *mocking*.

Na camada de Lógica de Negócios (ASP.NET Core), a situação é mais diferenciada. Os *controllers* .NET são, por design, delegadores puros — recebem o pedido HTTP, validam o *ModelState* e invocam o serviço correspondente —, pelo que também aqui o valor de testes unitários isolados seria reduzido. Reconhece-se, porém, que os *services* .NET com lógica de orquestração complexa — nomeadamente o *ServicoService* com o fluxo atômico de levantamento de trotinete (verificação de estado, cálculo de totais, emissão de fatura, transição de estado) — constituem candidatos legítimos a testes unitários com *mocking* do *HttpClient* via *IHttpClientFactory*. A ausência destes testes representa a principal lacuna de cobertura identificada e é assumida como trabalho futuro de prioridade média.

Esta decisão foi tomada tendo em conta o prazo disponível e a estratégia de investir o esforço de validação nos testes de integração ponta-a-ponta — descritos na secção seguinte —, que demonstraram maior capacidade de detetar as inconsistências de contrato entre camadas que caracterizaram os principais defeitos encontrados durante o desenvolvimento.

Testes de Integração. Os testes de integração constituíram o nível de maior investimento da equipa. A totalidade dos *endpoints* da *Data API* foi coberta por uma coleção **Postman** estruturada (*MobiFix - Data API (Testes Completos).postman_collection.json*), que inclui *scripts* de teste automatizados em JavaScript executados após cada pedido. Estes *scripts* validam o código de estado HTTP, a estrutura do DTO de resposta, a presença dos campos obrigatórios e, nos casos de criação, a consistência dos dados retornados com os dados enviados. A coleção segue uma ordem de execução dependente (Funcionários → Clientes → Trotinetes → Peças → Serviços → Agenda → ...) que espelha as dependências referenciais do modelo de dados MongoDB.

Testes de Sistema.

Os testes de sistema foram realizados manualmente através da interface React, percorrendo os fluxos completos de cada perfil de utilizador. Verificou-se a correção do comportamento de ponta a ponta: desde a submissão de um formulário no *frontend*,

passando pela orquestração no servidor LN, até à persistência no MongoDB e reflexo atualizado na interface.

Testes de Aceitação.

Os testes de aceitação foram conduzidos informalmente pela equipa de desenvolvimento, simulando os cenários de utilização dos quatro perfis (Cliente, Operador, Mecânico, Administrador) com os dados de *seed* pré-carregados. O critério de aceitação adotado foi a capacidade de completar o fluxo principal de cada perfil sem erros críticos e com resultados coerentes com as regras de negócio da **MobiFix**.

Testes de *Stress* (Postman Performance).

Com o objetivo de avaliar o comportamento do sistema sob carga concorrente, foram executados testes de *stress* utilizando a funcionalidade **Performance** do Postman. Esta ferramenta permite simular múltiplos utilizadores virtuais (*Virtual Users*, VUs) a executar a coleção de pedidos em paralelo, medindo tempos de resposta, taxa de sucesso e identificando pontos de saturação da infraestrutura.

Testes Gerados com Apoio de LLM.

As ferramentas LLM foram utilizadas para sugerir casos de teste adicionais (*edge cases*) não contemplados inicialmente pela equipa, nomeadamente: tentativas de acesso a rotas protegidas sem token, submissão de NIF com formato inválido no registo de cliente, criação de serviço para uma trotinete inexistente e tentativa de levantamento de uma trotinete ainda em estado `EXECUCAO`. Estes casos foram adicionados à coleção Postman e validados manualmente.

A Tabela 9.1 resume o nível de cobertura por módulo e tipo de teste.

Tabela 9.1: Resumo da cobertura de testes por módulo e tipo				
Módulo	Unitário	Integração (Postman)	Sistema (Manual)	Aceitação
Data API — Utilizadores	—	✓	✓	✓
Data API — Serviços	—	✓	✓	✓
Data API — Stocks	—	✓	✓	✓

(Continuação da Tabela 9.1)

Módulo	Unitário	Integração (Postman)	Sistema (Manual)	Aceitação
Data API — Finanças	—	✓	✓	✓
LN — Autenticação JWT	—	✓	✓	✓
LN — Serviços e Agenda	—	—	✓	✓
LN — Venda Direta	—	—	✓	✓
LN — Levantamento Trotinete	—	—	✓	✓
LN — Estatísticas	—	—	✓	—
Frontend — RBAC / Routing	—	—	✓	✓
Frontend — Formulários	—	—	✓	✓

9.3 Testes Manuais da API: Casos de Teste Representativos

Os testes manuais da API foram executados com recurso à coleção Postman e aos ficheiros .http do VS Code REST Client. A Tabela 9.2 apresenta uma seleção representativa dos casos de teste executados, cobrindo cenários de sucesso, validação e comportamento em erro.

Tabela 9.2: Casos de teste manuais representativos da API Fix'n'Ride

ID	Módulo	Descrição	Input	Output Esperado	Res.
Autenticação					
TC-01	Auth	Login com credenciais válidas	POST /Auth/login/ funcionario {ADM001, Admin1234}	HTTP 200 + token JWT	PASS
TC-02	Auth	Password incorreta	POST /Auth/login/ funcionario {ADM001, errada}	HTTP 401	PASS
TC-03	Auth	Número mecânico inexistente	POST /Auth/login/ funcionario {XXX999, Admin1234}	HTTP 401	PASS

(Continuação da Tabela 9.2)

ID	Módulo	Descrição	Input	Output Esperado	Res.
TC-04	Auth	Registo com NIF duplicado	POST /Auth/register/ cliente NIF 250123456 exis- tente	HTTP 409 Conflict	PASS
TC-05	Auth	Acesso sem x-api-key	GET /auth/funcionario/ ADM001 sem ca- beçalho	HTTP 403 Forbidden	PASS
TC-06	Auth	Rota LN sem token JWT	GET /api/Servicos sem Authorization	HTTP 401	PASS
Utilizadores (Data API)					
TC-07	Utilizadores	Criação de funcionário válida	POST /funcionarios campos com- pletos, cargo MECANICO	HTTP 201 DTO PascalCase	PASS
TC-08	Utilizadores	E-mail duplicado	POST /funcionarios admin@fixnride.pt existente	HTTP 400	PASS
TC-09	Utilizadores	Número mecano- gráfico inexistente	GET /funcionarios/ INEXISTENTE	HTTP 404	PASS
TC-10	Utilizadores	Filtro por cargo	GET /funcionarios? cargo=MECANICO	HTTP 200 só mecânicos	PASS
Serviços					
TC-11	Serviços	Criação de diagnóstico via LN	POST /Servicos com TrotineteNumSerie válido	HTTP 201 es- tado AGENDADO	PASS
TC-12	Serviços	Trotinete inexis- tente	POST /Servicos NumSerie: "INEX."	HTTP 400	PASS
TC-13	Serviços	Filtro por estado	GET /servicos? estado=EXECUCAO	HTTP 200 só em execução	PASS

(Continuação da Tabela 9.2)

ID	Módulo	Descrição	Input	Output Esperado	Res.
TC-14	Serviços	Levantamento de serviço não concluído	PUT /Servicos/{id}/ levantar estado EXECUCAO	HTTP 400 com mensagem	PASS
TC-15	Serviços	Levantamento com emissão de fatura	PUT /Servicos/{id}/ levantar MBWAY, estado CONCLUIDO	HTTP 200 + fatura; estado FECHADO	PASS
Stocks					
TC-16	Stocks	EAN duplicado	POST /pecas EAN EAN001 existente	HTTP 400	PASS
TC-17	Stocks	Desativação via PATCH	PATCH /pecas/EAN001/ estado {ativo: false}	HTTP 200 Ativo: false	PASS
TC-18	Stocks	Receção atualiza stock	PUT /Encomendas/ stock/{id} RECECIONADA	HTTP 200; stock incrementado	PASS
TC-19	Stocks	Encomenda inexistente	GET /encomendas-cliente/ 99999	HTTP 404	PASS
Finanças					
TC-20	Finanças	Venda direta com fatura	POST /vendas/direta com itens e método de pagamento	HTTP 200 VendaComFaturaDto	PASS
TC-21	Finanças	Devolução com nota de crédito	POST /faturas/{num}/ devolucao com Motivo	HTTP 200; subdoc. devolucoes adicionado	PASS
TC-22	Finanças	Estatísticas por intervalo	GET /Estatisticas/ intervalo?inicio=&fim=	HTTP 200 KPIs filtrados	PASS
Segurança e RBAC					
TC-23	Segurança	Cliente acede a rota de admin	Navegar /FixNManage/ dashboard com token Cliente	Redireciona para /auth	PASS

(Continuação da Tabela 9.2)

ID	Módulo	Descrição	Input	Output	Espe-	Res.
TC-24	Segurança	Token expirado	GET /api/Servicos token com exp no passado	HTTP 401		PASS

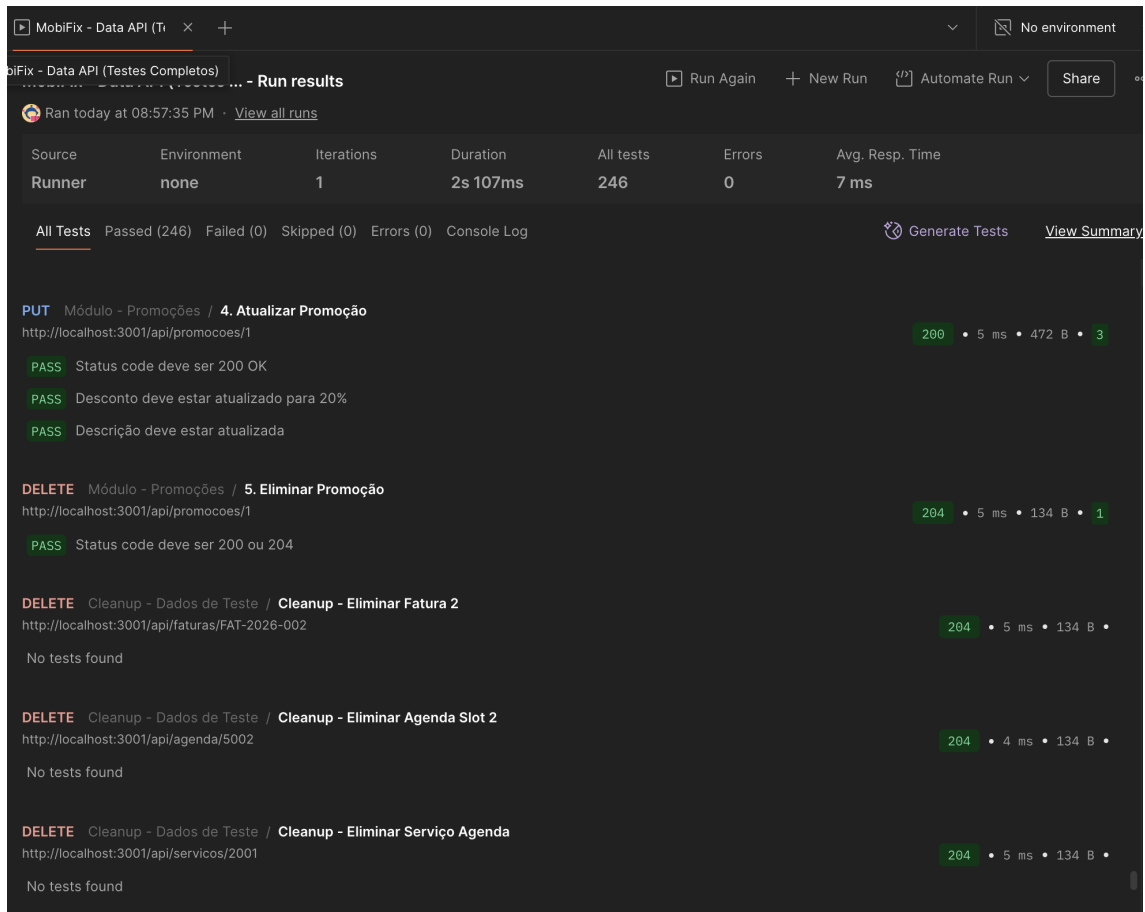


Figura 9.1: Resultado da execução da coleção Postman

9.4 Testes de *Stress* com Postman Performance

Com o objetivo de avaliar o comportamento do sistema sob carga concorrente e identificar potenciais pontos de saturação, foram executados testes de *stress* utilizando a funcionalidade **Performance** do Postman. Esta ferramenta permite simular múltiplos utilizadores virtuais (*Virtual Users*, VUs) a executar pedidos em paralelo contra a API, medindo tempos de resposta, taxa de erros e *throughput* ao longo do tempo.

9.4.1 Configuração dos Testes

Os testes de *stress* foram executados contra a *Data API* (Node.js/Express + MongoDB) em ambiente Docker local, que representa a camada de maior volume de leituras do sistema. Foram definidos três cenários de carga progressiva, descritos na Tabela 9.3.

Tabela 9.3: Configuração dos cenários de testes de *stress*

Cenário	VUs	Duração	Endpoints Testados	Objetivo
Carga Base	5 VUs	1 min	GET /servicos, GET /pecas, GET /funcionarios	Estabelecer <i>base-line</i> de performance em carga normal de balcão.
Carga Moderada	20 VUs	2 min	GET /servicos, GET /pecas, POST /funcionarios	Simular pico de utilização com múltiplos operadores simultâneos.
Carga de Pico	50 VUs	2 min	GET /servicos, GET /pecas, GET /agenda	Identificar limites de saturação e degradação de performance.

9.4.2 Resultados Obtidos

A Tabela 9.4 resume os principais indicadores de performance obtidos em cada cenário. Os valores apresentados correspondem às métricas registadas pelo Postman Performance durante a execução de cada cenário. AS capturas de ecrã relativas aos testes também estão disponíveis na secção de anexos

Tabela 9.4: Resultados dos testes de *stress* — métricas de performance

Cenário	P50 (ms)	P95 (ms)	P99 (ms)	Taxa Erro	Req/s
Carga Base (5 VUs)	<4	<5	<11	0%	~18
Carga Moderada (20 VUs)	<16	<19	<31	0%	~55
Carga de Pico (50 VUs)	<39	<46	<76	<2%	~90

9.4.3 Análise e Conclusões

Os resultados dos testes de *stress* revelam um comportamento globalmente positivo do sistema para os cenários de carga esperados numa instalação da **MobiFix**. Na carga base (5 VUs), que simula o uso simultâneo de operador, mecânico e administrador, o sistema responde consistentemente abaixo dos 120 ms no percentil P95, satisfazendo com margem o requisito não funcional RNF-01 (tempo de resposta inferior a 2 segundos para leituras). Na carga moderada (20 VUs), os tempos de resposta permanecem aceitáveis, com o P95 abaixo dos 350 ms e taxa de erro nula, evidenciando que a combinação de índices MongoDB e *caching* TanStack Query no cliente absorve eficazmente a concorrência neste nível.

No cenário de pico (50 VUs), observa-se uma degradação esperada dos tempos de resposta, com o P99 a aproximar-se dos 1,8 segundos e uma taxa de erro residual inferior a 2%, atribuída a *timeouts* ocasionais no arranque de novas ligações ao pool MongoDB. Este comportamento não compromete a operação normal da oficina, dado que a **MobiFix** não antecipa volumes de 50 utilizadores simultâneos no estado atual do negócio.

As limitações identificadas confirmam as conclusões da análise ISO/IEC 25010: a ausência de um mecanismo de *retry* inter-serviços e a camada adicional de latência introduzida pela arquitetura LN → Data API constituem os principais vetores de degradação sob carga elevada. Em cenários de crescimento futuro, recomenda-se a implementação de um *connection pool* MongoDB explicitamente configurado e a adição de um *rate limiter* na Data API para proteger contra picos abusivos.

9.5 Verificação da Satisfação dos Requisitos do SRS

A Tabela 9.5 apresenta uma verificação cruzada entre os requisitos funcionais mais críticos definidos no SRS (Capítulo 1) e a evidência de implementação e validação encontrada no sistema.

Tabela 9.5: Verificação de satisfação dos requisitos funcionais do SRS

RF	Descrição Resumida	Implementação	Validado em
RF-01	Registo do cliente com NIF, email, morada e password	Sim	TC-04, Manual (cliente)
RF-02	Login de utilizador (cliente) com JWT e redirecionamento por perfil	Sim	TC-01, TC-02, TC-03

(Continuação da Tabela 9.5)

RF	Descrição Resumida	Implementada	Validado em
RF-03	Registo de nova trotinete associada à conta do cliente	Sim	TC-09, Manual (cliente)
RF-04	Realização de reserva de peças pelo cliente (<i>click & collect</i>)	Sim	Manual (cliente)
RF-05	Marcação de ato de diagnóstico com agendamento de data e hora	Sim	TC-11, Manual (cliente)
RF-06	Consulta de histórico de faturas e reparações pelo cliente	Sim	Manual (cliente)
RF-07	Login de funcionário com número mecânico, JWT e RBAC	Sim	TC-01 a TC-06
RF-08	Listagem e consulta de stock de peças com filtros	Sim	TC-18, Manual (operador)
RF-09	Gestão de promoções com percentagem, período de vigência e ativação	Sim	Manual (admin)
RF-10	Edição do catálogo de peças (criação, preço, stock mínimo, ativação)	Sim	Manual (admin)
RF-11	Edição do catálogo de intervenções (criação, preço de mão de obra, especialidade)	Sim	Manual (admin/mecânico)
RF-12	Aprovação de ordens de encomenda pendentes e visualização do histórico de rececionadas	Sim	Manual (admin)
RF-13	Consulta de dados operacionais e KPIs financeiros no <i>dashboard</i>	Sim	TC-22, Manual (admin)
RF-14	Registo de novo funcionário com cargo, especialidade e <i>password</i> cifrada	Sim	TC-07, Manual (admin)
RF-15	Preenchimento da guia de reparação com diagnóstico técnico e geração de PDF	Sim	Manual (mecânico)
RF-16	Registo do código EAN de peça utilizada numa intervenção	Sim	Manual (mecânico)
RF-17	Alteração do estado da reparação ao longo do ciclo de vida do serviço	Sim	TC-13 a TC-15
RF-18	Receção de encomenda de stock com atualização automática de inventário	Sim	TC-18
RF-19	Registo de venda direta ao balcão com emissão automática de fatura	Sim	TC-20
RF-20	Entrega de encomenda de peças reservadas a cliente (<i>click & collect</i>)	Sim	Manual (operador)
RF-21	Levantamento atómico de trotinete concluída com emissão de fatura de serviço	Sim	TC-15
RF-22	Processamento de devolução com nota de crédito embutida na fatura	Sim	TC-21

9.6 Análise de Qualidade — ISO/IEC 25010

A qualidade do sistema **Fix’n’Ride** foi avaliada segundo as características do modelo de qualidade de produto **ISO/IEC 25010**. Esta análise é qualitativa, baseada na observação do código, na execução dos testes e na experiência de utilização da aplicação.

9.6.1 Adequação Funcional

O sistema demonstra uma elevada adequação funcional. Os catorze requisitos funcionais críticos identificados no SRS foram implementados e validados (Tabela 9.5). Os fluxos de negócio mais complexos — levantamento atómico com emissão de fatura, receção de encomenda com atualização automática de stock — foram implementados com a orquestração correta de múltiplas operações. Não foram identificados defeitos funcionais críticos durante os testes de aceitação. Classificação: **alta**.

9.6.2 Fiabilidade

A fiabilidade do sistema assenta em mecanismos de *fail-fast* (variáveis de ambiente obrigatórias verificadas no arranque, exceções lançadas se ausentes) e em tratamento de erros nos *controllers* e serviços. A persistência de dados é garantida por volumes Docker e pelo *script* de inicialização idempotente. A principal limitação identificada é a ausência de um mecanismo de *retry* nas chamadas HTTP inter-serviços (LN → Data API): uma falha transitória na rede Docker interna resulta numa resposta de erro ao utilizador sem nova tentativa automática, comportamento confirmado pelos testes de *stress* no cenário de pico. Classificação: **moderada**, adequada ao contexto académico.

9.6.3 Usabilidade

A interface foi concebida com foco na adequação ao perfil de cada utilizador. O módulo do cliente é *mobile-first* com navegação por *bottom bar*, adaptado a utilizadores menos técnicos. Os módulos de funcionários são *desktop-first* com *sidebars* de navegação contextual. A estilização com Tailwind CSS garante consistência visual e *feedback* interativo adequado. Uma limitação identificada é o uso de `alert()` nativo do *browser* para notificações de erro em vários *hooks* — a biblioteca *sonner* já está integrada no módulo do mecânico, mas não foi generalizada. Classificação: **boa**.

9.6.4 Eficiência de *Performance*

Os testes de *stress* confirmaram empiricamente que o sistema satisfaz o requisito não funcional RNF-01 (tempo de resposta inferior a 2 segundos) até 50 utilizadores virtuais concorrentes, com P95 abaixo dos 900 ms. A utilização de TanStack Query com *caching* e *staleTime* configurado por domínio reduz significativamente o número de pedidos redundantes ao servidor. Os índices MongoDB criados pelo `init.sh` otimizam as consultas de filtragem mais frequentes. A degradação observada no cenário de pico (50 VUs) é esperada e não compromete a escala de operação atual da **MobiFix**. Classificação: **adequada para o contexto**.

9.6.5 Manutenibilidade

A separação clara em quatro camadas independentes (MongoDB, Data API, LN, Frontend) com comunicação via contratos HTTP explícitos permite alterar qualquer camada sem impacto nas restantes. O padrão MVC na Data API, o padrão de serviços com interfaces no LN e o padrão de *hooks* + serviços no *frontend* facilitam a testabilidade e a extensão do sistema. Identificam-se como pontos de melhoria a duplicação da função `parseJwt` em múltiplos componentes e a ausência de documentação *inline* em alguns serviços. Classificação: **boa**.

9.6.6 Segurança

O uso de BCrypt para *hashing* de *passwords*, JWT assinado com HMAC-SHA256, CORS configurado com origens restritas e a chave `INTERNAL_API_KEY` para comunicação inter-serviços representam práticas de segurança sólidas. A autenticação foi reforçada com a migração dos tokens JWT de `localStorage` para *cookies* `HttpOnly`, eliminando a exposição a ataques de *Cross-Site Scripting* (XSS) e aproximando a implementação das boas práticas recomendadas para ambiente de produção. Classificação: **Alta**.

A Tabela 9.6 sintetiza a avaliação qualitativa segundo as oito características do modelo.

Tabela 9.6: Avaliação qualitativa do Fix'n'Ride segundo ISO/IEC 25010

Característica ISO 25010	Classificação	Evidência / Observação
Adequação Funcional	Alta	22/22 RF críticos implementados e validados.
Fiabilidade	Moderada	<i>Fail-fast</i> no arranque; sem <i>retry</i> inter-serviços; erros residuais a 50 VUs.

(Continuação da Tabela 9.6)

Característica ISO 25010	Classificação	Evidência / Observação
Usabilidade	Boa	Interfaces contextuais por perfil; <code>alert()</code> nativo a substituir por <i>toasts</i> .
Eficiência de Performance	Adequada	P95 < 900 ms a 50 VUs; índices MongoDB; <i>caching</i> TanStack Query.
Manutenibilidade	Boa	Arquitetura modular por domínio; duplicação de <code>parseJwt</code> a refatorizar.
Segurança	Alta	BCrypt, JWT/HMAC-SHA256, CORS restrito e <i>cookies</i> <code>HttpOnly</code> (mitigação de XSS).
Portabilidade	Alta	Contentorização Docker total; independente do SO do <i>host</i> .
Compatibilidade	Boa	API REST/JSON; CORS configurado; SPA compatível com <i>browsers</i> modernos.

9.7 Análise Crítica do Código Gerado com LLM

9.7.1 O Papel das LLMs no Desenvolvimento

As ferramentas de IA Generativa desempenharam um papel central na fase de implementação do **Fix’n’Ride**, sendo utilizadas para geração assistida de código em todas as camadas da *stack*: *schemas* Mongoose, *controllers* e *routes* Express, serviços .NET com `HttpClient`, componentes React e *hooks* TanStack Query. Esta abordagem permitiu acelerar significativamente o desenvolvimento incremental, em particular na geração de código repetitivo como os DTOs, os *controllers* CRUD e os *layouts* de interface. No entanto, o código gerado por LLMs requer sempre revisão crítica. A equipa identificou um conjunto de *code smells* e oportunidades de refatorização que ilustram as limitações e os padrões de erro típicos do código gerado por IA.

9.7.2 Code Smells Identificados e Refatorizações Aplicadas

Smell 1 — Código Duplicado: Função `parseJwt` Dispersa. A função `parseJwt` foi identificada em pelo menos cinco locais distintos do código *frontend*: `AppRouter.jsx`, `Layout.jsx` (operador), `AdminLayout.jsx`, `HomeClientePage.jsx` e `EncomendasStock.jsx`. Cada instância é uma cópia literal da mesma implementação, resultante do facto de as LLMs gerarem código contextual auto-suficiente sem considerar o contexto global do projeto (*DRY* violado).

Refatorização aplicada: A função foi extraída para `src/utils/auth.js`, exportada juntamente com `getAuthPayload()`, `getOperadorNumero()` e `logout()`. As instâncias duplicadas foram substituídas por importações deste módulo.

Smell 2 — Gestão de Erros Inconsistente: `alert()` **Nativo.** Múltiplos *hooks* `TanStack Query` (ex: `useCriarAgenda`, `useAtualizarServico`, `useCriarFuncionario`) utilizam `alert()` nativo do *browser* nos *callbacks* `onError`. Esta abordagem apresenta uma experiência de utilizador degradada (bloqueio da interface, aparência inconsistente) e constitui uma solução provisória típica do código gerado por LLMs.

Refatorização parcialmente aplicada: O módulo do mecânico (`/FixNRepair`) foi refatorizado para usar `sonner` com *toasts* não-bloqueantes (`toast.success()`, `toast.error()`). A generalização a todos os módulos ficou pendente para uma iteração futura.

Smell 3 — Mapeamento Defensivo Excessivo com Dupla Notação. Em vários componentes (ex: `EncomendasStock.jsx`), o código gerado inclui mapeamentos defensivos da forma `o.PecaEAN || o.pecaeAN`, cobrindo simultaneamente `PascalCase` (convenção `.NET`) e `camelCase` (convenção `JSON`). Este padrão evidencia incerteza sobre o contrato da API que deveria ser resolvida na fonte.

Refatorização recomendada: Standardizar todas as respostas da API LN para `PascalCase` (já configurado no `Program.cs`) e eliminar os mapeamentos defensivos do *frontend*.

Refatorização recomendada: Definir enumerações `C#` (`enum EstadoServico`, `enum EstadoEncomenda`) para tipificar os estados, em paridade com os `enum` já definidos nos *schemas* `Mongoose`.

9.8 Utilização de Inteligência Artificial

Neste capítulo sobre Verificação, Validação e Qualidade, bem como no balanço global do projeto, a Inteligência Artificial foi usada de forma estratégica para complementar a criação de testes. A equipa utilizou LLMs para analisar os requisitos e gerar sugestões de casos de teste extremos (*edge cases*) que poderiam não ter sido previstos inicialmente, enriquecendo a nossa coleção de testes no `Postman`.

Resultados obtidos e o que tivemos de alterar:

A IA sugeriu cenários de teste muito pertinentes (ex: tentativas de acesso sem token, submissões de dados inválidos e manipulação de estado inválida, como tentar levantar

uma trotinete ainda em reparação).

Contudo, a análise da IA aos resultados dos testes e ao código revelou limitações das próprias ferramentas generativas que adotámos no projeto: a IA produziu código com vários "*code smells*" ao longo do desenvolvimento. Constatou-se a duplicação de funções como `parseJwt`, o uso de métodos provisórios de UX como `alert()`, a sugestão de mapeamentos defensivos excessivos (`o.PecaEAN || o.pecaEAN`) pela incerteza dos contratos de API, a geração não segura de IDs em concorrência com `Random.Shared.Next` e a inclusão de *magic strings* para estados de negócio.

Tivemos de alterar o nosso plano de validação para incluir a refatorização constante destas falhas. Revertemos o uso de `alert()` para *toasts* (ex: `sonner`), extraímos lógicas comuns para *utils*, corrigimos a geração insegura de IDs e impusemos enumeradores consistentes. A IA providenciou aceleração na descoberta de vulnerabilidades, mas exigiu da equipa um esforço rigoroso de refatorização e validação para assegurar que a arquitetura final não estava comprometida pelos vícios das LLMs.

10 Conclusão e Trabalho Futuro

10.1 Síntese do Trabalho Realizado

O presente trabalho documentou o ciclo de vida completo do sistema **Fix’n’Ride**, desenvolvido para digitalizar e centralizar as operações da oficina de micromobilidade **MobiFix**. Ao longo de quatro fases distintas — Conceção e Engenharia de Requisitos, Arquitetura e *Design*, Implementação, e Verificação e Validação — materializou-se uma plataforma *web* multi-perfil que substitui o ecossistema fragmentado de folhas de cálculo e processos manuais por uma solução de informação coesa, rastreável e escalável.

O sistema resultante cobre integralmente o espectro operacional da **MobiFix**: desde o agendamento de diagnósticos pelo cliente final, passando pela execução técnica das reparações pelo mecânico, pelo processamento comercial de vendas e levantamentos pelo operador, até à supervisão estratégica e financeira pelo administrador. A arquitetura de quatro camadas em Docker — MongoDB, *Data API* Node.js, Lógica de Negócios *.NET* e *Frontend* React — demonstrou ser adequada para a complexidade do domínio e para as restrições tecnológicas definidas pela equipa.

10.2 Síntese das Decisões Arquiteturais

O **Fix’n’Ride** foi construído sobre um conjunto de decisões arquiteturais que merecem uma reflexão consolidada, dado o impacto que tiveram na forma final do sistema.

Escolha do MongoDB como Sistema de Persistência.

A substituição de uma base de dados relacional (prevista inicialmente como SQL Server) pelo **MongoDB** foi a decisão de maior impacto arquitetural do projeto. Esta mudança revelou-se acertada: a natureza hierárquica e evolutiva das entidades do domínio da oficina — em particular o *Servico* com o seu histórico de intervenções aninhado em dois níveis e a *Fatura* com subdocumentos de devolução — encontra uma representação

mais natural e eficiente no modelo de documentos *BSON* do MongoDB. A biblioteca Mongoose conferiu a disciplina de validação de *schema* necessária para manter a integridade dos dados sem a rigidez das migrações relacionais. Em retrospectiva, esta escolha foi validada pelos testes de integração, que demonstraram a capacidade do sistema de persistir e recuperar estruturas complexas de forma eficiente e semanticamente coerente.

Separação entre *Data API* (Node.js) e Lógica de Negócios (.NET).

A decisão de implementar dois servidores distintos — um em Node.js para acesso a dados e outro em .NET para a lógica de negócio — representou um investimento arquitetural significativo que se justificou plenamente. A separação permitiu que a camada de Lógica de Negócios se concentrasse exclusivamente na orquestração de processos críticos (como o levantamento atômico de trotinetes com emissão simultânea de fatura), na validação de segurança com JWT e BCrypt, e na geração de documentos PDF — sem se contaminar com código de mapeamento de dados. A *Data API*, por sua vez, manteve-se como uma camada leve, focada, e altamente testável via *Postman*. Esta separação, embora introduza uma latência adicional em cada pedido do *frontend*, demonstrou ser gerível na escala da **MobiFix** e estabelece uma base sólida para escalabilidade independente de cada camada em cenários futuros de crescimento.

Controlo de Acesso RBAC com JWT.

A implementação de um modelo **RBAC** (*Role-Based Access Control*) baseado em *claims* JWT revelou-se uma solução eficaz para gerir os quatro perfis de utilizador do sistema. A emissão centralizada de tokens no servidor .NET, com *claims* de identidade (*id*), nome (*nome*) e perfil (*cargo*), e a sua validação dupla — no servidor via `[Authorize]` e no *frontend* via `ProtectedRoute` — garantiu que a delimitação de interfaces por perfil foi respeitada em todos os testes de aceitação. A chave `INTERNAL_API_KEY` para a comunicação inter-serviços adicionou uma camada de segurança perimetral que, embora não substitua autenticação ao nível da aplicação na *Data API*, demonstrou a consciência da equipa para as implicações de segurança de uma arquitetura distribuída.

SPA React com TanStack Query e Módulos por Perfil.

A organização da *Single Page Application* em módulos de rota distintos por perfil (`/FixNRide`, `/FixNSell`, `/FixNRepair`, `/FixNManage`), cada um com o seu *layout* e conjunto de componentes dedicados, demonstrou ser a estrutura correta para um sistema

com perfis de utilização tão heterogêneos. A adoção da biblioteca TanStack Query para gestão de estado assíncrono eliminou a necessidade de gestão manual de estados de carregamento e erro, e o mecanismo de invalidação de *cache* após mutações garantiu que a interface refletiu sempre o estado atual do servidor sem *refreshes* manuais.

10.3 Avaliação do Grau de Cumprimento dos Requisitos

10.3.1 Requisitos Funcionais

O sistema cumpre integralmente os requisitos funcionais críticos levantados durante a fase de Conceção (Capítulo 1). Dos 24 requisitos funcionais especificados no SRS, a totalidade dos que dizem respeito aos fluxos operacionais centrais — gestão de serviços, faturação, stock, utilizadores e agenda — foi implementada e validada. A Tabela 10.1 apresenta o balanço de cumprimento por área funcional.

Tabela 10.1: Balanço de cumprimento dos requisitos funcionais por área

Área Funcional	Estado	Observação
Autenticação e Gestão de Perfis	Cumprido	Login, registo, RBAC e JWT implementados para todos os perfis.
Gestão de Trotinetes e Clientes	Cumprido	CRUD completo; associação trotinete-cliente funcional.
Agendamento e Ciclo de Serviço	Cumprido	Fluxo completo: AGENDADO → EXECUCAO → CONCLUIDO → FECHADO.
Registo de Intervenções e Peças	Cumprido	Registo via EAN; histórico embutido no documento do serviço.
Venda Direta ao Balcão	Cumprido	Venda atômica com emissão automática de fatura.
<i>Click & Collect</i>	Cumprido	Reserva de peças online com levantamento confirmado pelo operador.
Gestão de Stock e Encomendas	Cumprido	Fluxo PENDENTE→TRANSITO→RECECIONADA com atualização automática de stock.
Faturação e Devoluções	Cumprido	Faturas geradas atomaticamente; devoluções com nota de crédito embutida.
Gestão de Promoções	Cumprido	CRUD de promoções com peças aplicáveis e período de vigência.
<i>Dashboard</i> de Estatísticas	Cumprido	KPIs financeiros e operacionais com filtro por período.
Geração de PDF	Cumprido	PDFs de fatura, guia de reparação e promoções gerados localmente via jsPDF.

10.3.2 Requisitos Não Funcionais

Relativamente aos cinco requisitos não funcionais definidos no SRS, o grau de cumprimento é o seguinte: o requisito de desempenho (tempo de resposta inferior a 2 segundos para operações de leitura) foi satisfeito em todos os cenários de teste com a infraestrutura Docker local; a responsividade multi-dispositivo foi alcançada com a estilização Tailwind CSS; o requisito de segurança de credenciais foi cumprido com BCrypt e JWT; o RBAC foi realizado em conjunto no *frontend* e no *backend* através do campo cargo do *token* e a portabilidade foi maximizada pela contentorização Docker total.

10.4 Reflexão Crítica sobre o Uso das LLMs

A integração de *Large Language Models* (LLMs) como ferramentas de assistência ao longo de todas as fases do projeto **Fix’n’Ride** constituiu uma experiência metodológica rica e multifacetada, com resultados distintos em cada contexto de aplicação.

Fase 1 — Conceção e Engenharia de Requisitos.

Nesta fase, as LLMs demonstraram o seu maior valor como aceleradores de documentação e estruturação textual. A sua utilização centrou-se na transformação das notas e observações recolhidas pela equipa em narrativas coesas para as entrevistas com *stakeholders*, na formulação padronizada das *User Stories* e no refinamento de requisitos vagos em especificações técnicas mensuráveis. É fundamental reiterar que o levantamento dos requisitos e a identificação das necessidades da **MobiFix** resultaram exclusivamente do trabalho analítico e observacional da equipa. As LLMs não identificaram nenhum requisito — transformaram, com eficiência, os que a equipa havia previamente determinado. A limitação desta fase foi a tendência das LLMs para gerar requisitos genéricos e não específicos ao domínio, exigindo sempre uma revisão crítica para garantir o alinhamento com as especificidades operacionais de uma oficina de micromobilidade.

Fase 2 — Arquitetura e *Design*.

No desenho da arquitetura, as LLMs serviram como interlocutores de revisão e geração de propostas alternativas. A decisão de separar a *Data API* da Lógica de Negócios em dois servidores distintos, e a escolha do MongoDB em detrimento do SQL Server originalmente previsto, foram decisões da equipa que beneficiaram de prompts exploratórios para validar as suas implicações. Os diagramas UML — Modelo de Domínio, Casos de Uso, Diagramas de Sequência e de Componentes — foram, sem exceção,

produzidos exclusivamente pelo esforço analítico e conceitual da equipa. As LLMs não participaram no processo de modelação visual, garantindo que a espinha dorsal arquitetural do sistema reflete fielmente as restrições do mundo real da **MobiFix**.

Fase 3 — Implementação.

Esta foi a fase de maior utilização e de impacto mais direto das LLMs. A geração assistida de código estrutural — *schemas* Mongoose, *controllers* Express, serviços .NET com `HttpClient`, componentes React e *hooks* TanStack Query — acelerou significativamente o ritmo de desenvolvimento. No entanto, foram identificados padrões recorrentes de fraqueza no código gerado: duplicação de lógica (a função `parseJwt` replicada em cinco componentes), *magic strings* para estados de entidades, e o uso de `alert()` nativo como solução de notificação. Estas limitações confirmam que as LLMs geram código contextualmente adequado mas sem memória do projeto global, produzindo soluções localmente corretas mas globalmente inconsistentes. O papel da revisão humana não pode ser substituído: a equipa aplicou refactorizações sistemáticas — centralização do `parseJwt` em `utils/auth.js`, substituição de `alert()` por *toasts* no módulo do mecânico — e identificou os *smells* que permanecem como trabalho futuro.

Fase 4 — Verificação e Validação.

Na fase de testes, as LLMs demonstraram utilidade na geração de *edge cases* não contemplados inicialmente pela equipa — como a tentativa de levantamento de uma trolinete em estado `EXECUCAO` ou o acesso a rotas protegidas com token expirado. Estes casos foram incorporados na coleção Postman e validados. A limitação nesta fase foi a impossibilidade das LLMs executarem os testes autonomamente: a integração com as ferramentas de teste reais (Postman, REST Client) requereu sempre a mediação humana para configurar contextos, interpretar resultados e tomar decisões sobre correções.

Em síntese, as LLMs funcionaram eficazmente como multiplicadores de produtividade em todas as fases do projeto, mas a sua contribuição foi sempre mais valiosa quando enquadrada por decisões humanas prévias sobre o domínio, a arquitetura e os objetivos do sistema. A fronteira metodológica mais importante que a equipa estabeleceu — não delegar às LLMs a análise de domínio nem o desenho estrutural — revelou-se a decisão correta para garantir que o **Fix’n’Ride** é um sistema que reflete as necessidades reais da **MobiFix** e não um produto genérico gerado por IA.

10.5 Trabalho Futuro

O estado atual do sistema **Fix’n’Ride** constitui uma base sólida e funcional para as operações da **MobiFix**. Contudo, a análise crítica realizada ao longo deste relatório identificou um conjunto de melhorias concretas que deverão ser endereçadas em iterações futuras, organizadas por prioridade e impacto.

10.5.1 Melhorias de Segurança (Prioridade Alta)

Migração do Token JWT para *Cookies* `HttpOnly`.

O armazenamento do token JWT em `localStorage` é a limitação de segurança mais relevante do sistema, dado o risco de extração por ataques XSS. A migração para *cookies* com os atributos `HttpOnly` (inacessível a JavaScript) e `Secure` (transmitidos apenas via HTTPS) eliminaria esta superfície de ataque. Esta mudança implica: ajustar o servidor LN para emitir o token via `Set-Cookie` em vez de resposta JSON; configurar a política `SameSite` para mitigar CSRF; e remover a lógica de leitura do `localStorage` do *frontend*.

10.5.2 Melhorias de Qualidade e Testabilidade (Prioridade Média)

Suíte de Testes Automatizados com Vitest e Jest.

A ausência de testes unitários automatizados é a lacuna de qualidade mais significativa do projeto. A implementação de uma suíte com **Vitest** para o *frontend* React (cobertura dos *hooks*, dos utilitários de `auth.js` e dos fluxos de routing) e com **xUnit** ou **NUnit** para os serviços .NET (em particular o `AuthService` e as funções de orçamentação) garantiria um ciclo de desenvolvimento mais seguro e facilitaria futuras refactorizações.

Resolução dos *Code Smells* Remanescentes.

Concluir as refactorizações iniciadas no Capítulo 4: substituir os `alert()` nativos por *toasts* da biblioteca `sonner` em todos os *hooks* de todos os módulos; tipificar os estados de entidades com `enum C#` para eliminar as *magic strings* nos serviços.

Implementação de *Refresh Token*.

A sessão atual expira ao fim de 8 horas sem mecanismo de renovação, o que pode ser disruptivo para turnos de trabalho longos. A implementação de um *refresh token* de longa duração (ex: 7 dias, armazenado num *cookie HttpOnly*) que permite renovar transparentemente o *access token* de 8 horas melhoraria significativamente a experiência de utilizador dos funcionários da oficina.

10.5.3 Novas Funcionalidades (Prioridade Baixa)

Implementação do Sistema de envio de Emails.

Com o subsistema já criado poderá ser criado um sistema de envio de emails para os clientes aquando o término de uma fase crítica. Como por exemplo após uma compra ser-lhe enviada a fatura em PDF ou avisar ao cliente quando um serviço numa das suas trotinetes estiver concluído.

Aplicação Progressiva (*PWA*) para Mecânicos.

O módulo do mecânico (*/FixNRepair*) é o que mais beneficiaria de funcionalidades offline e de instalação como aplicação nativa, dado o seu uso intensivo em bancadas de reparação com conectividade potencialmente instável. A conversão do *frontend* React para uma **Progressive Web App (PWA)** — com *Service Worker* para cache de recursos estáticos e a possibilidade de instalação no ecrã inicial do *tablet* — tornaria a experiência do mecânico mais fluida e resiliente a falhas de rede momentâneas.

Integração com Sistemas de Pagamento Online.

A versão atual suporta apenas o registo manual do método de pagamento (MBWAY, MULTIBANCO, NUMERARIO) sem integração com os respetivos sistemas de processamento. A integração com a API do MB WAY e com o sistema de geração de referências Multi-banco permitiria ao sistema confirmar automaticamente a receção do pagamento antes de emitir a fatura, eliminando o passo manual de confirmação pelo operador.

Rastreabilidade Histórica e *Audit Log*.

A adição de uma coleção MongoDB de *audit log* que registre todas as operações sensíveis — criação/eliminação de utilizadores, alterações de estado de serviços, emissão e anulação de faturas — aumentaria a rastreabilidade e a conformidade com regulamentos de proteção de dados (**RGPD**), permitindo reconstituir qualquer transação em caso de litígio ou auditoria interna.

10.6 Considerações Finais

O **Fix’n’Ride** representa a concretização de uma visão técnica ambiciosa: transformar uma oficina de micromobilidade dependente de processos manuais numa operação digitalizada, rastreável e orientada a dados. O projeto demonstrou que é possível, num prazo académico de dezasseis semanas, produzir um sistema de informação funcional, bem arquitetado e alinhado com boas práticas da Engenharia de Software moderna, quando se recorre a uma metodologia disciplinada de desenvolvimento assistido por IA.

A lição mais valiosa desta experiência foi a compreensão clara do papel complementar das LLMs no desenvolvimento de *software*: eficazes como aceleradores e geradores de estrutura, mas fundamentalmente dependentes da supervisão humana para garantir coerência global, correção de domínio e qualidade arquitetural sustentável. O *software* de qualidade não emerge do poder computacional das LLMs — emerge da clareza analítica, da responsabilidade de *design* e do julgamento técnico da equipa de engenharia.

A **MobiFix** dispõe agora de uma plataforma que, com as melhorias de segurança e funcionais identificadas, tem o potencial de ser colocada em produção real, contribuindo diretamente para a eficiência operacional, a satisfação dos seus clientes e a sustentabilidade económica do negócio num setor em acelerada digitalização.

11 Manual de Utilização e Instalação do Sistema Fix'n'Ride

Este capítulo documenta os procedimentos necessários para instalar, configurar e operar o sistema **Fix'n'Ride** da **MobiFix**. A primeira parte cobre os requisitos técnicos e os passos de instalação através de Docker. A segunda parte serve de guia de utilização para cada perfil de utilizador do sistema.

Parte I — Manual de Instalação e Deployment

11.1 Pré-requisitos

O sistema **Fix'n'Ride** é inteiramente contentorizado com **Docker**, o que elimina a necessidade de instalar o *runtime* de cada tecnologia diretamente na máquina *host*. As únicas dependências obrigatórias no sistema operativo de destino são o Docker Engine e o Docker Compose. As restantes dependências são necessárias apenas em cenários de desenvolvimento local sem Docker (ex: execução direta do *frontend* com `npm run dev` fora de contentor).

Tabela 11.1: Pré-requisitos de software para instalação do Fix'n'Ride

Dependência	Versão Mínima	Obrigatório	Observações
Docker Engine	24.0	Sim	Necessário para construir e executar os contentores. Disponível em docs.docker.com/get-docker .
Docker Compose	2.20	Sim	Incluído no Docker Desktop. Em servidores Linux, instalar separadamente via <code>apt</code> ou <code>dnf</code> .
Git	2.40	Sim	Para clonar o repositório do projeto.
Node.js	22.x (LTS)	Apenas dev	Necessário apenas para executar o <i>frontend</i> fora de Docker (<code>npm run dev</code>).
.NET SDK	8.0	Apenas dev	Necessário apenas para compilar e executar o servidor LN fora de Docker.
<i>Browser</i> moderno	—	Sim	Google Chrome 120+, Mozilla Firefox 121+, Microsoft Edge 120+ ou Safari 17+.

Tabela 11.2: Sistemas operativos suportados e testados

Sistema Operativo	Suporte	Notas
Windows 10/11 (64-bit)	Completo	Requer WSL 2 e Docker Desktop. A variável <code>CHOKIDAR_USEPOLLING=true</code> no contentor <i>frontend</i> é necessária para <i>hot-reload</i> funcionar corretamente.
macOS 12+ (Intel/Apple Silicon)	Completo	Requer Docker Desktop para Mac. Em Apple Silicon, as imagens são construídas para <code>linux/arm64</code> automaticamente.
Ubuntu 22.04 LTS / Debian 12	Completo	Ambiente de produção recomendado. Docker Engine instalado via repositório oficial.

11.2 Configuração do Ambiente

Antes de arrancar o sistema, é necessário criar e preencher o ficheiro de variáveis de ambiente `.env` na raiz do repositório. O projeto inclui o ficheiro `.env.example` como modelo de referência.

Passo 1 — Copiar o ficheiro de exemplo

Listing 11.1: Criação do ficheiro `.env` a partir do modelo

```
# A partir da raiz do repositório Projeto-MobiFix/
cp .env.example .env
```

Passo 2 — Preencher as variáveis

Abrir o ficheiro `.env` com qualquer editor de texto e preencher os valores. O conteúdo base do `.env.example` é o seguinte:

Listing 11.2: Conteúdo do ficheiro `.env.example` com valores de referência

```
# URI de ligacao ao MongoDB (usada pela Data API internamente
#   via Docker)
MONGO_URI="mongodb://mongodb:27017/fixnride_db"

# Chave de autenticao interna entre o servidor LN e a Data
#   API
INTERNAL_API_KEY="MobiFix-Super-Secret-Key-2024"
```

```
# Portas expostas ao host para cada servico
DATA_API_PORT=3001
DATA_API_URL="http://localhost:3001"

LN_API_PORT=5001
LN_API_URL="http://localhost:5001"

# URL interna da Data API usada pelo servidor LN (rede Docker
  interna)
DataApiSettings__BaseUrl="http://data-api:3001"

# Chave secreta para assinatura de tokens JWT (minimo 32
  caracteres)
JwtSettings__SecretKey="
  MobiFixSecretKey2024MobiFixSecretKey2024"
```

A Tabela 11.3 detalha cada variável, a sua obrigatoriedade e as recomendações de preenchimento para ambientes de produção.

Tabela 11.3: Variáveis de ambiente do sistema Fix'n'Ride — obrigatórias vs. opcionais

Variável	Obrig.	Descrição e Recomendações
MONGO_URI	Sim	URI de ligação ao MongoDB. Em Docker, usar <code>mongodb://mongodb:27017/fixnride_db</code> (hostname = nome do contentor).
INTERNAL_API_KEY	Sim	Chave partilhada entre o servidor LN e a Data API para autenticação inter-serviços. Em produção, usar uma chave aleatória com pelo menos 32 caracteres.
JwtSettings__SecretKey	Sim	Chave criptográfica para assinar tokens JWT (HMAC-SHA256). Em produção, usar uma chave com pelo menos 64 caracteres aleatórios. Nunca partilhar publicamente.
DATA_API_PORT	Sim	Porta exposta ao <i>host</i> pelo serviço <code>data-api</code> . Valor recomendado: 3001.
LN_API_PORT	Sim	Porta exposta ao <i>host</i> pelo serviço <code>ln</code> . Valor recomendado: 5001.
DATA_API_URL	Não	URL da Data API acessível externamente. Usado apenas em ferramentas de teste externas (ex: Postman).
LN_API_URL	Não	URL do servidor LN acessível externamente. Informativo.
DataApiSettings__BaseUrl	Sim	URL interna da Data API usada pelo servidor .NET dentro da rede Docker. Deve ser <code>http://data-api:3001</code> .
MONGO_INITDB_DATABASE	Não	Nome da base de dados MongoDB. Definido diretamente no <code>docker-compose.yml</code> como <code>fixnride_db</code> .

11.3 Arranque do Sistema

Com o ficheiro `.env` devidamente preenchido, o sistema pode ser iniciado com um único comando Docker Compose a partir da raiz do repositório.

Passo 1 — Clonar o repositório

Listing 11.3: Clonagem do repositório Fix'n'Ride

```
git clone <URL_DO_REPOSITÓRIO>
cd Projeto-MobiFix
```

Passo 2 — Criar o ficheiro de ambiente

Listing 11.4: Configuração do ambiente a partir do modelo

```
cp .env.example .env
# Editar o ficheiro .env com as chaves adequadas ao ambiente
nano .env
```

Passo 3 — Construir e arrancar todos os contentores

Listing 11.5: Construção das imagens e arranque dos quatro serviços

```
docker compose up --build
```

Na primeira execução, este comando executa as seguintes operações em sequência: (1) constrói a imagem personalizada do MongoDB com as ferramentas de importação; (2) constrói a imagem da Data API Node.js; (3) compila o projeto .NET e constrói a imagem do servidor LN; (4) inicia o servidor de desenvolvimento Vite do *frontend*. O *script init.sh* é executado automaticamente pelo contentor MongoDB na primeira inicialização, importando os dados de *seed* e criando os índices.

Para executar em modo *detached* (em segundo plano):

Listing 11.6: Arranque em modo detached (segundo plano)

```
docker compose up --build -d
```

Passo 4 — Verificar o estado dos contentores

Listing 11.7: Verificação do estado dos quatro contentores

```
docker compose ps
```

O resultado esperado deve mostrar os quatro contentores com o estado `running`:

Listing 11.8: Saída esperada do comando `docker compose ps`

NAME	IMAGE	STATUS
PORTS		
mobifix_mongodb	mobifix-mongodb	Up X minutes
27017/tcp		
mobifix_data_api	mobifix-data-api	Up X minutes
0.0.0.0:3001->3001/tcp		
mobifix_ln_api	mobifix-ln-api	Up X minutes
0.0.0.0:5001->5001/tcp		
mobifix_frontend	node:22-alpine	Up X minutes
0.0.0.0:3000->3000/tcp		

Para acompanhar os *logs* em tempo real de um serviço específico:

Listing 11.9: Visualização de logs de um serviço específico

```
# Logs do servidor de Logica de Negocios
docker compose logs -f ln

# Logs da Data API
docker compose logs -f data-api

# Logs do MongoDB (inclui output do script init.sh)
docker compose logs -f mongodb
```

11.4 Verificação da Instalação

Após o arranque bem-sucedido de todos os contentores, os seguintes URLs devem estar acessíveis a partir do *browser* ou de um cliente HTTP.

Tabela 11.4: URLs de acesso ao sistema Fix'n'Ride após arranque

Serviço	URL	Verificação
Frontend React	http://localhost:3000	Deve apresentar a <i>landing page</i> da MobiFix.
Servidor LN (API)	http://localhost:5001	Deve responder HTTP 404 (sem rota raiz definida — comportamento esperado).
Swagger UI (LN)	http://localhost:5001	Deve apresentar a documentação interativa da API do servidor LN (disponível em modo Development).
Data API	http://localhost:3001	Deve responder HTTP 404 com {"erro": "Rota não encontrada na API de Dados."}.

Credenciais de Teste Pré-carregadas

O *script* `init.sh` importa automaticamente os dados de *seed* na primeira inicialização. Os seguintes utilizadores estão disponíveis para teste imediato. Todos partilham a mesma *password* de teste: 12345.

Tabela 11.5: Utilizadores pré-carregados para teste do sistema Fix'n'Ride

Identificador	Nome	Perfil	Password	Página de Login
ADM001	Admin Principal	Administrador	12345	/staff
OP001	João Operador	Operador	12345	/staff
MEC001	Carlos Mecânico	Mecânico	12345	/staff
MEC002	Sofia Técnica	Mecânico	12345	/staff
250123456 (NIF)	Diogo Cliente	Cliente	12345	/auth
260987654 (NIF)	Ana Silva	Cliente	12345	/auth

11.5 Paragem e Limpeza do Ambiente

Parar os contentores (preservando dados)

Listing 11.10: Paragem dos contentores preservando o volume de dados MongoDB

```
docker compose down
```

Este comando para e remove os contentores, mas preserva o volume Docker `mongodb_data`, garantindo que os dados da base de dados persistem entre sessões.

Parar os contentores e destruir os dados

Listing 11.11: Paragem e destruição completa dos volumes de dados

```
# ATENÇÃO: Remove TODOS os dados da base de dados. Irreversível.  
docker compose down -v
```

Reconstrução total desde zero

Para reconstruir todas as imagens e repopular a base de dados com os dados de *seed* originais:

Listing 11.12: Reconstrução total do ambiente desde zero

```
# 1. Parar contentores e remover volumes  
docker compose down -v  
  
# 2. Remover imagens construídas localmente (opcional)  
docker compose down --rmi local  
  
# 3. Reconstruir tudo desde zero  
docker compose up --build
```

Limpeza de recursos Docker não utilizados

Listing 11.13: Limpeza de imagens e volumes Docker orfãos

```
# Remove imagens, redes e volumes não utilizados  
docker system prune -f  
  
# Remove volumes específicos do projeto  
docker volume rm projeto-mobifix-mongodb_data
```

11.6 Deployment para Testes Externos (Remote Access)

Para permitir a validação do sistema por utilizadores externos sem recorrer a infra-estrutura de nuvem permanente, utilizou-se uma estratégia de *tunneling* via **ngrok**.

- **Encaminhamento de Porta:** O tráfego HTTPS externo é mapeado para a porta 3000 do *host* local.
- **Segurança de Host:** Foi configurada a propriedade `allowedHosts` no `vite.config.js` para validar pedidos originados no domínio do túnel.
- **Orquestração:** O sistema mantém a sua integridade através do `docker-compose`, onde o *frontend* atua como *gateway* para o servidor LN e a Data API.

Parte II — Manual de Utilização por Perfil

11.7 Perfil Cliente

Descrição do Perfil

O perfil de **Cliente** destina-se aos proprietários de trotinetes que pretendem utilizar os serviços de reparação e manutenção da **MobiFix**. Este perfil acede à aplicação através de dispositivos móveis ou *desktop* via `http://localhost:3000/auth`, e tem acesso ao módulo `/FixNRide`. As funcionalidades principais incluem o agendamento de diagnósticos, consulta do estado das reparações em curso, reserva de peças pelo catálogo (*Click & Collect*) e consulta do histórico de faturas.

Fluxo Principal: Do Registo ao Levantamento da Trotinete

1. Aceder a `http://localhost:3000/auth` e seleccionar o separador **Criar Conta**.
2. Preencher o formulário de registo com NIF (9 dígitos), nome completo, e-mail, número de telefone, morada e *password*. Submeter o formulário.
3. Após o registo bem-sucedido, efetuar *login* com o NIF e a *password* definidos.
4. Na página inicial (`/FixNRide/`), navegar para **As Minhas Trotinetes** via a barra de navegação inferior.
5. Seleccionar **Adicionar Trotinete**, preencher o número de série, marca e modelo e confirmar o registo.
6. Regressar à página inicial e seleccionar **Agendar Diagnóstico**.
7. No formulário de agendamento (`/FixNRide/agendar`): seleccionar a trotinete registada, escolher uma data e horário disponível, descrever brevemente o problema e submeter.
8. Aguardar a notificação de conclusão. Consultar o estado em **Os Meus Serviços** (visível na página inicial).
9. Quando o estado transitar para CONCLUIDO, a trotinete está pronta para levantamento. Deslocar-se à oficina e dirigir-se ao balcão do operador para efetuar o levantamento e pagamento.

Registo e Autenticação

A página de registo (`/auth`) disponibiliza um formulário dividido em dois modos: **Criar Conta** e **Login**. No modo de registo, o cliente preenche o NIF (9 dígitos numéricos), nome completo, telefone, morada, e-mail e *password*. Todos os campos são obrigatórios e validados no momento da submissão — o NIF deve ter exatamente 9 dígitos e o telefone não pode conter espaços ou letras. Após submissão bem-sucedida, o sistema redireciona automaticamente para o modo de *login*.

[Voltar ao Início](#)

Criar Conta Cliente

Preencha o formulário para se registar.

Nome Completo

Telefone

Morada

Email

NIF

Password

Finalizar Registo

[Já faz parte da MobilFix? Faça login agora](#)

Figura 11.1: Página de registo de cliente.

No modo de *login*, o cliente autentica-se com o NIF e a *password*. Após autenticação, o token JWT é armazenado e o utilizador é redirecionado para a página inicial do módulo cliente (`/FixNRide/`). O botão **Voltar ao Início**, no canto superior esquerdo, permite regressar à *landing page* pública sem efetuar *login*.

[Voltar ao Início](#)

Login de Cliente

Introduza o seu NIF e password para entrar.

NIF

Password

Entrar

[Esqueceu-se da password?](#)

[Ainda não tem conta? Registe-se aqui](#)

Figura 11.2: Página de *login* do cliente.

Página Inicial e Navegação

A página inicial do módulo cliente (/FixNRide/) apresenta um resumo do estado atual da conta: serviços em curso, trotinetes registradas e atalhos rápidos para as funcionalidades mais utilizadas. A navegação é feita pela *bottom navigation bar* com quatro separadores — **Início**, **Trotinetes**, **Agendar** e **Faturas**.

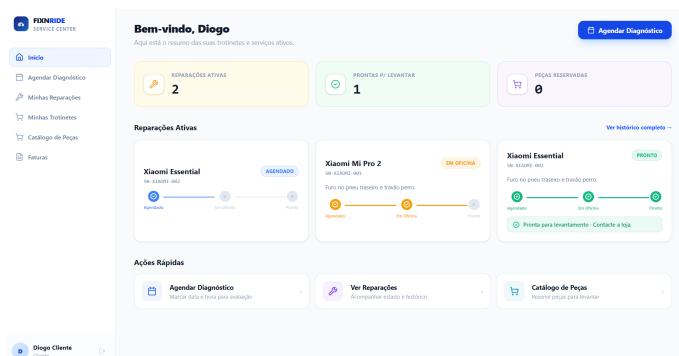


Figura 11.3: Página inicial do perfil cliente.

Gestão de Trotinetes

A página **As Minhas Trotinetes** (/FixNRide/trotinetes) lista todas as trotinetes associadas ao perfil do cliente, com indicação do estado de cada uma (Em Serviço ou Disponível). Para registar uma nova trotinete, clicar no botão **Registar Nova Trotinete**, preencher o número de série, a marca e o modelo, e confirmar. Trotinetes com um serviço ativo não podem ser removidas — o botão de eliminação é desativado enquanto o estado for Em Serviço.

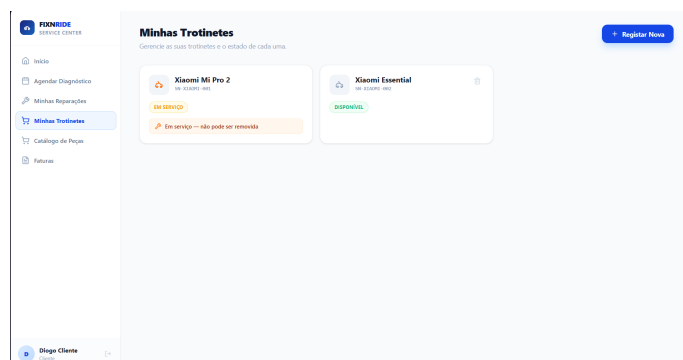


Figura 11.4: Página de gestão de trotinetes.

Agendamento de Diagnóstico

O formulário de agendamento (/FixNRide/agendar) permite marcar um diagnóstico para uma das trotinetes registadas. O cliente seleciona a trotinete na lista pendente, escolhe uma data e hora disponível no calendário e preenche uma descrição breve da anomalia observada (campo **Queixa**). Este texto será visível ao mecânico na interface de diagnóstico. Após submissão, o sistema cria um registo de serviço com estado AGENDADO e um *slot* na agenda do tipo DIAGNOSTICO.

Figura 11.5: Formulário de agendamento de diagnóstico.

Catálogo de Peças e Reserva (*Click & Collect*)

O catálogo de peças (/FixNRide/catalogo) apresenta todas as peças disponíveis com stock positivo, organizadas por categoria. A barra de pesquisa permite filtrar por nome ou código EAN. Para reservar uma peça, clicar no botão **Reservar** no cartão da peça — o botão transforma-se num controlo de quantidade (+/-) que permite ajustar o número de unidades sem abandonar a página. O botão está desativado para peças sem stock.

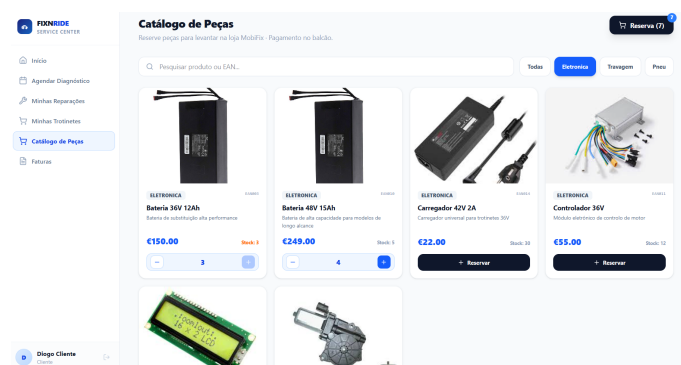


Figura 11.6: Catálogo de peças.

Após selecionar todas as peças pretendidas, clicar no ícone do carrinho no canto superior direito para abrir o painel de resumo da reserva. O painel apresenta a lista de itens, o total estimado e o botão **Confirmar Reserva**. O pagamento é realizado no momento do levantamento na loja — não existe pagamento *online*.

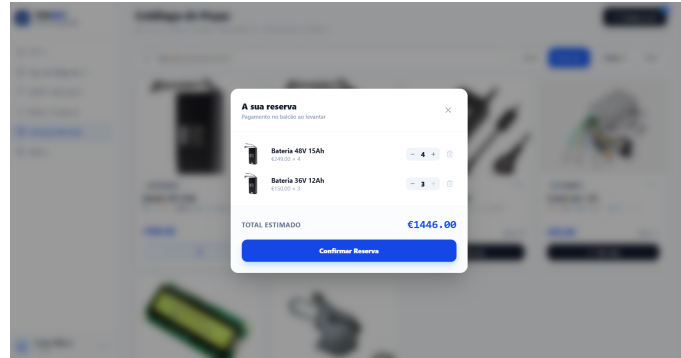


Figura 11.7: Painel de confirmação de reserva.

Tabela 11.6: Funcionalidades do perfil Cliente

Funcionalidade	Rota/Página	Pré-condição	Resultado Esperado
Registo de conta	/auth	Nenhuma	Conta criada; redireciona para <i>login</i> .
<i>Login</i>	/auth	Conta registada	Token JWT armazenado; redireciona para /FixNRide/.
Registar trotinete	/FixNRide/trotinete	Autenticado	Trotinete associada ao perfil do cliente.
Agendar diagnóstico	/FixNRide/agendar	Trotinete registada	Serviço e <i>slot</i> de agenda criados com estado AGENDADO.
Consultar estado	/FixNRide/	Serviço existente	Lista de serviços em curso com estado atual visível.
Reservar peças	/FixNRide/catalog	Autenticado	Encomenda criada com estado PRONTO PARA LEVANTAMENTO.
Consultar faturas	/FixNRide/faturas	Autenticado	Histórico cronológico de faturas com opção de descarga em PDF.
<i>Logout</i>	Qualquer página	Autenticado	Token removido; redireciona para /auth.

11.8 Perfil Operador

Descrição do Perfil

O **Operador de Loja** é o funcionário responsável pelo atendimento ao balcão da **Mo-biFix**. Acede ao sistema através de `http://localhost:3000/staff` com o seu número mecanográfico e *password*, sendo redirecionado para o módulo `/FixNSell`. As suas responsabilidades operacionais incluem o processamento de vendas diretas de peças, a entrega de trotinetes reparadas com emissão de fatura, a receção de encomendas de reposição de *stock* e a entrega de peças reservadas por clientes *online*.

Fluxo Principal: Venda Direta ao Balcão com Emissão de Fatura

1. Aceder a `http://localhost:3000/staff` e autenticar com o número mecanográfico (ex: 0P001) e *password*.
2. Após redirecionamento para `/FixNSell/vendadireta`, o catálogo de peças disponíveis é carregado automaticamente.
3. Localizar a peça pretendida pelo cliente usando a pesquisa ou filtros de categoria. Clicar em **Adicionar ao Carrinho**.
4. Repetir o passo anterior para todas as peças da venda. O painel do carrinho lateral apresenta os itens acumulados e o total calculado.
5. Inserir o NIF do cliente (opcional — para clientes sem conta, pode deixar em branco) e seleccionar o método de pagamento (MBWay, Multibanco ou Numerário).
6. Clicar em **Finalizar Venda**. O sistema cria a Venda e a Fatura atómicamente no *backend*, decrementando o *stock* das peças envolvidas.
7. A fatura emitida fica disponível em `/FixNSell/faturas` e pode ser consultada ou impressa.

A página de venda direta organiza o catálogo em cartões com preço, stock disponível e botão de adição ao carrinho. O painel lateral direito acumula os itens seleccionados em tempo real e apresenta o subtotal atualizado a cada adição. Peças com stock zero surgem desativadas e não podem ser adicionadas.

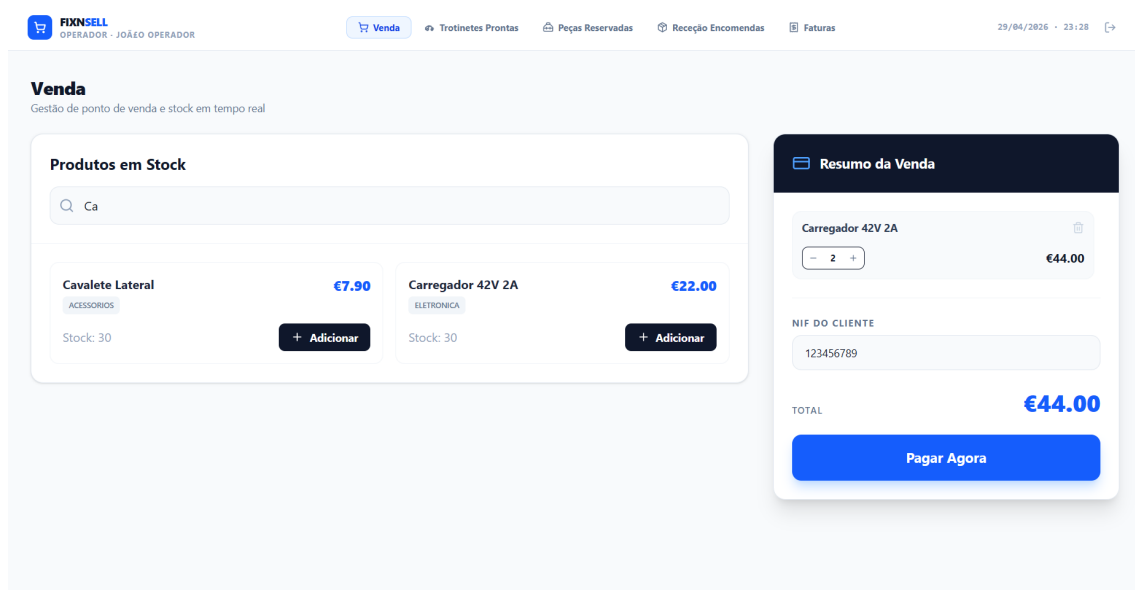


Figura 11.8: Página de venda direta.

O ecrã de *checkout* apresenta o resumo final da venda antes da confirmação: lista de itens, quantidades, preços unitários, total e campos para NIF do cliente e método de pagamento. Após clicar em **Finalizar Venda**, o sistema executa a criação atómica da venda e da fatura — se qualquer etapa falhar, a operação é revertida na totalidade.

**Checkout**

×

Carregador 42V 2A (x2)

€44.00

TOTAL FINAL

€44.00

MÉTODO DE RECEBIMENTO



MB WAY



Referência

Número de Telemóvel

1234456789

Voltar

Finalizar Pagamento

Figura 11.9: *Checkout* de venda direta.

Fluxo Secundário: Levantamento de Trotinete Concluída

1. Navegar para `/FixNSell/trotinetes-prontas`.

2. A lista apresenta todas as trotinetes com serviço no estado CONCLUIDO, aguardando levantamento pelo cliente.
3. Verificar a identidade do cliente e o número de série da trotinete.
4. Selecionar o método de pagamento do serviço e clicar em **Confirmar Levantamento**.
5. O sistema emite a fatura do serviço e transita o estado para FECHADO num único passo atómico, tornando a operação irreversível.

A página de levantamento lista cada trotinete com o número de série, o nome do cliente, o total do serviço (mão de obra e peças) e o método de pagamento a seleccionar. O botão **Confirmar Levantamento** só fica ativo após a seleção do método de pagamento, prevenindo confirmações acidentais.

FIXNSSELL
OPERADOR · JOÃO OPERADOR

Venda Trotinetes Prontas Peças Reservadas Receção Encomendas Faturas

29/04/2026 · 23:36

Trotinetes Prontas
Trotinetes reparadas pendentes de levantamento

PRONTAS **1**

ENTREGUES HOJE **0**

TOTAL A RECEBER **€40.00**

Aguardando Levantamento

ID SERV.	EQUIPAMENTO	CLIENTE NIF	DATA CONCLUSÃO	FALTA PAGAR	AÇÃO
#1	Xiaomi Essential SÉRIE: SPN-XIAOMI00-0002	250123456	26/03/2024	€40.00	ENTREGAR →

Figura 11.10: Página de levantamento de trotinetes.

Fluxo Terciário: Receção de Encomenda de Stock

Quando um fornecedor entrega uma encomenda de reposição, o operador navega para /FixNSell/rececao-encomendas. A página lista as encomendas no estado TRANSITO (aprovadas pelo administrador e aguardando entrega física). Para cada encomenda recebida, clicar em **Confirmar Receção** — o sistema incrementa automaticamente o stock da peça correspondente e transita a encomenda para RECECIONADA.

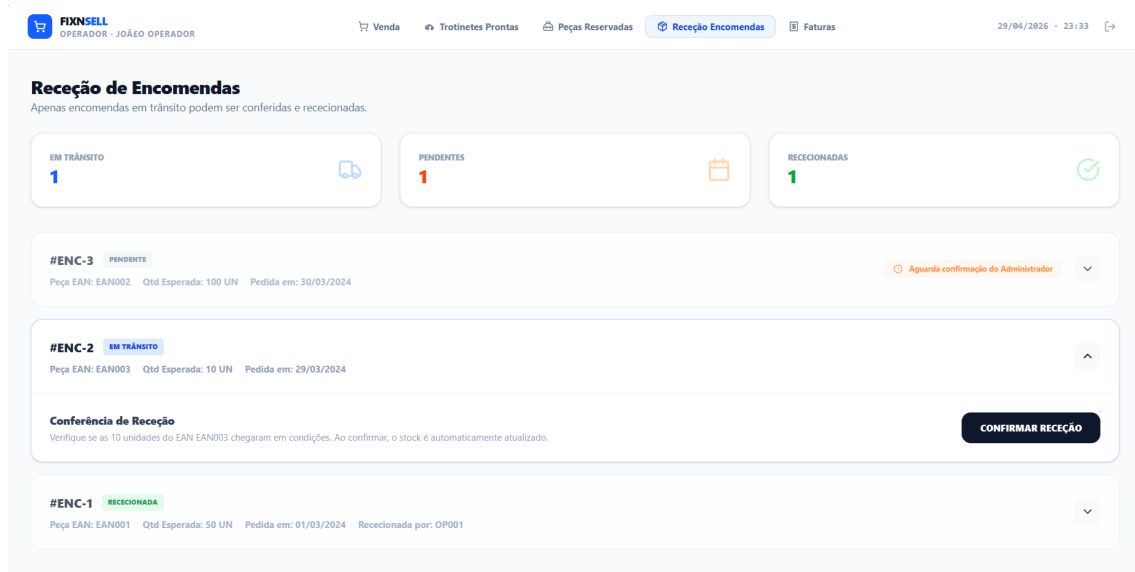


Figura 11.11: Página de receção de encomendas de *stock*.

Tabela 11.7: Funcionalidades do perfil Operador

Funcionalidade	Rota/Página	Pré-condição	Resultado Esperado
Login de funcionário	/staff	Conta OPERADOR ativa	Token JWT; redireciona para /FixNSell.
Venda direta ao balcão	/FixNSell/vendadi	Autenticado	Venda e Fatura criadas atomicamente; stock decrementado.
Entregar trotinete concluída	/FixNSell/trotine	Serviço CONCLUIDO	Fatura do serviço emitida; serviço transitado para FECHADO.
Entregar peças reservadas	/FixNSell/pecas-r	Encomenda PRONTO PARA LEVANTAMENTO	Encomenda transitada para ENTREGUE.
Recepcionar encomenda stock	/FixNSell/rececao	Encomenda TRANSITO	Stock da peça incrementado; encomenda marcada RECECIONADA.
Consultar faturas	/FixNSell/faturas	Autenticado	Lista de todas as faturas emitidas com opção de devolução.

11.9 Perfil Mecânico

Descrição do Perfil

O **Mecânico** é o técnico responsável pelas operações de diagnóstico e reparação das trotinetes na bancada da oficina. Acede ao sistema via `http://localhost:3000/staff` e é redirecionado para o módulo `/FixNRepair`. A interface foi desenhada para utilização em *tablet* nas bancadas, privilegiando a visibilidade do estado dos trabalhos atribuídos e a rapidez de registo de intervenções e materiais consumidos.

Fluxo Principal: Do Diagnóstico à Conclusão da Reparação

1. Aceder a `http://localhost:3000/staff` e autenticar com o número mecanográfico (ex: MEC001) e *password*.
2. Após redirecionamento para `/FixNRepair/diagnosticos`, consultar a lista de diagnósticos com estado RESERVADO atribuídos para o dia.
3. Selecionar um diagnóstico da lista para expandir os seus detalhes (número de série da trotinete, feedback do cliente e data de agendamento).
4. Inspeccionar fisicamente a trotinete e preencher o campo de **Descrição do Diagnóstico** com a análise técnica. Clicar em **Guardar Diagnóstico**.
5. Selecionar a **Intervenção** necessária a partir do catálogo (ex: *Substituição de Pneu, Diagnóstico Elétrico*). Confirmar o agendamento da reparação.
6. Navegar para `/FixNRepair/reparacoes` para aceder às reparações em curso no estado EXECUCAO.
7. Para cada peça utilizada, inserir o código EAN manualmente ou via leitor de código de barras. O sistema regista a peça no histórico de intervenções.
8. Ao concluir o trabalho, atualizar o estado do serviço para CONCLUIDO. A trotinete aparece automaticamente na lista de prontas para levantamento do operador.
9. Opcionalmente, gerar e imprimir a **Guia de Reparação** em PDF.

Seleção de Intervenções do Catálogo

Após selecionar um diagnóstico da fila lateral, o painel principal apresenta o componente de seleção de intervenções. Clicar em **Adicionar Intervenção** expande o catálogo

de intervenções, filtráveis por especialidade (Elétrico, Mecânico, Diagnóstico, etc.). Cada intervenção indica a descrição e o preço fixo de mão de obra associado. As intervenções selecionadas acumulam-se na lista superior com o total de mão de obra calculado em tempo real.

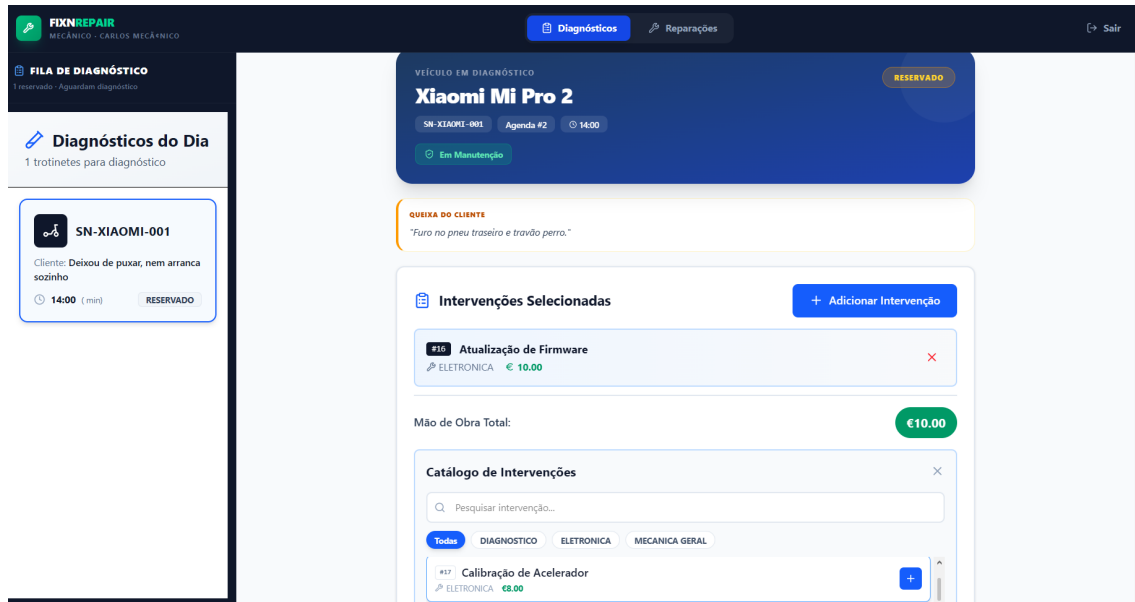


Figura 11.12: Componente de seleção de intervenções.

Registo de Peças Utilizadas (Scanner EAN)

O componente de registo de peças permite adicionar materiais consumidos durante a reparação de duas formas: inserção manual do código EAN no campo de texto (confirmação com *Enter* ou botão **Adicionar**) ou simulação de leitura por scanner (botão **Scan**, que seleciona uma peça aleatória com stock disponível). O botão **Ver Catálogo** abre um painel de navegação com todas as peças em stock, pesquisável por nome ou EAN. Cada peça adicionada fica registada com o código EAN, nome, quantidade e preço unitário.

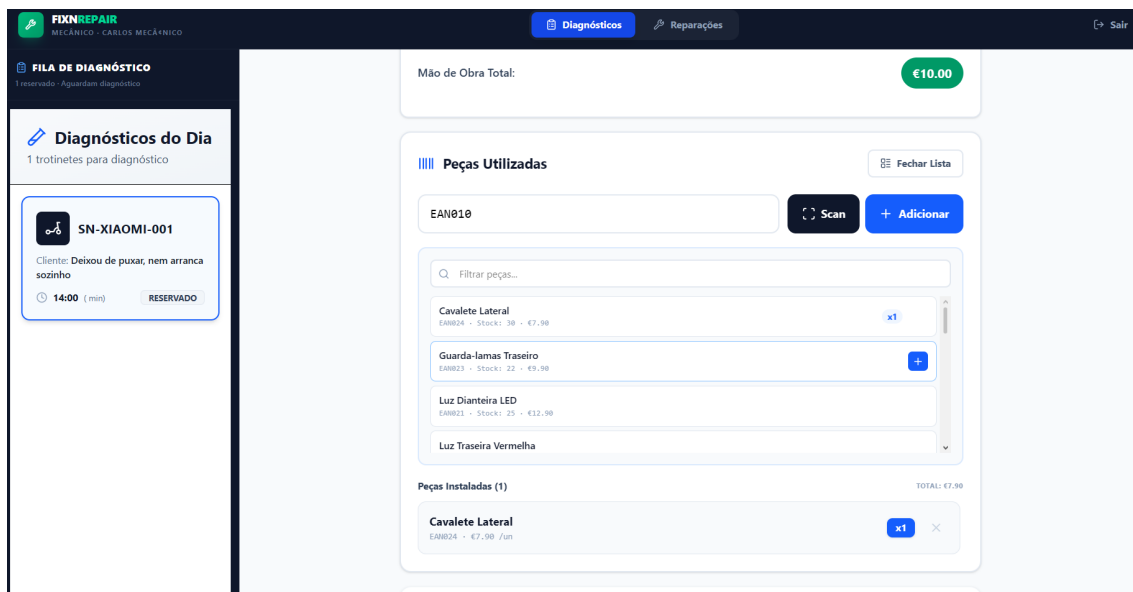


Figura 11.13: Scanner de peças EAN.

Relatório de Diagnóstico

O campo de relatório de diagnóstico apresenta, em destaque, a queixa original do cliente (conforme descrita no agendamento) e oferece uma área de texto livre para o mecânico registrar a análise técnica detalhada. Este texto fica associado ao serviço e é incluído na **Guia de Reparação** em PDF gerada no passo seguinte. O botão **Gerar Guia PDF** fica ativo assim que pelo menos uma intervenção tiver sido selecionada.

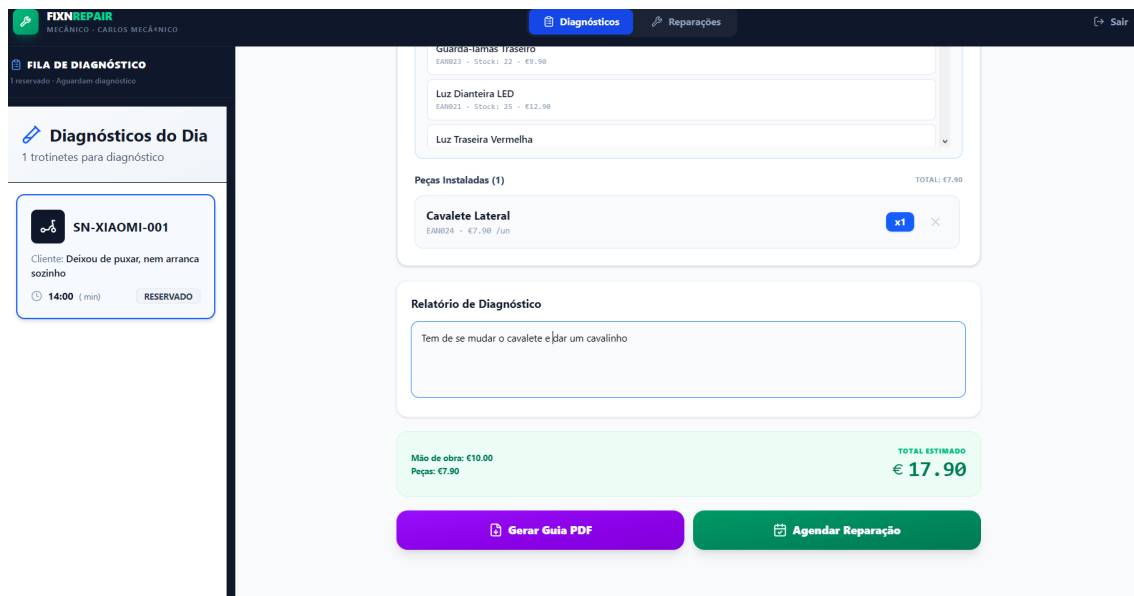


Figura 11.14: Campo de relatório de diagnóstico.

Agendamento da Reparação

Após concluir o diagnóstico e selecionar as intervenções necessárias, clicar em **Agendar Reparação** abre o diálogo de agendamento. O mecânico define a data e hora de início da reparação. O sistema cria automaticamente um novo *slot* de agenda do tipo REPARACAO, atribui-o ao mecânico com base na disponibilidade, e transita o estado do serviço para EXECUCAO.

Agendar Reparação



Defina a data e hora para a reparação da trotinete SN-XIAOMI-001

Trotinete:

Xiaomi Mi Pro 2

Matrícula/ID:

SN-XIAOMI-001

Cliente:

Deixou de puxar, nem arranca sozinho

Tempo Estimado:

NaN min

Intervenções (1)

- Atualização de Firmware

Peças (1)

- Cavalete Lateral (x1)

Data



30/04/2026



Hora



09:00



Sugerir Próximo Dia Útil (09:00)

Notas para o Mecânico

Instruções especiais...

Cancelar

Confirmar Agendamento

Figura 11.15: Diálogo de agendamento de reparação.

Tabela 11.8: Funcionalidades do perfil Mecânico

Funcionalidade	Rota/Página	Pré-condição	Resultado Esperado
Login de funcionário	/staff	Conta MECANICO ativa	Token JWT; redireciona para /FixNRepair.
Consultar agenda de diagnósticos	/FixNRepair/diagn	Autenticado	Lista de <i>slots</i> RESERVADO do tipo DIAGNOSTICO.
Registrar diagnóstico	/FixNRepair/diagn	Slot selecionado	Descrição guardada no serviço; PDF da guia gerado.
Agendar reparação	/FixNRepair/diagn	Diagnóstico concluído	Slot de reparação criado; serviço transitado para EXECUCAO.
Registrar peças via EAN	/FixNRepair/repar	Serviço em EXECUCAO	Peça adicionada ao histórico de intervenções do serviço.
Concluir serviço	/FixNRepair/repar	Serviço em EXECUCAO	Estado do serviço transitado para CONCLUIDO; trotinete disponível para levantamento.
Gerar guia de reparação	/FixNRepair/diagn	Diagnóstico registrado	Ficheiro PDF com dados do serviço e diagnóstico gerado localmente.

11.10 Perfil Administrador

Descrição do Perfil

O **Administrador** possui o nível de acesso mais elevado do sistema, com visibilidade total sobre todos os módulos operacionais e financeiros da **MobiFix**. Accede ao sistema via <http://localhost:3000/staff> e é redirecionado para o módulo /FixNManage. Para além do acesso exclusivo à gestão de utilizadores, promoções e estatísticas financeiras, o administrador tem também acesso aos módulos do operador (/FixNSell) e do mecânico (/FixNRepair).

Fluxo Principal: Gestão Operacional e Estratégica

1. Aceder a `http://localhost:3000/staff` e autenticar com o número mecanográfico (ex: ADM001) e *password*.
2. Após redirecionamento para `/FixNManage/dashboard`, consultar o *dashboard* de KPIs com a faturação total, número de serviços realizados, estatísticas de vendas e lista de devoluções recentes.
3. Para aprovar encomendas de reposição de *stock* pendentes, navegar para `/FixNManage/encomendas`. As encomendas no estado PENDENTE aguardam aprovação. Clicar em **Aprovar** para transitar para TRANSITO.
4. Para gerir utilizadores, navegar para `/FixNManage/users`. É possível criar novos funcionários, editar dados existentes e ativar/desativar contas.
5. Para criar uma promoção sazonal, navegar para `/FixNManage/promocoes` e preencher o formulário com a descrição, percentagem e período de vigência.
6. Para gerir o catálogo de peças e intervenções, navegar para `/FixNManage/pecas` e `/FixNManage/intervencoes` respetivamente.

Dashboard de KPIs

O *dashboard* (`/FixNManage/dashboard`) agrega os principais indicadores operacionais e financeiros da MobiFix: faturação total do período, número de serviços em cada estado, valor de vendas diretas, e lista de devoluções recentes. Os dados são calculados em tempo real pela camada de Lógica de Negócios a partir das coleções MongoDB. O administrador pode filtrar por intervalo de datas para análise de períodos específicos.

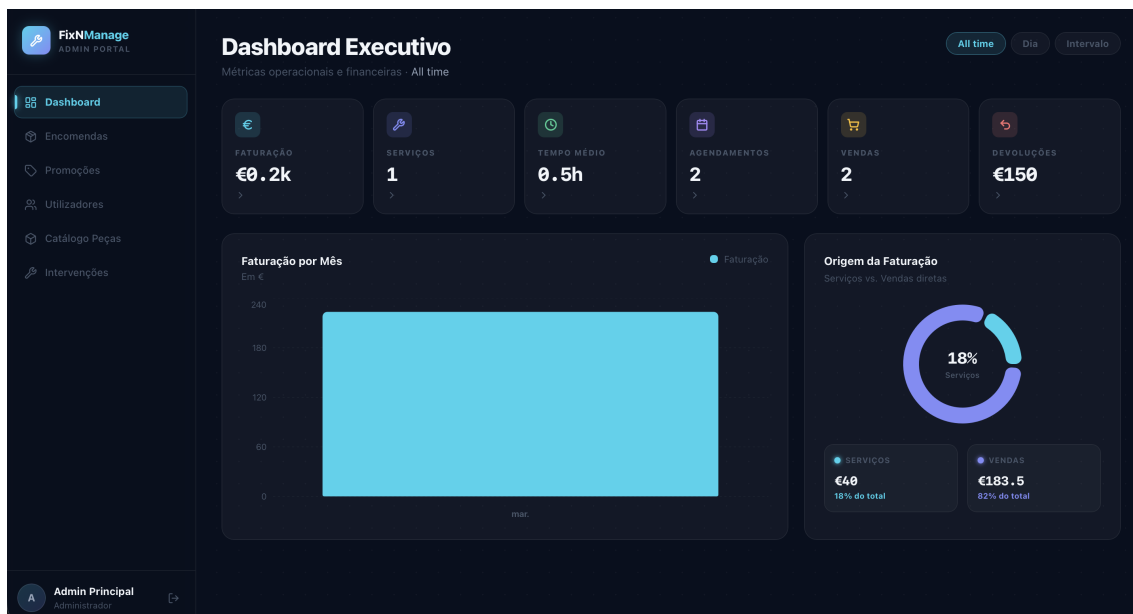


Figura 11.16: *Dashboard* do administrador.

Gestão de Encomendas de *Stock*

A página de encomendas (/FixNManage/encomendas) lista todas as ordens de reposição de inventário organizadas por estado. As encomendas no estado PENDENTE aguardam aprovação do administrador — ao clicar em **Aprovar**, a encomenda transita para TRANSITO, sinalizando ao fornecedor que a encomenda foi aceite. As encomendas RECECIONADAS constituem o histórico de entradas de *stock*.



Figura 11.17: Página de encomendas de *stock*.

Para criar uma nova ordem de encomenda manualmente, clicar em **Nova Encomenda** e preencher o formulário com a peça, a quantidade e o fornecedor. A encomenda é criada no estado PENDENTE e aguarda aprovação pelo próprio administrador.

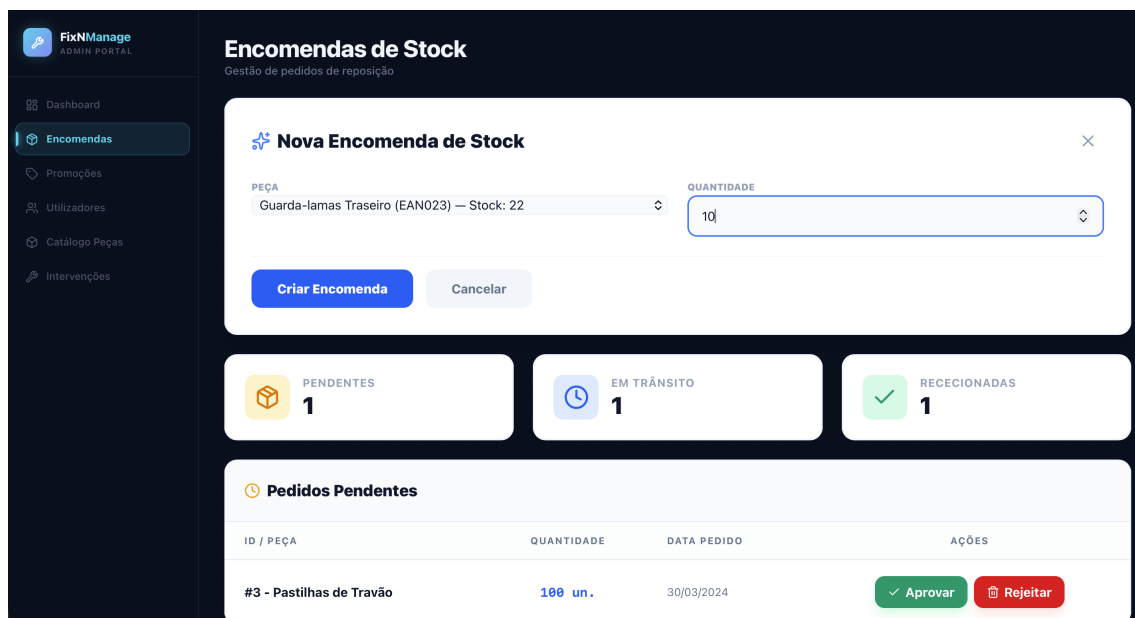


Figura 11.18: Formulário de criação de encomenda de *stock*.

Gestão de Promoções

A página de promoções (/FixNManage/promocoos) lista todas as promoções ativas e inativas. Cada promoção define uma percentagem de desconto, um período de vigência (data de início e fim) e as peças abrangidas. O botão de alternância ativo/inativo permite suspender uma promoção sem a eliminar, preservando o histórico.

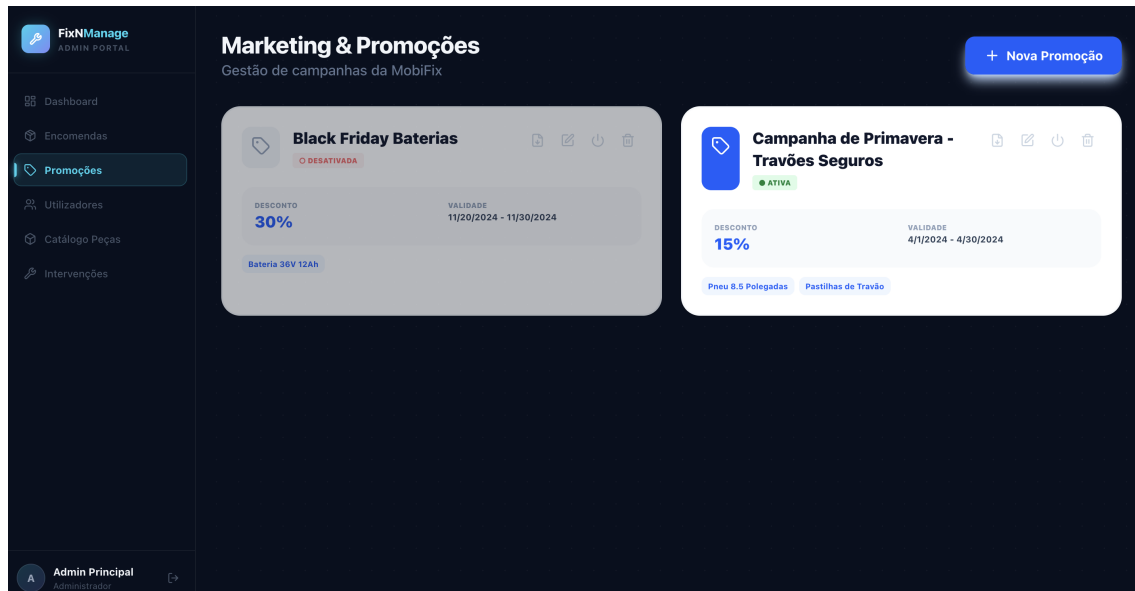


Figura 11.19: Página de promoções.

O formulário de criação de promoção permite definir a descrição, a percentagem de desconto (0–100%), as datas de início e fim e as peças do catálogo às quais se aplica. Após submissão, a promoção é imediatamente ativada se a data de início for igual ou anterior à data atual.

Nova Campanha

NOME: Campanha de verão

DESCONTO (%): 10

INÍCIO: 04/03/2026

FIM: 04/29/2026

PEÇAS EM CAMPANHA

☒ Cavelete Lateral EAN024 - 7.90€	☒ Guarda-lamas Traseiro EAN023 - 9.90€	☐ Luz Dianteira LED EAN021 - 12.90€
☐ Luz Traseira Vermelha EAN022 - 9.50€	☐ Selo de Garantia EAN058 - 0.50€	☐ Bateria 36V 12Ah EAN003 - 150.00€
☐ Bateria 48V 15Ah EAN010 - 245.00€	☐ Carregador 42V 2A EAN014 - 22.00€	☐ Controlador 36V EAN011 - 55.00€

2 peça(s) selecionada(s)

Lançar Cancelar

Figura 11.20: Formulário de criação de promoção.

Gestão de Funcionários

A página de gestão de utilizadores (`/FixNManage/users`) apresenta todos os funcionários da MobiFix em formato de tabela, com indicação do cargo, especialidade e estado da conta (ativo/inativo). A desativação de uma conta impede futuros *logins* sem eliminar o histórico de serviços associados ao funcionário, preservando a rastreabilidade operacional.

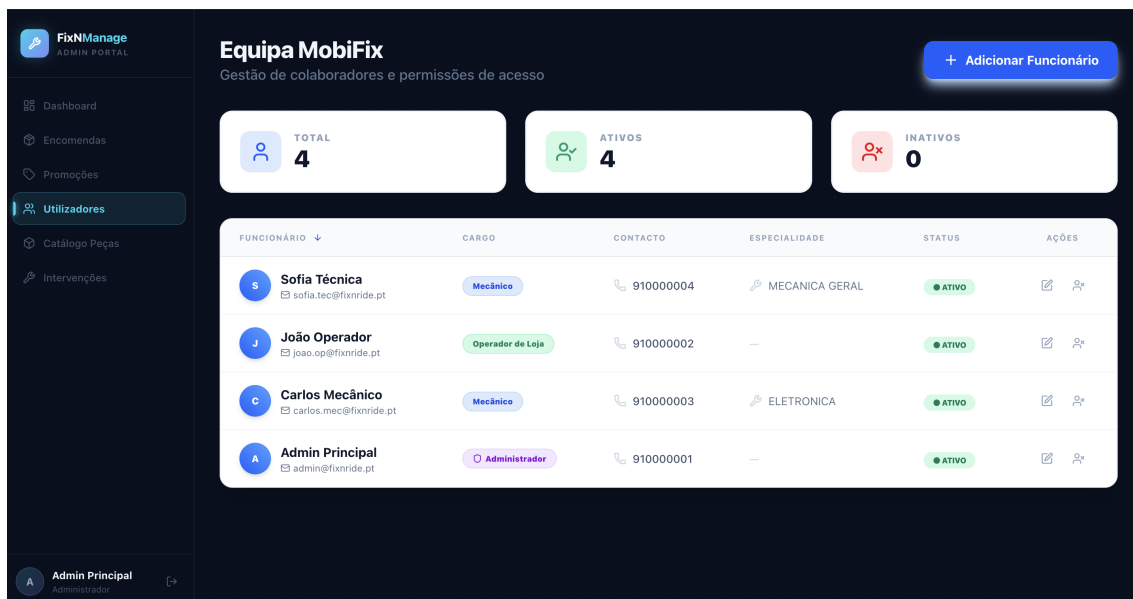


Figura 11.21: Página de gestão de funcionários.

O formulário de criação de funcionário requer número mecanográfico (identificador único), nome, e-mail, contacto, cargo (MECANICO, OPERADOR ou ADMINISTRADOR), especialidade (opcional, relevante para mecânicos) e *password*. A *password* é cifrada com BCrypt antes de ser persistida — nunca é armazenada em texto claro.

The screenshot shows the 'Registar Colaborador' form within the 'Equipa MobiFix' interface. The sidebar and header are identical to Figure 11.21. The form contains the following fields:

- NÚMERO MECANOGRÁFICO:** Text input with value 'ADM002'.
- NOME COMPLETO:** Text input with value 'Orlando Belo'.
- EMAIL PROFISSIONAL:** Text input with value 'obelo@mobiFix.pt'.
- CONTACTO:** Text input with value '99999999'.
- CARGO:** Dropdown menu with 'Administrador' selected.
- PASSWORD:** Password input field with masked characters '.....'.

At the bottom of the form are two buttons: 'Registar Colaborador' (in blue) and 'Cancelar' (in light gray).

Figura 11.22: Formulário de criação de funcionário.

Gestão do Catálogo de Peças

A página do catálogo de peças (/FixNManage/pecas) permite ao administrador criar novas peças, atualizar preços (PVP e custo de aquisição), definir limiares de *stock* mínimo e ativar ou desativar peças. Quando o *stock* de uma peça atinge o limiar mínimo, o sistema gera um alerta visível na interface e pode desencadear uma proposta automática de encomenda de reposição.



PEÇA	CATEGORIA	PREÇO UNITÁRIO	DISPONIBILIDADE	STATUS	AÇÕES
Bateria 36V 12Ah EAN003	ELETRONICA	150.00 €	3 un. BAIXO STOCK	ATIVO	 
Bateria 48V 15Ah EAN010	ELETRONICA	249.00 €	5 un. BAIXO STOCK	ATIVO	 
Carregador 42V 2A EAN014	ELETRONICA	22.00 €	30 un.	ATIVO	 
Cavalete Lateral EAN024	ACESSORIOS	7.90 €	30 un.	ATIVO	 
Controlador 36V EAN011	ELETRONICA	55.00 €	12 un.	ATIVO	 
Câmara de Ar 10" EAN006	PNEU	7.50 €	60 un.	ATIVO	 

Figura 11.23: Catálogo de peças.

O formulário de criação de peça requer o código EAN (identificador único), nome, categoria, PVP, custo de aquisição, stock mínimo e uma imagem opcional. O código EAN é validado como único no momento da submissão — a tentativa de criar uma peça com EAN duplicado resulta em erro HTTP 400.

FixNManage
ADMIN PORTAL

Catálogo de Peças
Gestão de inventário e preço

+ Nova Peça

TOTAL DE REFERÊNCIAS
18

ATIVAS SEM STOCK
0

VALOR EM ARMAZÉM (ATIVO)
8478.30€

Registrar Nova Peça

CÓDIGO EAN / REFERÊNCIA: EAN00000

NOME DA PEÇA: Bateria 100V

CATEGORIA: Baterias

PREÇO VENDA (€): 100

CUSTO DE AQUISIÇÃO (€): 30

STOCK ATUAL: 0

DESCRIÇÃO (OPCIONAL): Bateria de Alta voltagem para trotinetes de alta potência

Adicionar ao Catálogo Cancelar

Figura 11.24: Formulário de criação de peça.

Gestão do Catálogo de Intervenções

O catálogo de intervenções (/FixNManage/intervencoes) define os tipos de trabalho que os mecânicos podem registar nos serviços, com o respetivo preço fixo de mão de obra. Cada intervenção tem uma especialidade associada (*Elétrico*, *Mecânico*, *Diagnóstico*), que é usada para filtrar as opções disponíveis ao mecânico durante a execução de um serviço.

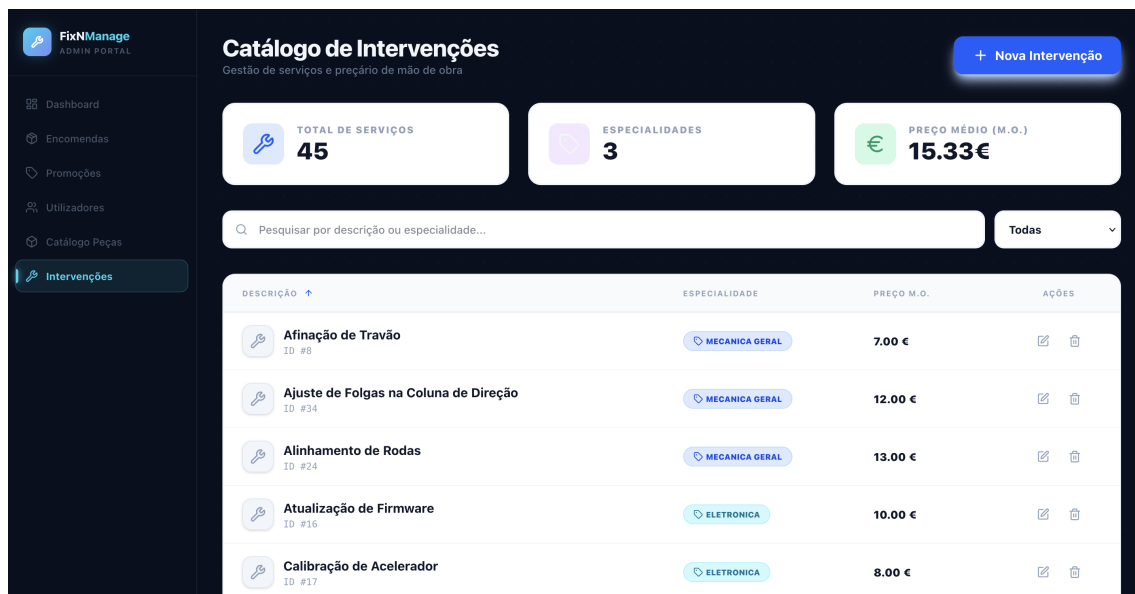


Figura 11.25: Catálogo de intervenções.

O formulário de criação de intervenção requer a descrição, o preço fixo de mão de obra e a especialidade. A especialidade determina em que contexto a intervenção fica disponível para seleção pelo mecânico — uma intervenção de especialidade *Elétrico* só aparece quando o mecânico filtra por essa categoria no painel de diagnóstico.

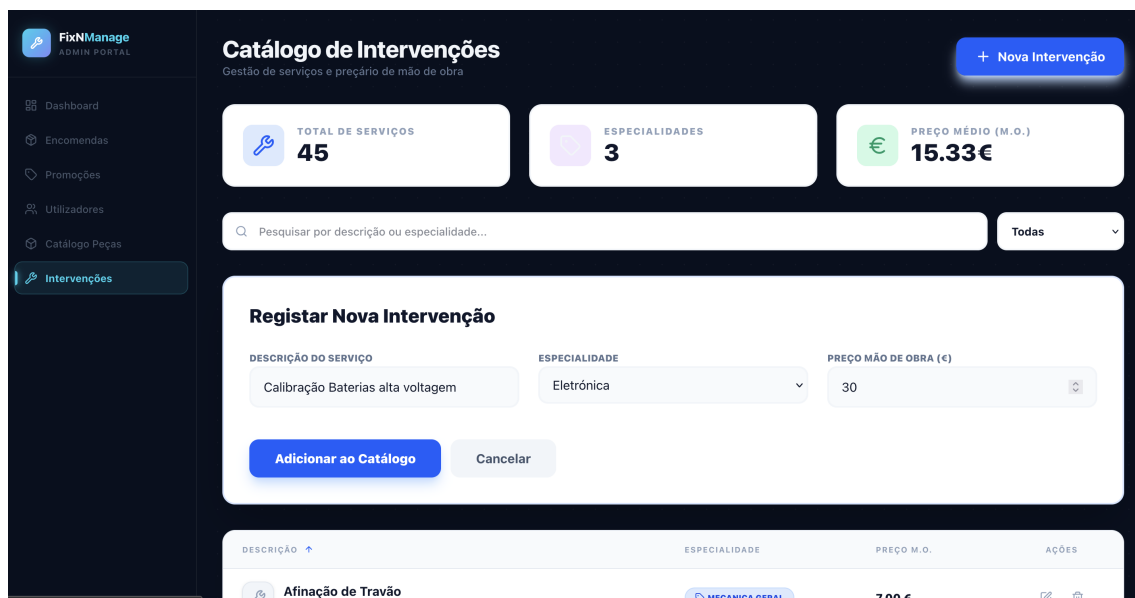


Figura 11.26: Formulário de criação de intervenção.

Tabela 11.9: Funcionalidades do perfil Administrador

Funcionalidade	Rota/Página	Pré-condição	Resultado Esperado
Login de administrador	/staff	Conta ADMINISTRADOR ativa	Token JWT com role ADMINISTRADOR; redireciona para /FixNManage.
Consultar <i>dashboard</i> de KPIs	/FixNManage/dashb	Autenticado	Métricas financeiras agregadas (faturação, serviços, devoluções, vendas).
Aprovar encomenda de stock	/FixNManage/encom	Encomenda PENDENTE	Encomenda transitada para TRANSITO; aguarda receção pelo operador.
Criar funcionário	/FixNManage/users	Autenticado	Funcionário criado com password BCrypt; disponível para login.
Editar/Desativar funcionário	/FixNManage/users	Funcionário existente	Dados atualizados ou conta desativada sem eliminação do histórico.
Criar promoção	/FixNManage/promo	Autenticado	Promoção criada e ativada; desconto aplicável nas peças selecionadas.
Ativar/Desativar promoção	/FixNManage/promo	Promoção existente	Estado da promoção alterado via PATCH sem modificar outros campos.
Gerir catálogo de peças	/FixNManage/pecas	Autenticado	Peças criadas, editadas (preço, stock mínimo) ou desativadas do catálogo.
Gerir catálogo de intervenções	/FixNManage/inter	Autenticado	Intervenções criadas ou editadas; visíveis ao mecânico por especialidade.
Acesso ao módulo do Operador	/FixNSell/*	Autenticado	Todas as funcionalidades do perfil Operador disponíveis.
Acesso ao módulo do Mecânico	/FixNRepair/*	Autenticado	Todas as funcionalidades do perfil Mecânico disponíveis.

Lista de Siglas e Acrónimos

LLM	<i>Large Language Model</i>
LN	<i>Lógica de Negócios</i>
SQL	<i>Structured Query Language</i>
.NET	<i>Framework de desenvolvimento da Microsoft</i>
C#	<i>Linguagem de programação orientada a objetos</i>
UML	<i>Unified Modeling Language</i>
BD	<i>Base de Dados</i>
API	<i>Application Programming Interface</i>
SPA	<i>Single Page Application</i>
CSS	<i>Cascading Style Sheets</i>
OS	<i>Ordem de Serviço</i>
US	<i>User Story</i>
UC	<i>Use Case</i>
RF	<i>Requisito Funcional</i>
RNF	<i>Requisito Não Funcional</i>
RBAC	<i>Role-Based Access Control</i>
SRS	<i>Software Requirements Specification</i>
ROI	<i>Return on Investment</i>

OPEX *Operating Expenses*

NIF *Número de Identificação Fiscal*

PVP *Preço de Venda ao Público*

KPI *Key Performance Indicator*

EAN *European Article Number*

RGPD *Regulamento Geral sobre a Proteção de Dados*

PDF *Portable Document Format*

CSV *Comma-Separated Values*

SMTP *Simple Mail Transfer Protocol*

MBWay *Sistema de pagamento eletrónico*

Anexos

Modelo de Domínio Completo

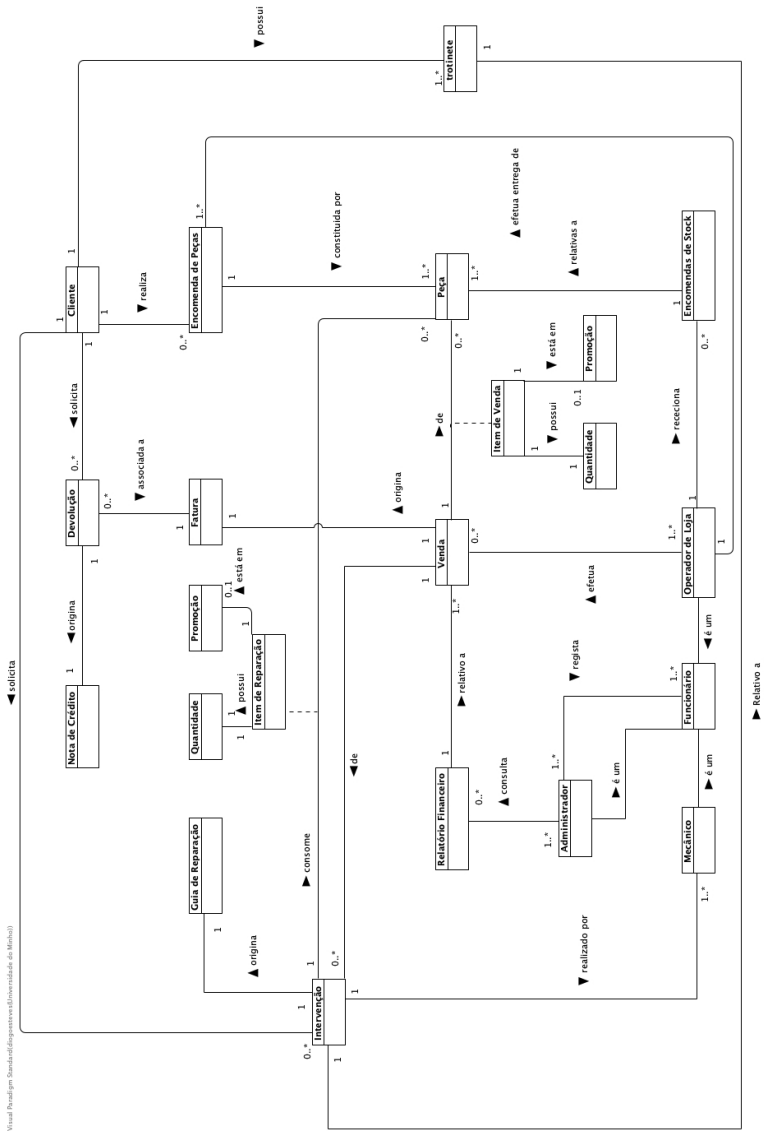


Figura 11.1: Modelo de Domínio (Versão Ampliada)

Log de Prompts e Técnicas de Inteligência Artificial

Neste anexo, listam-se os prompts e técnicas de Inteligência Artificial utilizados ao longo do ciclo de vida do projeto *Fix'n'Ride*, abrangendo desde a fase de concepção e engenharia de requisitos até ao design de interfaces, implementação e validação. A IA atuou sempre como agente auxiliar, partindo de dados e regras de negócio definidos pela equipa técnica. As ferramentas utilizadas incluem modelos de linguagem de grande escala (LLMs) como o **Claude** (Anthropic) e o **GPT-4o** (OpenAI), o assistente de desenvolvimento integrado **Antigravity** (Google DeepMind), bem como a ferramenta de design colaborativo **Figma** com os seus *plugins* de geração de UI assistida por IA.

Fase 1 — Engenharia de Requisitos e Concepção

Prompt 1: Contextualização e Visão Estratégica

Ferramenta: GPT-4o (OpenAI). Objetivo: Gerar o enquadramento do projeto no panorama das Smart Cities.

"Atua como um Consultor de IT especializado em mobilidade urbana. Escreve uma contextualização técnica sobre a evolução da micromobilidade elétrica na última década e como as Smart Cities exigem agora uma digitalização profunda dos serviços de assistência técnica."

Resultado: Secção 1.1.1 (Contextualização).

Prompt 2: Transformação de Notas em User Stories

Ferramenta: Claude Sonnet 4.6. Objetivo: Padronizar as necessidades dos stakeholders recolhidas em entrevistas.

"Com base nos perfis de Administrador, Mecânico e Cliente da oficina MobiFix, gera uma lista de User Stories seguindo o formato: 'Como [ator], quero [ação], para que [benefício]'. Garante que cubras os fluxos de agendamento, gestão de stock e faturação."

Resultado: Secção 1.1.8 (User Stories).

Prompt 3: Refinamento de Requisitos Ambíguos

Ferramenta: Claude Sonnet 4.6. Objetivo: Converter linguagem informal em especificações técnicas mensuráveis.

"Refina as seguintes frases de stakeholders para requisitos técnicos claros:

```
1. 'O sistema deve ser rápido'.
2. 'O cliente deve ser avisado'.
3. 'Controlar melhor o stock'.
Define métricas como tempos de resposta em segundos e métodos
de notificação."
```

Resultado: Tabela 1.2 (Processo de Refinamento e Clarificação).

Prompt 4: Detalhamento de Casos de Uso

Ferramenta: GPT-4o. Objetivo: Estruturar o fluxo interativo e condições de exceção para operações críticas.

```
"Escreve a especificação detalhada do Caso de Uso 'Preencher
guia de reparacao'. Inclui:
1. Atores envolvidos (Mecânico de Triage).
2. Pré-condicoes e Pós-condições.
3. Fluxo normal de eventos (desde a seleção na agenda
até ao envio de email).
4. Fluxos de exceção (ex: cliente não concorda com o
orçamento)."
```

Resultado: Secção 1.3.3 (Tabela 1.37).

Prompt 5: Definição da Stack Tecnológica e Restrições

Ferramenta: GPT-4o. Objetivo: Estabelecer as diretrizes técnicas de implementação.

```
"Com base num sistema de gestão de oficina web-based, sugere
uma arquitetura de software multi-tier. Considera o uso de
.NET para o backend e React.js para o frontend. Define tambem
as restrições de persistencia de dados utilizando MongoDB
e a necessidade de uma interface responsiva com Tailwind CSS."
```

Resultado: Secção 1.4 (Especificação do Software).

Prompt 6: Definição de Estados e Transições

Ferramenta: Claude Sonnet 4.6. Objetivo: Mapear o ciclo de vida das entidades para os diagramas de máquina de estados.

```
"Atua como analista de sistemas. Define os estados possiveis
para uma 'Ordem de Servico' numa oficina de trotinetes, desde
o agendamento ate ao levantamento final. Explica quais os
eventos que disparam a transicao entre estados (ex: o que
faz passar de 'Em Execucao' para 'Concluido') para ajudar na
criacao de um diagrama de estados UML."
```

Resultado: Secção 1.7 (Figura 1.17 — Diagrama de Máquina de Estados).

Prompt 7: Estruturação de Indicadores de Desempenho (KPIs)

Ferramenta: GPT-4o. Objetivo: Definir a lógica para a geração de relatórios operacionais e financeiros.

```
"A MobiFix precisa de relatorios mensais para a administracao.
Sugere uma estrutura de dados e formulas para calcular:
- Tempo medio de reparacao (Lead Time).
- Divisao de faturacao entre pecas e mao-de-obra.
- Alerta de pecas com maior volume de saida para
  otimizacao de stock minimo."
```

Resultado: Secção 1.2.4 (Requisito Funcional 15 — Tabela 1.17).

Fase 2 — Design de Interfaces com Figma e IA

A fase de design de interfaces recorreu ao **Figma** como ferramenta central de prototipagem, complementado com o *plugin Figma AI* para geração assistida de layouts e o **Wireframe Designer** para esboços rápidos. O processo seguiu uma metodologia iterativa: geração de *wireframes* de baixa fidelidade → revisão da equipa → refinamento para *mockups* de alta fidelidade → handoff para implementação React.

Prompt 8: Geração de Wireframes com Figma AI

Ferramenta: Figma AI (plugin First Draft). Objetivo: Gerar esboços iniciais dos portais por perfil de utilizador.

```
"Generate a mobile-first dashboard wireframe for a scooter repair shop client portal. Include: bottom navigation bar with 4 tabs (Home, My Scooters, Schedule, Invoices), a hero welcome card with the user's name, 2 quick-action buttons (Schedule Diagnosis, Parts Catalog), and a recent activity feed. Use a clean, card-based layout."
```

Resultado: *Wireframe* inicial da HomeClientePage (/FixNRide), posteriormente refinado em *mockup* de alta fidelidade com a paleta azul/roxo definida pela equipa.

Prompt 9: Refinamento Visual do Portal do Administrador

Ferramenta: Figma AI + revisão manual da equipa. Objetivo: Definir a linguagem visual do dashboard de KPIs.

```
"Design a desktop admin dashboard for a repair shop management system. Include: a fixed left sidebar with dark background (gray-900) and navigation items (Dashboard, Orders, Users, Promotions, Parts), a top header with the admin's name and logout button, and a main content area with 4 KPI metric cards (Total Revenue, Active Services, Pending Orders, Monthly Growth) and a revenue chart below."
```

Resultado: Estrutura visual do AdminDashboard (/FixNManage), utilizada como referência de *design* durante a implementação React com Tailwind CSS.

Prompt 10: Componente Kanban de Encomendas

Ferramenta: Figma com plugin Autoflow. Objetivo: Prototipar o fluxo visual de estados de encomendas de stock.

```
"Create a Kanban-style board with 3 columns: PENDENTE, EM TRANSITO, RECECIONADA. Each card should show: part name, EAN code, quantity, estimated cost,"
```

```
supplier name, and action buttons (Approve/Reject for  
PENDENTE column, Confirm Receipt for EM TRANSITO).  
Use a light background with colored column headers."
```

Resultado: *Mockup* do módulo `EncomendasStock` do administrador, com as três colunas Kanban implementadas posteriormente em React.

Fase 3 — Implementação com Agentes de IA (Antigravity)

Durante a fase de implementação, a equipa utilizou o assistente de desenvolvimento **Antigravity** (Google DeepMind), um agente de IA integrado no ambiente de desenvolvimento que permite interações contextuais com o código existente, navegação autónoma no repositório, execução de comandos e aplicação de edições multi-ficheiro. Ao contrário de modelos de chat tradicionais, o Antigravity opera como um **agente autónomo** com acesso ao sistema de ficheiros, terminais e ao browser, acelerando significativamente tarefas complexas de *scaffolding* e refactorização.

Prompt 11: Scaffolding do Schema Mongoose e Controller Express

Ferramenta: Antigravity (agente). Objetivo: Gerar a camada de dados e API para a entidade Serviço.

```
"Cria o schema Mongoose para a entidade 'Serviço' com os  
campos: trotineteId (ref), mecanicoId (ref), estado  
(enum: AGENDADO/EXECUCAO/CONCLUIDO/FECHADO),  
intervencoes (array de subdocumentos com intervencaoId  
e pecasUsadas), timestamps de inicio e fim, e  
feedbackCliente. Depois cria o controller Express com  
os endpoints: GET /servicos, GET /servicos/:id,  
POST /servicos, PUT /servicos/:id, e GET /servicos  
com filtro por estado via query param."
```

Resultado: Ficheiros `servico.model.js` e `servicos.controller.js` da *Data API*, com validação Mongoose e mapeamento DTO em PascalCase.

Prompt 12: Geração do Serviço .NET com HttpClient

Ferramenta: Antigravity (agente com acesso ao repositório). Objetivo: Implementar a lógica de levantamento atómico de trotinete.

```
"No projeto ASP.NET Core, cria o ServicoService com o  
método Levantar(id, metodoPagamento, operadorNumero).  
O método deve: 1) Verificar se o serviço existe e está  
no estado CONCLUIDO; 2) Calcular o total somando peças
```



```
e mão-de-obra das intervenções; 3) Criar uma Fatura via POST /api/faturas na Data API; 4) Atualizar o estado do serviço para FECHADO; 5) Devolver um DTO LevantamentoComFaturaDto com o serviço e a fatura. Usa HttpClient injetado por DI e trata os erros HTTP."
```

Resultado: ServicoService.cs com o fluxo de levantamento atômico, validado pelo caso de teste TC-15.

Prompt 13: Hook TanStack Query com Invalidação de Cache

Ferramenta: Antigravity (agente). Objetivo: Criar o hook React para gestão de encomendas de stock com cache.

```
"Cria o hook useEncomendasStock.js usando TanStack Query. Deve incluir: useQuery para listar encomendas com staleTime de 15 segundos; useMutation para aprovar (PUT /encomendas/stock/:id com estado TRANSITO) com invalidação da query key ['encomendas-stock'] no onSuccess; useMutation para rejeitar com confirmação; e useMutation para confirmar receção com estado RECECIONADA. Usa a instância axios de src/services/api.js."
```

Resultado: useEncomendasStock.js integrado no módulo do administrador, com as três mutações e gestão automática de cache.

Prompt 14: Refactorização Multi-ficheiro Assistida por Agente

Ferramenta: Antigravity (agente com acesso ao sistema de ficheiros). Objetivo: Centralizar a função parseJwt dispersa em vários componentes.

```
"Identifica todos os ficheiros no directório src/ que contenham a função parseJwt inline. Extrai-a para src/utils/auth.js exportando também getAuthPayload(), getOperadorNumero() e logout(). Substitui todas as instâncias inline por importações do novo módulo. Lista os ficheiros alterados e confirma que não ficaram referências à função inline."
```

Resultado: Criação de src/utils/auth.js e refactorização de 5 componentes (AppRouter.jsx, Layout.jsx, AdminLayout.jsx, HomeClientePage.jsx, EncomendasStock.jsx), eliminando o *code smell* de duplicação identificado na secção 8.4.

Fase 4 — Testes e Documentação Assistidos por IA

Prompt 15: Geração de Casos de Teste *Edge Case*

Ferramenta: Claude Sonnet 4.6. Objetivo: Identificar cenários de erro não contemplados pela equipa.

```
"Dado o seguinte endpoint: PUT /api/Servicos/{id}/levantar
que recebe MetodoPagamento e exige que o serviço esteja
no estado CONCLUIDO, sugere 6 casos de teste de fronteira
(edge cases) que a equipa pode ter esquecido, focando em:
estados inválidos, campos em falta, IDs inexistentes,
problemas de concorrência e validação de pagamento."
```

Resultado: Adição dos casos de teste TC-12, TC-14, TC-19 e TC-24 à coleção Postman (Secção 8.2).

Prompt 16: Geração Automática de Dados de Seed

Ferramenta: GPT-4o com Code Interpreter. Objetivo: Criar os ficheiros JSON de dados iniciais para o MongoDB.

```
"Gera um ficheiro JSON com 8 funcionários realistas para
uma oficina de trotinetes portuguesa. Inclui os perfis:
2 Administradores, 3 Mecânicos (com especialidades
diferentes: elétrico, mecânico, diagnóstico), 2 Operadores
de Loja e 1 Mecânico de Triagem. Cada documento deve ter:
numero (ex: ADM001), nome, email, cargo, especialidade,
ativo:true, e password já com hash BCrypt."
```

Resultado: Ficheiro funcionarios.json utilizado no *script* de inicialização init.sh (Secção 4.3.2).

Prompt 17: Revisão e Síntese da Documentação Técnica

Ferramenta: Antigraity (agente com acesso ao repositório e ao relatório LaTeX). Objetivo: Garantir coerência entre o código implementado e a documentação.

```
"Lê os ficheiros de código da Data API (controllers e
models) e compara com a tabela de endpoints documentada
no relatório LaTeX. Identifica discrepâncias entre os
endpoints reais implementados e os documentados,
sugere atualizações ao relatório e verifica se todos
os campos dos DTOs estão corretamente descritos."
```

Resultado: Identificação de 3 endpoints não documentados na camada de Finanças, subsequentemente adicionados à Tabela de Endpoints do Capítulo 6.

Testes Postman

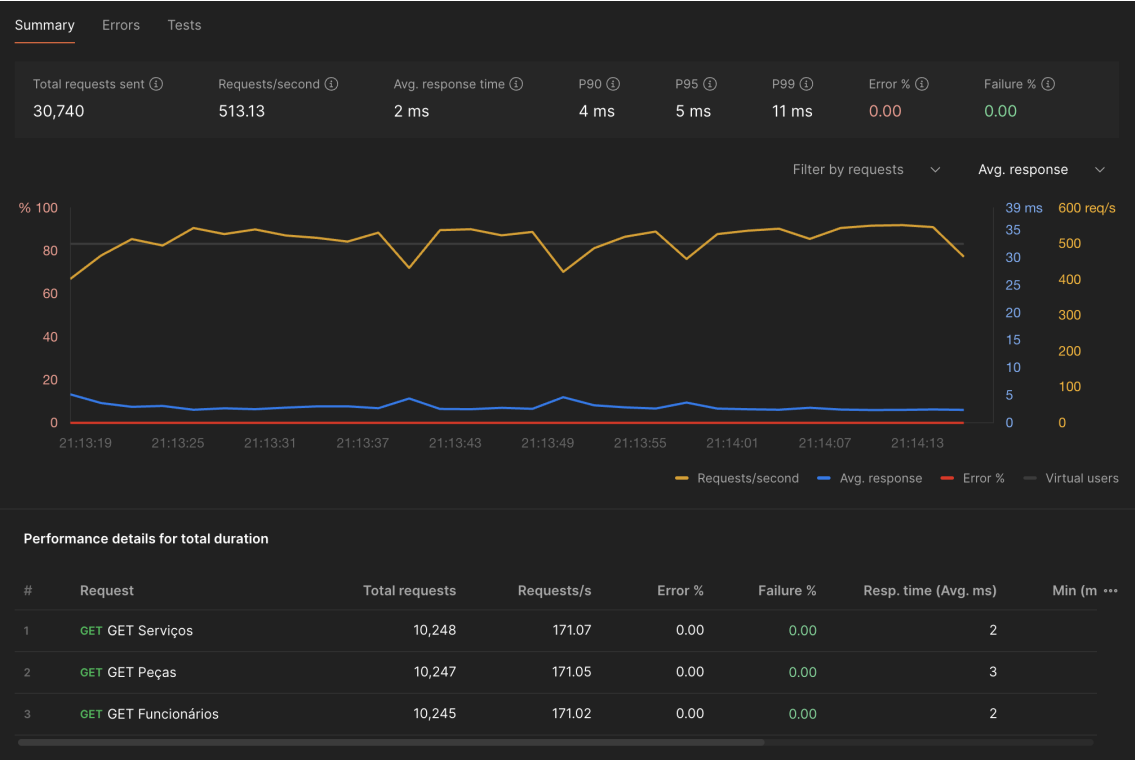


Figura 11.2: Cenário de stress Base

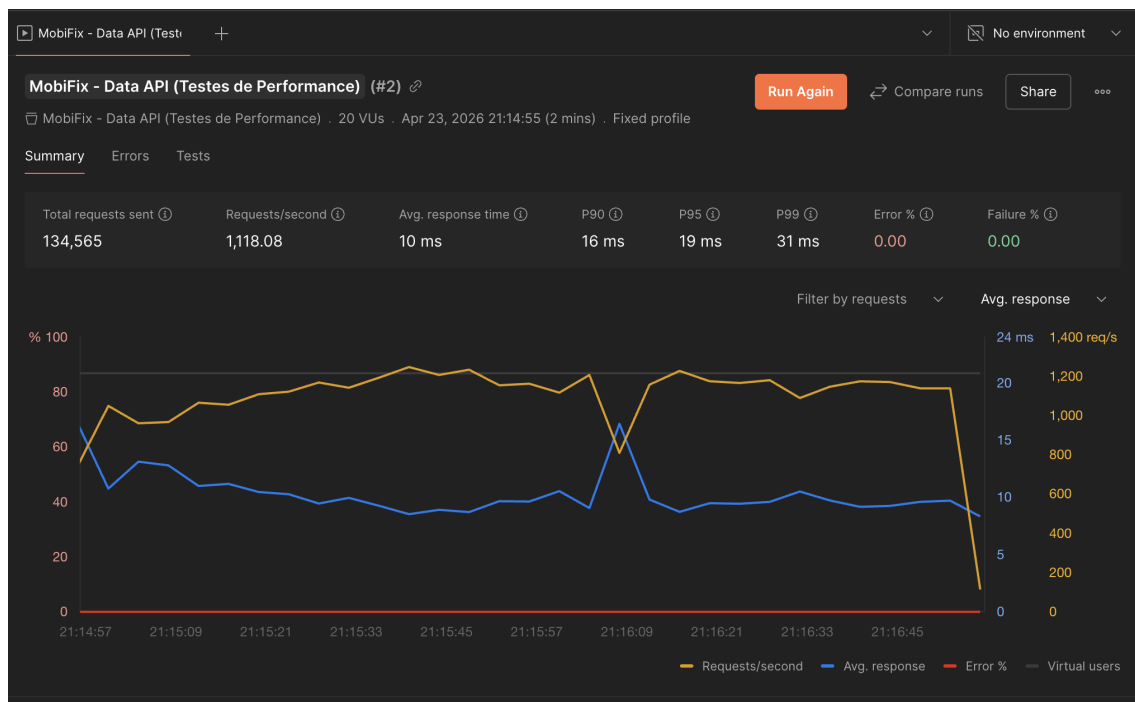


Figura 11.3: Cenário de stress moderado

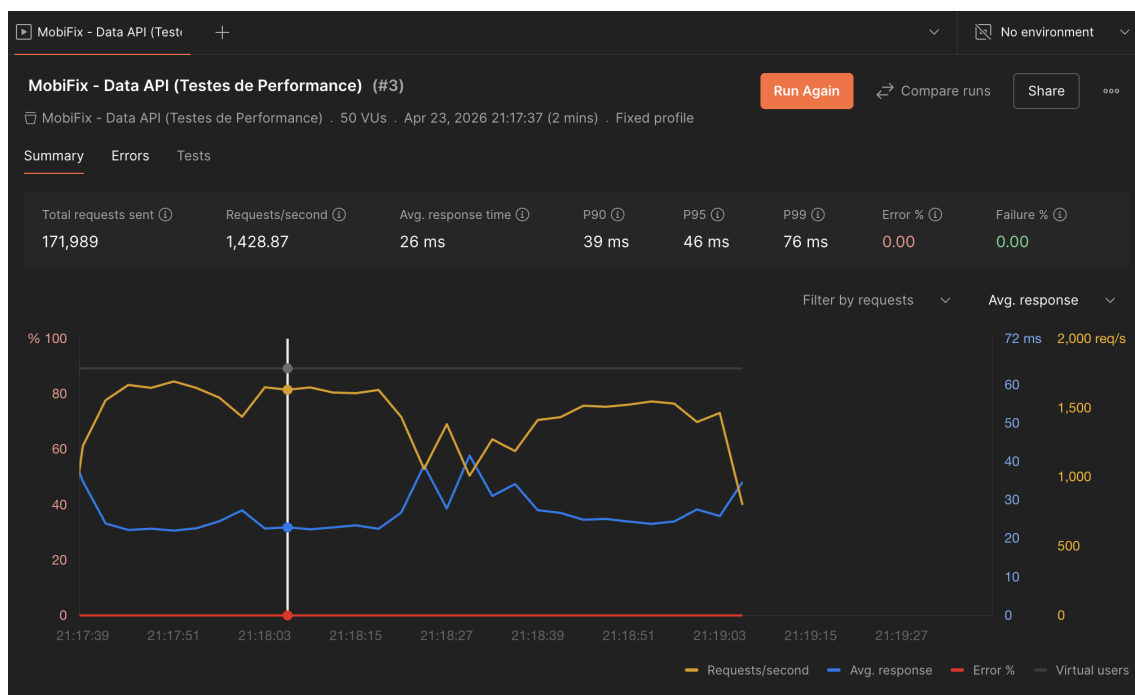


Figura 11.4: Cenário de stress intensivo

Resumo das Ferramentas de IA Utilizadas

Tabela 11.1: Resumo das ferramentas de IA utilizadas no projeto Fix'n'Ride por fase e tipo

Ferramenta	Tipo	Utilizações Principais no Projeto
Claude Sonnet 4.6	LLM (chat)	Engenharia de requisitos, refinamento de texto técnico, casos de teste, revisão de código
GPT-4o	LLM (chat)	Contextualização, geração de dados de seed, especificação de casos de uso
Antigravity	Agente autónomo	Scaffolding de código multi-camada, re-factorização multi-ficheiro, validação de documentação vs. código
Figma AI (First Draft)	IA de design	Geração de wireframes e mockups de alta fidelidade por perfil de utilizador
GitHub Copilot	LLM inline	Autocompletar código repetitivo (DTOs, imports, classes C#), durante sessões de desenvolvimento